

Skip List – Introduction

Skip Lists

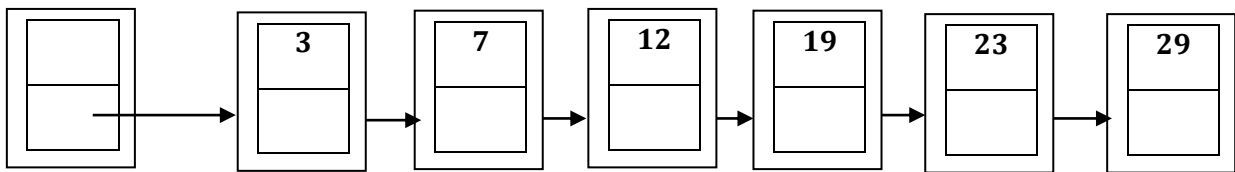
Binary trees can be used for representing abstract data types such as dictionaries and ordered lists. They work well when the elements are inserted in a random order. Some sequences of operations, such as inserting the elements in order, produce degenerate data structures that give very poor performance. If it were possible to randomly permute the list of items to be inserted, trees would work well with high probability for any input sequence. In most cases queries must be answered on – line, so randomly permuting the input is impractical. Balanced tree algorithms re – arrange the tree as operations are performed to maintain certain balance conditions and assure good performance.

Skip lists are a probabilistic alternative to balanced trees. Skip list is a data structure that can be used as an alternative to balanced binary trees . As compared to a binary tree, skip lists allow quick search, insertion and deletion of elements. This is achieved by using probabilistic balancing rather than strictly enforce balancing. It is basically a linked list with additional pointers such that intermediate nodes can be skipped. It uses a random number generator to make some decisions.

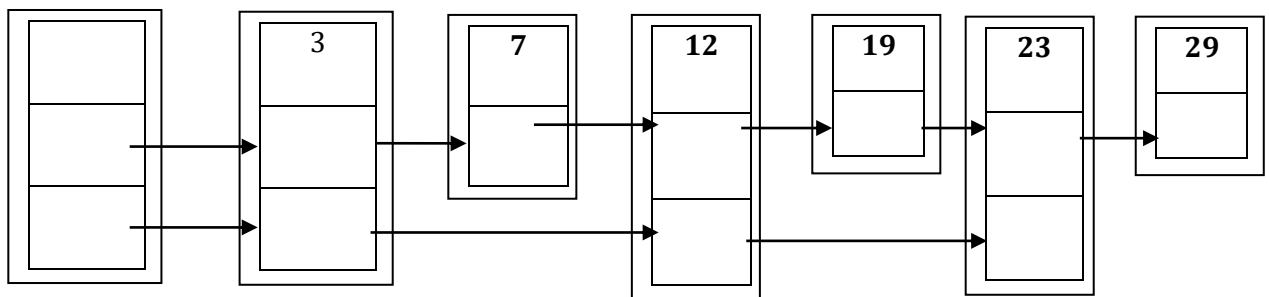
In an ordinary sorted linked list, search, insert, and delete are in $O(n)$ because the list must be scanned node – by – node from the head

to find the relevant node. If somehow we could scan down the list in bigger steps (skip down, as it were), we would reduce the cost of scanning. This is the fundamental idea behind Skip Lists.

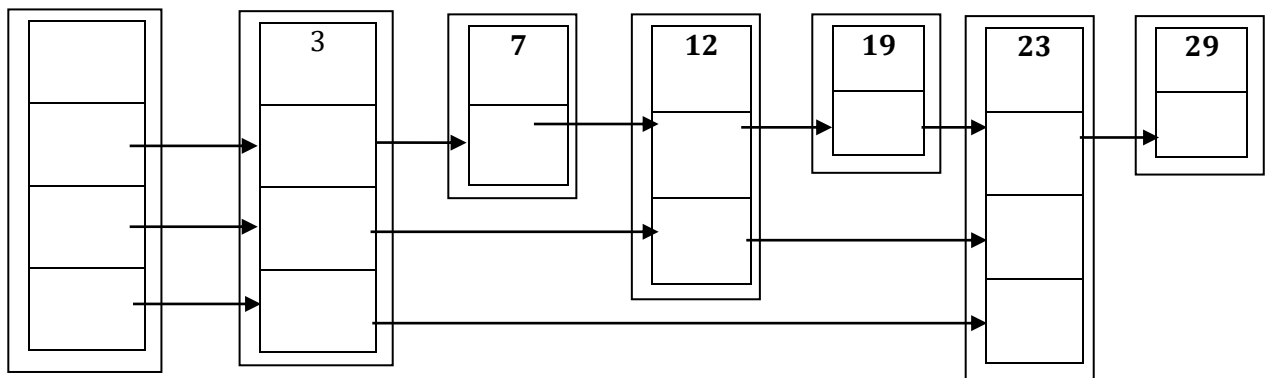
Skip Lists with One Level



Skip Lists with Two Level



Skip Lists with Three Level



Performance

In a simple linked list that consists of n elements, to perform a search n comparisons are required in the worst case. If a second pointer pointing two nodes ahead is added to every node, the number of comparisons goes down to $\frac{n}{2} + 1$ in the worst case.

Adding one more pointer to every fourth node and making them point to the fourth node ahead reduces the number of comparisons to $\left\lceil \frac{n}{4} \right\rceil + 2$. If this strategy is continued so that every node with i pointers points to 2^{i-1} nodes ahead, $O(\log n)$ performance is obtained and the number of pointers has only doubled $\left(n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \frac{n}{16} + \dots = 2n \right)$.

Level	Pointer spacing [No. of Pointers]	Number of nodes having that pointer
1	1	$\frac{n}{1}$
2	2	$\frac{n}{2}$
3	4	$\frac{n}{4}$
4	8	$\frac{n}{8}$
5	16	$\frac{n}{16}$
...	...	...

So the i – th level skips approximately:

$$\boxed{2^{i-1}}$$

nodes.

For example:

$$i = 1 \Rightarrow 2^0 = 1$$

$$i = 2 \Rightarrow 2^1 = 2$$

$$i = 3 \Rightarrow 2^2 = 4$$

$$i = 4 \Rightarrow 2^3 = 8$$

and so on.

Suppose there are $n = 16$ elements.

The levels can jump:

$$1, 2, 4, 8, 16$$

The number of levels needed is approximately:

$$\log_2 n$$

because,

$$2^k \approx n$$

gives,

$$k \approx \log_2 n \left[\begin{array}{c} \text{By Logarithmic} \\ \text{Definitions} \\ \text{if } a^n = b, \text{ then:} \\ \log_a b = n \end{array} \right]$$

Therefore, the search can be performed in approximately

$$\boxed{O(\log n)}$$

expected time.

At each level, fewer nodes need the extra pointer:

$$n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \frac{n}{16} + \dots$$

Factor out n:

$$n \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots \right)$$

The expression inside the parentheses is an infinite geometric series:

$$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots = 2$$

Therefore:

$$n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots = 2n$$

So the total number of forward – pointer slots is approximately 2n.

We have increased the pointer storage from approximately ' n ' pointers in an ordinary linked list to approximately ' $2n$ ' pointers in the skip – list structure. Yet the search performance improves from ' $O(n)$ ' in an ordinary linked list to ' $O(\log n)$ ' in the skip – list.

In simple terms, Skip Lists are sorted linked lists with two differences:

- *The nodes in an ordinary list have one next reference. The nodes in a Skip List have many next references (also called forward references).*
- *The number of forward references for a given node is determined probabilistically.*

We speak of a Skip List node having levels, one level per forward reference.

The number of levels in a node is called the size of the node.

*In an ordinary sorted list, **insert**, **remove**, and **find** operations require sequential traversal of the list. This results in $O(n)$ performance per operation. Skip Lists allow intermediate nodes in the list to be skipped during a traversal – resulting in an expected performance of $O(\log n)$ per operation.*

Skip List – Some Important Information

```
#include <iostream>
#include <cstdlib>
#include <cstring>
#include <ctime>

using namespace std;

// Maximum level for this skip list.
// Level 3 means nodes can exist on levels 0, 1, 2, and 3.
#define MAX_LEVEL 3
```

Program Analysis

< iostream >

– It Handles standard input and output streams in C ++.

–In our code: It gives us access to cin and cout.

We need this to print the interactive menu to the screen and read the user's keyboard input .

(like choosing option "1" to insert, or typing a number to search for).

< cstdlib > (C Standard Library)

–Contains general – purpose functions, including dynamic memory management and random number generation.

–In our code: This is a heavy lifter for this specific program.

It provides malloc() and free() to create and destroy the nodes in memory (which was one of your original requirements).

It also provides rand() and srand(), which we use to flip the coin and generate the random levels for the express lanes.

< cstring > (C String)

–Provides functions for manipulating arrays of characters and blocks of memory.

–In our code: It gives us the memset() function.

When we create the update array (the "breadcrumbs" used during insertion and deletion), we need to ensure all

the pointers inside it start as completely empty (NULL).

memset lets us wipe that entire array clean in one fast line of code instead of writing a for loop.

< ctime > (C Time)

–Provides functions for fetching and manipulating the current date and time.

–It gives us time(0), which grabs the current system time.

In main(), we pass this time to srand(). Because the time is always changing, this seeds the random number generator with a unique starting point every time you run the program. If we didn't include this, rand() would generate the exact same sequence of coin flips every single time you launched the app, destroying the randomness of the Skip List.

RAND() or RANDOM

How it works: Computers cannot generate true randomness.

Instead, rand() uses a complex mathematical formula. It takes a starting number, applies a bunch of math to it, and spits out a new number. When you call it again, it takes that new number, applies the same math, and spits out the next one.

The Problem: Because it is just a fixed mathematical formula, if the starting number is always the same, the sequence of random numbers will be exactly the same every time we run the program.

By default, C++ always starts with a starting number of 1.

The Formula of RAND()

The standard rand() function in most C/C++ compilers uses an algorithm called a Linear Congruential Generator (LCG).

$$X_{n+1} = (a \cdot X_n + c)(\text{mod } m)$$

X_{n+1} → The new random number being generated.

a → The multiplier (a massive, carefully chosen constant).

X_n → The previous random number. (If this is the very first time rand() is called, this is the seed value you set with srand()).

c → The increment (another carefully chosen constant).

m → The modulus (the maximum possible limit, usually tied to the memory limit of an integer, like 2^{31}).

Lets say:

$$a = 1103515245$$

$$c = 12345$$

$$m = 2^{31} \text{ (which is 2,147,483,648).}$$

And X_n by default 1 at starting .

$$X_{n+1} = (1103515245 \cdot 1 + 12345)(\text{mod } 2^{31})$$

$$X_{n+1} = (1103527590)(\text{mod } 2147483648)$$

As, $25 \text{ mod } 10 = 5$ (remainder) [$25 > 10$] but when,

$$1 \text{ mod } 10 = 1 [1 < 10] [\text{Remainder}]$$

$$2 \text{ mod } 10 = 2 [2 < 10] [\text{Remainder}]$$

$$3 \text{ mod } 10 = 3 [3 < 10] [\text{Remainder}]$$

.

.

$$10 \text{ mod } 10 = 0 [10 = 10] [\text{Remainder}]$$

$$\therefore 1103527590 < 2147483648$$

$$\text{Hence, } X_{n+1} = 1103527590$$

Next call to rand() will become:

$$\text{Now, } X_n = 1103527590$$

$$X_{n+1} = (1103515245 \cdot 1103527590 + 12345)(\text{mod } 2^{31})$$

$$X_{n+1} = (1103515245 \cdot 1103527590 + 12345)(\text{mod } 2147483648)$$

$$1103515245 \times 1103527590 = 1,217,759,518,843,109,550$$

$$1,217,759,518,843,109,550 + 12345 = 1,217,759,518,843,121,895$$

$$1,217,759,518,843,121,895 \text{ mod } 2,147,483,648, \text{ and:}$$

$$2,147,483,648 < 1,217,759,518,843,121,895$$

$$\therefore 1,217,759,518,843,121,895 \text{ mod } 2,147,483,648 \\ = 377401575 [\text{Remainder}]$$

$$\therefore X_{n+1} = 377401575$$

TIME(0)

It fetches the current system time.

Under the hood, computers track time by counting the exact number of seconds that have passed since January 1, 1970 (known as the Unix Epoch).

In the late 1960s, a team of legendary programmers at Bell Labs (most notably Ken Thompson and Dennis Ritchie) were building a brand – new operating system called Unix. (Unix is the grandfather of Linux, macOS, Android, and iOS).

They needed a way for Unix to track time. Initially, in early drafts of the system, they actually set the zero point to January 1, 1970.

If we run time(0) to ask your C++ program what time it is, it will calculate: "How many seconds have passed since January 1, 1970, up until this exact moment today at random time say 8:52 PM?"

- ***When we call time(0), it doesn't give you 8:52 PM. It gives us a massive integer, like 1817046727.***
- ***If we call it one second later, it gives you 1817046728.***

(Note: The 0 or NULL just tells the function to return the value directly rather than trying to store it in a separate variable).

So, time(0) is always measuring the distance between January 1, 1970 and Right Now, no matter what time of day it currently is!

SRAND() stands for SEED RAND() or SEED RANDOM.

*It allows you to manually set the starting number
(the "seed") that rand() uses for its mathematical formula.*

SRAND(TIME(0)):

*It means "Take the exact number of seconds since 1970
and use THAT as the starting point for the random math."*

*Suppose current date and time is : 01: 11: 00 A.M of 31 – 07 – 2026
And it will be calculated from : January 1, 1970*

Counting Years : $2025 - 1970 + 1 = 56$ years

Days without leap year : $56 \times 365 = 20440$

**Leap years : (1972, 1976, 1980, 1984, 1988, 1992, 1996, 2000, 2004, 2008, 2012)
, 2016, 2020, 2024)**

= 14 leap years

$\therefore 20440 + 14 = 20454$

Step 3: Count days in 2026 before July 30

Since 2026 is not a leap year,

Month	Days
January	31
February	28
March	31
April	30
May	31
June	30
July (before the 30th)	29

Now add them:

$$31 + 28 = 59$$

$$59 + 31 = 90$$

$$90 + 30 = 120$$

$$120 + 31 = 151$$

$$151 + 30 = 181$$

$$181 + 29 = 210$$

Therefore,

210 days

have elapsed before 30 July 00:00 UTC.

Step 4: Total days

$$20454 + 210 = 20664$$

So,

20664 days have elapsed.

Step 5: Convert days into seconds

One day contains

$$24 \text{ hrs} \times 60 \text{ minute} \times 60 \text{ seconds} = 86400 \text{ seconds.}$$

$$\text{Therefore, } 20664 \times 86400 = 1,785,369,600 \text{ seconds.}$$

Step 6: Convert the current time to UTC

Local time (IST):

01:11:00

IST is 5 hours 30 minutes ahead of UTC.

Subtract 5 hours 30 minutes:

$$01:11 - 05:30 = 19:41 \text{ on the previous day.}$$

So UTC time is:

19:41:00

Step 7: Convert 19:41:00 to seconds

Hours: 19 hrs $\times$ 60 minutes $\times$ 60 seconds = 68,400seconds

Minutes: 41minutes $\times$ 60 seconds = 2,460 seconds

Seconds: 0

Total: 68,400 + 2,460 + 0 = 70,860 seconds

Step 8: Final addition

Base seconds: 1,785,369,600

Current day's elapsed seconds: 70,860

Adding them: 1,785,369,600 + 70,860 = 1,785,440,460

If we run it just one single second later at 01:11:01 AM, the seed becomes 1785440461.

Therefore for Random Equation:

$$X_{n+1} = (a \cdot X_n + c)(\text{mod } m)$$

Here are the constants used by some of the most popular compilers in the world:

- ***Linux (GNU C Library / glibc):***
 - ***a = 1103515245***
 - ***c = 12345***

- ***Windows (Microsoft Visual Studio):***
 - ***a = 214013***
 - ***c = 2531011***

- ***Older Systems (Borland C/C++):***
 - ***a = 22695477***
 - ***c = 1***

If we go with ,Linux (GNU C Library / glibc), we get:

$$X_{n+1} = (1103515245 \times 1,785,440,460 + 12345)(\text{mod } 2^{31})$$

memset(): The Memory Wiper

“memset” stands for Memory Set. It takes a block of memory and rapidly fills it with a specific value.

In our Skip List code, it looks like this:

```
struct Node *update[MAX_LEVEL + 1];
```

```
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

And

```
#define MAX_LEVEL 3
```

Then levels are:

Level → 0

Level → 1

Level → 2

Level → 3

hence, MAX_LEVEL + 1

*sizeof(struct Node *) → 8 bytes for 64 bit system architecture.*

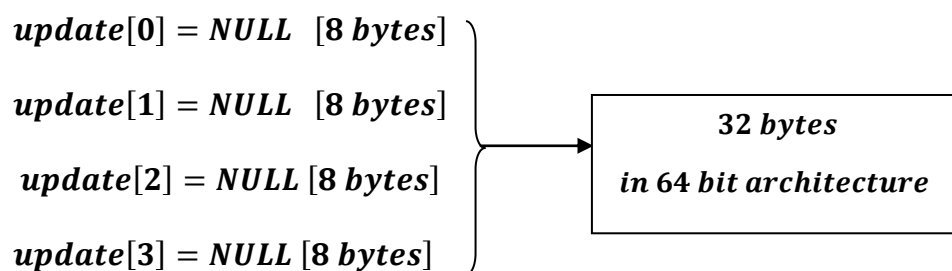
*sizeof(struct Node *) → 4 bytes for 32 bit system architecture.*

*if we take 64 bit , it will be : 8 bytes * (3 + 1) = 8 × 4 = 32 bytes.*

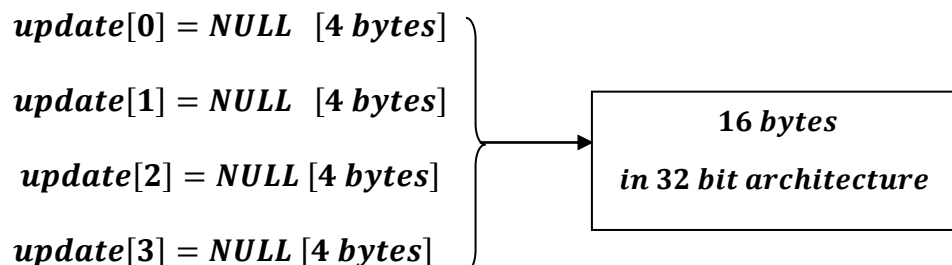
*if we take 32 bit , it will be : 4 bytes * (3 + 1) = 4 × 4 = 16 bytes.*

The Value: 0 (In C ++, setting a pointer to 0 is the exact same as setting it to NULL).

That is it will set:



[OR]



Why use it?

When we declare an array in C ++ like `struct Node * update[4];`, the computer just grabs raw memory for it. That memory might contain leftover garbage data from other programs that were recently closed. If we accidentally try to follow a garbage pointer, our program crashes.

We could clean it out using a loop:

```
for (int i = 0; i <= MAX_LEVEL; i++)  
{  
    update[i] = NULL;  
}
```

However, loops process data one item at a time. `memset()` is a highly optimized, low – level instruction that communicates directly with the processor to wipe the entire block of memory in one massive swoop. It is the fastest, safest way to ensure your update array is completely blank before we start tracking our Skip List path.

2. Skip List – Fundamentals

```
#define MAX_LEVEL 3
```

1. The Ceiling: `#define MAX_LEVEL 3`

This is a rule for the compiler. It says: "Any time you see the word `MAX_LEVEL`, replace it with the number 3."

- *As we start counting at Level 0, `MAX_LEVEL 3` means nodes can exist on Level 0, Level 1, Level 2, or Level 3.*
- *Without a limit, the coin – flip function might flip 100 heads in a row, building a massive 100 – level tower for a single number, which would waste a huge amount of memory.*

```
struct Node
{
    int key;
    // Array of pointers to next nodes, one for each level
    struct Node **forward;
};
```

- *`int key`; This is the actual data we are saving (e. g., the number 42).*
- *`struct Node ** forward`; This is the most complex line in the program.*

In a standard linked list, we just have `struct Node * next` (one pointer to the next node).

But in a Skip List, a node might be on Level 0, Level 1, and Level 2 simultaneously. Therefore, it needs multiple pointers.

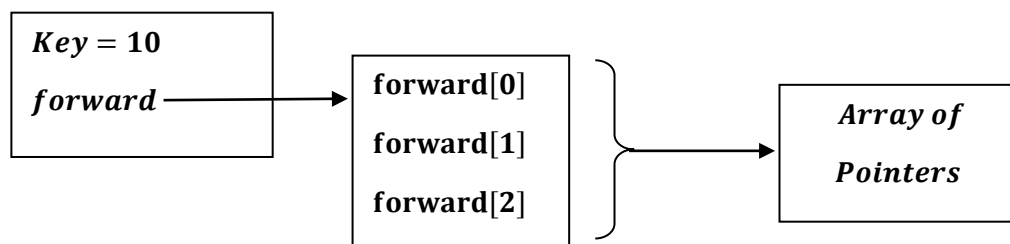
- The double asterisk (**) tells C++ to treat forward as a dynamic array of pointers.

If the keys are 10, 20, 30, 40 then there are exactly :

1 × Node 10
 1 × Node 20
 1 × Node 30
 1 × Node 40

A total of 4 node objects.

Now, say Node 10, it has :



Level 2: [Node 10] -----> [Node 40] -> NULL

Level 1: [Node 10] -----> [Node 30] ----> [Node 40] -> NULL

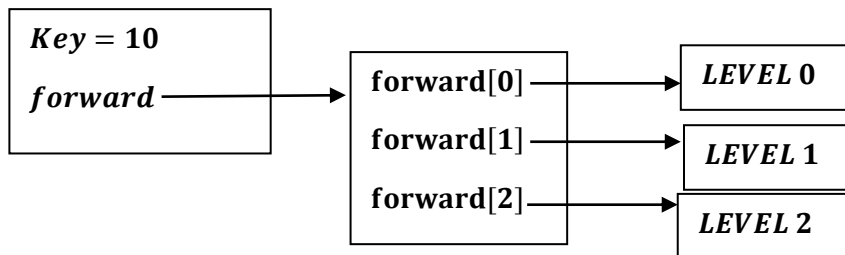
Level 0: [Node 10] ---> [Node 20] -> [Node 30] ---> [Node 40] -> NULL

This does not mean there are three copies of Node 10.

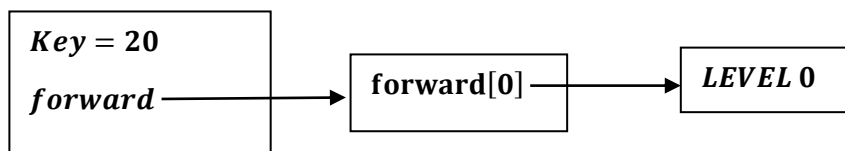
Keys and its corresponding Nodes are arranged in random levels suppose:

<i>Key</i>	<i>Level</i>
10	2
20	0
30	1
40	2

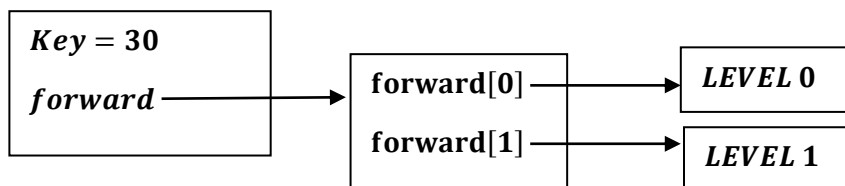
Node 10 , it has :



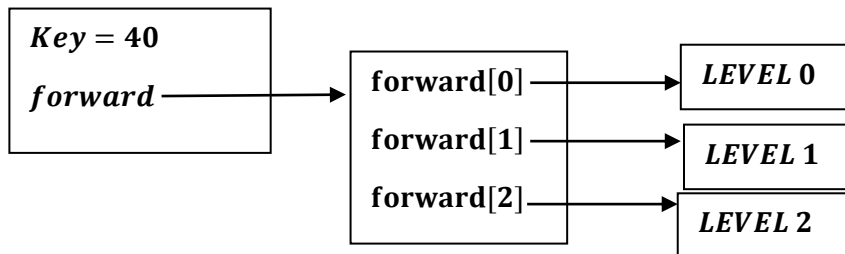
Node 20 , it has :



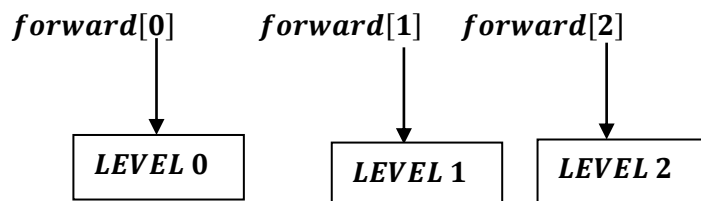
Node 30 , it has :



Node 40 , it has :



Note , Level is by default : $MAX_LEVEL = 3$. *We will know it during create node module.*



forward[0], here `0` means Level 0 .

forward[1], here `1` means Level 1 , It means Level 0 i.e. forward[0] also present.

forward[2], here `2` means Level 2, It means Level 1 and Level 0 also present,

i.e. forward[0], forward[1] also present .

```
// Skip List structure
struct SkipList
{
    int level;           // Current maximum level in the list
    struct Node *header; // Pointer to the header node
};
```

Here `level` is the current maximum level and during creation of list it is assigned to 0 and is not define MAX_LEVEL 3 .

```
lst->level = 0;
```

Now, header pointer:

```
struct Node *header; // Pointer to the header node
```

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level + 1));
```

And

```
lst->header = createNode(-1, MAX_LEVEL);
```

These shows :

header = createNode(-1, MAX_LEVEL = 3)

*forward = (Struct Node **)malloc(sizeof(struct Node *) * (level = 3 + 1));*

or,

*forward = (Struct Node **)malloc(sizeof(struct Node *) * (level = 4));*

∴ At creation of header , we get:

header → *forward*[3] [LEVEL 3] – 8 bytes

4 bytes * 4 = 16 bytes according to 32 bit architecture.

Level 0: [Node 10] ---> [Node 20] -> [Node 30] ---> [Node 40] -> NULL

LEVEL 3 is not reached, therefore header → forward[3] points to NULL.

As,

```
for (int i = 0; i <= level; i++)
{
    n->forward[i] = NULL;
}
```

forward[3] [LEVEL 3] = NULL

During creation of the list.

Header is a pointer variable that points to a special node called the header node (also called a head node or sentinel node) of the skip list.

```
struct SkipList
{
    int level;           // Current maximum level in the list
    struct Node *header; // Pointer to the header node
};
```

- *struct Node *header; means:*
 - *header is a variable.*
 - *Its type is pointer to struct Node.*
 - *It stores the memory address of the header node.*

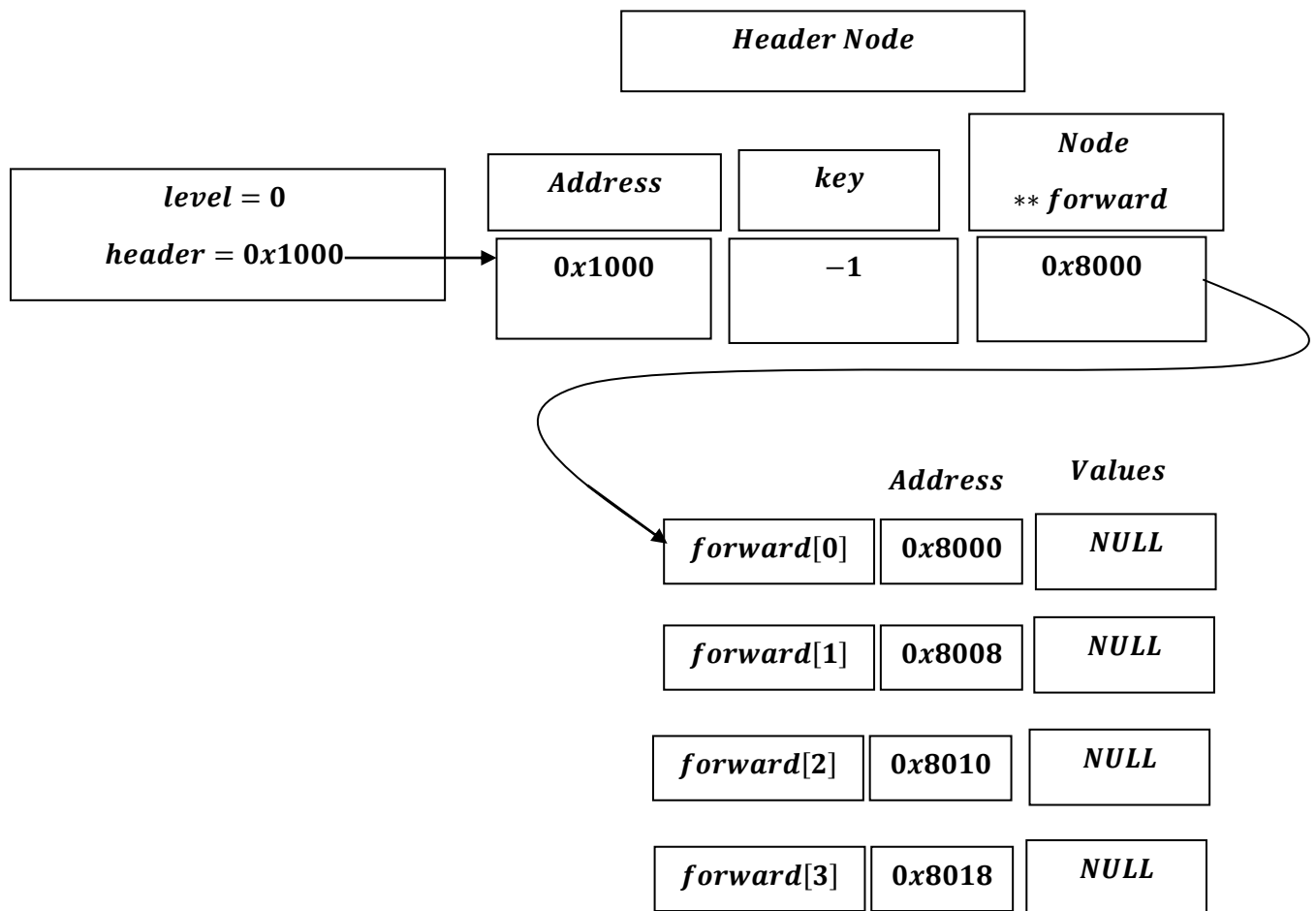
Where is `header` initialized?

In `createSkipList()`:

```
lst->header = createNode(-1, MAX_LEVEL);
```

Suppose the header node is allocated at address 0x1000.

Then:



So header simply stores the address 0x1000.

key = -1 [Dummy value]

This value is just a placeholder. We never search for or display the header's key.

The header exists so that every level has a starting point.

Initially:

Level 3:

Header --forward[3] --> NULL

Level 2:

Header --forward[2] --> NULL

Level 1:

Header --forward[1] --> NULL

Level 0:

Header --forward[0] --> NULL

From here we can get some picture clarity about the program.

3. Skip List – randomlevel

Program :

```
int randomLevel() {
    int lvl = 0;
    // 50% probability to increase the level
    while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {
        lvl++;
    }
    return lvl;
}
...
int main() {
    // Seed random number generator for random levels
    srand((unsigned)time(0));
    ...
}
```

Program Analysis

Introduction:

This is a random level generation , that decides node's height

- if it returns `0`, then node gets 1 pointer[LEVEL: 0].*
- if it returns `1`, then node gets 2 pointers[LEVELS: 0, 1].*
- if it returns `2`, then node gets 3 pointers[LEVELS: 0, 1, 2].*
- etc.*

The randomLevel() function is only ever used during the insertion of a new element.

It is never used when searching for an element, and it is never used when deleting an element.

Analysis

```
int randomLevel() {... }
```

First to start with – there is no parameters.

```
int lvl = 0;
```

So, lvl = 0.

```
while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {  
    lvl++;  
}
```

MAX_LEVEL = 3 as,

```
#define MAX_LEVEL 3
```

say 1st, rand() = 42

1st loop run:

while $\left((rand() = 42 \% 2) = 0 == 0 \left[\frac{42}{2} = 0[Remainder] \right] [True] \right.$
 $\left. AND\ lvl = 0 < MAX_LEVEL = 3 [TRUE] \right):$

$lvl = 0 + 1 = 1$

$$2^{nd}, rand() = 998$$

2nd loop run:

$$while \left(\begin{array}{l} (rand() = 998 \% 2) = 0 == 0 \left[\frac{998}{2} = 0[Remainder] \right] [True] \\ AND \text{ } lvl = 1 < MAX_LEVEL = 3 [TRUE] \end{array} \right):$$

$$lvl = 1 + 1 = 2$$

$$3^{rd}, rand() = 8471$$

3rd loop run:

$$while \left(\begin{array}{l} (rand() = 8471 \% 2) = 1 \neq 0 \left[\frac{8471}{2} = 1[Remainder] \right] [False] \\ AND \text{ } lvl = 2 < MAX_LEVEL = 3 [TRUE] \end{array} \right):$$

Condition fails , hence :

```
return lvl;
```

return , lvl = 2

How srand helps rand to choose random numbers?

```
int main() {  
    // Seed random number generator for random levels  
    srand((unsigned)time(0));  
    ...  
}
```

When main function runs, **srand ((unsigned)time(0))** executes.

Later on, the program calls rand() inside the randomLevel() function:

```
int randomLevel() {  
    int lvl = 0;  
    // 50% probability to increase the level  
    while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {  
        lvl++;  
    }  
    return lvl;  
}
```

Computers cannot generate truly random numbers easily; they use mathematical formulas to generate sequences of numbers that look random (pseudo – random numbers).

If you use rand() without srand(), the generator will start with a default seed (usually 1). This means that every time we run our program, rand() will produce the exact same sequence of numbers.

Example:

```
#include <stdio.h>  
#include <stdlib.h> // Required for rand()  
  
int main() {  
    printf("Generating 5 random numbers:\n");
```

```

    for (int i = 0; i < 5; i++) {
        // rand() generates a pseudo-random integer
        printf("%d ", rand());
    }

    printf("\n");
    return 0;
}

```

Output of Run 1:

```

Generating 5 random numbers:
1804289383 846930886 1681692777 1714636915 1957747793

```

Output of Run 2:

```

Generating 5 random numbers:
1804289383 846930886 1681692777 1714636915 1957747793

```

Output of Run 3:

```

Generating 5 random numbers:
1804289383 846930886 1681692777 1714636915 1957747793

```

Same sequence of numbers at each output .

But for SRAND example:

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>    // Required for time()

int main() {
    // Seed the random number generator with the current system
    time.
    // You only need to call this ONCE at the start of your program.
    srand(time(NULL));

    printf("Generating 5 TRULY random numbers:\n");

    for (int i = 0; i < 5; i++) {
        printf("%d ", rand());
    }
}

```

```

    }

    printf("\n");
    return 0;
}

```

Output of Run 1:

```

Generating 5 TRULY random numbers:
142839210 18493021 984920184 48291048 2948194

```

Output of Run 2:

```

Generating 5 TRULY random numbers:
958392019 39481023 11849302 48920184 7583921

```

Here NULL and 0 is same thing and we used :

```

srand((unsigned)time(0));

```

The srand()function, however, specifically requires an unsigned int (which is usually a 32 – bit positive integer).

*If we write **srand(time(NULL));**, we are handing a signed integer to a function that wants an unsigned integer. The compiler will handle the conversion for us automatically, but modern compilers might throw a warning like this:*

Warning: conversion from 'time_t' to 'unsigned int', possible loss of data.

*By adding **(unsigned)**, we are explicitly telling the compiler, "I know I am converting these data types, do it anyway." This is called type casting, and it suppresses that warning.*

In the context of our Skip List:

- *Skip Lists rely on randomness (a coin toss) to decide how many levels a new node should have.*
- *If rand() produced the exact same sequence every time we ran the program, our Skip List would build itself in the exact same shape for a given set of inputs every single time, which defeats the purpose of testing a probabilistic data structure.*

That is:

Output of Run 1:

```
Generating 5 random numbers:  
1804289383 846930886 1681692777 1714636915 1957747793
```

Output of Run 2:

```
Generating 5 random numbers:  
1804289383 846930886 1681692777 1714636915 1957747793
```

Output of Run 3:

```
Generating 5 random numbers:  
1804289383 846930886 1681692777 1714636915 1957747793
```

It defeats the purpose of testing a probabilistic data structure.

By calling srand((unsigned)time(0)), we are setting the starting seed to the current time (the number of seconds since January 1, 1970). Because the time changes every second, the starting seed is different every time we run the program. This guarantees that rand() will produce a different sequence of random numbers each time, causing our Skip List nodes to genuinely have unpredictable heights on every run.

Best Case

1st, rand() = 8471

1st loop run:

$\text{while} \left(\begin{array}{l} (\text{rand}() = 8471 \% 2) = 1 \neq 0 \left[\frac{8471}{2} = 1[\text{Remainder}] \right] [\text{False}] \\ \text{AND } \text{lvl} = 0 < \text{MAX_LEVEL} = 3 [\text{TRUE}] \end{array} \right):$

Condition fails , hence :

```
return lvl;
```

return , lvl = 0

In the best – case scenario, the very first call to rand()% 2 returns 1.

***The condition (rand()% 2) == 0 immediately evaluates to false,
and the while loop terminates without executing even once.***

The function simply returns 0. This takes constant time.

Time Complexity : O(1).

Worst Case

$$\mathbf{1^{st}, rand() = 42}$$

1st loop run:

$$\mathbf{while \left(\begin{array}{l} (rand() = 42 \% 2) = 0 == 0 \left[\frac{42}{2} = 0[Remainder] \right] [True] \\ AND \text{ } lvl = 0 < MAX_LEVEL = 3 [TRUE] \end{array} \right):}$$

$$\mathbf{lvl = 0 + 1 = 1}$$

$$\mathbf{2^{nd}, rand() = 998}$$

2nd loop run:

$$\mathbf{while \left(\begin{array}{l} (rand() = 998 \% 2) = 0 == 0 \left[\frac{998}{2} = 0[Remainder] \right] [True] \\ AND \text{ } lvl = 1 < MAX_LEVEL = 3 [TRUE] \end{array} \right):}$$

$$\mathbf{lvl = 1 + 1 = 2}$$

$3^{rd}, rand() = 1042$

2nd loop run:

$while \left(\begin{array}{l} (rand() = 1042 \% 2) = 0 == 0 \left[\frac{1042}{2} = 0[Remainder] \right] [True] \\ AND \text{ } lvl = 2 < MAX_LEVEL = 3 [TRUE] \end{array} \right):$

$lvl = 2 + 1 = 3$

Now , either

$4^{rth}, rand() = 8471$

4^{rth} loop run:

$while \left(\begin{array}{l} (rand() = 8471 \% 2) = 1 \neq 0 \left[\frac{8471}{2} = 1[Remainder] \right] [False] \\ AND \text{ } lvl = 3 < MAX_LEVEL = 3 [False] \end{array} \right):$

Condition fails , hence :

```
return lvl;
```

$return, lvl = 3$

OR,

$$4^{th}, rand() = 8472$$

4th loop run:

$$while \left(\begin{array}{l} (rand() = 8472 \% 2) = 0 == 0 \left[\frac{8472}{2} = 0[Remainder] \right] [True] \\ AND \text{ lvl} = 3 < MAX_LEVEL = 3 [False] \end{array} \right):$$

Condition fails , hence :

```
return lvl;
```

return , lvl = 3

In the absolute worst – case scenario, rand() % 2 repeatedly returns 0, causing the loop to execute until the second condition lvl < MAX_LEVEL forces it to stop. The loop will run exactly MAX_LEVEL times.

(Note: Since MAX_LEVEL is usually a predefined constant , here MAX_LEVEL = 3, in most Skip List implementations – the worst – case is technically bounded by a constant, which means it can be strictly argued as O(1). However, expressing it as O(MAX_LEVEL) is the most accurate way to show what bounds the worst – case performance).

Time Complexity : O(MAX_LEVEL) ≈ O(1).

Average Case

$rand() \% 2 = 0$, when 0 means Head .

$rand() \% 2 = 1$, when 1 means Tail. When Tail comes ,then loop condition is False.

0 times, which flip Tails immediately. Probability $= \frac{1}{2} = \left(\frac{1}{2}\right)^{0+1}$.

1 times, which flip Heads then Tails. Probability $= \frac{1}{2} \times \frac{1}{2} = \frac{1}{4} = \left(\frac{1}{2}\right)^{1+1}$.

2 times, which flip Heads, Heads then Tails. Probability $= \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} = \frac{1}{8}$
 $= \left(\frac{1}{2}\right)^{2+1}$.

3 times, which flip Heads, Heads, Heads then Tails. Probability
 $= \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} = \frac{1}{16} = \left(\frac{1}{2}\right)^{3+1}$

.

.

i^{th} times, which flip i times Heads and then Tails . Probability $= \left(\frac{1}{2}\right)^{i+1}$

$S = (0 \text{ runs} \cdot \text{prob}) + (1 \text{ run} \cdot \text{prob}) + (2 \text{ run} \cdot \text{prob}) + \dots$

Plugging in our numbers from Step 1, we get:

$$S = 0 \left(\frac{1}{2}\right) + 1 \left(\frac{1}{4}\right) + 2 \left(\frac{1}{8}\right) + 3 \left(\frac{1}{16}\right) + 4 \left(\frac{1}{32}\right) + \dots$$

$$S = 0 + \frac{1}{4} + \frac{2}{8} + \frac{3}{16} + \frac{4}{32} + \dots$$

$$\therefore S = \frac{1}{4} + \frac{2}{8} + \frac{3}{16} + \frac{4}{32} + \dots$$

Multiplky entire equation by 2 .

$$2S = \frac{2 \times 1}{4} + \frac{2 \times 2}{8} + \frac{2 \times 3}{16} + \frac{2 \times 4}{32} + \frac{2 \times 5}{64} \dots$$

$$\therefore 2S = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \frac{5}{32} + \dots$$

$$2S = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \frac{5}{32} + \dots$$

—

$$S = 0 + \frac{1}{4} + \frac{2}{8} + \frac{3}{16} + \frac{4}{32} + \dots$$

— — — — —

$$\therefore (2S - S) = \left(\frac{1}{2} - 0\right) + \left(\frac{2}{4} - \frac{1}{4}\right) + \left(\frac{3}{8} - \frac{2}{8}\right) + \left(\frac{4}{16} - \frac{3}{16}\right) + \left(\frac{5}{32} - \frac{4}{32}\right) + \dots$$

$$\Rightarrow S = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \dots$$

It is infinite series : $S = \sum_{i=1}^{\infty} ar^{i-1} = a + ar + ar^2 + ar^3 + \dots = \frac{a}{1-r}$

$$S = \sum_{i=1}^{\infty} ar^{i-1} = \sum_{i=1}^{\infty} \frac{1}{2} \left(\frac{1}{2}\right)^{i-1} = \frac{1}{2} \times \left(\frac{1}{2}\right)^{1-1=0} + \frac{1}{2} \times \left(\frac{1}{2}\right)^{2-1=1} + \frac{1}{2} \times \left(\frac{1}{2}\right)^{3-1=2} + \dots$$

→ a is very first number in the series, here $\frac{1}{2}$.

→ r is the common ratio, here, $r = \frac{\text{Term 2}}{\text{Term 1}}$, say $\frac{\frac{1}{4}}{\frac{1}{2}} = \frac{1}{4} \times \frac{2}{1} = \frac{1}{2}$

similarly, $r = \frac{\frac{1}{8}}{\frac{1}{4}} = \frac{1}{8} \times \frac{4}{1} = \frac{1}{2}$

Hence, common ratio, $r = \frac{1}{2}$

$$S = \sum_{i=1}^{\infty} ar^{i-1} = a + ar + ar^2 + ar^3 + \dots = \frac{a}{1-r} = \frac{\frac{1}{2}}{1-\frac{1}{2}} = \frac{\frac{1}{2}}{\frac{1}{2}} = 1$$

i. e. Average Time Complexity = $O(1)$.

<i>Skip List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Random Level</i>	<i>Best Case</i>	$O(1)$
	<i>Worst Case</i>	$O(\text{MAX_LEVEL})$ $= O(c) \approx O(1)$.
	<i>Average Case</i>	$O(1)$

Space Complexity

1. Input Space Complexity: $O(1)$

This measures the memory required to store the inputs provided to the function.

- **Parametric Space Complexity: 0 bytes ($O(1)$)**

The function signature is `int randomLevel()`. It takes absolutely zero parameters. No arguments are being passed into it, so it consumes no memory for parameters.

- **FULL Data Input (Holistic Input Space): $O(1)$**

The function does not take in or process any external data structures (like arrays, linked lists, or trees). The only external data it looks at is `MAX_LEVEL` (which is likely a global constant or a preprocessor macro `#define`) and the internal state of the `rand()` function. Reading a single global constant takes constant, negligible space.

2. Auxiliary Space Complexity: $O(1)$

This measures the temporary, extra memory the algorithm needs to do its work during execution.

- **Local Variables:** The algorithm creates exactly one local variable: `int lvl = 0;`. This consumes a fixed amount of memory (usually 4 bytes in C/C++), regardless of how large the Skip List gets.
- **Execution Stack:** There is no recursion happening here. The function is called once, runs a simple while loop, and finishes. It only takes up a single, small stack frame in memory.
- **Dynamic Allocation:** There is no `malloc` or `new` keyword inside this function. It creates no dynamic memory on the heap.

Because the memory used does not grow or change based on the size of the Skip List or the number of elements being inserted, the auxiliary space is strictly $O(1)$.

Total Space Complexity = Input Space + Auxiliary Space.

1) Total Space Complexity = Parametric Input Space + Auxiliary Space.

$$O(1) + O(1) = O(1).$$

2) Total Space Complexity = Full Data Input Space + Auxiliary Space.

$$O(1) + O(1) = O(1).$$

$\therefore$ in both cases , Total Space Complexity: $O(1)$.

Algorithm

CONSTANT MAX_ Level= 3

1. Function RandomLevel():

2. SET lvl := 0.

// Loop with a 50% chance of continuing, bounded by MAX_LEVEL = 3

3. Repeat while Random() Mod 2 = 0 AND lvl < MAX_LEVEL :

3. a. lvl = lvl + 1.

[END of Loop]

4. Return lvl

5. End of Function RandomLevel()

[END of Algorithm]

Skip List – Create Node And Create Skip List

–Program Analysis

Program:

```
#define MAX_LEVEL 3

// Node structure
struct Node {
    int key;
    // Array of pointers to next nodes, one for each level
    struct Node **forward;
};

// Skip List structure
struct SkipList {
    int level;           // Current maximum level in the list
    struct Node *header; // Pointer to the header node
};

// Function to create a new node using malloc
struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
    // elements)
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
    + 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}
```

```

// Function to initialize the skip list
struct SkipList *createSkipList() {
    struct SkipList *lst = (struct SkipList *)malloc(sizeof(struct
SkipList));
    lst->level = 0;

    // The header node doesn't store real data, so we use -1 as a
placeholder.
    // It must have the maximum possible level.
    lst->header = createNode(-1, MAX_LEVEL);
    return lst;
}

...
int main() {
    // Seed random number generator for random levels
    ...

    struct SkipList *list = createSkipList();
...
}

```

Program Analysis

```

struct SkipList *list = createSkipList();

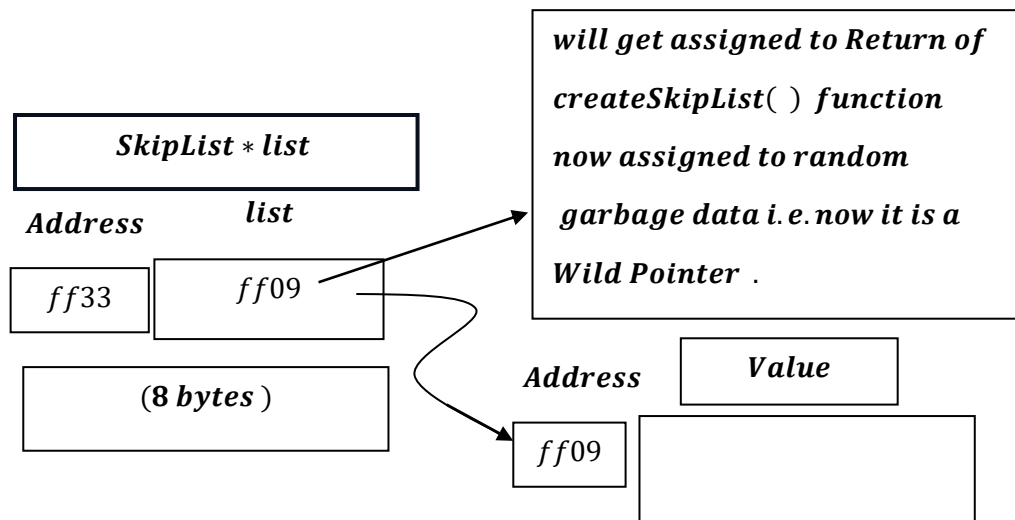
```

*Now, we have *list which is a pointer variable of struct SkipList ,
hence *list allocate 8 bytes , which can access member variable of
struct SkipList → int level and header pointer.*

```

// Skip List structure
struct SkipList {
    int level;           // Current maximum level in the list
    struct Node *header; // Pointer to the header node
};

```



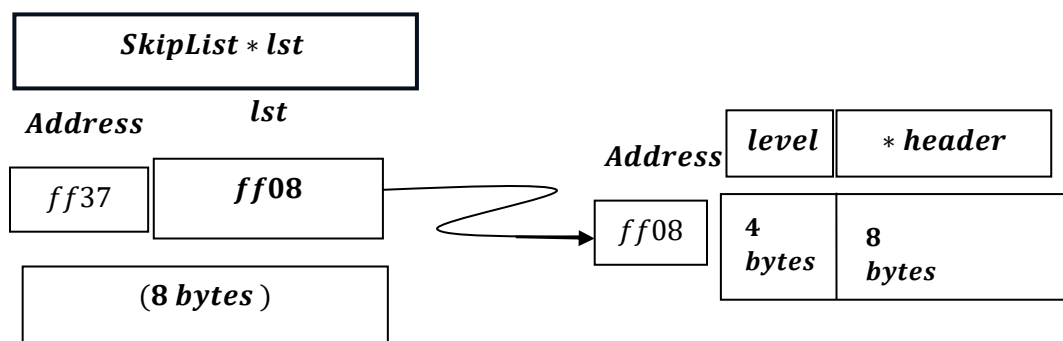
```
struct SkipList *createSkipList() {
    struct SkipList *lst = (struct SkipList *)malloc(sizeof(struct SkipList));
    lst->level = 0;

    // The header node doesn't store real data, so we use -1 as a placeholder.
    // It must have the maximum possible level.
    lst->header = createNode(-1, MAX_LEVEL);
    return lst;
}
```

Now,

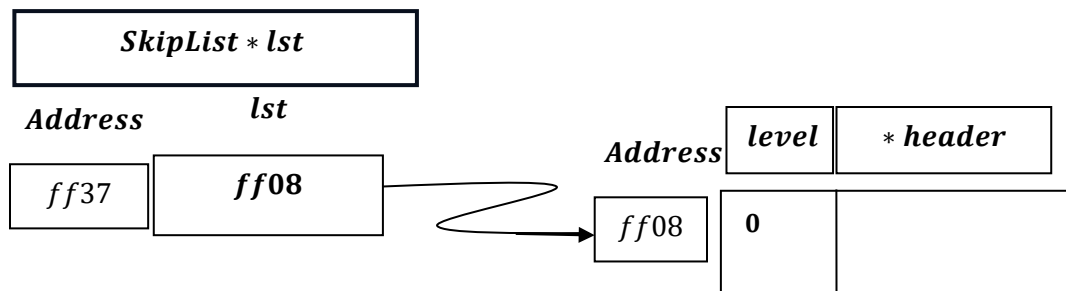
```
struct SkipList *lst = (struct SkipList *)malloc(sizeof(struct SkipList));
```

We have **lst* which is a pointer variable of struct *SkipList*,
hence **lst* allocate 8 bytes ,which can access member variable of
struct *SkipList* → int level and header pointer.



```
lst->level = 0;
```

list → *level* is assigned to 0 .



```
lst->header = createNode(-1, MAX_LEVEL);
```

header will assigned to 8 bytes pointer variable which is an address return by *createNode(-1, MAX_LEVEL)* passing values - 1, and *MAX_LEVEL* = 3 .

```
// Node structure
struct Node {
    int key;
    // Array of pointers to next nodes, one for each level
    struct Node **forward;
};
```

```
// Function to create a new node using malloc
struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
    elements)
```

```

    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}

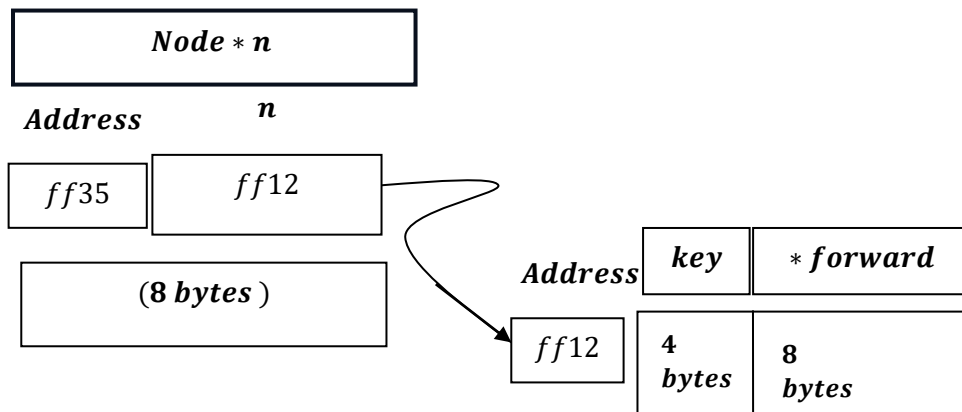
```

Now,

```

struct Node *n = (struct Node *)malloc(sizeof(struct Node));

```



```

struct Node *createNode(int key, int level) {

```

```

    n->key = key;

```

*This 'key' is
local variable.*

*This 'key' is
member variable
of struct Node.*

$n \rightarrow key = -1.$

```
struct Node {  
    int key;  
    // Array of pointers to next nodes, one for each level  
    struct Node **forward;  
};
```

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level  
+ 1));
```

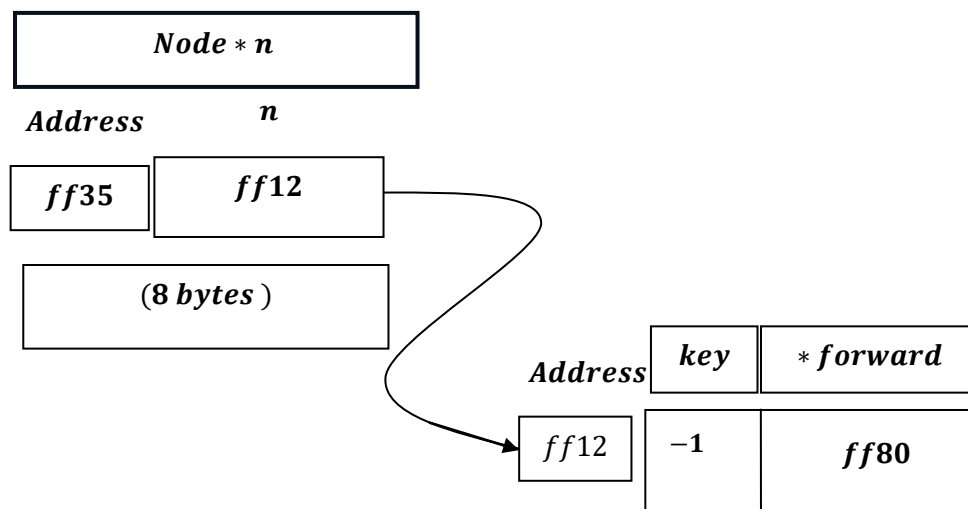
*It will allocate, $(struct Node * = 8 \text{ bytes}) \times (level + 1)$*

$= 8 \text{ bytes} \times (3 + 1)$

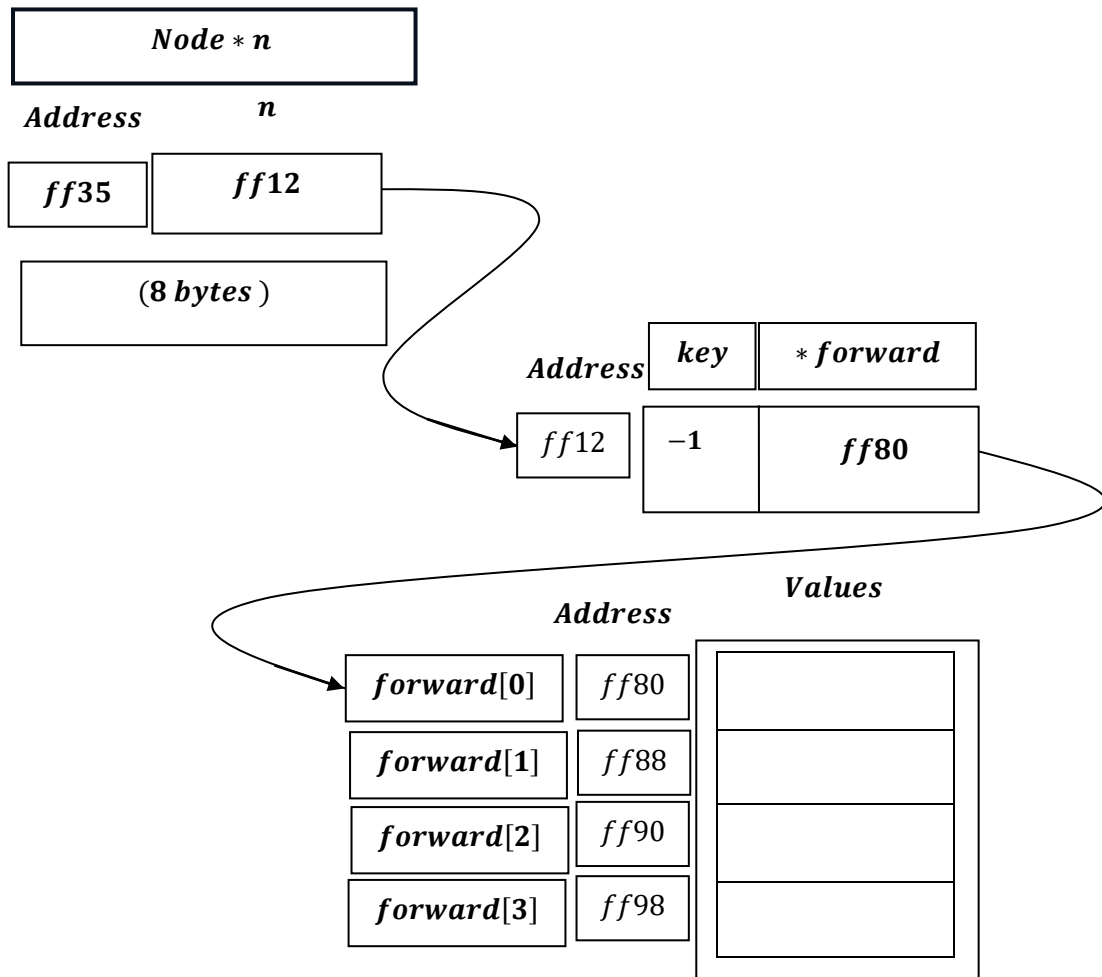
$= 8 \text{ bytes} \times 4$

$= 32 \text{ bytes}$

Which means ,it allocates 4 slots of memory of 8 bytes ,like: –



And now,



Actual structure

Address at which value is stored	Value	offset used
ff20	key = -1	key[ff20 – ff23] 4 bytes for int
		padding → [ff24 – ff27] 4 bytes for padding
ff28	Address: ff80	* forward → [ff28 – ff2f] 8 bytes for pointer variable
ff80	NULL	forward[0] → [ff80 – ff87] 8 bytes for pointer variable
ff88	NULL	forward[1] → [ff88 – ff8f] 8 bytes for pointer variable
ff90	NULL	forward[2] → [ff90 – ff97] 8 bytes for pointer variable
ff98	NULL	forward[3] → [ff98 – ff9f] 8 bytes for pointer variable

This will be discussed in memory allocation section.

```
for (int i = 0; i <= level; i++) {  
    n->forward[i] = NULL;  
}
```

Note , level = MAXL_EVEL = 3 .

Iteration 1:

$i = 0; 0 \leq \text{level} = 3$ [Condition True]

$n \rightarrow \text{forward}[0] = \text{NULL}$

$i = 0 + 1 = 1$

Iteration 2:

$i = 1; 1 \leq \text{level} = 3$ [Condition True]

$n \rightarrow \text{forward}[1] = \text{NULL}$

$i = 1 + 1 = 2$

Iteration 3:

$i = 2; 2 \leq \text{level} = 3$ [Condition True]

$n \rightarrow \text{forward}[2] = \text{NULL}$

$i = 2 + 1 = 3$

Iteration 4:

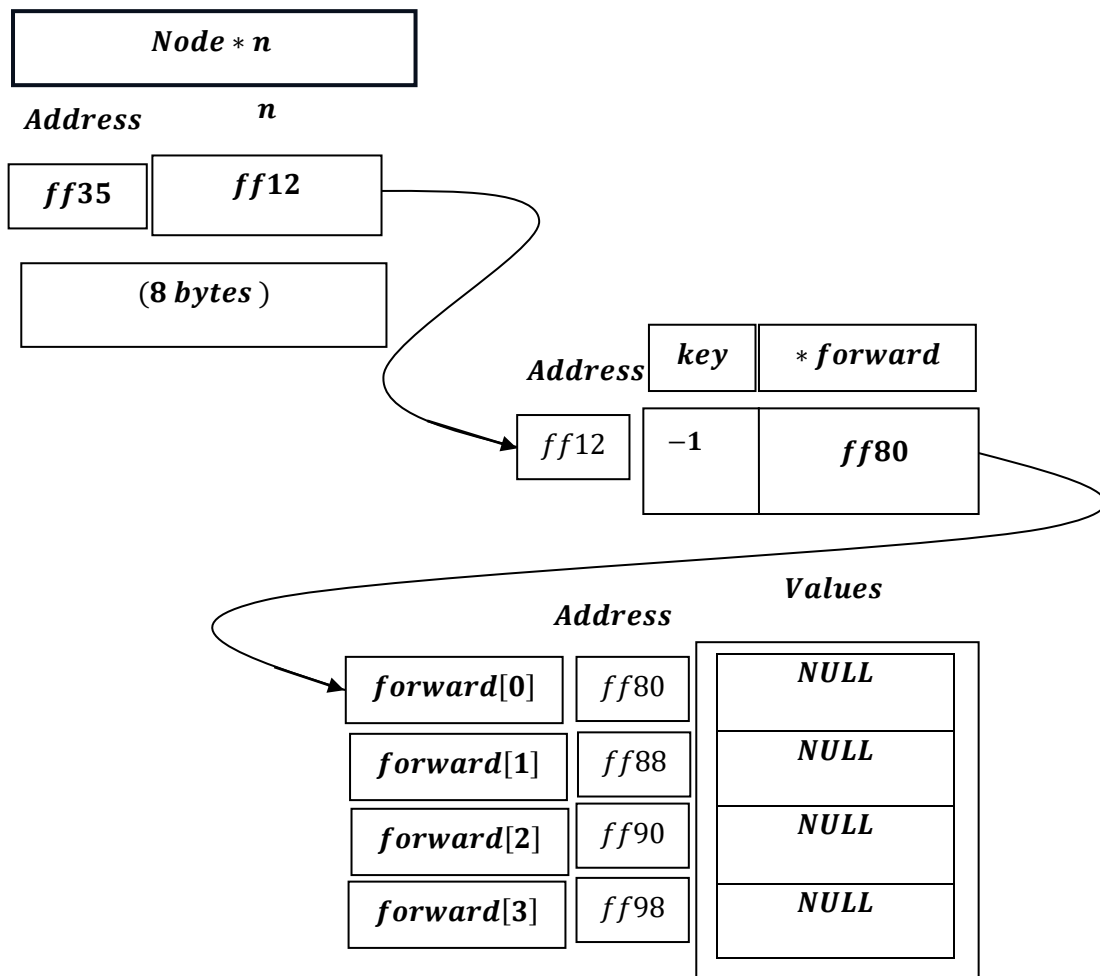
$i = 3; 3 \leq \text{level} = 3$ [Condition True]

$n \rightarrow \text{forward}[3] = \text{NULL}$

$i = 3 + 1 = 4$

Iteration 5:

$i = 4; 4 \leq \text{level} = 3$ [Condition is False]
hence, loop exits

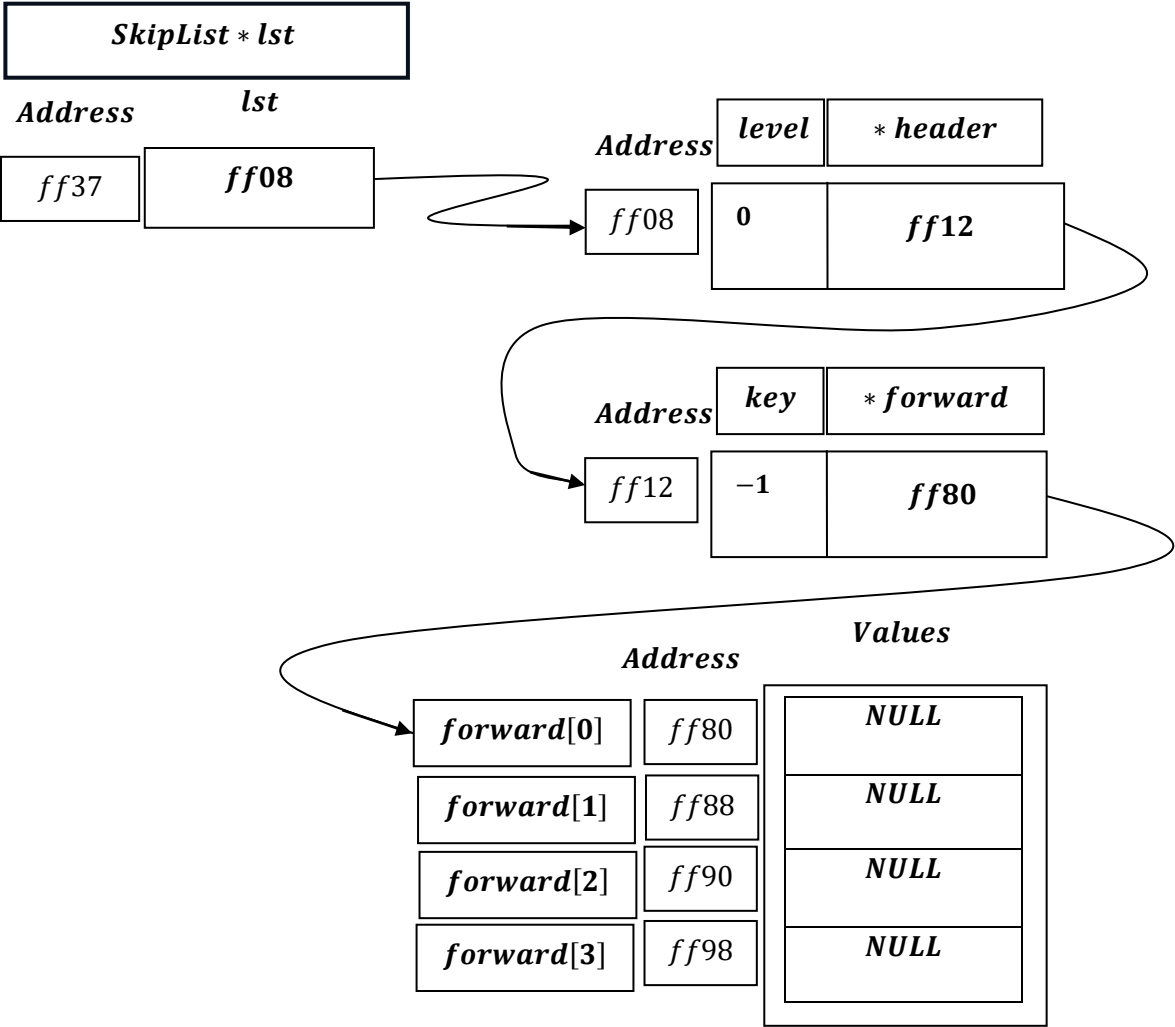


```
return n;
```

It returns address value that pointer variable n has stored : ff12

```
lst->header = createNode(-1, MAX_LEVEL);
```

i.e. lst → header = ff12.

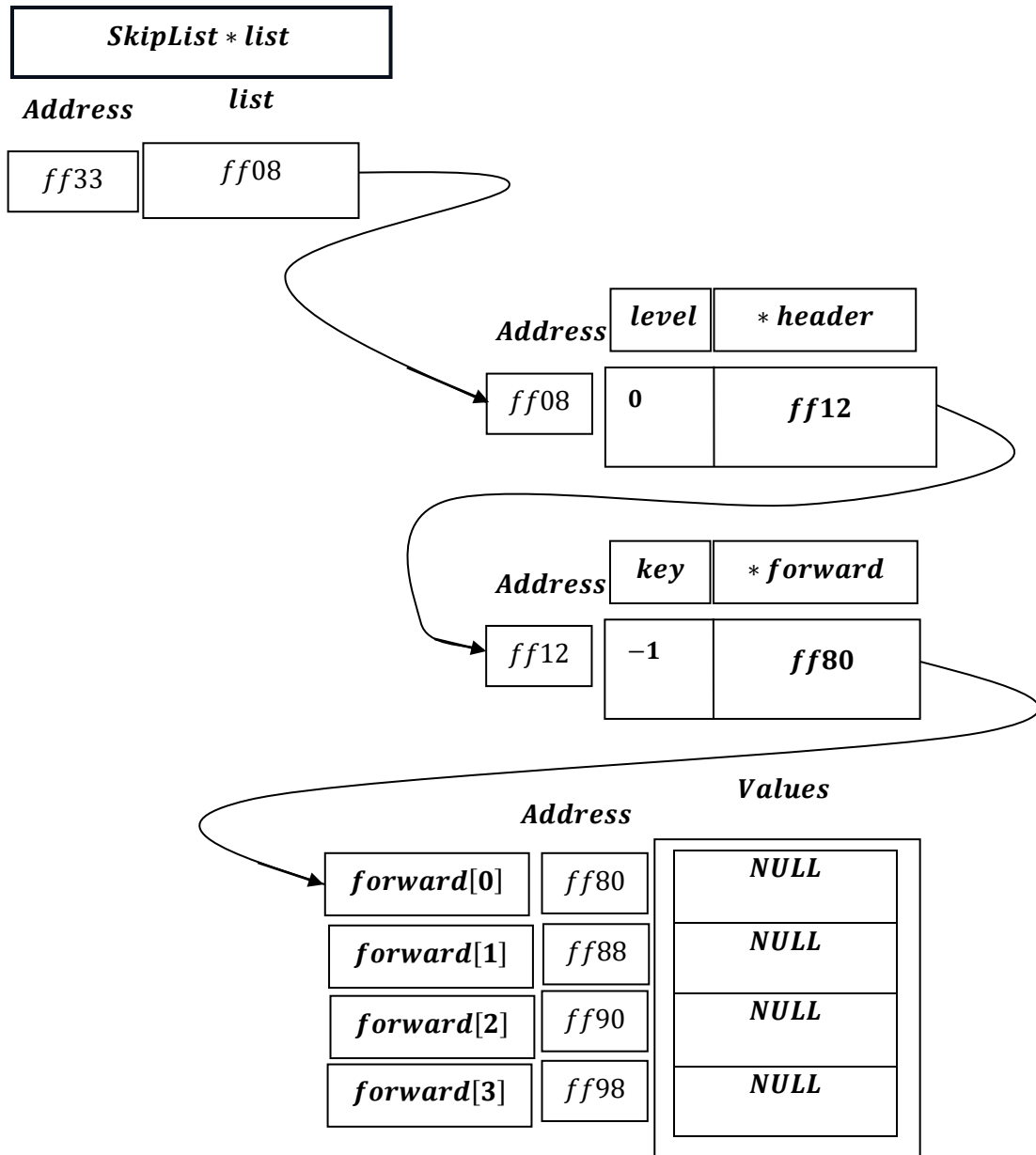


```
return lst;
```

return lst = ff08.

```
struct SkipList *list = createSkipList();
```

*struct SkipList * list = ff08.*



Time Complexity

```
#define MAX_LEVEL 3
```

#define is a preprocessor directive, not an executable statement.

Before compilation, the C preprocessor simply replaces every occurrence of MAX_LEVEL with 3.

There is no runtime execution, so there is no runtime time complexity, and the compiler simply records it.

Preprocessor Directive, Not Runtime Code: #define is handled by the C/C++ preprocessor before the compiler even sees the code. It acts essentially like a find and replace operation. Every time we use MAX_LEVEL in your code, the preprocessor swaps it out with the literal number 3.

Time Complexity: $O(1)$ (or zero runtime cost). Because the substitution happens at compile time, there is no code generated to execute or evaluate this definition while the program is running.

The runtime time complexity of the definition itself is practically zero.

Space Complexity: $O(1)$. Unlike declaring a variable (e.g., `const int MAX_LEVEL = 3;` which might allocate memory depending on compiler optimizations), #define does not allocate any memory in the RAM for the value 3. The literal number 3 is just baked directly into the machine code instructions wherever it is used. Therefore, it requires no extra runtime memory allocation, making the space complexity $O(1)$.

```
// Node structure
struct Node {
    int key;
    // Array of pointers to next nodes, one for each level
    struct Node **forward;
};
```

*Declaring int level; and struct Node ** forward; takes constant time complexity : $O(1)$, no runtime work is performed, the compiler simply records the layout of the structure.*

Space Complexity

The structure contains:

- *int key* → (4 bytes for both 64/32 bit architecture) $O(1)$
- *struct Node ** forward (a pointer)* → (8 bytes for 64 bit architecture and 4 bytes for 32 bit architecture) $O(1)$

Therefore, the space complexity of Node objects are: $O(1)$.

```
struct SkipList {
    int level;           // Current maximum level in the list
    struct Node *header; // Pointer to the header node
};
```

*Declaring int level; and struct Node * header; takes constant time complexity : $O(1)$, no runtime work is performed, the compiler simply records the layout of the structure.*

Space Complexity

The structure contains:

- *int level* → (4 bytes for both 64/32 bit architecture) $O(1)$
- *struct Node * header (a pointer)* → (8 bytes for 64 bit architecture and 4 bytes for 32 bit architecture) $O(1)$

Therefore, the space complexity of SkipList objects are: $O(1)$.

```

// Function to create a new node using malloc
struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
elements)
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}

// Function to initialize the skip list
struct SkipList *createSkipList() {
    struct SkipList *lst = (struct SkipList *)malloc(sizeof(struct
SkipList));
    lst->level = 0;

    // The header node doesn't store real data, so we use -1 as a
placeholder.
    // It must have the maximum possible level.
    lst->header = createNode(-1, MAX_LEVEL);
    return lst;
}

```

Taking into account the constant #define MAX_LEVEL 3.

```

struct SkipList *lst = (struct SkipList *)malloc(sizeof(struct
SkipList));

```

Allocation of memory for SkipList struct takes constant time. $\rightarrow O(1)$

```

lst->level = 0;

```

This assignment ``lst  $\rightarrow$  level = 0`` takes constant amount of time. $\rightarrow O(1)$

```

lst->header = createNode(-1, MAX_LEVEL);

```

createNode(-1, MAX_LEVEL)

```
// Function to create a new node using malloc
struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
    elements)
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}
```

$key = -1$ and $level = MAX_LEVEL = 3$ $\left[\begin{matrix} \text{constant time} \\ \text{complexity} \end{matrix} \right] \rightarrow \text{Takes } O(c) \text{ or } O(1)$

```
struct Node *n = (struct Node *)malloc(sizeof(struct Node));
```

Allocation of memory for SkipList struct takes constant time. $\rightarrow$ Takes $O(c)$ or $O(1)$

```
n->key = key;
```

$n \rightarrow key = key = -1$

This assignment takes constant amount of time. $\rightarrow O(c)$ or $O(1)$

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));
```

Allocation of memory for SkipList struct takes constant time. $\rightarrow$ Takes $O(c)$ or $O(1)$

```
// Initialize pointers to NULL
for (int i = 0; i <= level; i++) {
    n->forward[i] = NULL;
}
```

Here, we can say $\rightarrow O(LEVEL)$, but we know, $level = MAX_LEVEL = 3$ $\left[\begin{matrix} LEVEL \\ is \\ Constant \end{matrix} \right]$

As, i runs from 0 to $level = MAX_LEVEL = 3$ (which is a constant)

Loop becomes constant i.e. $O(c)$ or $O(1)$.

Retrun `n` which also takes constant `O(c)` or $O(1)$.

Hence in both the cases :

```
// Function to initialize the skip list
struct SkipList *createSkipList() {
    struct SkipList *lst = (struct SkipList *)malloc(sizeof(struct
SkipList));
    lst->level = 0;

    // The header node doesn't store real data, so we use -1 as a
placeholder.
    // It must have the maximum possible level.
    lst->header = createNode(-1, MAX_LEVEL);
    return lst;
}
```

Time Complexity $\rightarrow O(1)$.

```
// Function to create a new node using malloc
struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
elements)
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}
```

Time Complexity $\rightarrow O(LEVEL) \approx O(MAX_LEVEL) \approx O(c) \approx O(1)$.

Space Complexity of createNode

```
// Function to create a new node using malloc
struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
elements)
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}
```

- **Parametric Space (Input Space): $O(1)$**

The function takes two integer parameters (key and level) by value, which require a fixed amount of stack memory regardless of what those values are.

- **Full Data Input Space: $O(1)$**

Because the function accepts only primitive types and no externally allocated arrays or structures, the full input space is identical to the parametric space.

- **Auxiliary Space: $O(1)$**

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level + 1));
```

The function allocates heap memory dynamically. While the Node wrapper requires constant $O(1)$ space, the forward array requires memory proportional to $level + 1$.

As it require `level + 1`, we can say space complexity = $O(level)$, but :

As, $level = MAX_LEVEL = 3$ is a constant ,

The function allocates dynamic memory on the heap.

Because $level \leq 3$, the forward array will allocate a maximum of 4 pointers.

Allocating a fixed maximum amount of memory , regardless of the overall size of the skip list, is an $O(1)$ operation.

```
// Initialize pointers to NULL
for (int i = 0; i <= level; i++) {
    n->forward[i] = NULL;
}
```

here int `i` and `level` as well as $n \rightarrow forward[i] = NULL$ using same constant amount of space. Hence no new space is created like recursion , therefore this loop also consumes constant space = $O(c)$ or $O(1)$.

Hence Auxiliary space also becomes $O(1)$.

- **Total Space Complexity = Input Space + Auxiliary Space = $O(1)$.**

Space Complexity of createSkipList

```
// Function to initialize the skip list
struct SkipList *createSkipList() {
    struct SkipList *lst = (struct SkipList *)malloc(sizeof(struct
SkipList));
    lst->level = 0;

    // The header node doesn't store real data, so we use -1 as a
placeholder.
    // It must have the maximum possible level.
    lst->header = createNode(-1, MAX_LEVEL);
    return lst;
}
```

Because MAX_LEVEL is defined as a fixed constant (3), the mathematical complexity of this specific function simplifies to constant time and space.

Space Complexity Breakdown:

- *Parametric Space (Input Space): $O(1)$*

The function takes no arguments, meaning parameter space is strictly zero/constant.

- *Full Data Input Space: $O(1)$*

No inputs are passed, so this remains constant.

- *Auxiliary Space: $O(1)$*

The function allocates a fixed – size SkipList struct on the heap.

The call to createNode(-1, MAX_LEVEL) allocates memory for a Node and an array of 4 pointers ($\text{MAX_LEVEL} + 1$).

Because this allocation size is fixed at compile – time and does not grow, the auxiliary space is strictly $O(1)$.

Total Space Complexity: Input Space + Auxiliary Space = $O(1)$.

Algorithm

CONSTANT MAX_LEVEL = 3

1. Function createSkipList()

**((SkipList *)malloc(sizeof(SkipList)) is availability of space for insertion of Data,
hence, lets name it AVAIL.**

2. SET LST_PTR := AVAIL

3. SET LEVEL[LST_PTR] := 0

4. SET HEADER[LST_PTR] = createNode(-1, MAX_LEVEL)

Translatin createNode(-1, MAX_LEVEL)

1. Function createNode(Set Key := -1, Set Level := MAX_LEVEL)

**(Node *)malloc(sizeof(Node)) is availability of space for insertion of Data,
hence, lets name it AVAIL.**

2. SET n_ptr := AVAIL.

3. SET key[n_ptr] := Key

4. SET forward[n_ptr] := AVAIL * (Level + 1)

5. Repeat for $i = 0$ and $i \leq \text{Level}$:

5.a. Set forward[n_ptr][i] := NULL

[End of For Loop]

6. Return n_ptr

[End of Function createNode]

5. Return LST_PTR

[End of Function createSkipList]

[End of Algorithm]

Note :

We can also write in algorithm:

1. **SET $n_ptr.key := Key$**
 2. **SET $n_ptr.forward := AVAIL * (Level + 1)$**
 3. **Repeat for $i = 0$ and $i \leq Level$:**
 5. a. **Set $n_ptr.forward[i] := NULL$**
- [End of For Loop]**

Also fine .

Skip – List

–Insert Element(At Beginning)

–Program Analysis

Program Analysis:

```
void insertElement(struct SkipList *lst, int key) {
    struct Node *current = lst->header;

    // Array to keep track of the path we took (nodes needing pointer
updates)
    struct Node *update[MAX_LEVEL + 1];
    memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));

    // Start from the highest current level and move down
    for (int i = lst->level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key <
key) {
            current = current->forward[i];
        }
        update[i] = current;
    }

    // Move to level 0 and look at the next node
    current = current->forward[0];

    // If the key doesn't already exist, insert it
    if (current == NULL || current->key != key) {
        int rlevel = randomLevel();

        // If the random level is greater than the list's current max
level,
        // initialize the update array for the new levels to point to
the header.
        if (rlevel > lst->level) {
            for (int i = lst->level + 1; i <= rlevel; i++) {
                update[i] = lst->header;
            }
            lst->level = rlevel;
        }
    }
}
```

```

        // Create the new node and rewire pointers
        struct Node *n = createNode(key, rlevel);
        for (int i = 0; i <= rlevel; i++) {
            n->forward[i] = update[i]->forward[i];
            update[i]->forward[i] = n;
        }
        cout << "Successfully inserted key " << key << endl;
    } else {
        cout << "Key " << key << " already exists in the list." << endl;
    }
}

int main() {
    // Seed random number generator for random levels
    srand((unsigned)time(0));
    struct SkipList *list = createSkipList();
    int choice, key;

    while (true) {

        cout << "\n==== Skip List Menu =====" << endl;
        cout << "1. Insert" << endl;
        cout << "Enter your choice: ";

        if (!(cin >> choice)) {
            cout << "Invalid input. Exiting." << endl;
            break;
        }

        switch (choice) {

        case 1:
            cout << "Enter key to insert: ";
            cin >> key;
            insertElement(list, key);
            break;
        }
    }
    return 0;
}

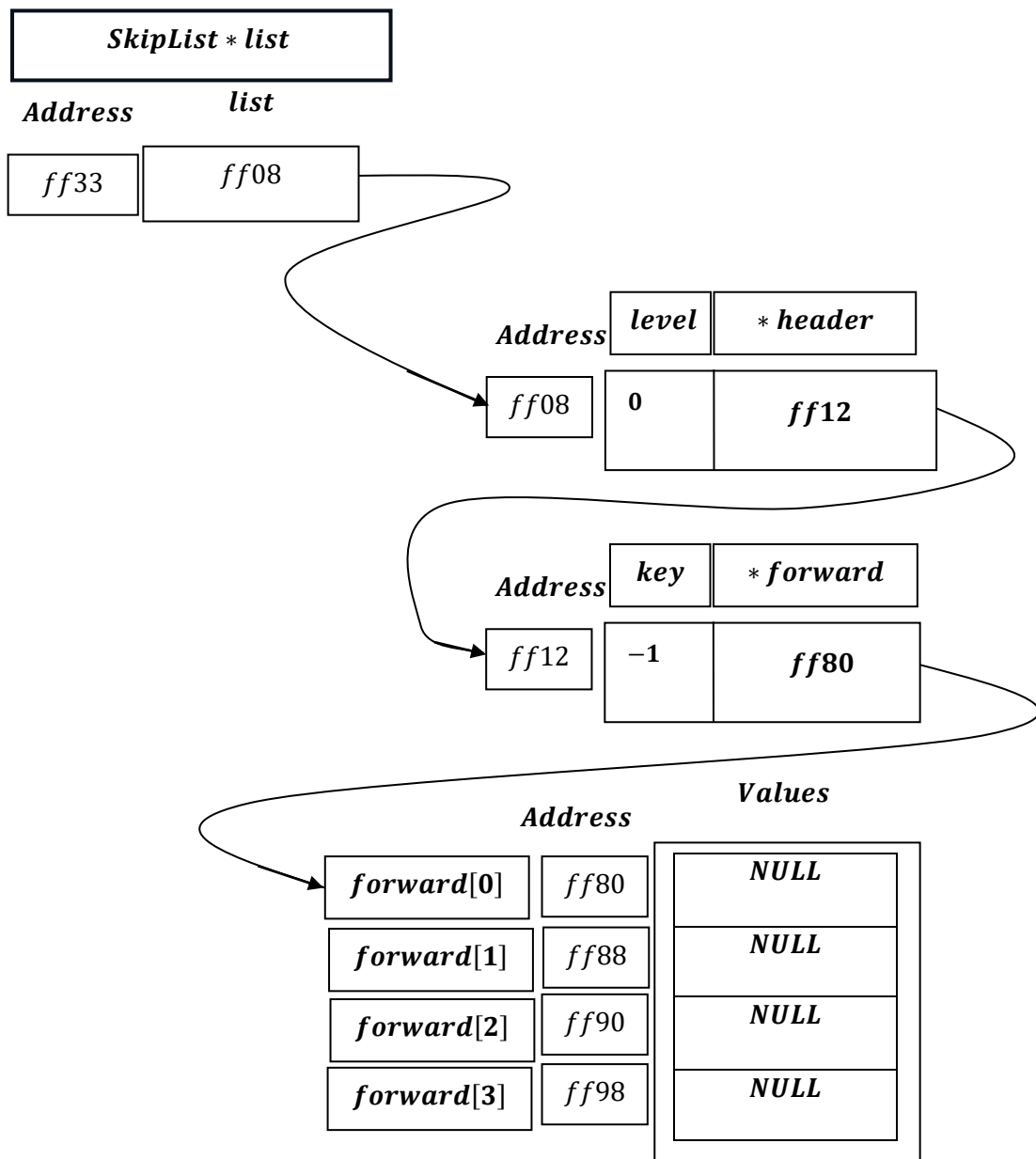
```


Program Analysis

```
struct SkipList *list = createSkipList();
```

We know from previous analysis, that :

*struct SkipList * list = createSkipList() = ff08*



```

if (!(cin >> choice)) {
    cout << "Invalid input. Exiting." << endl;
    break;
}

```

- *cin >> choice: This attempts to read a value from the standard input (the keyboard) and store it in the variable choice.*
- *The return value: The >> operator actually returns a reference to the stream itself (cin).*
- *The boolean check: In C++, when a stream object like cin is evaluated in a boolean context (like inside an if statement), it returns true if the read operation was successful and the stream is in a "good" state. It returns false if the read operation failed.*
- *The ! (NOT) operator: The ! negates the result.*
 - *If the input was successful, (cin >> choice) is true, so !(cin >> choice) is false, and the if block is skipped.*
 - *If the input failed, (cin >> choice) is false, so !(cin >> choice) becomes true, and the if block executes.*

Why would the input fail?

The most common reasons for cin to fail here are:

- *Type mismatch: For example, if choice is an int, but the user types a letter like "a".*
- *End of file (EOF): If the input stream is closed (e.g., the user presses Ctrl + D on Linux/ Mac*

or Ctrl + Z on Windows).

Inside the if block: If the input fails (e.g., the user enters letters instead of a number for a menu choice), the program will print ``Invalid input. Exiting.`` and then break out of the current loop (usually a while or for loop that is repeatedly asking for input).

∴ Input choice for program menu here → 1, which represent insert element.

As we see program menu :

```
cout << "\n==== Skip List Menu =====" << endl;
cout << "1. Insert" << endl;
cout << "Enter your choice: ";
```

```
switch (choice) {

case 1:
    cout << "Enter key to insert: ";
    cin >> key;
    insertElement(list, key);
    break;
```

Entering : Key = 20 .

insertElement(list: ff08 ,key: 20);

Now ,we have :

```
void insertElement(struct SkipList *lst, int key)
```

*list = struct SkipList * lst = ff08 and int key: 20.*

*SkipList * lst*

Address

list

<i>ff37</i>	<i>ff08</i>
-------------	-------------

*SkipList * list*

Address

list

<i>ff33</i>	<i>ff08</i>
-------------	-------------

Address

level

** header*

ff08

0

ff12

Address

key

** forward*

ff12

-1

ff80

Address

Values

forward[0]

ff80

forward[1]

ff88

forward[2]

ff90

forward[3]

ff98

NULL

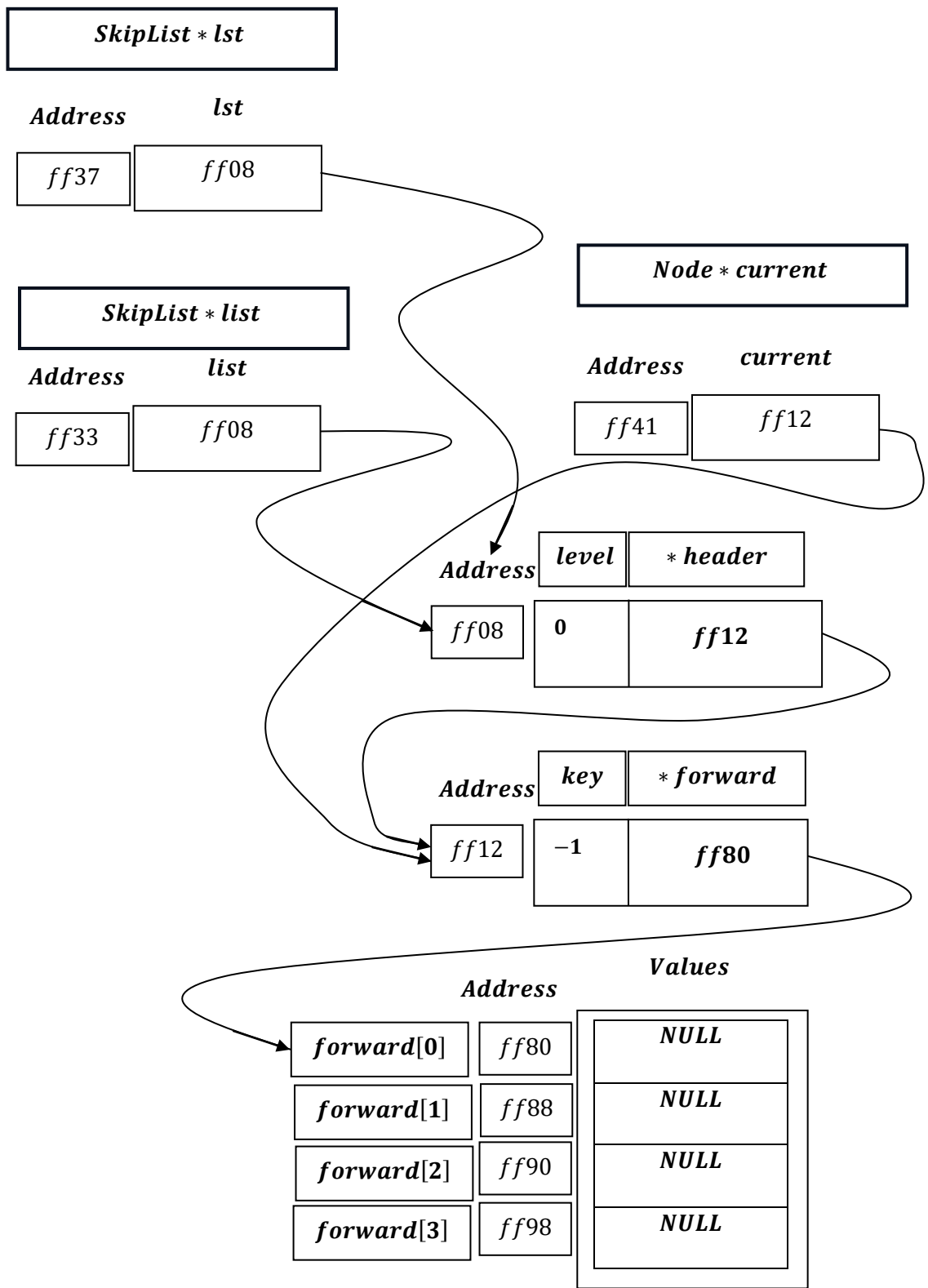
NULL

NULL

NULL

```
struct Node *current = lst->header;
```

current = *lst* → *header* : *ff12*.



```
struct Node *update[MAX_LEVEL + 1];
```

update[3 + 1 = 4],

MAX_LEVEL evaluates to 3. Consequently, the array size calculation is update[3 + 1 = 4].

This means the array is allocated to hold exactly 4 elements.

Before memset call, the update array may look like :

Array Element	Hypothetical Stack Address	Value (Garbage Data)	Pointer Status
update[0]	0x7fff0000 to 0x7fff0007	0x4a3f8921bce45099	Wild Pointer (Points to a random, invalid address)
update[1]	0x7fff0008 to 0x7fff000f	0x0000000000000002	Wild Pointer
update[2]	0x7fff0010 to 0x7fff0017	0x7ffee1245a00881c	Wild Pointer
update[3]	0x7fff0018 to 0x7fff001f	0x0000000000000000	Accidentally NULL (Just by chance)

More simpler way:

	Address	Values	
update[0]	0x7fff0000	0x4a3f8921bce45099	[8 bytes]
update[1]	0x7fff0008	0x0000000000000002	[8 bytes]
update[2]	0x7fff0010	0x7ffee1245a00881c	[8 bytes]
update[3]	0x7fff0018	0x0000000000000000	[8 bytes]

```
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

“memset” stands for Memory Set. It takes a block of memory and rapidly fills it with a specific value. Here `0` or NULL.

	Address	Values
update[0]	0x7fff0000	NULL
update[1]	0x7fff0008	NULL
update[2]	0x7fff0010	NULL
update[3]	0x7fff0018	NULL

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

Before continuing this part , lets clarify form earlier sections of the program:

```
struct SkipList {
    int level;           // Current maximum level in the list
    struct Node *header; // Pointer to the header node
};
```

```

struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
elements)
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}

// Function to initialize the skip list
struct SkipList *createSkipList() {
    struct SkipList *lst = (struct SkipList *)malloc(sizeof(struct
SkipList));
    lst->level = 0;

    // The header node doesn't store real data, so we use -1 as a
placeholder.
    // It must have the maximum possible level.
    lst->header = createNode(-1, MAX_LEVEL);
    return lst;
}

```

```

struct Node *createNode(int key, int level)

```

This level is local variable of the function createNode.

```

lst->level = 0;

```

This level variable is member variable of struct SkipList.


```
struct SkipList {
    int level;           // Current maximum level in the list
    struct Node *header; // Pointer to the header node
};
```

*∴ here $lst \rightarrow level$ is member variable of struct *SkipList*, which is assigned to 0.*

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

there it is :

$i = 0$ and $i \geq 0$; i.e. Condition is true but will run single time:

And we know :

$forward[0] = forward[1] = forward[2] = forward[3] = NULL$.

Therefore,

```
while (current->forward[i] != NULL && current->forward[i]->key <
key)
```

*here, $current \rightarrow forward[i] = NULL$ as it is *NULL* ,no further checking of $current \rightarrow forward[i] \rightarrow key$ as $current \rightarrow forward[i] = NULL$, hence whole condition of the while loop becomes false .*

```
update[i] = current;
```

update[0] = current = ff12.

And $i = i - 1 = 0 - 1 = -1$ [Update]

$i = -1$ and $i \geq 0$ [Condition is false], hence, Loop execution ends.

```
current = current->forward[0];
```

current = current $\rightarrow$ forward[0] = NULL

*SkipList * lst*

Address

lst

<i>ff37</i>	<i>ff08</i>
-------------	-------------

*SkipList * list*

Address

list

<i>ff33</i>	<i>ff08</i>
-------------	-------------

*Node * current*

Address

current

<i>ff41</i>	<i>NULL</i>
-------------	-------------

Address

level

** header*

<i>ff08</i>	<i>0</i>	<i>ff12</i>
-------------	----------	-------------

Address

key

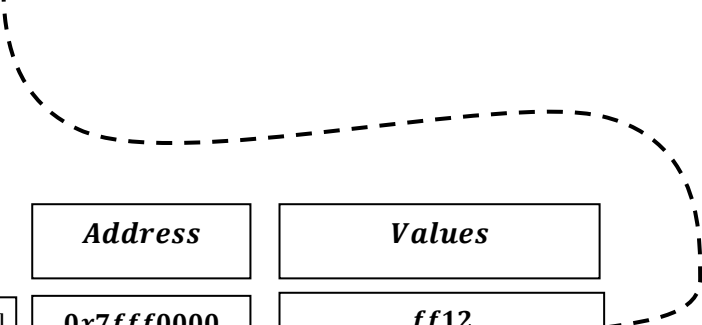
** forward*

<i>ff12</i>	<i>-1</i>	<i>ff80</i>
-------------	-----------	-------------

Address

Values

<i>forward[0]</i>	<i>ff80</i>	<i>NULL</i>
<i>forward[1]</i>	<i>ff88</i>	<i>NULL</i>
<i>forward[2]</i>	<i>ff90</i>	<i>NULL</i>
<i>forward[3]</i>	<i>ff98</i>	<i>NULL</i>



	<i>Address</i>	<i>Values</i>
<i>update[0]</i>	0x7fff0000	ff12
<i>update[1]</i>	0x7fff0008	NULL
<i>update[2]</i>	0x7fff0010	NULL
<i>update[3]</i>	0x7fff0018	NULL

```
current = current->forward[0];
```

current = *current* → *forward*[0] = *NULL*

Now,

```
// If the key doesn't already exist, insert it
if (current == NULL || current->key != key) {
    int rlevel = randomLevel();

    // If the random level is greater than the list's current max
    level,
    // initialize the update array for the new levels to point to
    the header.
    if (rlevel > lst->level) {
        for (int i = lst->level + 1; i <= rlevel; i++) {
            update[i] = lst->header;
        }
        lst->level = rlevel;
    }

    // Create the new node and rewire pointers
    struct Node *n = createNode(key, rlevel);
    for (int i = 0; i <= rlevel; i++) {
        n->forward[i] = update[i]->forward[i];
        update[i]->forward[i] = n;
    }
}
```

```

    }
    cout << "Successfully inserted key " << key << endl;
} else {
    cout << "Key " << key << " already exists in the list." << endl;
}
}

```

*if current = NULL is true , and as current = NULL , therefore ,
current → key ≠ key , this is not considered.*

```
int rlevel = randomLevel();
```

RANDOMLEVEL() executes:

```
int randomLevel() {... }
```

First to start with – there is no parameters.

```
int lvl = 0;
```

So, lvl = 0.

```

while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {
    lvl++;
}

```

MAX_LEVEL = 3 as,

```
#define MAX_LEVEL 3
```

say 1st, rand() = 42

1st loop run:

while $\left(\begin{array}{l} (rand() = 42 \% 2) = 0 == 0 \left[\frac{42}{2} = 0[Remainder] \right] [True] \\ AND\ lvl = 0 < MAX_LEVEL = 3 [TRUE] \end{array} \right):$

lvl = 0 + 1 = 1

2nd, rand() = 998

2nd loop run:

while $\left(\begin{array}{l} (rand() = 998 \% 2) = 0 == 0 \left[\frac{998}{2} = 0[Remainder] \right] [True] \\ AND\ lvl = 1 < MAX_LEVEL = 3 [TRUE] \end{array} \right):$

lvl = 1 + 1 = 2

3rd, rand() = 8471

3rd loop run:

while $\left(\begin{array}{l} (rand() = 8471 \% 2) = 1 \neq 0 \left[\frac{8471}{2} = 1[Remainder] \right] [False] \\ AND\ lvl = 2 < MAX_LEVEL = 3 [TRUE] \end{array} \right):$

Condition fails , hence :

```
return lvl;
```

return , lvl = 2

```
int rlevel = randomLevel();
```

int rlevel = randomLevel() = 2 .

```
if (rlevel > lst->level) {  
    for (int i = lst->level + 1; i <= rlevel; i++) {  
        update[i] = lst->header;  
    }  
    lst->level = rlevel;  
}
```

if rlevel = 2 > lst → level = 0 is true:

Loop: int i = lst → level = (0 + 1 = 1); i = 1 ≤ rlevel = 2 $\left[\begin{smallmatrix} \text{condition is} \\ \text{true} \end{smallmatrix} \right]$

update[1] = lst → header := ff12

i = i + 1 = 1 + 1 = 2

Loop: int i = 2 ≤ rlevel = 2 $\left[\begin{smallmatrix} \text{condition is} \\ \text{true} \end{smallmatrix} \right]$

update[2] = lst → header := ff12

i = i + 1 = 2 + 1 = 3

Loop: int i = 3 ≤ rlevel = 2 $\left[\begin{smallmatrix} \text{condition is} \\ \text{False} \end{smallmatrix} \right]$

lst → level = rlevel = 2.

*SkipList * lst*

Address

lst

ff37

ff08

*SkipList * list*

Address

list

ff33

ff08

*Node * current*

Address

current

ff41

NULL

Address

level

** header*

ff08

2

ff12

Address

key

** forward*

ff12

-1

ff80

Address

Values

forward[0]

ff80

NULL

forward[1]

ff88

NULL

forward[2]

ff90

NULL

forward[3]

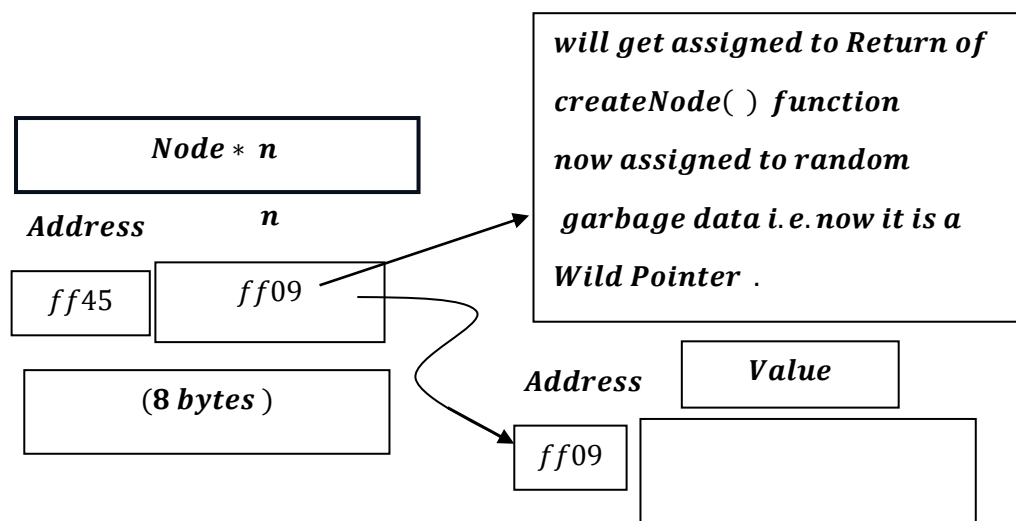
ff98

NULL

	Address	Values
update[0]	0x7fff0000	ff12
update[1]	0x7fff0008	ff12
update[2]	0x7fff0010	ff12
update[3]	0x7fff0018	NULL

```
struct Node *n = createNode(key, rlevel){...};
```

*struct Node * n = createNode(key = 20,rlevel = 2)*



```
// Function to create a new node using malloc
struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
    elements)
```

```

    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}

```

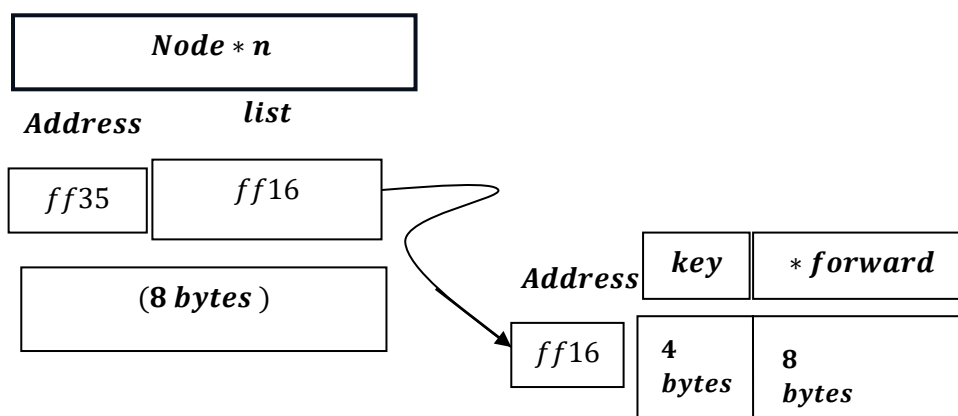
struct Node * createNode(int key = 20, level = 2)

Now,

```

struct Node *n = (struct Node *)malloc(sizeof(struct Node));

```



```

struct Node *createNode(int key, int level) {
    n->key = key;
}

```

*This 'key' is
local variable.*

*This 'key' is
member variable
of struct Node.*

$n \rightarrow \text{key} = 20$.

```
struct Node {  
    int key;  
    // Array of pointers to next nodes, one for each level  
    struct Node **forward;  
};
```

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level  
+ 1));
```

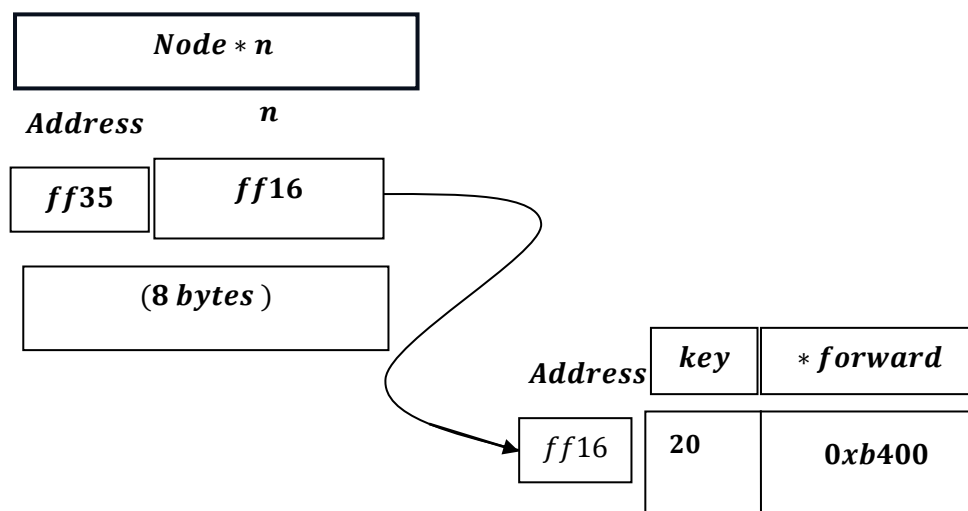
*It will allocate, $(\text{struct Node} * = 8 \text{ bytes}) \times (\text{level} + 1)$*

$= 8 \text{ bytes} \times (2 + 1)$

$= 8 \text{ bytes} \times 3$

$= 24 \text{ bytes}$

Which means , it allocates 3 slots of memory of 8 bytes , like: –



*Node * n*

Address *n*

ff35 *ff16*

(8 bytes)

Address *key* ** forward*

ff16 20 0xb400

Address *Values*

<i>forward[0]</i>	0xb400	
<i>forward[1]</i>	0xb408	
<i>forward[2]</i>	0xb410	

Actual structure

Address at which value is stored	Value	offset used
<i>ff16</i>	<i>key = 20</i>	<i>key[ff16 – ff19] 4 bytes for int</i>
		<i>padding → [ff1a – ff1d] 4 bytes for padding</i>
<i>ff1e</i>	<i>Address: 0xb400</i>	<i>* forward → [ff1e – ff25] 8 bytes for pointer variable</i>
<i>0xb400</i>	<i>NULL</i>	<i>forward[0] → [0xb400 – 0xb407] 8 bytes for pointer variable</i>
<i>0xb408</i>	<i>NULL</i>	<i>forward[1] → [0xb408 – 0xb40f] 8 bytes for pointer variable</i>
<i>0xb410</i>	<i>NULL</i>	<i>forward[2] → [0xb410 – 0xb417] 8 bytes for pointer variable</i>

This will be discussed in memory allocation section.

```
for (int i = 0; i <= level; i++) {  
    n->forward[i] = NULL;  
}
```

Note , level = 2 .

Iteration 1:

i = 0; 0 ≤ level = 2 [Condition True]

n → forward[0] = NULL

i = 0 + 1 = 1

Iteration 2:

i = 1; 1 ≤ level = 2 [Condition True]

n → forward[1] = NULL

i = 1 + 1 = 2

Iteration 3:

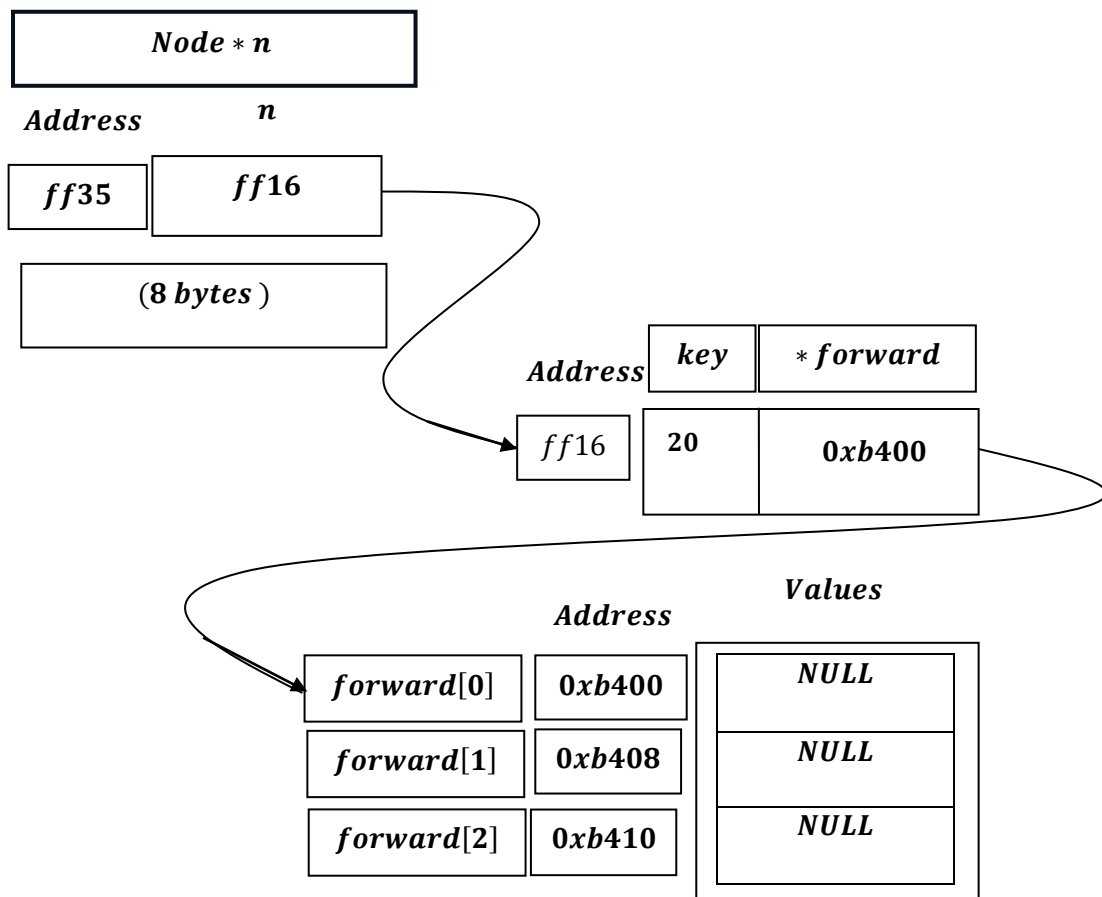
$i = 2; 2 \leq \text{level} = 2$ [Condition True]

$n \rightarrow \text{forward}[2] = \text{NULL}$

$i = 2 + 1 = 3$

Iteration 4:

$i = 3; 3 \leq \text{level} = 2$ [Condition is False]
hence , loop exits]

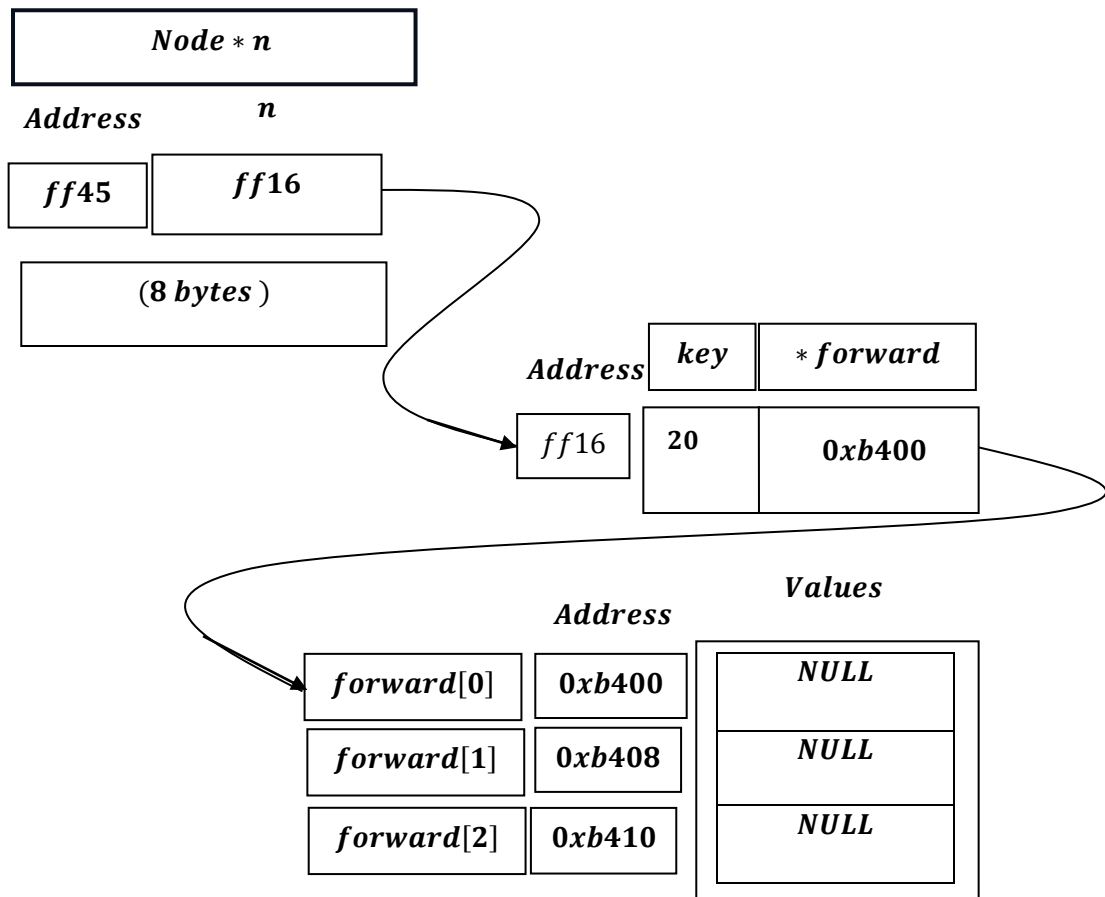


```
return n;
```

i. e. Return $n = \text{ff16}$.

struct Node * n = createNode(key = 20,rlevel = 2)

struct Node * n = ff16



```
for (int i = 0; i <= rlevel; i++) {
    n->forward[i] = update[i]->forward[i];
    update[i]->forward[i] = n;
}
```

int i = 0; i ≤ rlevel = 2

n → forward[0] = update[0] → forward[0] = NULL

update[0] → forward[0] = n = ff16

i = i + 1 = 0 + 1 = 1

int i = 1; i ≤ rlevel = 2

n → forward[1] = update[1] → forward[1] = NULL

update[1] → forward[1] = n = ff16

i = i + 1 = 1 + 1 = 2

int i = 2; i ≤ rlevel = 2

n → forward[2] = update[2] → forward[2] = NULL

update[2] → forward[2] = n = ff16

i = i + 1 = 2 + 1 = 3

int i = 3; i ≤ rlevel = 2 $\left[\begin{array}{l} \textit{Condition} \\ \textit{is False} \end{array} \right]$

*SkipList * lst*

Address

lst

<i>ff37</i>	<i>ff08</i>
-------------	-------------

*SkipList * list*

Address

list

<i>ff33</i>	<i>ff08</i>
-------------	-------------

*Node * current*

Address

current

<i>ff41</i>	<i>NULL</i>
-------------	-------------

Address

level

** header*

<i>ff08</i>

2

ff12

Address

key

** forward*

<i>ff12</i>

-1

ff80

Address

Values

forward[0]

ff80

forward[1]

ff88

forward[2]

ff90

forward[3]

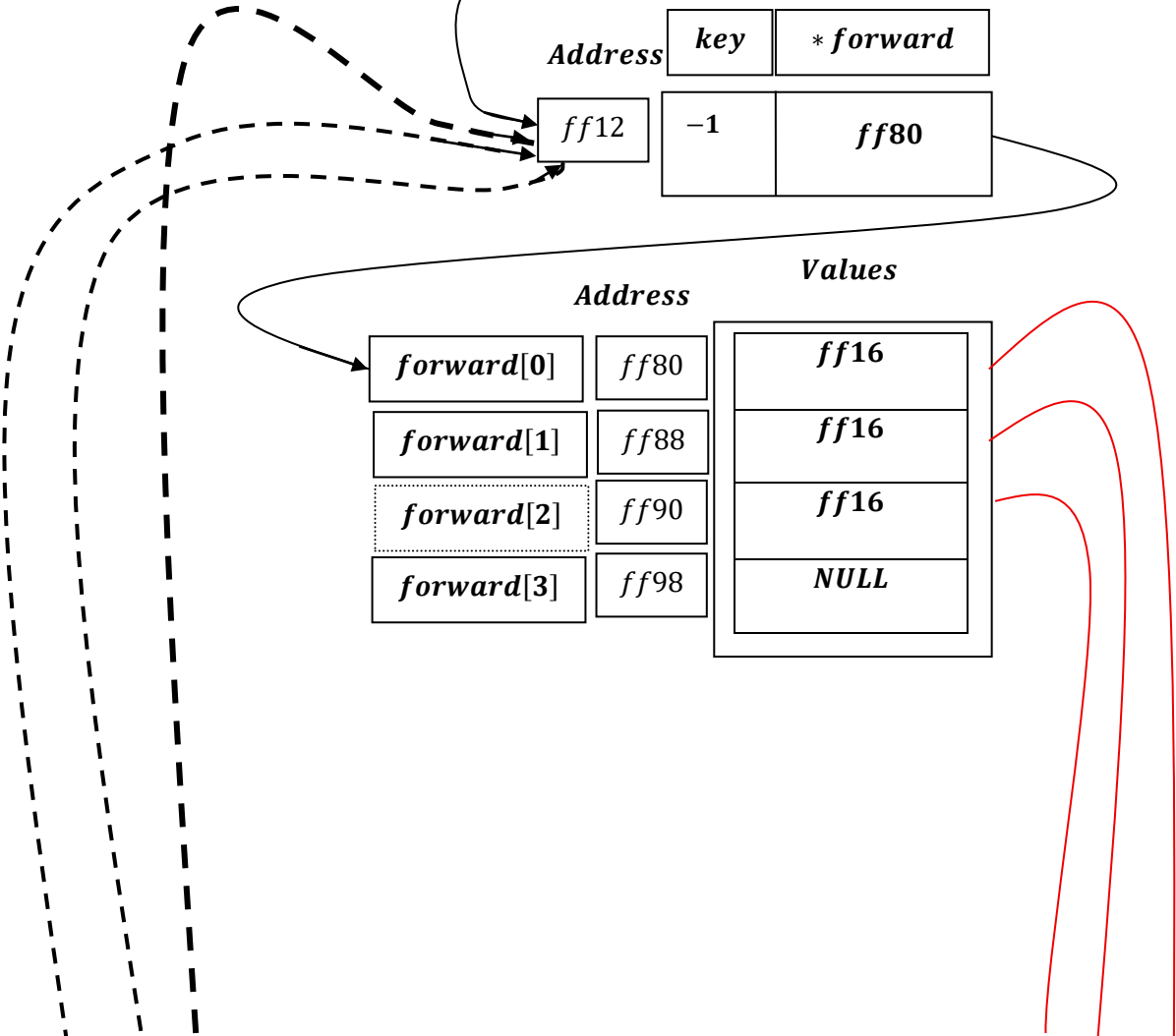
ff98

ff16

ff16

ff16

NULL



	Address	Values
<i>update</i> [0]	0x7fff0000	<i>ff12</i>
<i>update</i> [1]	0x7fff0008	<i>ff12</i>
<i>update</i> [2]	0x7fff0010	<i>ff12</i>
<i>update</i> [3]	0x7fff0018	NULL

*Node * n*

Address *n*

ff45 *ff16*

(8 bytes)

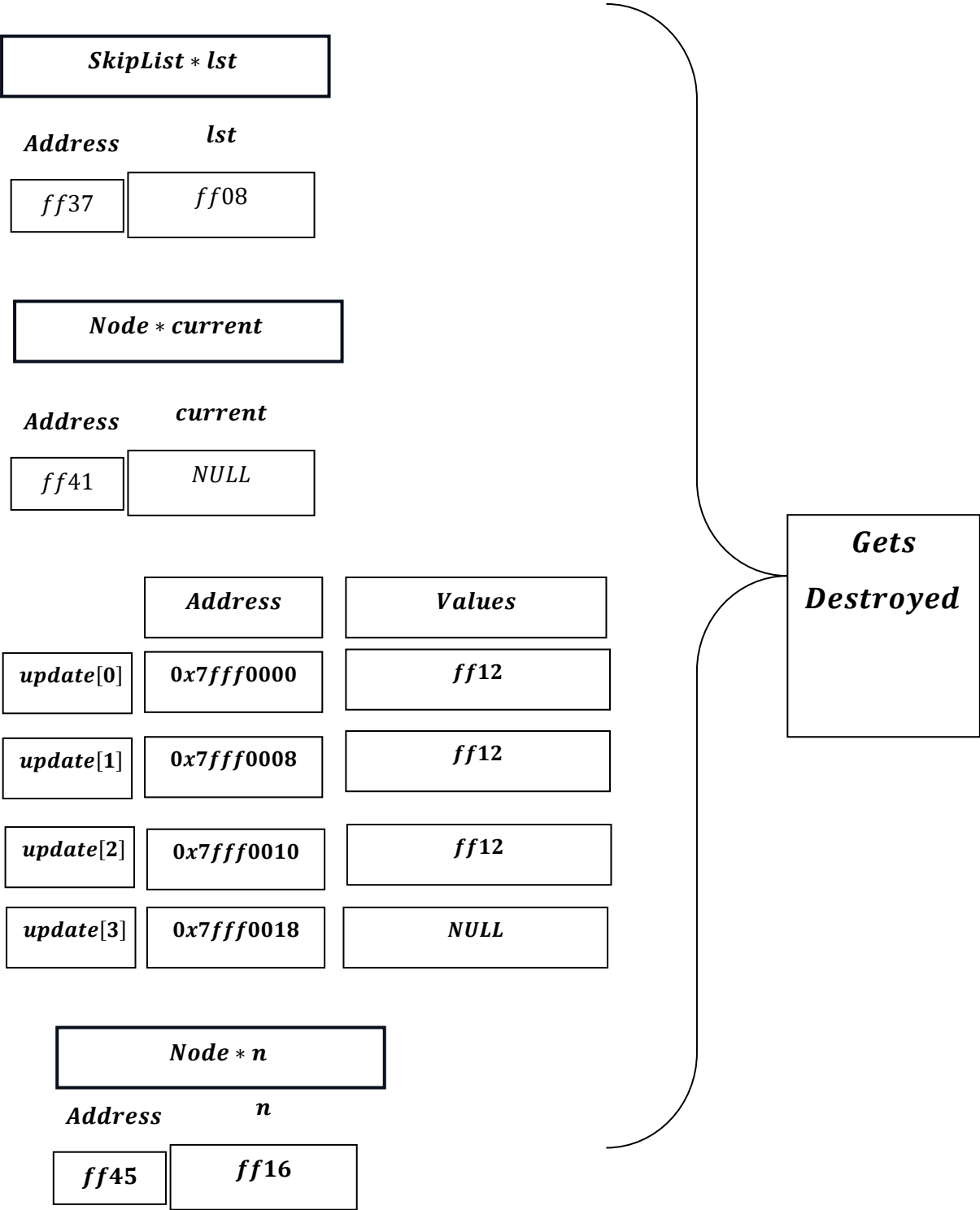
Address	key	* <i>forward</i>
<i>ff16</i>	20	0xb400

	Address	Values
<i>forward</i> [0]	0xb400	NULL
<i>forward</i> [1]	0xb408	NULL
<i>forward</i> [2]	0xb410	NULL

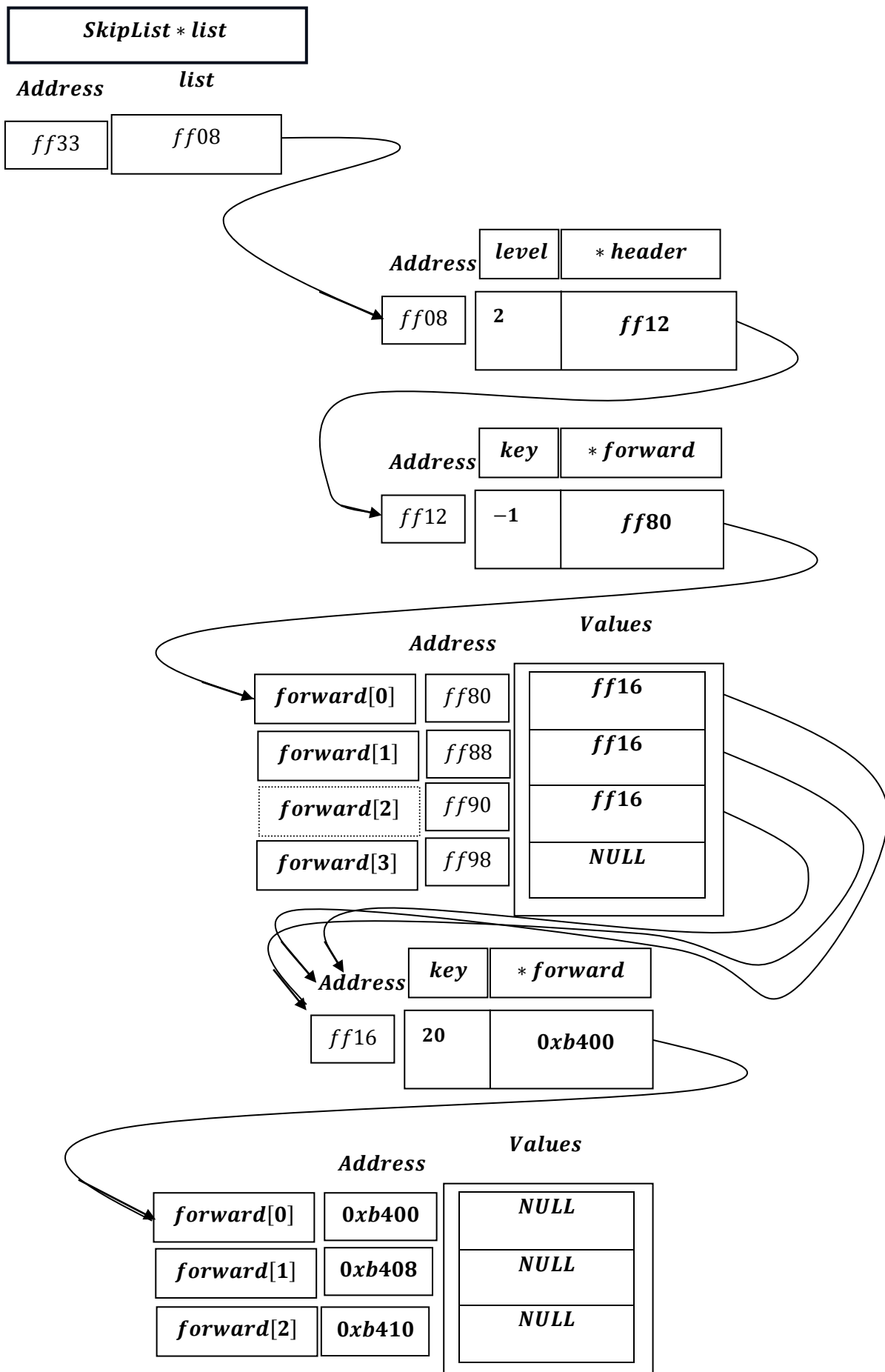
```
cout << "Successfully inserted key " << key << endl;
```

Successfully inserted key : 20 and function ends .

As , the function ends :



And we are left with:



Skip List – Insert Element – Program Analysis
–Insertion (Insert At End)– Part 2

After First Insertion we have:

*SkipList * list*

Address list

ff33 ff08

*Address level * header*

ff08 2 ff12

*Address key * forward*

ff12 -1 ff80

Address Values

<i>forward</i>	<i>Address</i>	<i>Values</i>
<i>forward[0]</i>	<i>ff80</i>	<i>ff16</i>
<i>forward[1]</i>	<i>ff88</i>	<i>ff16</i>
<i>forward[2]</i>	<i>ff90</i>	<i>ff16</i>
<i>forward[3]</i>	<i>ff98</i>	<i>NULL</i>

*Address key * forward*

ff16 20 0xb400

Address Values

<i>forward</i>	<i>Address</i>	<i>Values</i>
<i>forward[0]</i>	<i>0xb400</i>	<i>NULL</i>
<i>forward[1]</i>	<i>0xb408</i>	<i>NULL</i>
<i>forward[2]</i>	<i>0xb410</i>	<i>NULL</i>

```
case 1:
    cout << "Enter key to insert: ";
    cin >> key;
    insertElement(list, key);
    break;
```

Entering : Key = 30 .

insertElement(list: ff08 ,key: 30);

Now ,we have :

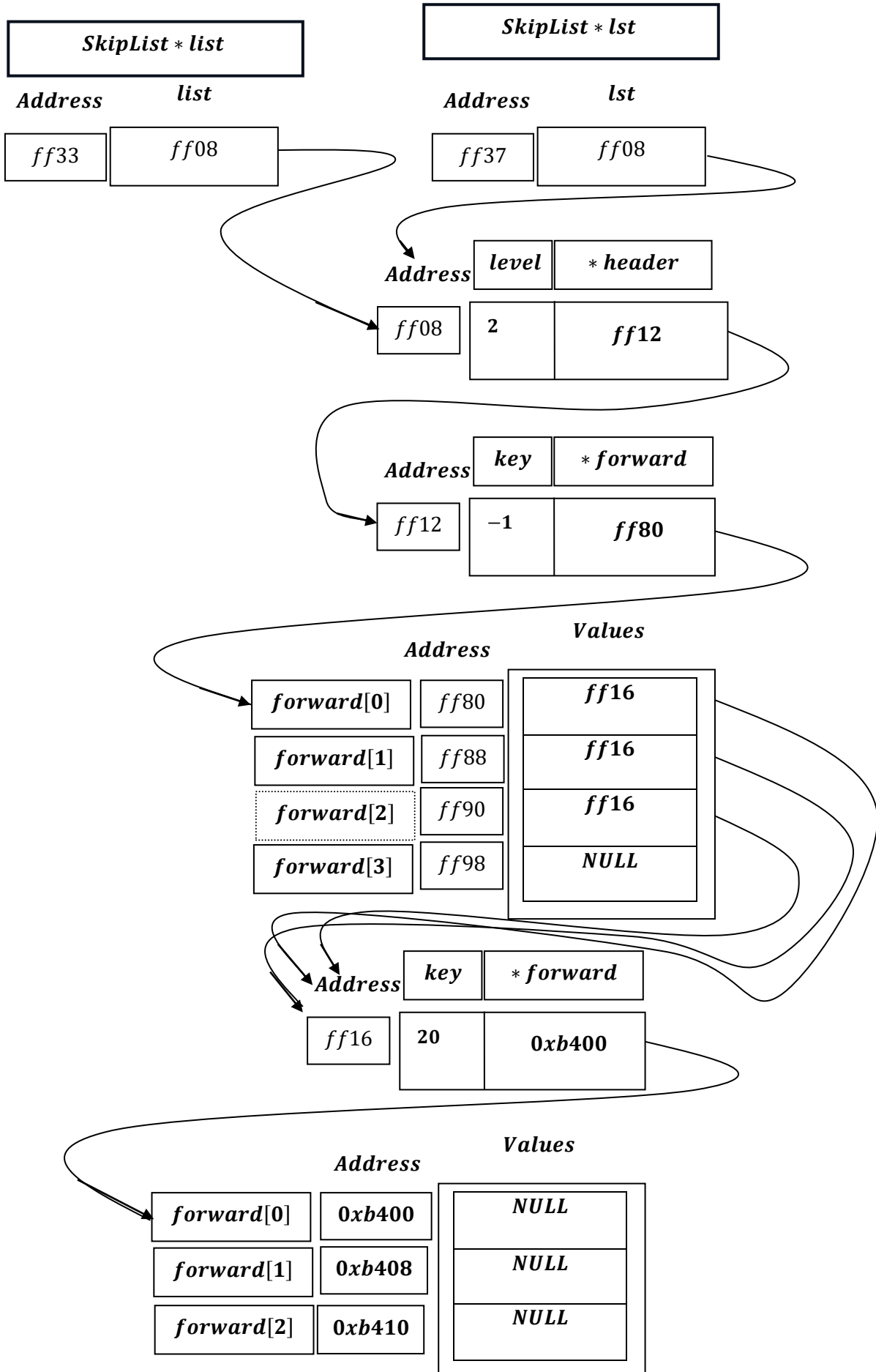
```
void insertElement(struct SkipList *lst, int key)
```

*list = struct SkipList * lst = ff08 and int key: 30.*

Now ,we have :

```
void insertElement(struct SkipList *lst, int key)
```

*list = struct SkipList * lst = ff08 and int key: 30.*




```
struct Node *current = lst->header;
```

current = lst → header : ff12.

```
struct Node *update[MAX_LEVEL + 1];
```

update[3 + 1 = 4] ,

MAX_LEVEL evaluates to 3. Consequently, the array size calculation is update[3 + 1 = 4].

This means the array is allocated to hold exactly 4 elements.

Before memset call, the update array may look like :

Array Element	Hypothetical Stack Address	Value (Garbage Data)	Pointer Status
update[0]	0x7fff0000 to 0x7fff0007	0x4a3f8921bce45099	Wild Pointer (Points to a random, invalid address)
update[1]	0x7fff0008 to 0x7fff000f	0x0000000000000002	Wild Pointer
update[2]	0x7fff0010 to 0x7fff0017	0x7ffee1245a00881c	Wild Pointer
update[3]	0x7fff0018 to 0x7fff001f	0x0000000000000000	Accidentally NULL (Just by chance)

More simpler way:

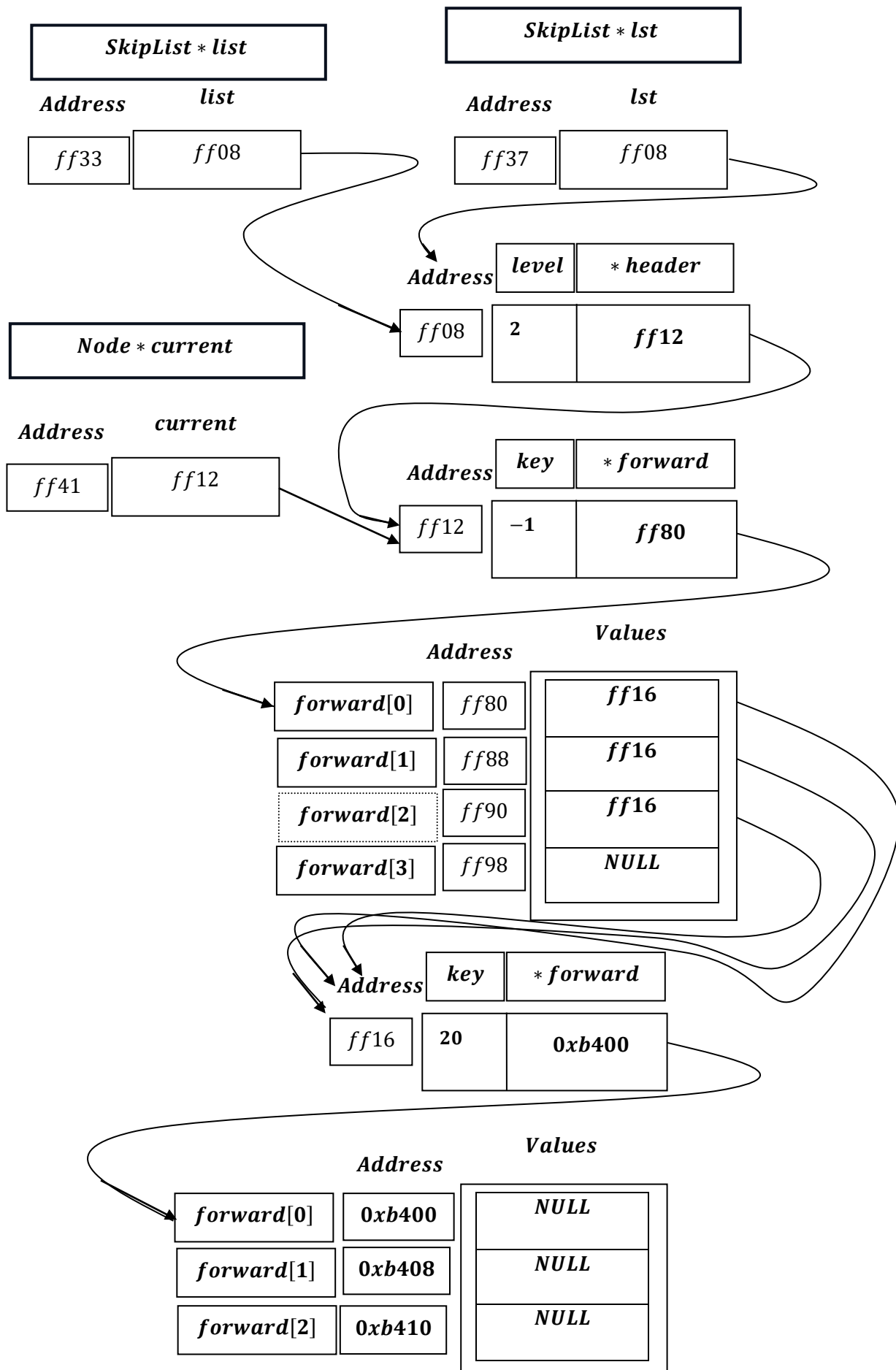
	Address	Values	
update[0]	0x7fff0000	0x4a3f8921bce45099	[8 bytes]
update[1]	0x7fff0008	0x0000000000000002	[8 bytes]
update[2]	0x7fff0010	0x7ffee1245a00881c	[8 bytes]
update[3]	0x7fff0018	0x0000000000000000	[8 bytes]

```
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

``memset`` stands for Memory Set. It takes a block of memory and rapidly fills it with a specific value. Here `0` or NULL.

	Address	Values
update[0]	0x7fff0000	NULL
update[1]	0x7fff0008	NULL
update[2]	0x7fff0010	NULL
update[3]	0x7fff0018	NULL

Here we kept addresses same like: 0x7fff0000 , just for an example , but it may not be same , in reality.



	<i>Address</i>	<i>Values</i>
<i>update[0]</i>	0x7fff0000	<i>NULL</i>
<i>update[1]</i>	0x7fff0008	<i>NULL</i>
<i>update[2]</i>	0x7fff0010	<i>NULL</i>
<i>update[3]</i>	0x7fff0018	<i>NULL</i>

```

for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}

```

For : $i = lst \rightarrow level = 2 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

while: $current \rightarrow forward[2] = ff16 \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$ And

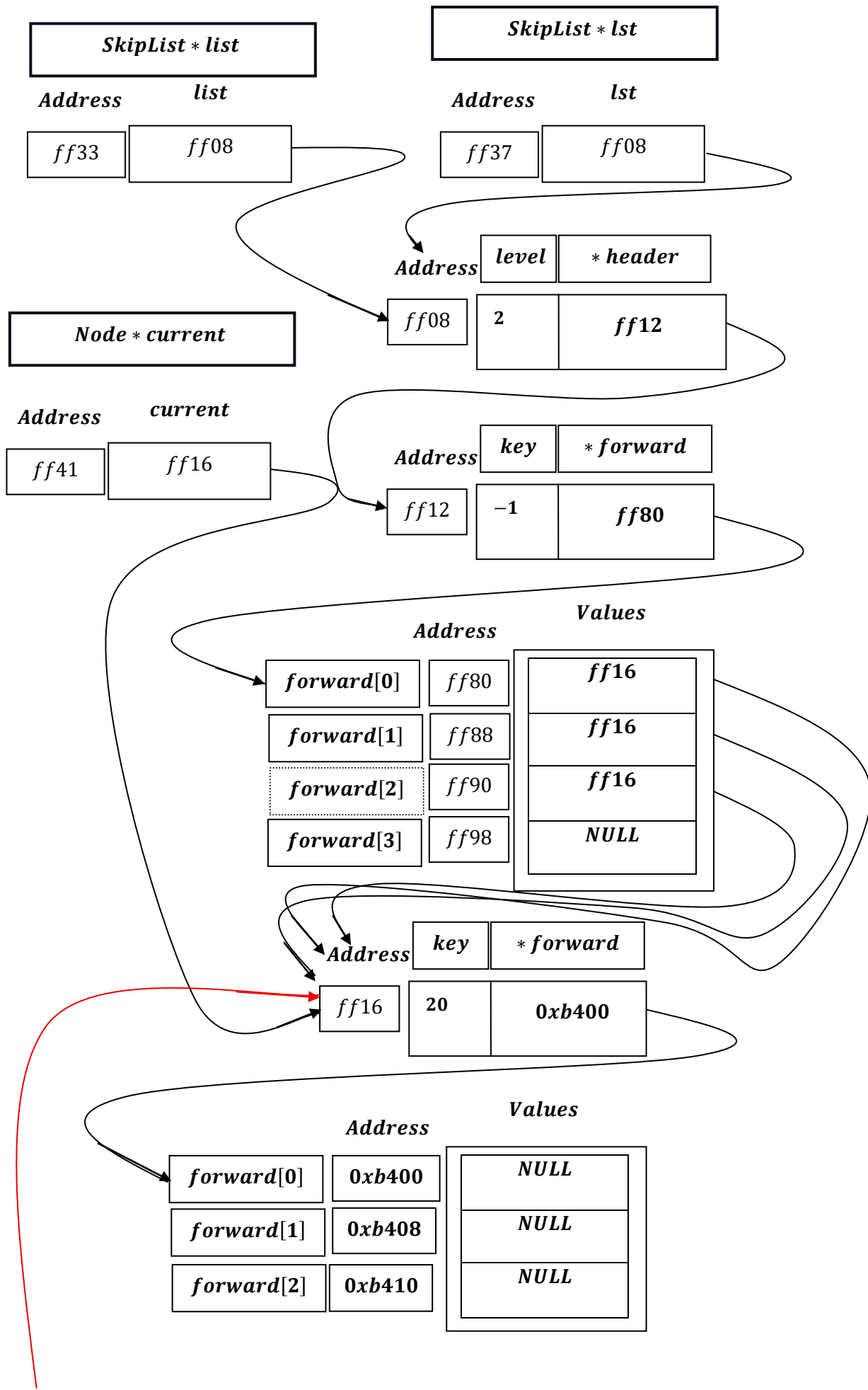
$current \rightarrow forward[2] \rightarrow key: 20 < key: 30$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

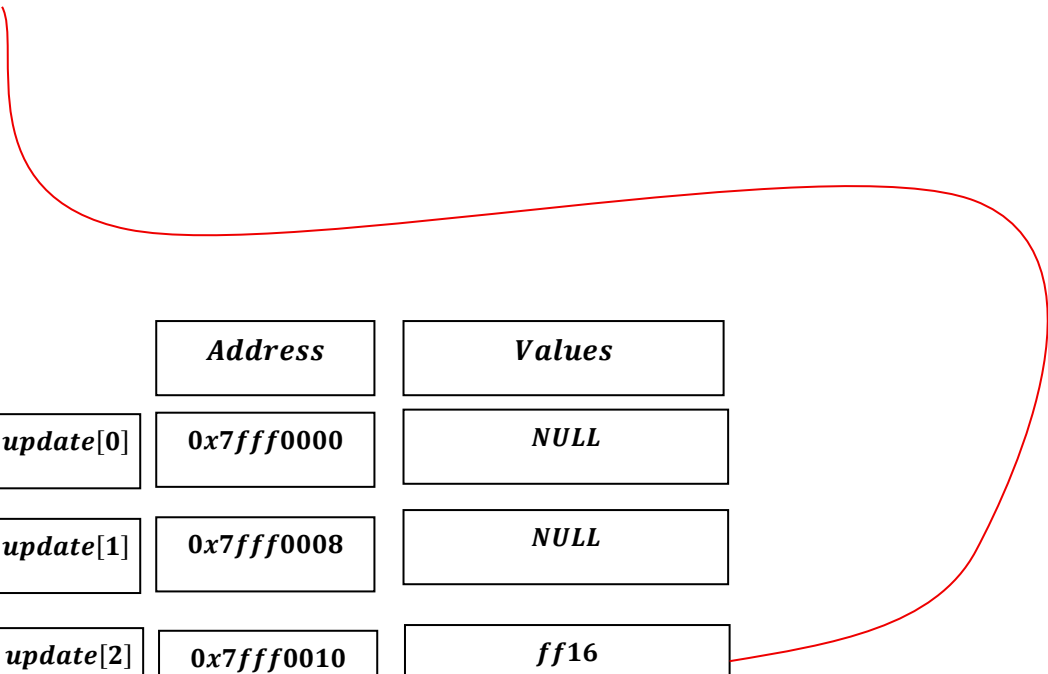
$current = current \rightarrow forward[2] = ff16$

while: $current \rightarrow forward[2] = NULL \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False, while} \\ \text{loop exits} \end{array} \right]$

$update[2] = current = ff16.$

$i = i - 1 = 2 - 1 = 1.$





	<i>Address</i>	<i>Values</i>
<i>update[0]</i>	<i>0x7fff0000</i>	<i>NULL</i>
<i>update[1]</i>	<i>0x7fff0008</i>	<i>NULL</i>
<i>update[2]</i>	<i>0x7fff0010</i>	<i>ff16</i>
<i>update[3]</i>	<i>0x7fff0018</i>	<i>NULL</i>

For : $i = 1 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

while: $current \rightarrow forward[1] = NULL \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$

[hence, while loop exits]

$update[1] = current = ff16.$

$i = i - 1 = 1 - 1 = 0.$

SkipList * list

Address list

ff33	ff08
------	------

SkipList * lst

Address lst

ff37	ff08
------	------

Node * current

Address current

ff41	ff16
------	------

Address level * header

ff08	2	ff12
------	---	------

Address key * forward

ff12	-1	ff80
------	----	------

Address Values

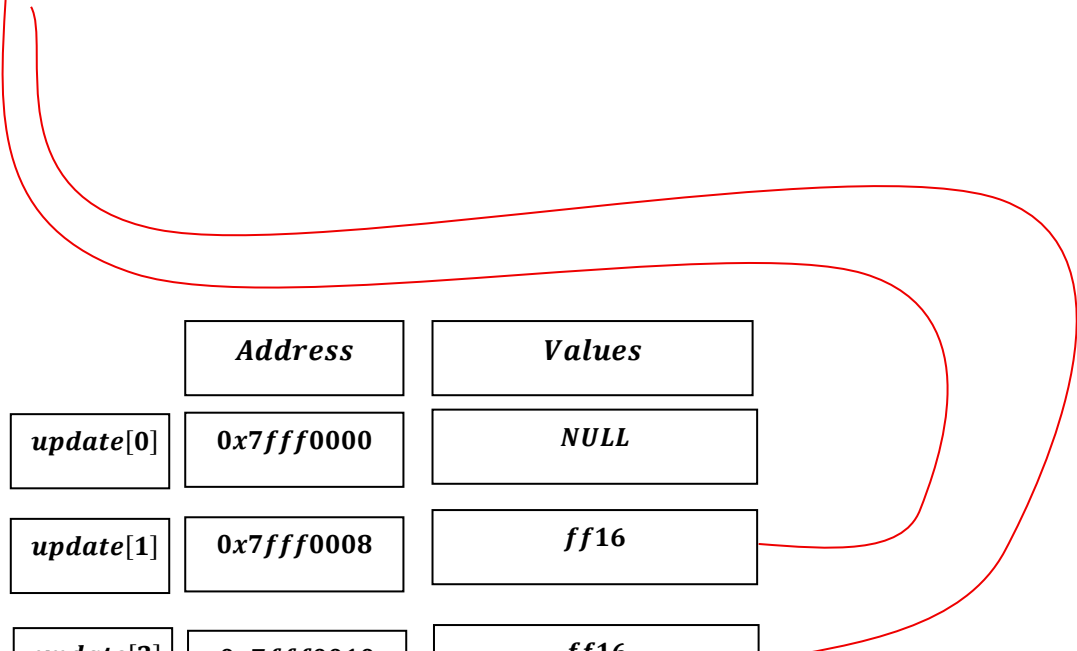
Address	Values	
forward[0]	ff80	ff16
forward[1]	ff88	ff16
forward[2]	ff90	ff16
forward[3]	ff98	NULL

Address key * forward

ff16	20	0xb400
------	----	--------

Address Values

Address	Values	
forward[0]	0xb400	NULL
forward[1]	0xb408	NULL
forward[2]	0xb410	NULL



	<i>Address</i>	<i>Values</i>
<i>update[0]</i>	<i>0x7fff0000</i>	<i>NULL</i>
<i>update[1]</i>	<i>0x7fff0008</i>	<i>ff16</i>
<i>update[2]</i>	<i>0x7fff0010</i>	<i>ff16</i>
<i>update[3]</i>	<i>0x7fff0018</i>	<i>NULL</i>

For : $i = 0 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

while: $current \rightarrow forward[0] = NULL \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$

[hence, while loop exits]

$update[0] = current = ff16.$

$i = i - 1 = 0 - 1 = -1.$

*SkipList * list*

Address list

<i>ff33</i>	<i>ff08</i>
-------------	-------------

*SkipList * lst*

Address lst

<i>ff37</i>	<i>ff08</i>
-------------	-------------

*Node * current*

Address current

<i>ff41</i>	<i>ff16</i>
-------------	-------------

*Address level * header*

<i>ff08</i>	2	<i>ff12</i>
-------------	---	-------------

*Address key * forward*

<i>ff12</i>	-1	<i>ff80</i>
-------------	----	-------------

Address Values

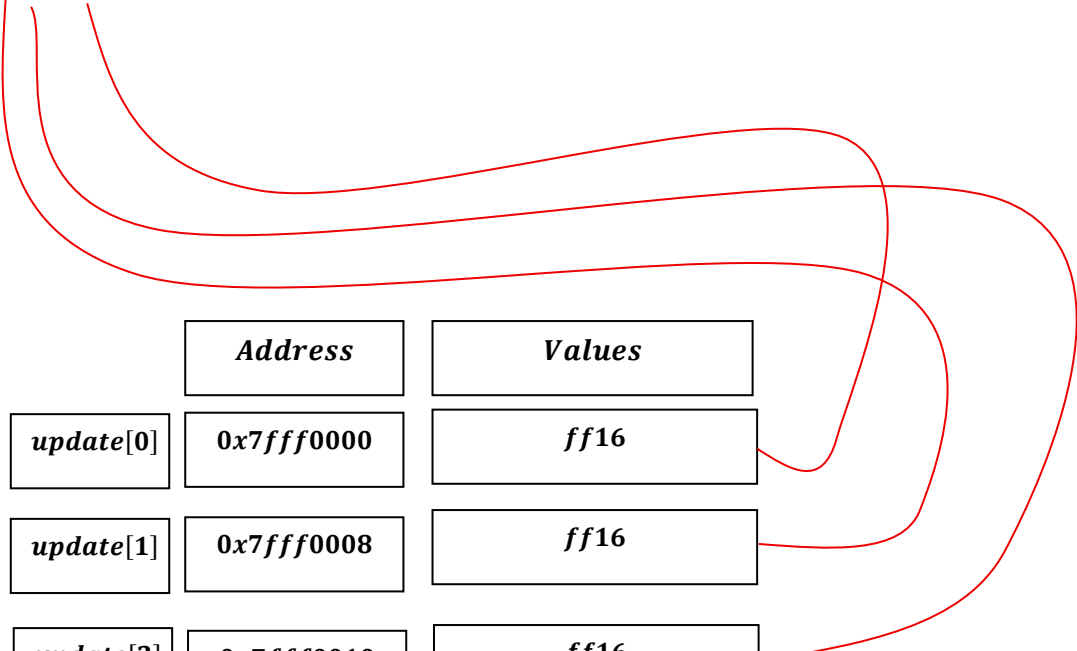
<i>forward</i>	<i>Address</i>	<i>Values</i>
<i>forward[0]</i>	<i>ff80</i>	<i>ff16</i>
<i>forward[1]</i>	<i>ff88</i>	<i>ff16</i>
<i>forward[2]</i>	<i>ff90</i>	<i>ff16</i>
<i>forward[3]</i>	<i>ff98</i>	<i>NULL</i>

*Address key * forward*

<i>ff16</i>	20	<i>0xb400</i>
-------------	----	---------------

Address Values

<i>forward</i>	<i>Address</i>	<i>Values</i>
<i>forward[0]</i>	<i>0xb400</i>	<i>NULL</i>
<i>forward[1]</i>	<i>0xb408</i>	<i>NULL</i>
<i>forward[2]</i>	<i>0xb410</i>	<i>NULL</i>



	Address	Values
update[0]	0x7fff0000	ff16
update[1]	0x7fff0008	ff16
update[2]	0x7fff0010	ff16
update[3]	0x7fff0018	NULL

For : $i = -1 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$, condition is false and loop exits.

```
current = current->forward[0];
```

$current = current \rightarrow forward[0] = NULL$.

```
if (current == NULL || current->key != key) {...}
```

$current = NULL$ (True)

```
int rlevel = randomLevel();
```

$rlevel = randomlevel() = 1$ (say)

```
if (rlevel > lst->level) {
    for (int i = lst->level + 1; i <= rlevel; i++) {
        update[i] = lst->header;
    }
    lst->level = rlevel;
}
```

if $rlevel = 1 > lst \rightarrow level = 2$ is false $\left[\begin{array}{c} \text{Condition is} \\ \text{false} \end{array} \right]$, if block exits.

*SkipList * list*

Address list

ff33	ff08
------	------

*SkipList * lst*

Address lst

ff37	ff08
------	------

*Node * current*

Address current

ff41	NULL
------	-------------

Address level * header

ff08	2	ff12
------	---	------

Address key * forward

ff12	-1	ff80
------	----	------

Address Values

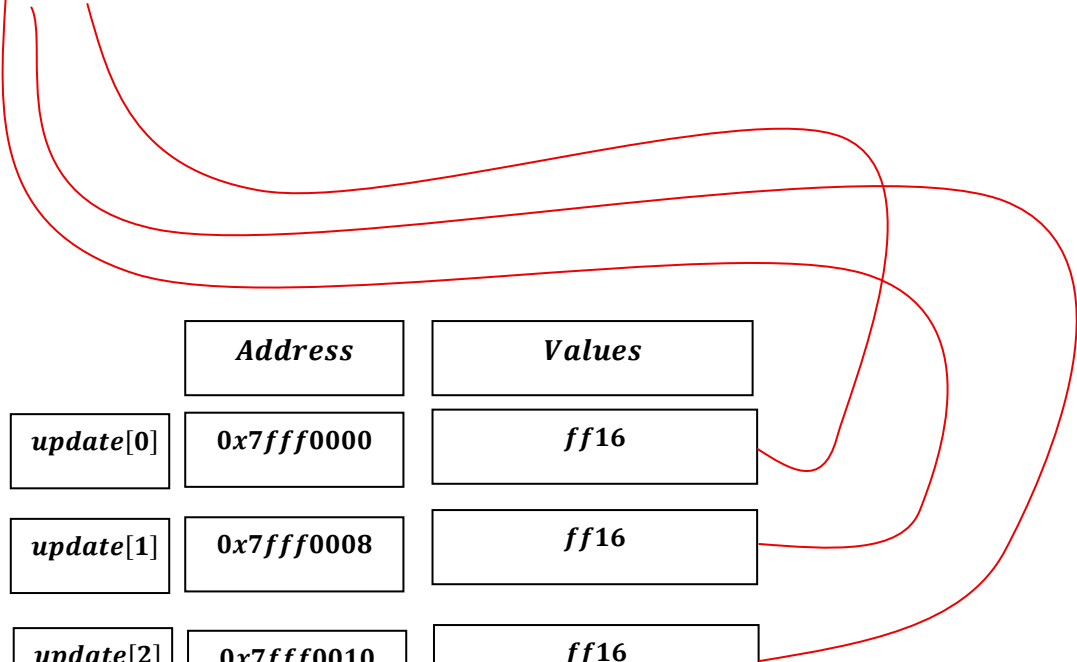
Address	Values	
forward[0]	ff80	ff16
forward[1]	ff88	ff16
forward[2]	ff90	ff16
forward[3]	ff98	NULL

Address key * forward

ff16	20	0xb400
------	----	--------

Address Values

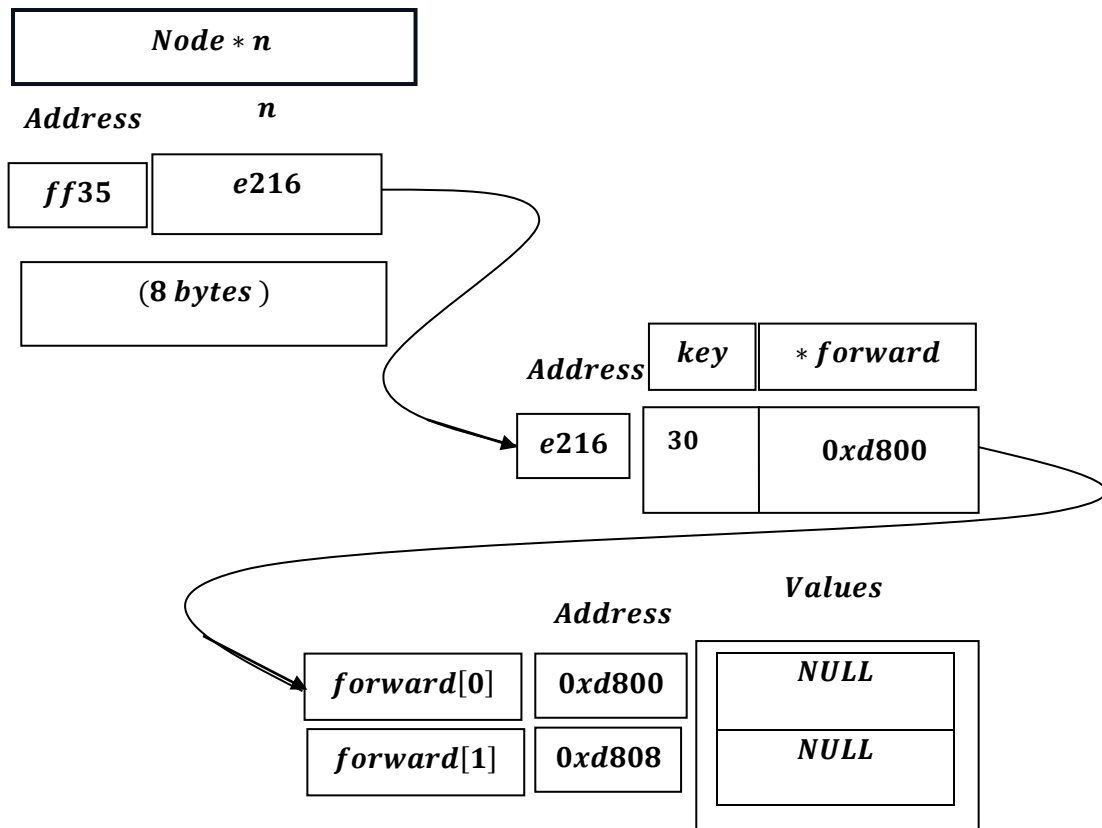
Address	Values	
forward[0]	0xb400	NULL
forward[1]	0xb408	NULL
forward[2]	0xb410	NULL



	<i>Address</i>	<i>Values</i>
<i>update[0]</i>	<i>0x7fff0000</i>	<i>ff16</i>
<i>update[1]</i>	<i>0x7fff0008</i>	<i>ff16</i>
<i>update[2]</i>	<i>0x7fff0010</i>	<i>ff16</i>
<i>update[3]</i>	<i>0x7fff0018</i>	<i>NULL</i>

```
struct Node *n = createNode(key, rlevel){...};
```

Let rlevel = 1 and key = 30.



```
return n;
```

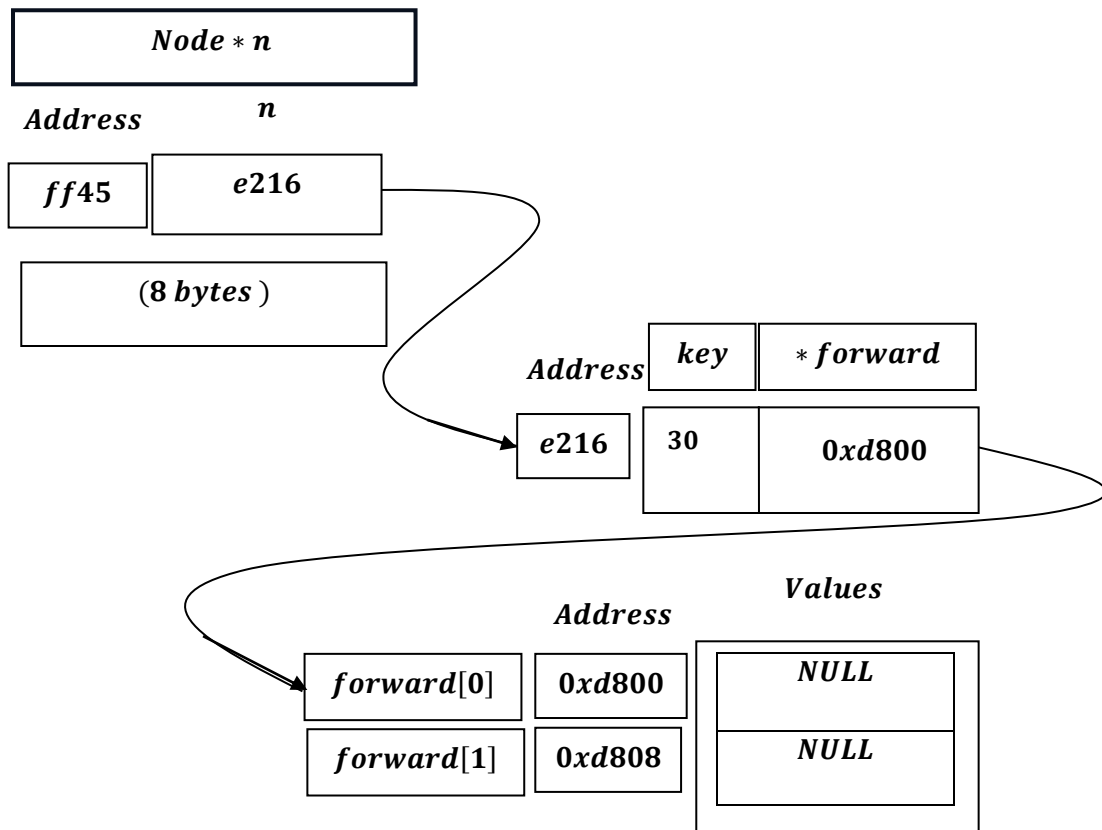
i. e. Return $n = e216$.

Actual structure

Address at which value is stored	Value	offset used
e216	key = 30	key[e216 – e219] 4 bytes for int
		padding → [e21a – e21d] 4 bytes for padding
e21e	Address: 0xd800	* forward → [e21e – e225] 8 bytes for pointer variable
0xd800	NULL	forward[0] → [0xd800 – 0xd807] 8 bytes for pointer variable
0xd808	NULL	forward[1] → [0xd808 – 0xd80f] 8 bytes for pointer variable

struct Node * n = createNode(key = 30,rlevel = 1)

struct Node * n = e216



```

for (int i = 0; i <= rlevel; i++) {
    n->forward[i] = update[i]->forward[i];
    update[i]->forward[i] = n;
}

```

int i = 0; i ≤ rlevel = 1

n → forward[0] = update[0] → forward[0] = NULL

update[0] → forward[0] = n = e216

i = i + 1 = 0 + 1 = 1

int i = 1; i ≤ rlevel = 1

n → forward[1] = update[1] → forward[1] = NULL

update[1] → forward[1] = n = e216

i = i + 1 = 1 + 1 = 2

int i = 2; i ≤ rlevel = 1 $\left[\begin{array}{l} \textit{Condition} \\ \textit{is False} \end{array} \right]$

*SkipList * list*

Address list

<i>ff33</i>	<i>ff08</i>
-------------	-------------

*SkipList * lst*

Address lst

<i>ff37</i>	<i>ff08</i>
-------------	-------------

*Node * current*

Address current

<i>ff41</i>	<i>NULL</i>
-------------	-------------

*Address level * header*

<i>ff08</i>	<i>2</i>	<i>ff12</i>
-------------	----------	-------------

*Address key * forward*

<i>ff12</i>	<i>-1</i>	<i>ff80</i>
-------------	-----------	-------------

Address Values

<i>forward</i>	<i>Address</i>	<i>Values</i>
<i>forward[0]</i>	<i>ff80</i>	<i>ff16</i>
<i>forward[1]</i>	<i>ff88</i>	<i>ff16</i>
<i>forward[2]</i>	<i>ff90</i>	<i>ff16</i>
<i>forward[3]</i>	<i>ff98</i>	<i>NULL</i>

*Address key * forward*

<i>ff16</i>	<i>20</i>	<i>0xb400</i>
-------------	-----------	---------------

Address Values

<i>forward</i>	<i>Address</i>	<i>Values</i>
<i>forward[0]</i>	<i>0xb400</i>	<i>e216</i>
<i>forward[1]</i>	<i>0xb408</i>	<i>e216</i>
<i>forward[2]</i>	<i>0xb410</i>	<i>NULL</i>

	<i>Address</i>	<i>Values</i>
<i>update[0]</i>	0x7fff0000	<i>ff16</i>
<i>update[1]</i>	0x7fff0008	<i>ff16</i>
<i>update[2]</i>	0x7fff0010	<i>ff16</i>
<i>update[3]</i>	0x7fff0018	<i>NULL</i>

*Node * n*

Address n

ff45 e216

(8 bytes)

<i>Address</i>	<i>key</i>	<i>* forward</i>
<i>e216</i>	30	0xd800

	<i>Address</i>	<i>Values</i>
<i>forward[0]</i>	0xd800	<i>NULL</i>
<i>forward[1]</i>	0xd808	<i>NULL</i>

```
cout << "Successfully inserted key " << key << endl;
```

Successfully inserted key : 30 and function ends .

As , the function ends :

*SkipList * lst*

Address	lst
ff37	ff08

*Node * current*

Address	current
ff41	NULL

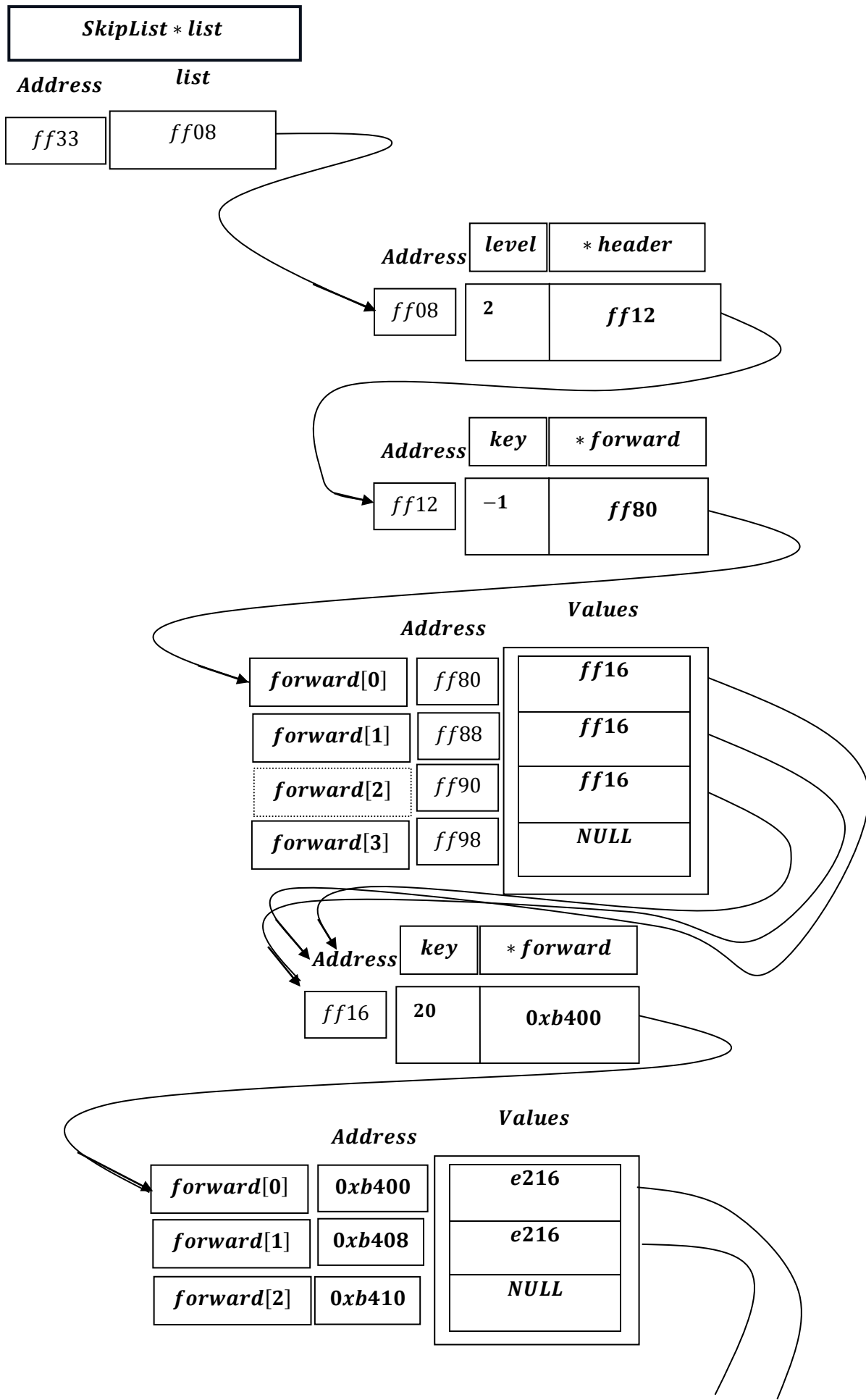
	Address	Values
update[0]	0x7fff0000	ff16
update[1]	0x7fff0008	ff16
update[2]	0x7fff0010	ff16
update[3]	0x7fff0018	NULL

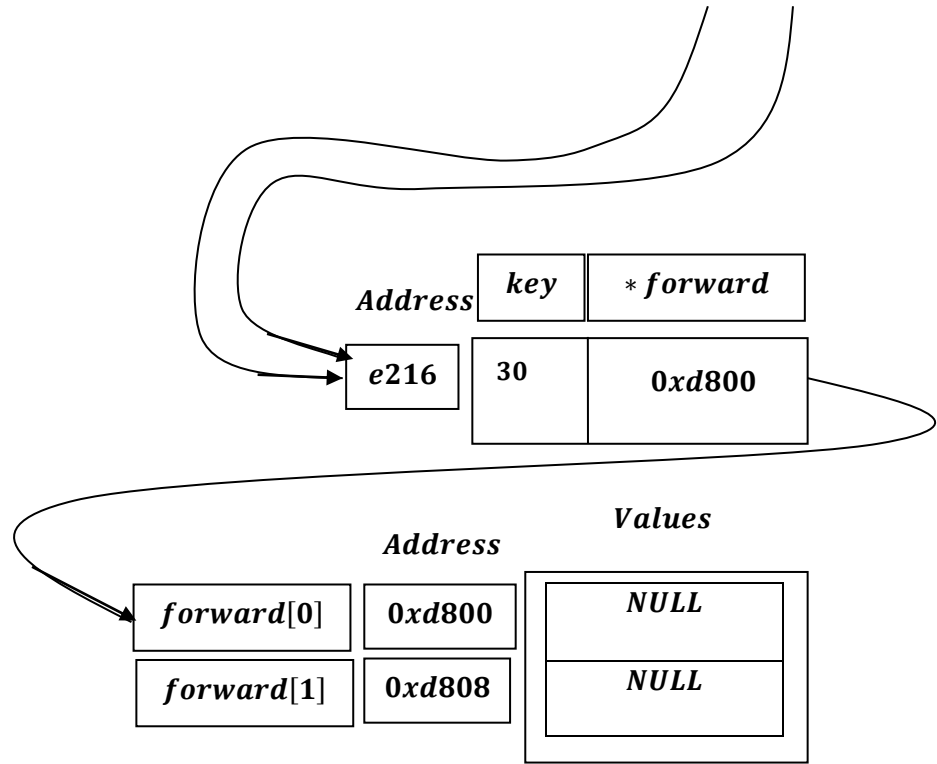
*Node * n*

Address	n
ff45	e216

*Gets
Destroyed*

And we are left with :





Skip List – Insert Element

–Program Analysis

–Insertion(3)[Insert In Between]

*SkipList * list*

Address *list*

ff33 *ff08*

Address *level* ** header*

ff08 2 *ff12*

Address *key* ** forward*

ff12 -1 *ff80*

Address *Values*

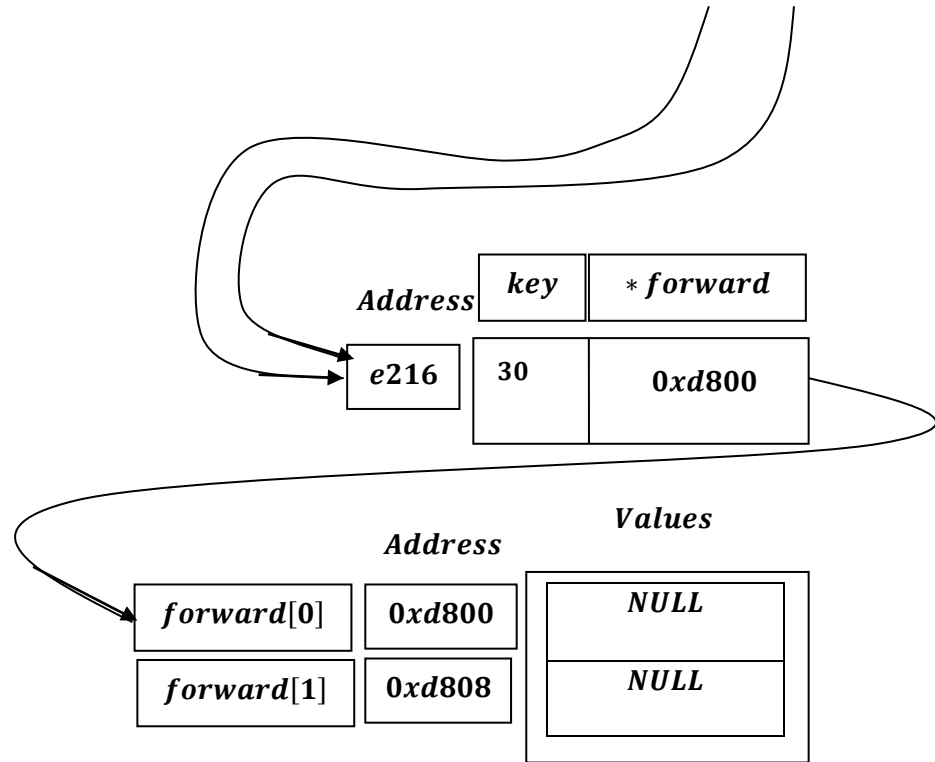
<i>forward</i>	<i>Address</i>	<i>Values</i>
<i>forward</i> [0]	<i>ff80</i>	<i>ff16</i>
<i>forward</i> [1]	<i>ff88</i>	<i>ff16</i>
<i>forward</i> [2]	<i>ff90</i>	<i>ff16</i>
<i>forward</i> [3]	<i>ff98</i>	<i>NULL</i>

Address *key* ** forward*

ff16 20 *0xb400*

Address *Values*

<i>forward</i>	<i>Address</i>	<i>Values</i>
<i>forward</i> [0]	<i>0xb400</i>	<i>e216</i>
<i>forward</i> [1]	<i>0xb408</i>	<i>e216</i>
<i>forward</i> [2]	<i>0xb410</i>	<i>NULL</i>



```
case 1:
    cout << "Enter key to insert: ";
    cin >> key;
    insertElement(list, key);
    break;
```

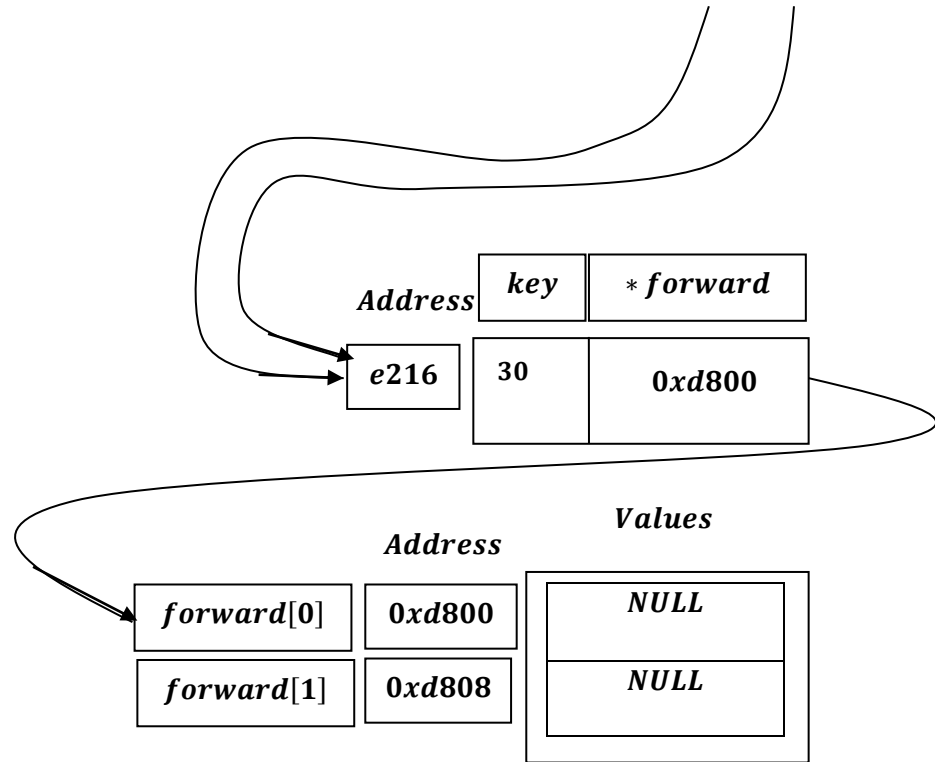
Entering : Key = 25 .

insertElement(list: ff08 ,key: 25);

Now ,we have :

```
void insertElement(struct SkipList *lst, int key)
```

*list = struct SkipList * lst = ff08 and int key: 25.*



```
struct Node *current = lst->header;
```

current = lst → header : ff12.

```
struct Node *update[MAX_LEVEL + 1];
```

update[3 + 1 = 4],

MAX_LEVEL evaluates to 3. Consequently, the array size calculation is update[3 + 1 = 4].

This means the array is allocated to hold exactly 4 elements.

Before memset call, the update array may look like :

<i>Array Element</i>	<i>Hypothetical Stack Address</i>	<i>Value (Garbage Data)</i>	<i>Pointer Status</i>
<i>update[0]</i>	<i>0x7fff0000 to 0x7fff0007</i>	<i>0x4a3f8921bce45099</i>	<i>Wild Pointer (Points to a random, invalid address)</i>
<i>update[1]</i>	<i>0x7fff0008 to 0x7fff000f</i>	<i>0x0000000000000002</i>	<i>Wild Pointer</i>
<i>update[2]</i>	<i>0x7fff0010 to 0x7fff0017</i>	<i>0x7ffee1245a00881c</i>	<i>Wild Pointer</i>
<i>update[3]</i>	<i>0x7fff0018 to 0x7fff001f</i>	<i>0x0000000000000000</i>	<i>Accidentally NULL (Just by chance)</i>

More simpler way:

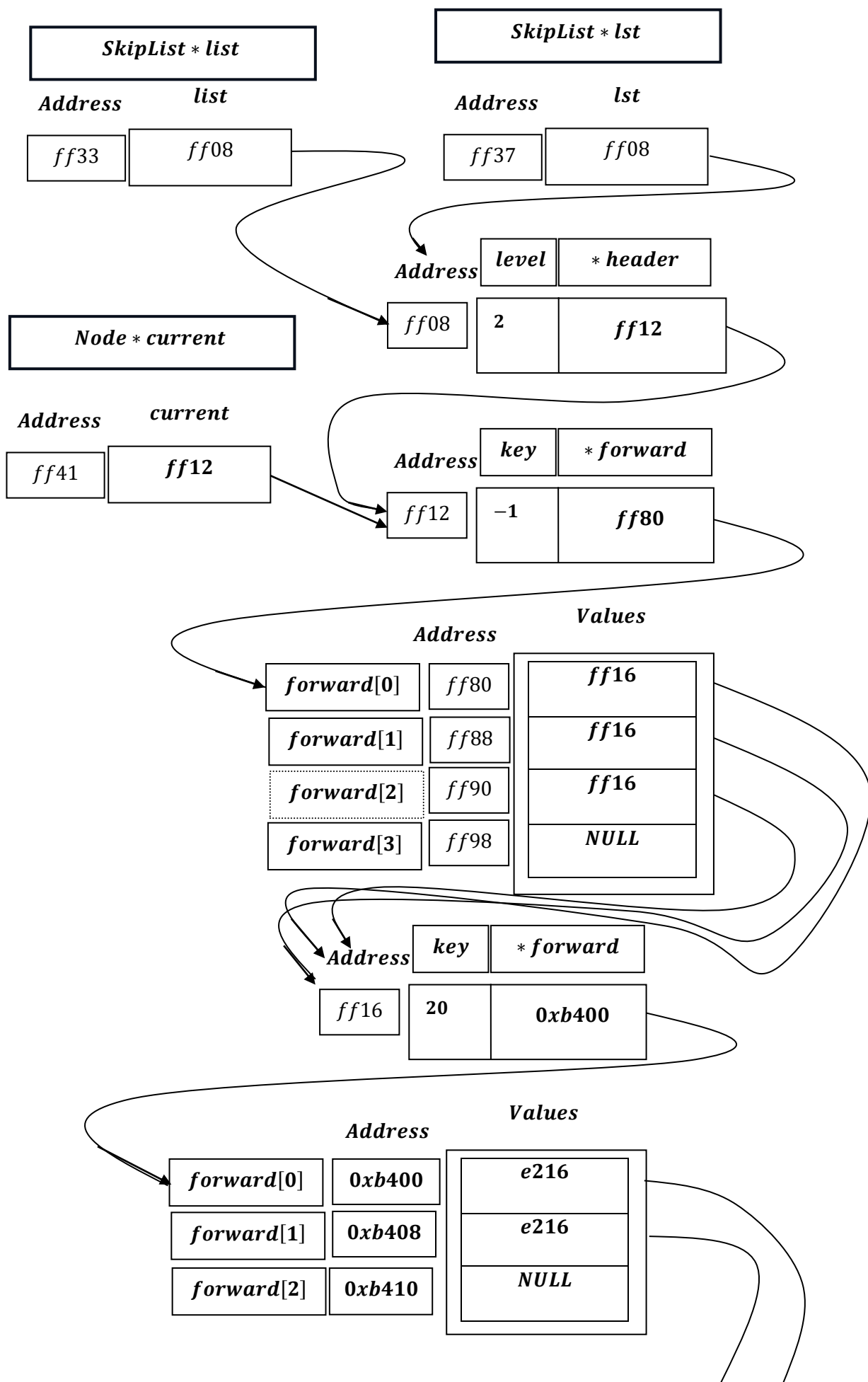
	<i>Address</i>	<i>Values</i>	
<i>update[0]</i>	<i>0x7fff0000</i>	<i>0x4a3f8921bce45099</i>	<i>[8 bytes]</i>
<i>update[1]</i>	<i>0x7fff0008</i>	<i>0x0000000000000002</i>	<i>[8 bytes]</i>
<i>update[2]</i>	<i>0x7fff0010</i>	<i>0x7ffee1245a00881c</i>	<i>[8 bytes]</i>
<i>update[3]</i>	<i>0x7fff0018</i>	<i>0x0000000000000000</i>	<i>[8 bytes]</i>

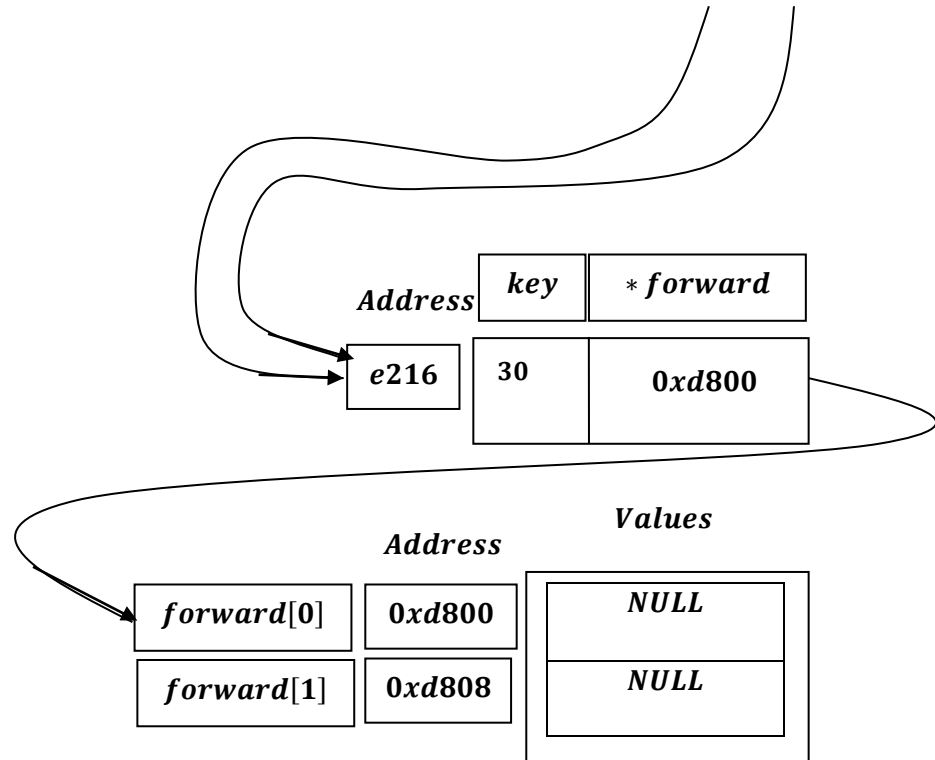
```
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

``memset`` stands for Memory Set. It takes a block of memory and rapidly fills it with a specific value. Here `0` or NULL.

	<i>Address</i>	<i>Values</i>
<i>update[0]</i>	<i>0x7fff0000</i>	<i>NULL</i>
<i>update[1]</i>	<i>0x7fff0008</i>	<i>NULL</i>
<i>update[2]</i>	<i>0x7fff0010</i>	<i>NULL</i>
<i>update[3]</i>	<i>0x7fff0018</i>	<i>NULL</i>

*Here we kept addresses same like: 0x7fff0000
, just for an example , but it may not be same ,
in reality.*





	Address	Values
<i>update</i> [0]	0x7fff0000	NULL
<i>update</i> [1]	0x7fff0008	NULL
<i>update</i> [2]	0x7fff0010	NULL
<i>update</i> [3]	0x7fff0018	NULL

```

for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}

```

For : $i = lst \rightarrow level = 2 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

while: $current \rightarrow forward[2] = ff16 \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$ And

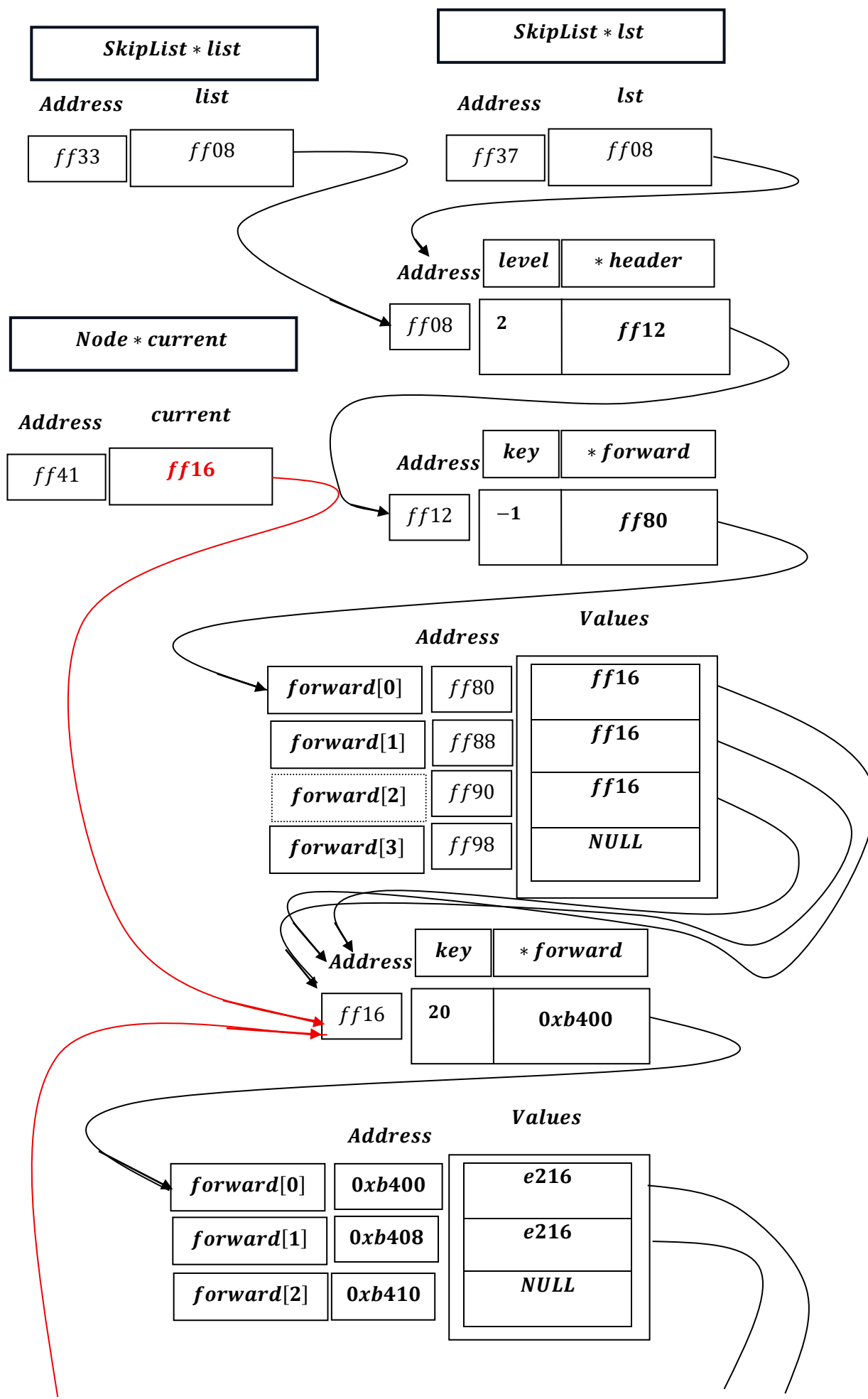
$current \rightarrow forward[2] \rightarrow key: 20 < key: 25$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

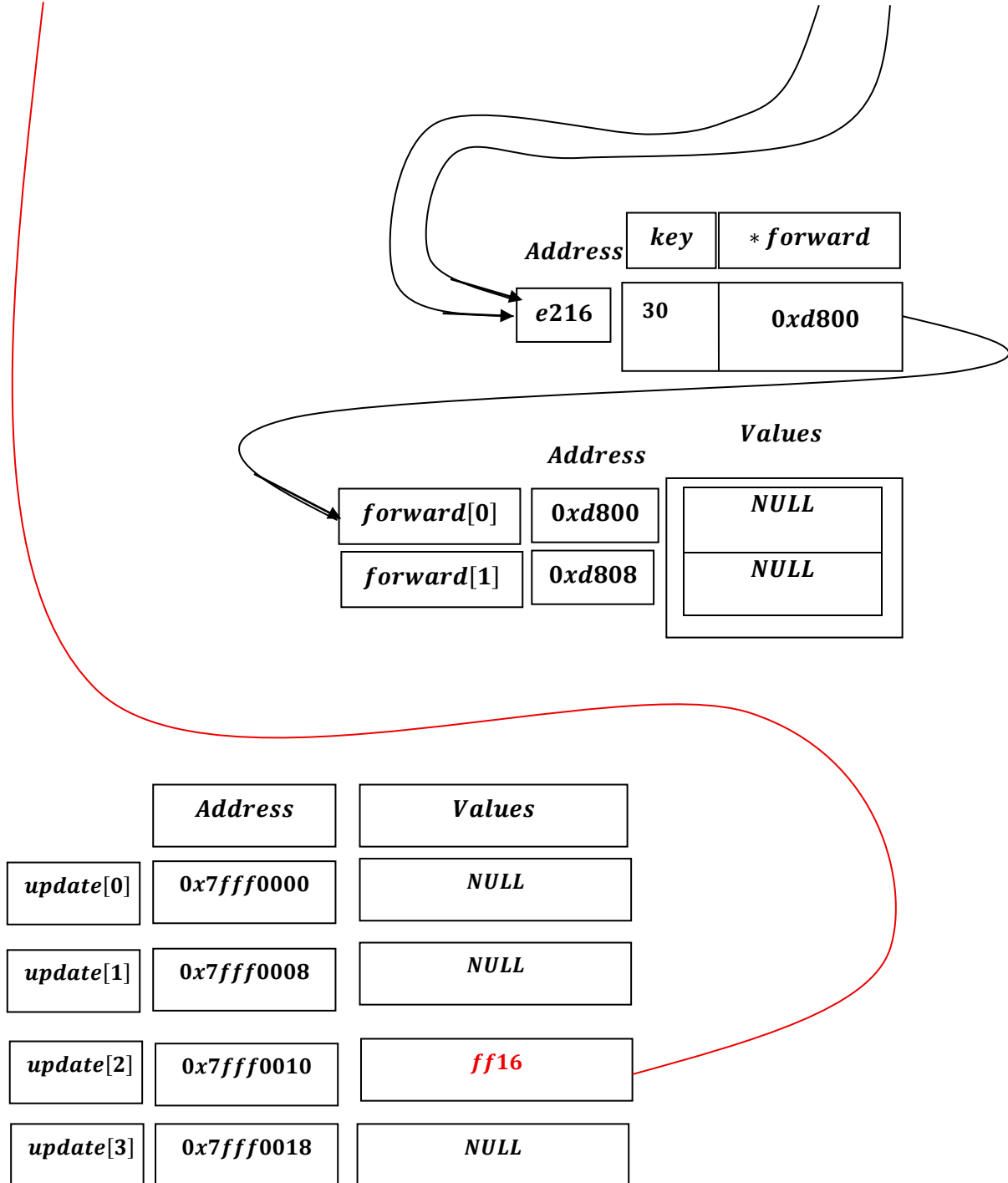
$current = current \rightarrow forward[2] = ff16$

while: $current \rightarrow forward[2] = NULL \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False, while} \\ \text{loop exits} \end{array} \right]$

$update[2] = current = ff16.$

$i = i - 1 = 2 - 1 = 1.$





For : $i = 1 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

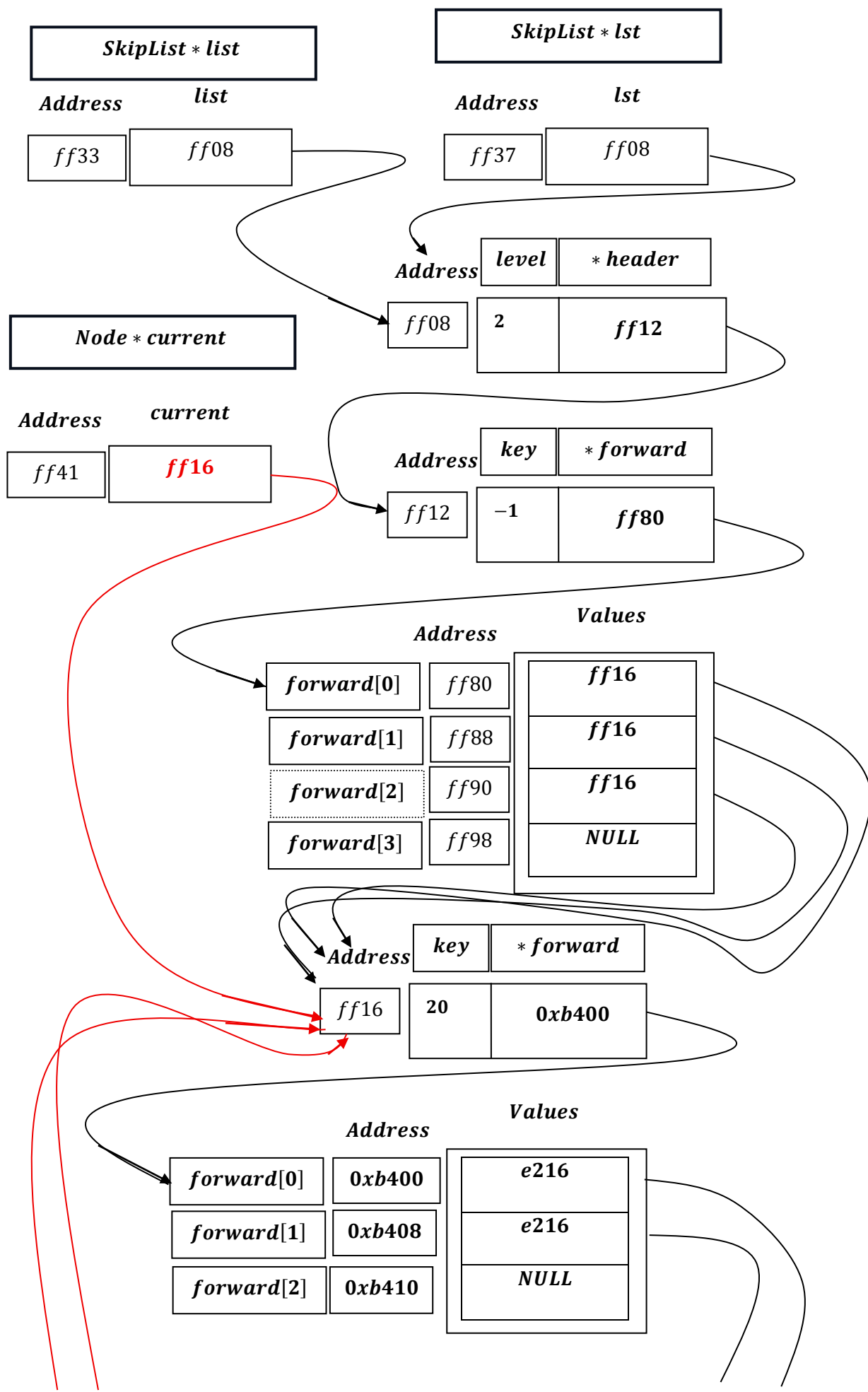
while: $current \rightarrow forward[1] = e216 \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is true} \end{array} \right]$ And

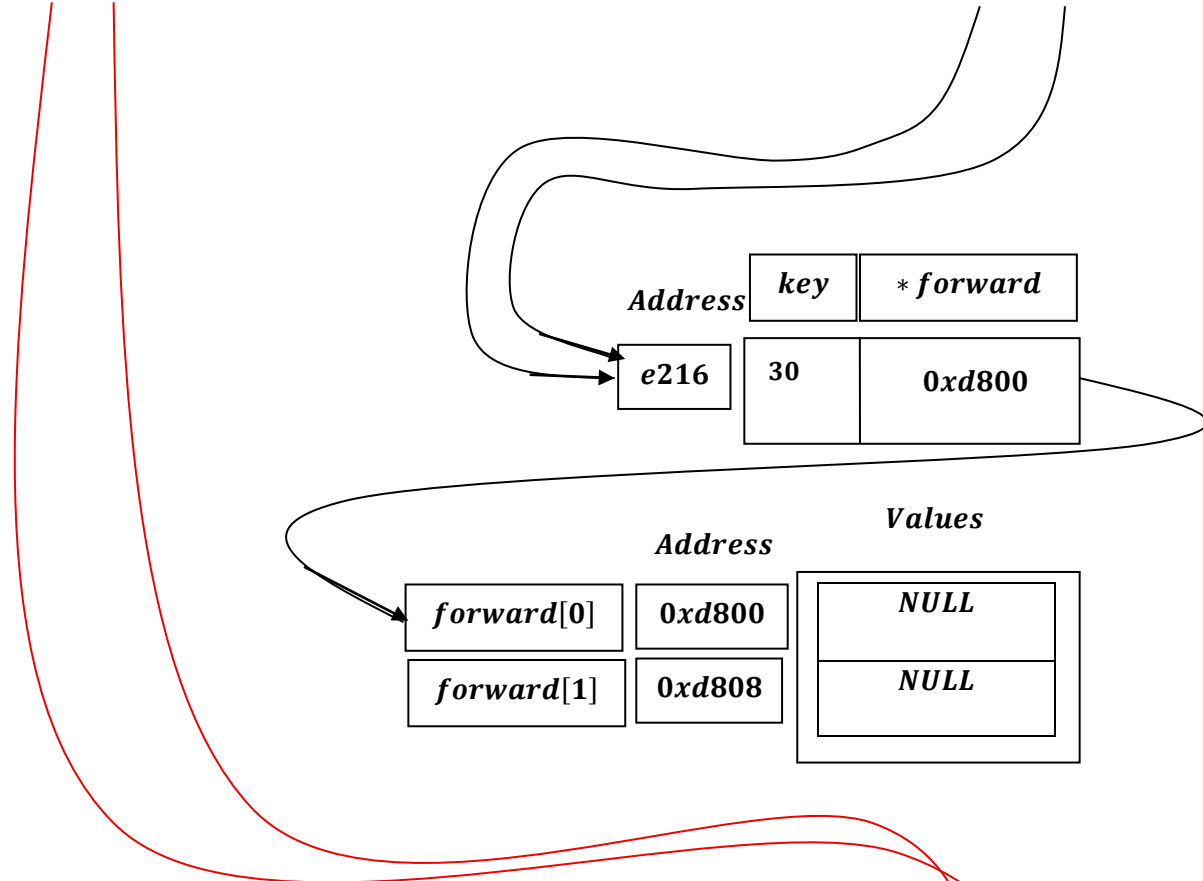
$current \rightarrow forward[1] \rightarrow key: 30 < key: 25$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$

[hence, while loop exits]

$update[1] = current = ff16.$

$i = i - 1 = 1 - 1 = 0.$





	Address	Values
update[0]	0x7fff0000	NULL
update[1]	0x7fff0008	ff16
update[2]	0x7fff0010	ff16
update[3]	0x7fff0018	NULL

For : $i = 0 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

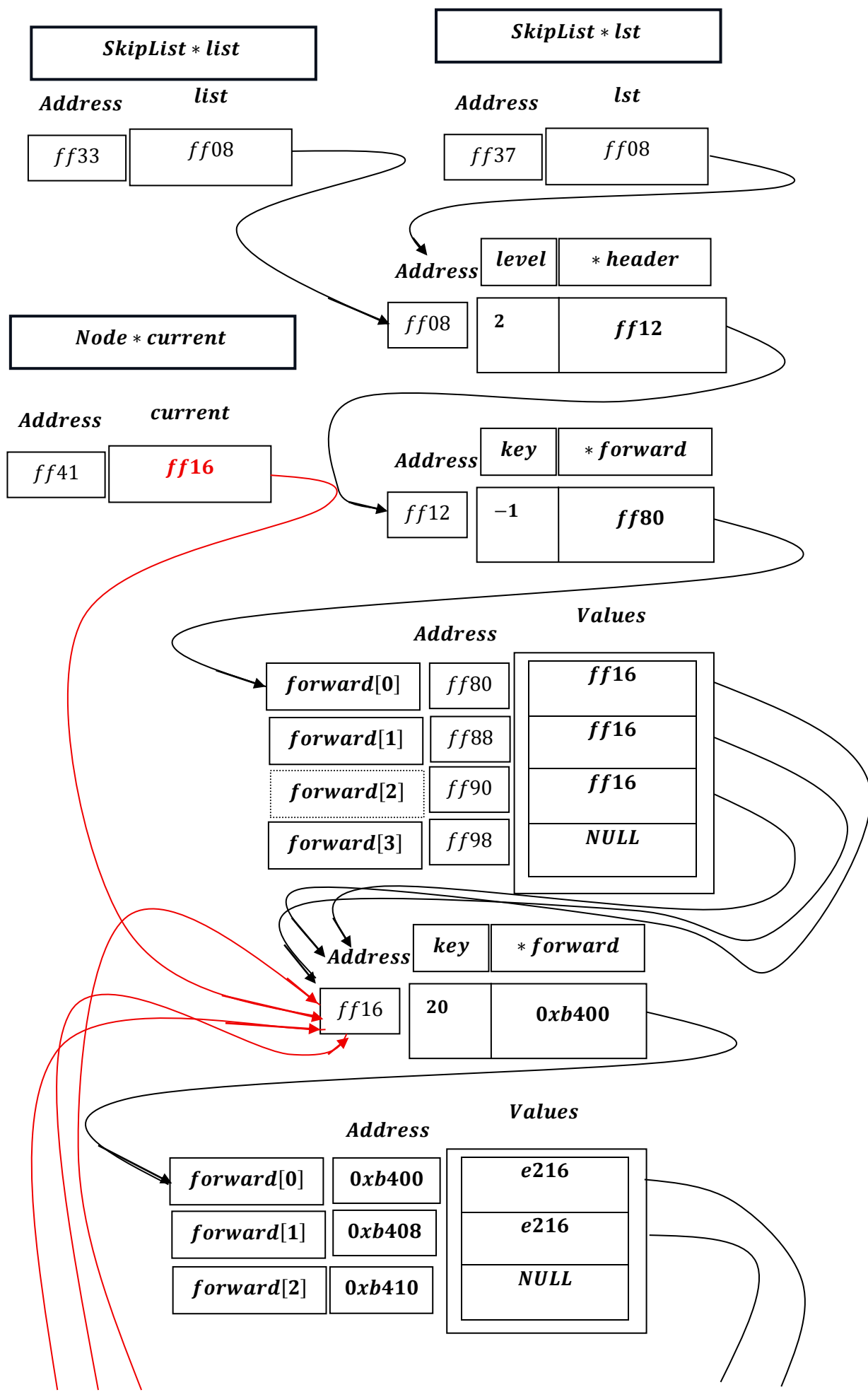
while: $\text{current} \rightarrow \text{forward}[0] = \text{e216} \neq \text{NULL}$ $\left[\begin{array}{l} \text{Condition} \\ \text{is true} \end{array} \right]$ And

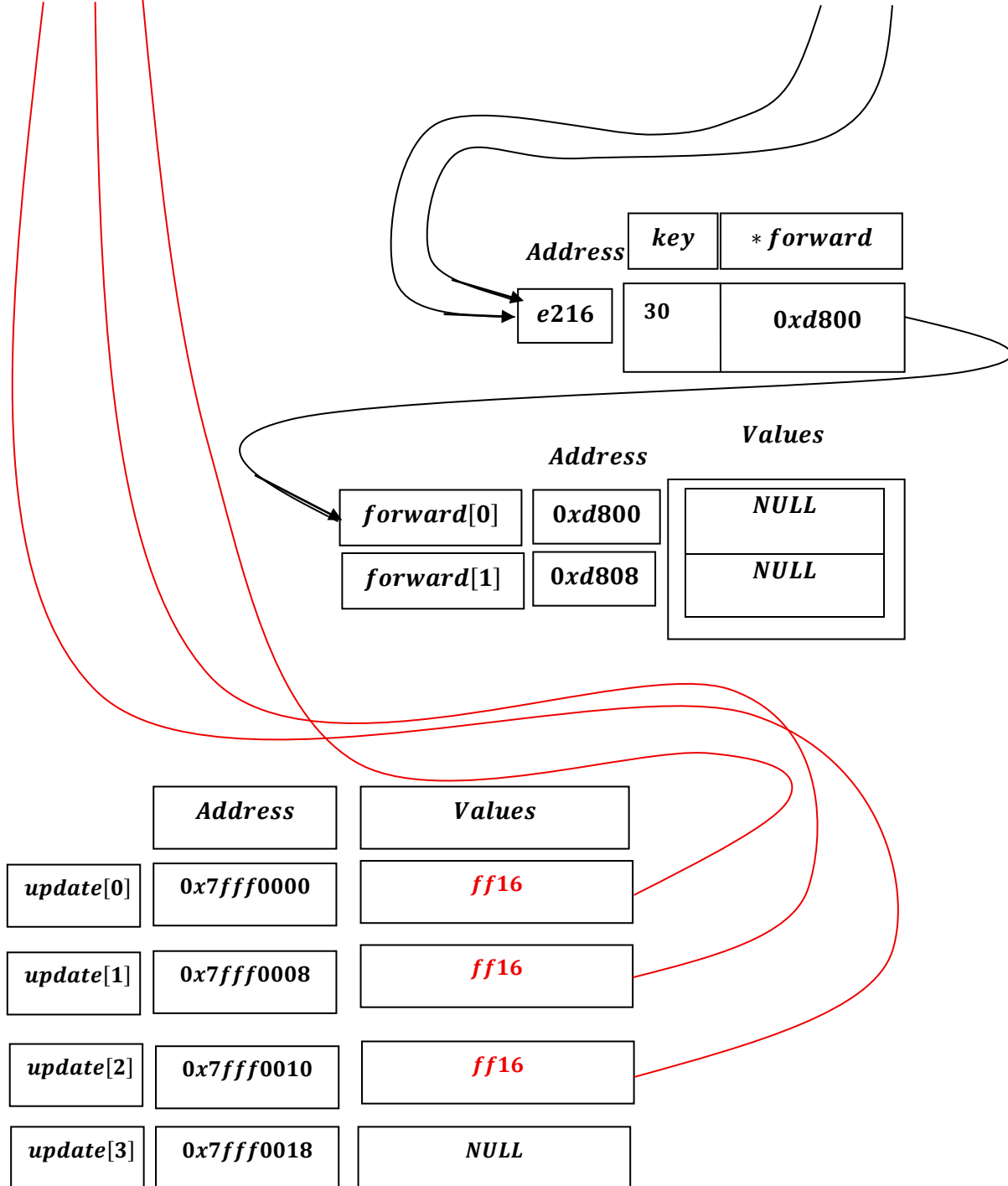
$\text{current} \rightarrow \text{forward}[0] \rightarrow \text{key: } 30 < \text{key: } 25$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$

[hence, while loop exits]

$\text{update}[0] = \text{current} = \text{ff16}.$

$i = i - 1 = 0 - 1 = -1.$

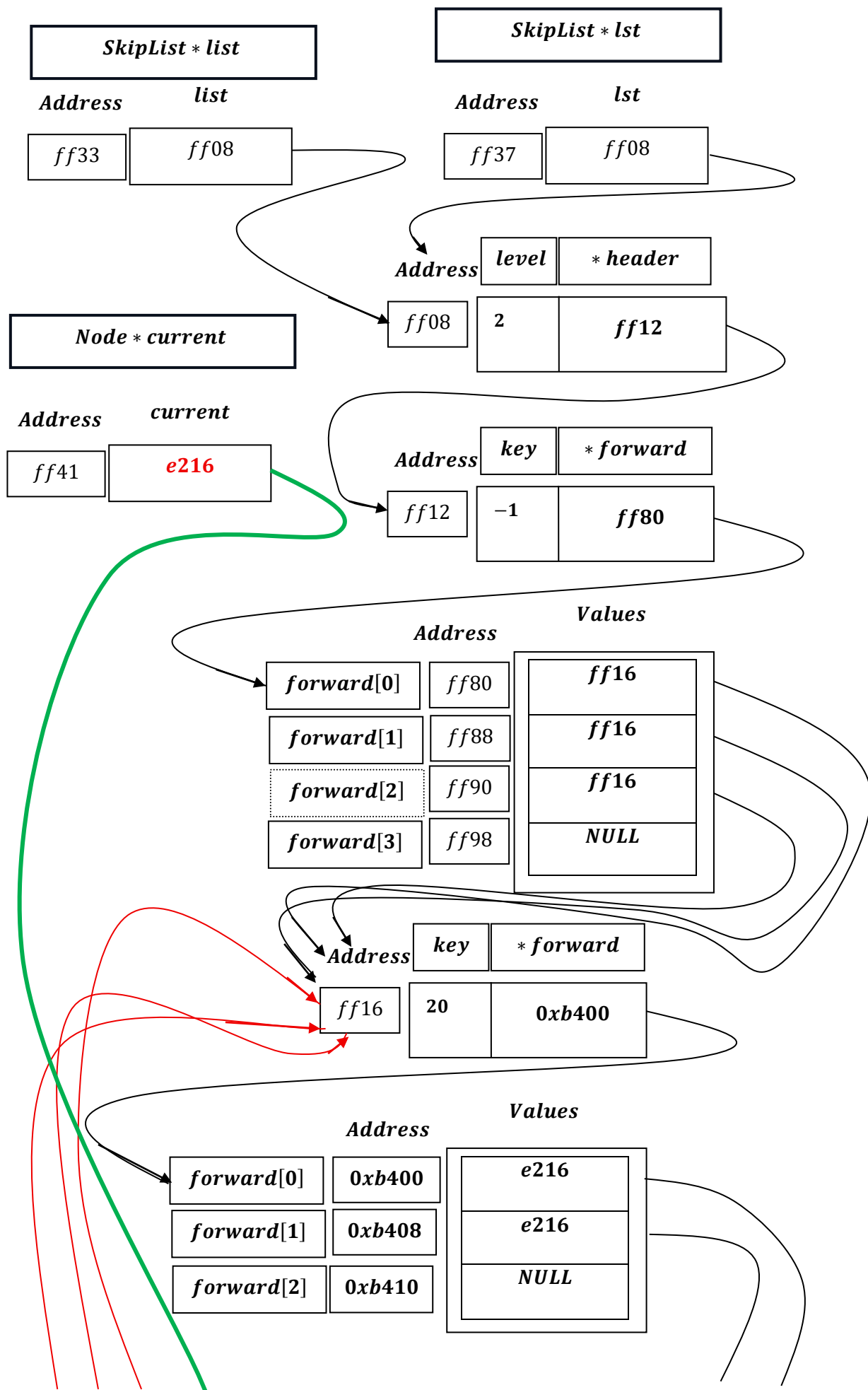


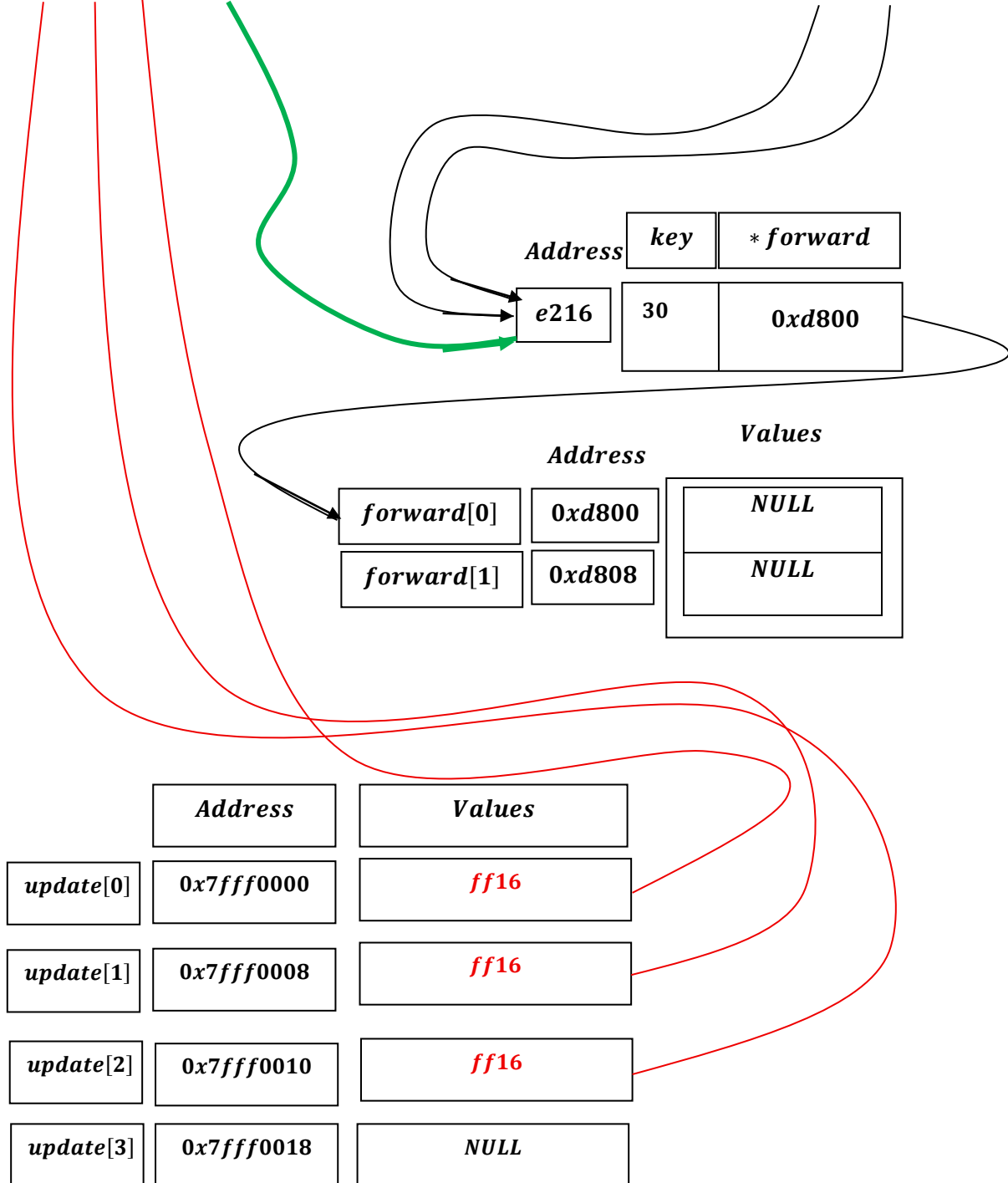


For : $i = -1 ; i \geq 0$ $\left[\begin{matrix} \text{Condition} \\ \text{is False} \end{matrix} \right]$, condition is false and loop exits.

```
current = current->forward[0];
```

current = current → forward[0] = e216.





```
if (current == NULL || current->key != key) {...}
```

current = *e216* $\neq$ *NULL* (*false*) OR *current* $\rightarrow$ *key* = *30* $\neq$ *key* = *25* (*True*),
 hence if block executes.

```
int rlevel = randomLevel();
```

rlevel = *randomlevel*() = *1* (say)

```

if (rlevel > lst->level) {
    for (int i = lst->level + 1; i <= rlevel; i++) {
        update[i] = lst->header;
    }
    lst->level = rlevel;
}

```

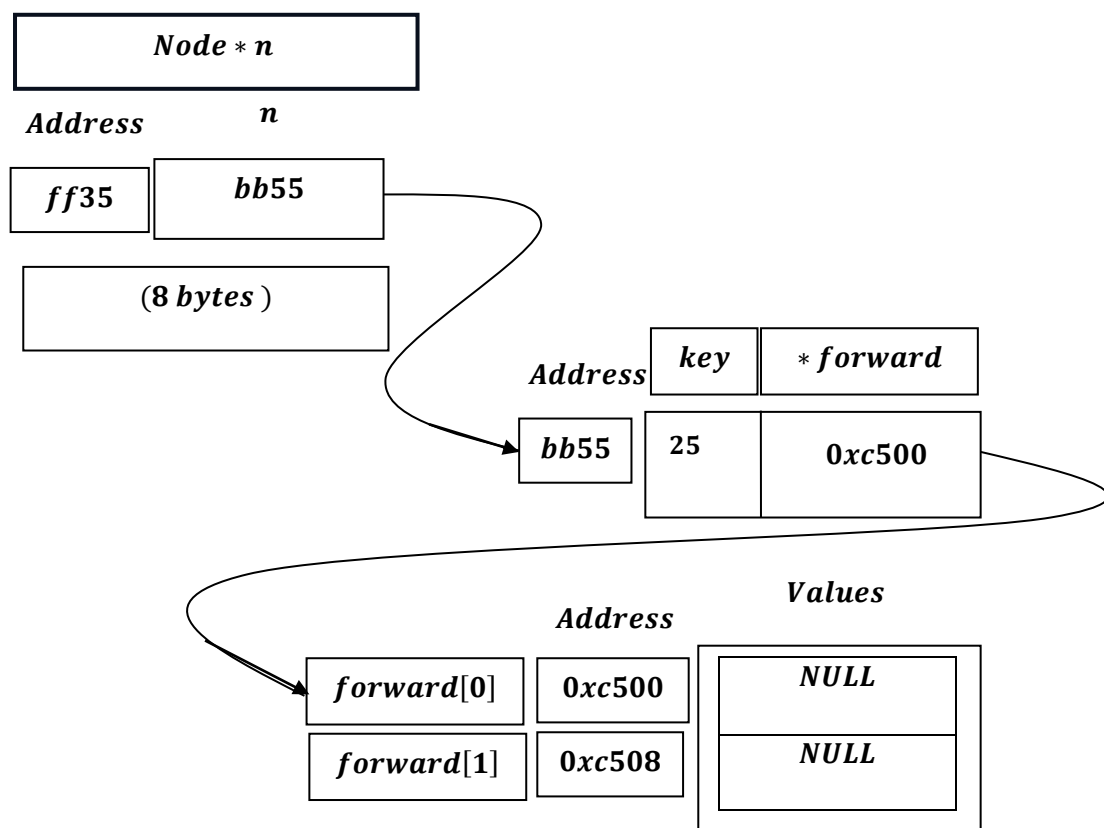
if $rlevel = 1 > lst \rightarrow level = 2$ is false $\left[\begin{array}{c} \text{Condition is} \\ \text{false} \end{array} \right]$, if block exits.

```

struct Node *n = createNode(key, rlevel){...};

```

Let $rlevel = 1$ and $key = 25$.



```

return n;

```

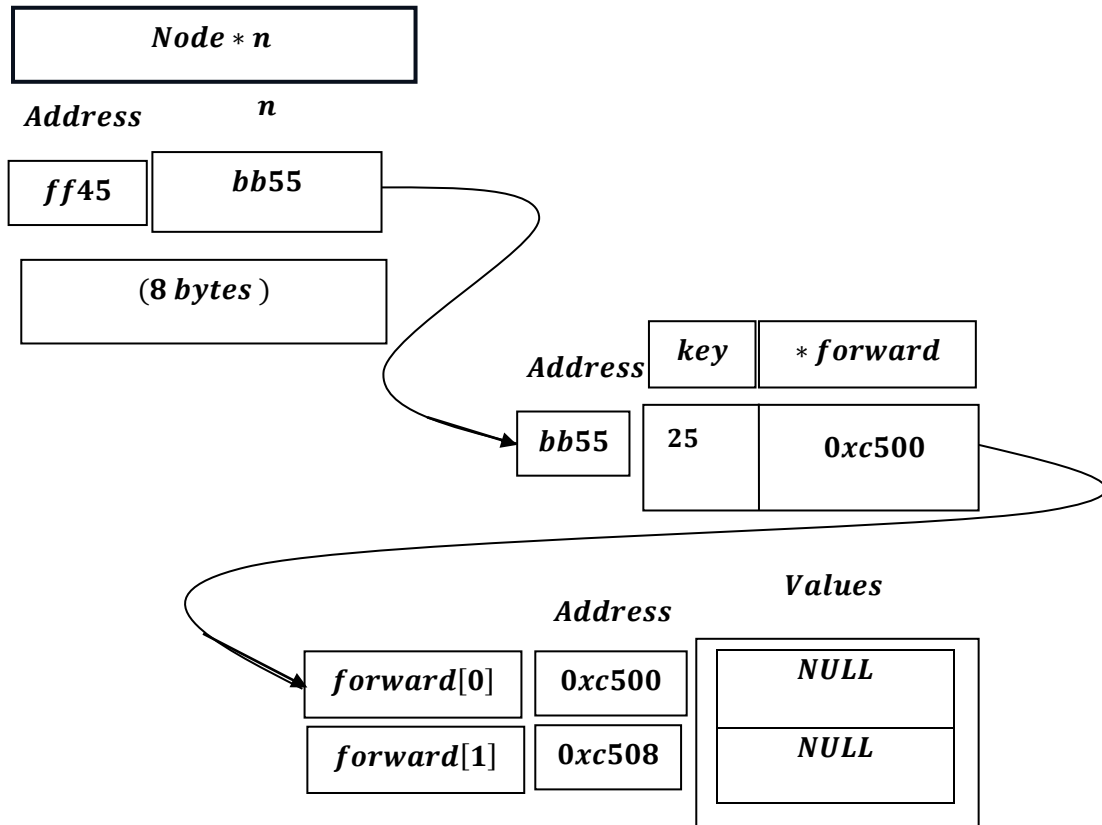
i. e. Return $n = bb55$

Actual structure

<i>Address at which value is stored</i>	<i>Value</i>	<i>offset used</i>
<i>bb55</i>	<i>key = 25</i>	<i>key[bb55 – bb58] 4 bytes for int</i>
		<i>padding → [bb59 – bb5c] 4 bytes for padding</i>
<i>bb5d</i>	<i>Address: 0xc500</i>	<i>* forward → [bb5d – bb64] 8 bytes for pointer variable</i>
<i>0xc500</i>	<i>NULL</i>	<i>forward[0] → [0xc500 – 0xc507] 8 bytes for pointer variable</i>
<i>0xc508</i>	<i>NULL</i>	<i>forward[1] → [0xc508 – 0xc50f] 8 bytes for pointer variable</i>

struct Node * n = createNode(key = 25,rlevel = 1)

struct Node * n = bb55



```
for (int i = 0; i <= rlevel; i++) {
    n->forward[i] = update[i]->forward[i];
    update[i]->forward[i] = n;
}
```

int i = 0; i ≤ rlevel = 1

n → forward[0] = update[0] → forward[0] = e216

update[0] → forward[0] = n = bb55

i = i + 1 = 0 + 1 = 1

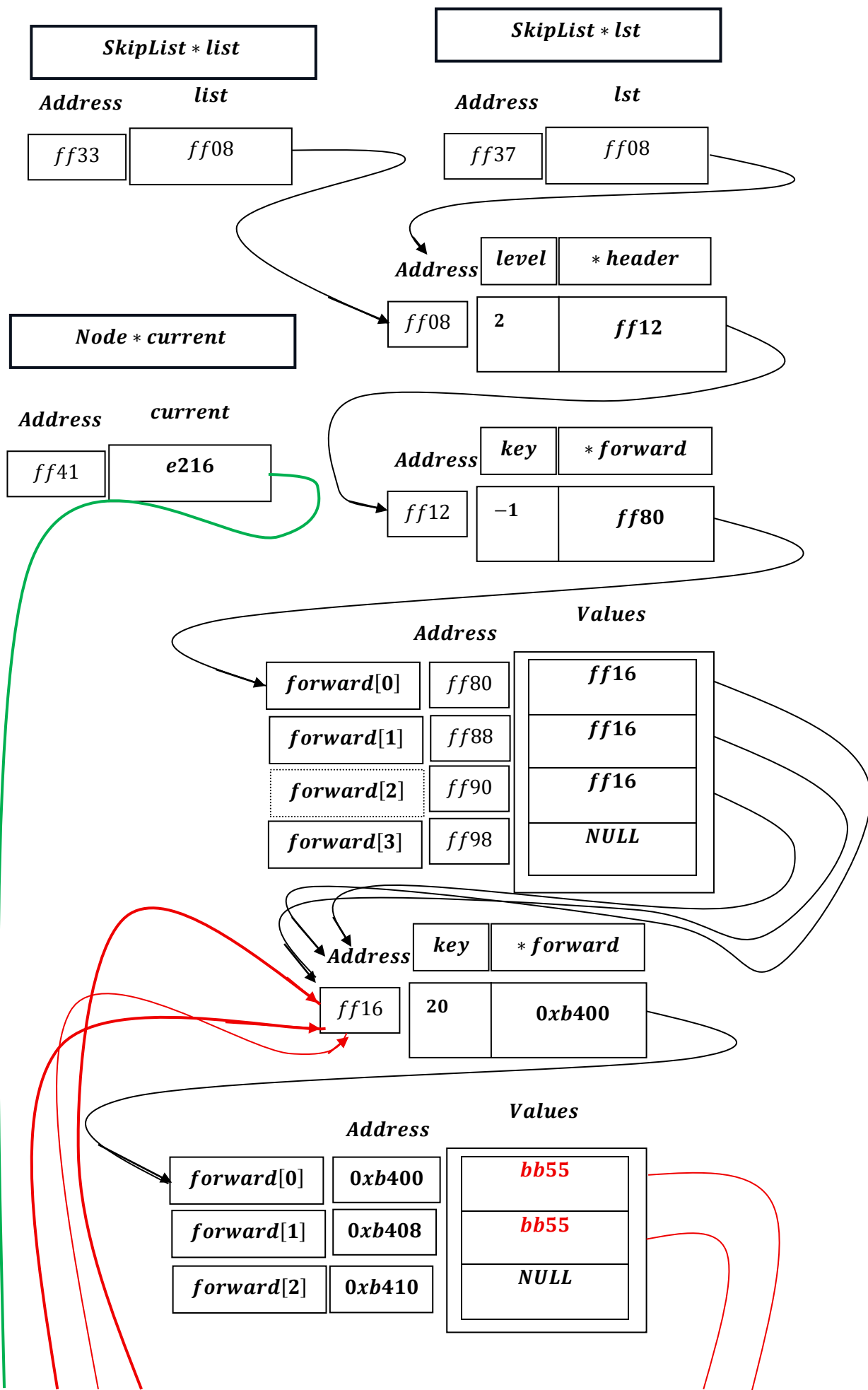
int i = 1; i ≤ rlevel = 1

n → forward[1] = update[1] → forward[1] = e216

update[1] → forward[1] = n = bb55

i = i + 1 = 1 + 1 = 2

int i = 2; i ≤ rlevel = 1 $\left[\begin{array}{l} \textit{Condition} \\ \textit{is False} \end{array} \right]$



*Node * n*

Address

n

ff45

bb55

Address

key

** forward*

(8 bytes)

bb55

25

0xc500

Address

Values

forward[0]

0xc500

e216

forward[1]

0xc508

e216

Address

key

** forward*

e216

30

0xd800

Address

Values

forward[0]

0xd800

NULL

forward[1]

0xd808

NULL

Address

Values

update[0]

0x7ff0000

ff16

update[1]

0x7ff0008

ff16

update[2]

0x7ff0010

ff16

update[3]

0x7ff0018

NULL

```
cout << "Successfully inserted key " << key << endl;
```

Successfully inserted key : 25 and function ends .

As , the function ends :

*SkipList * lst*

Address	lst
ff37	ff08

*Node * current*

Address	current
ff41	e216

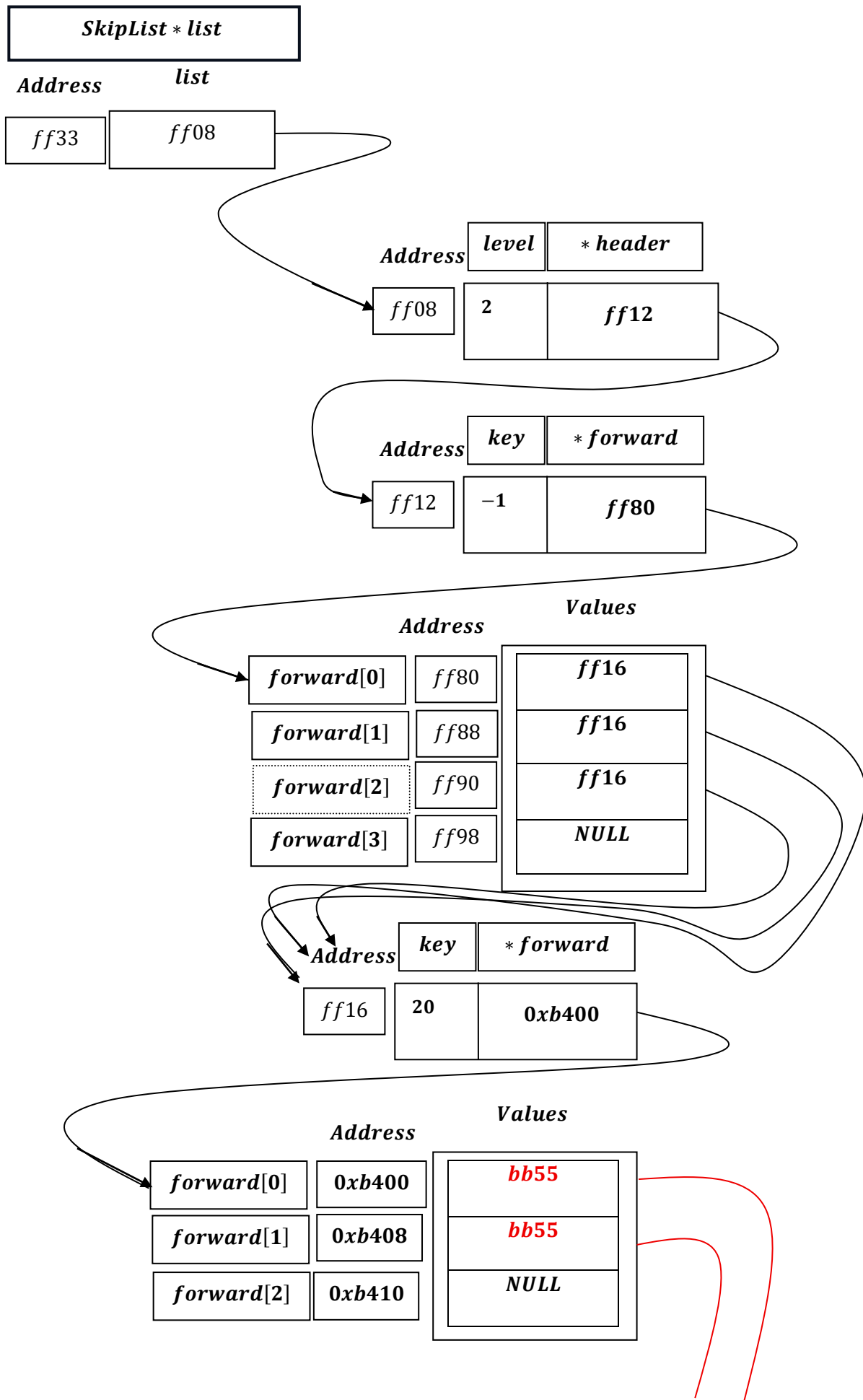
	Address	Values
update[0]	0x7fff0000	ff16
update[1]	0x7fff0008	ff16
update[2]	0x7fff0010	ff16
update[3]	0x7fff0018	NULL

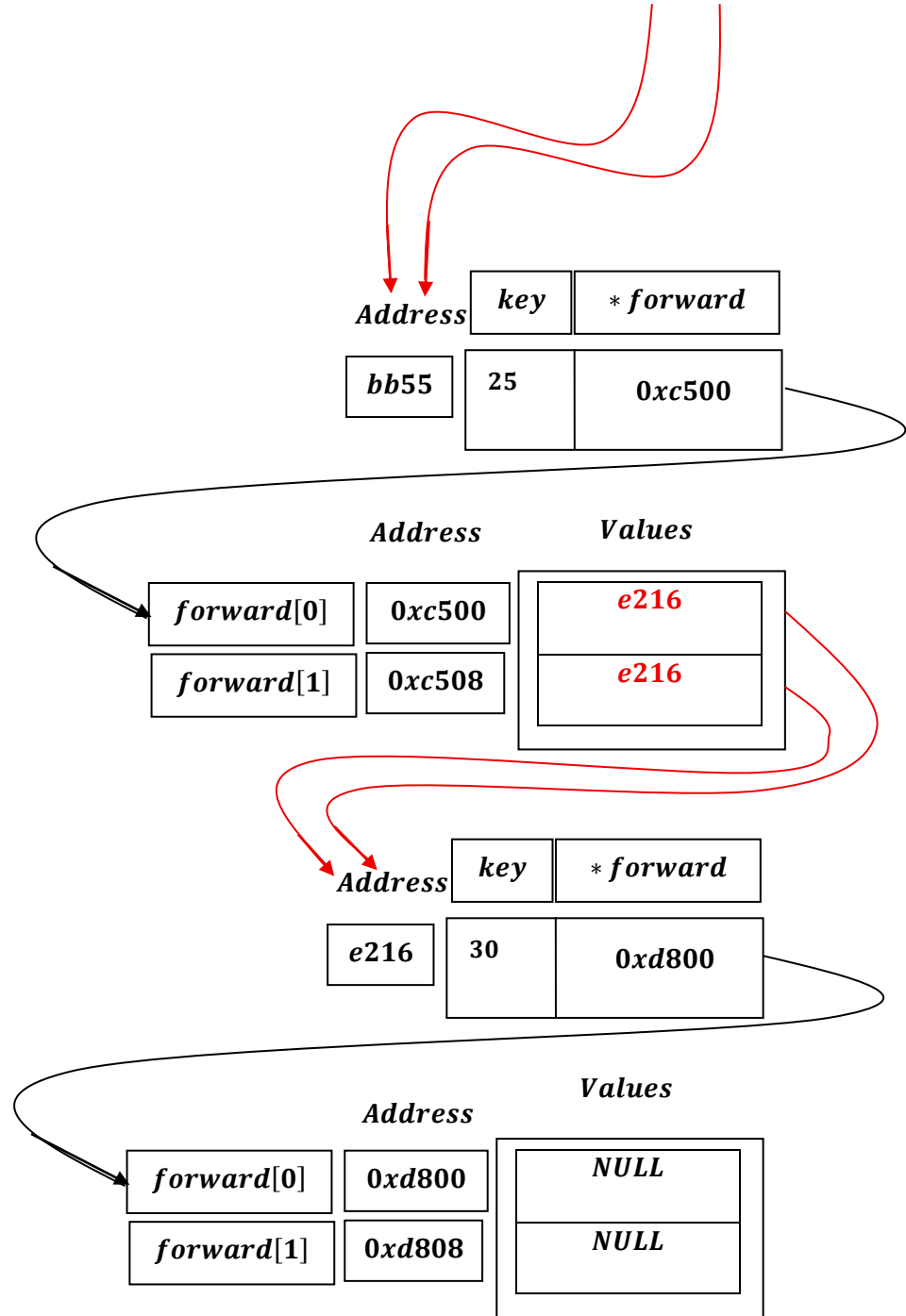
*Node * n*

Address	n
ff45	bb55

*Gets
Destroyed*

And we are left with:





But ,what happens,when ,we increase rand() value i.e. 3 > lst → level: 2, then what happens? Lets discuss this on next

Skip List – Insert Element

–Program Analysis

–Insertion(4) [rlevel>level]

But ,what happens,when ,we increase rand()value i.e.3 > lst → level: 2,then what happens? Let begin this from previous document observation(from middle):

```
if (current == NULL || current->key != key) {...}
```

*current = e216 ≠ NULL (false)OR **current → key = 30 ≠ key = 25(True)**,
hence if block executes.*

```
int rlevel = randomLevel();
```

rlevel = randomlevel() = 3 (say)

```
if (rlevel > lst->level) {  
    for (int i = lst->level + 1; i <= rlevel; i++) {  
        update[i] = lst->header;  
    }  
    lst->level = rlevel;  
}
```

*if **rlevel = 3 > lst → level = 2 is True** [Condition is True], if block executes:*

for int i = lst → level + 1 = 2 + 1 = 3; i ≤ rlevel = 3:

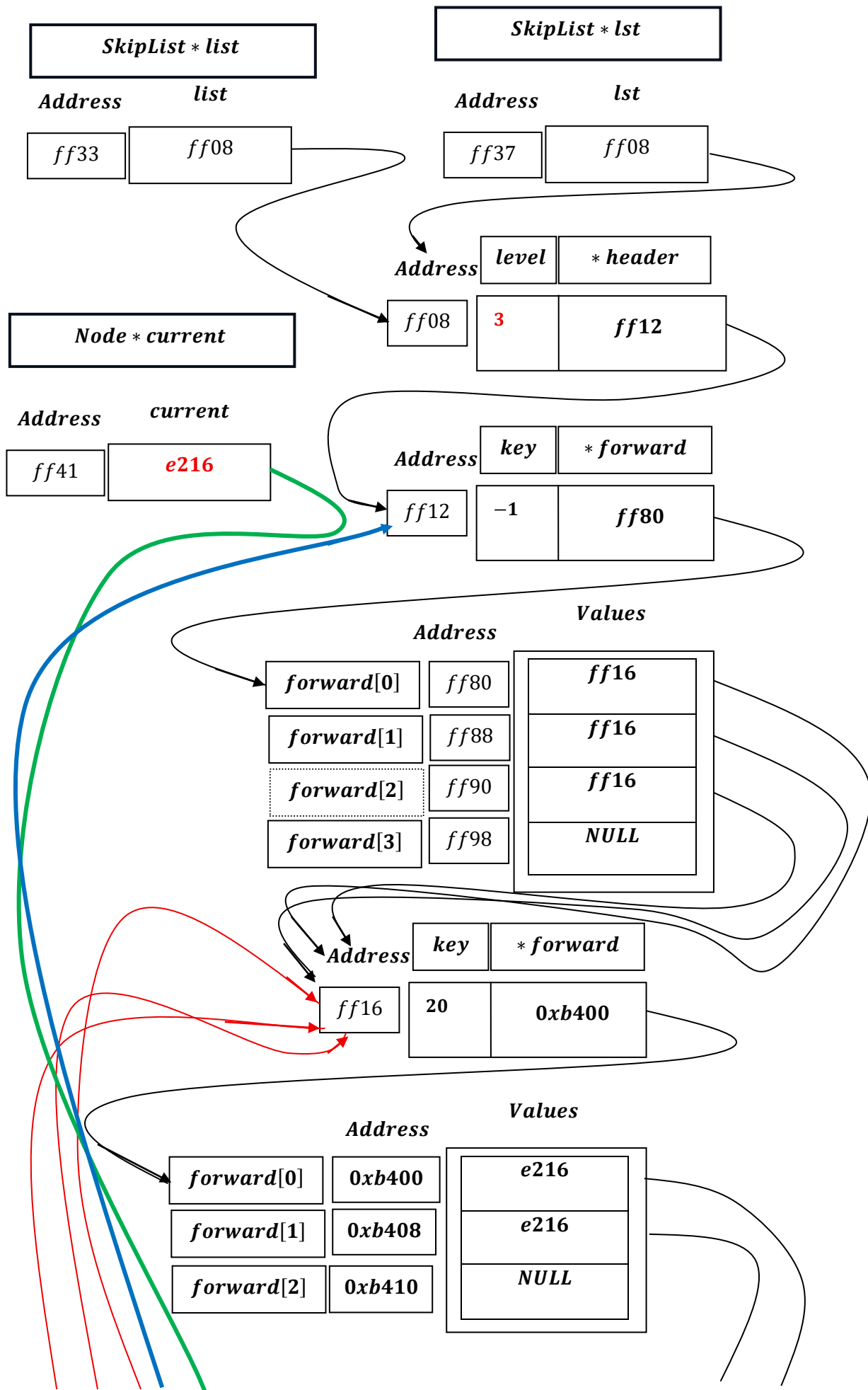
update[3] = lst → header = ff12

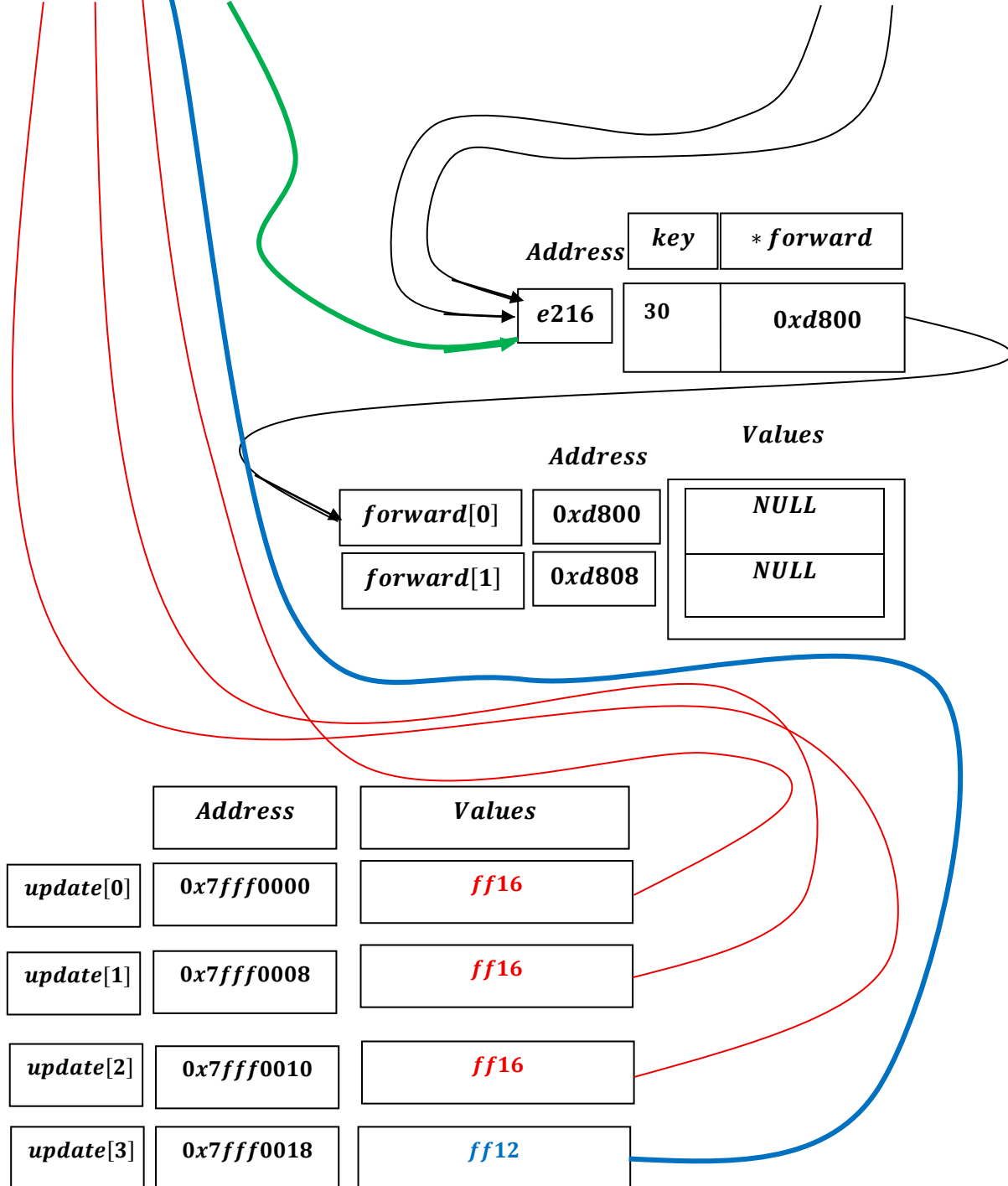
i = i + 1 = 3 + 1 = 4

for int i = 4; i ≤ rlevel = 3 $\left[\begin{array}{c} \text{Condition is} \\ \text{false} \end{array} \right]$ loop exists.

lst → level = rlevel = 3.

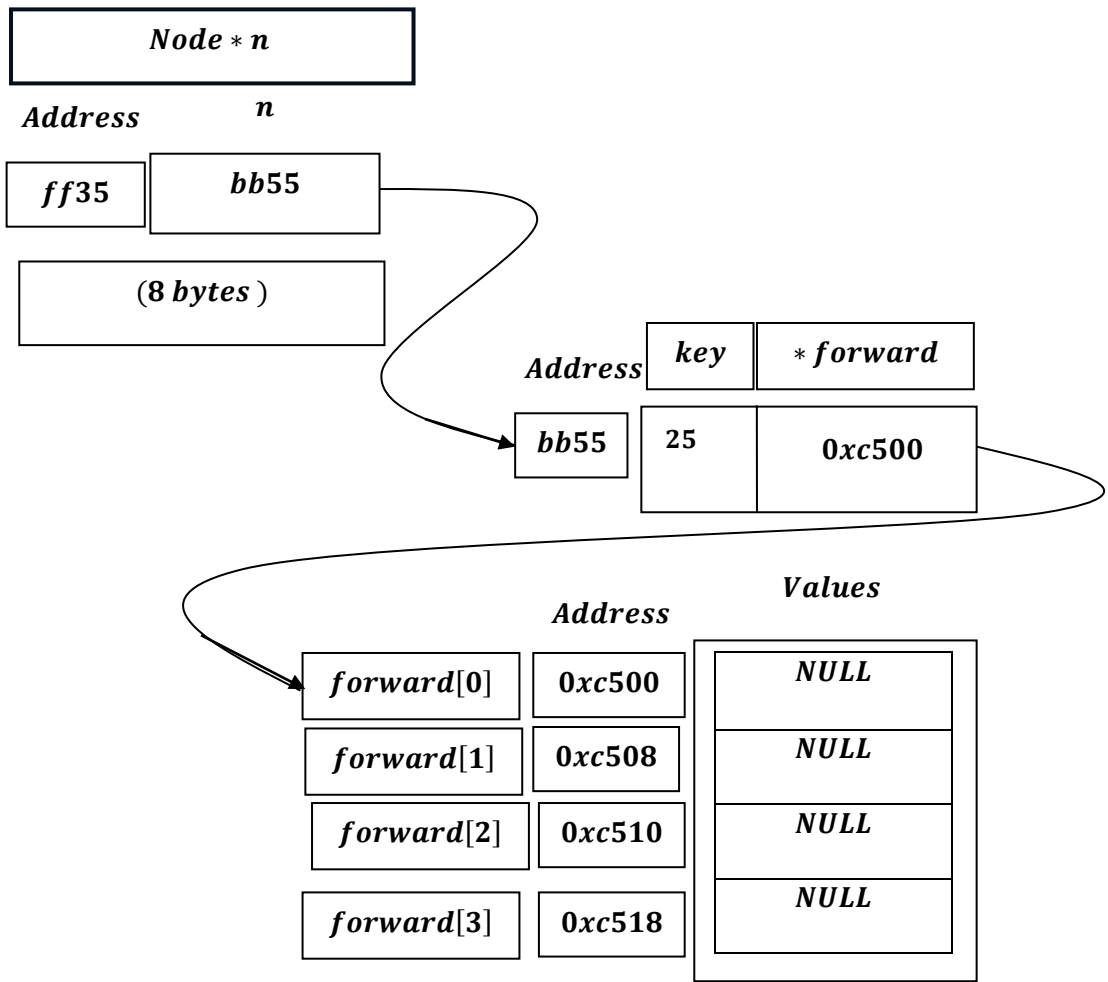
[end of if block]





```
struct Node *n = createNode(key, rlevel){...};
```

Let *rlevel* = 3 and *key* = 25.



```
return n;
```

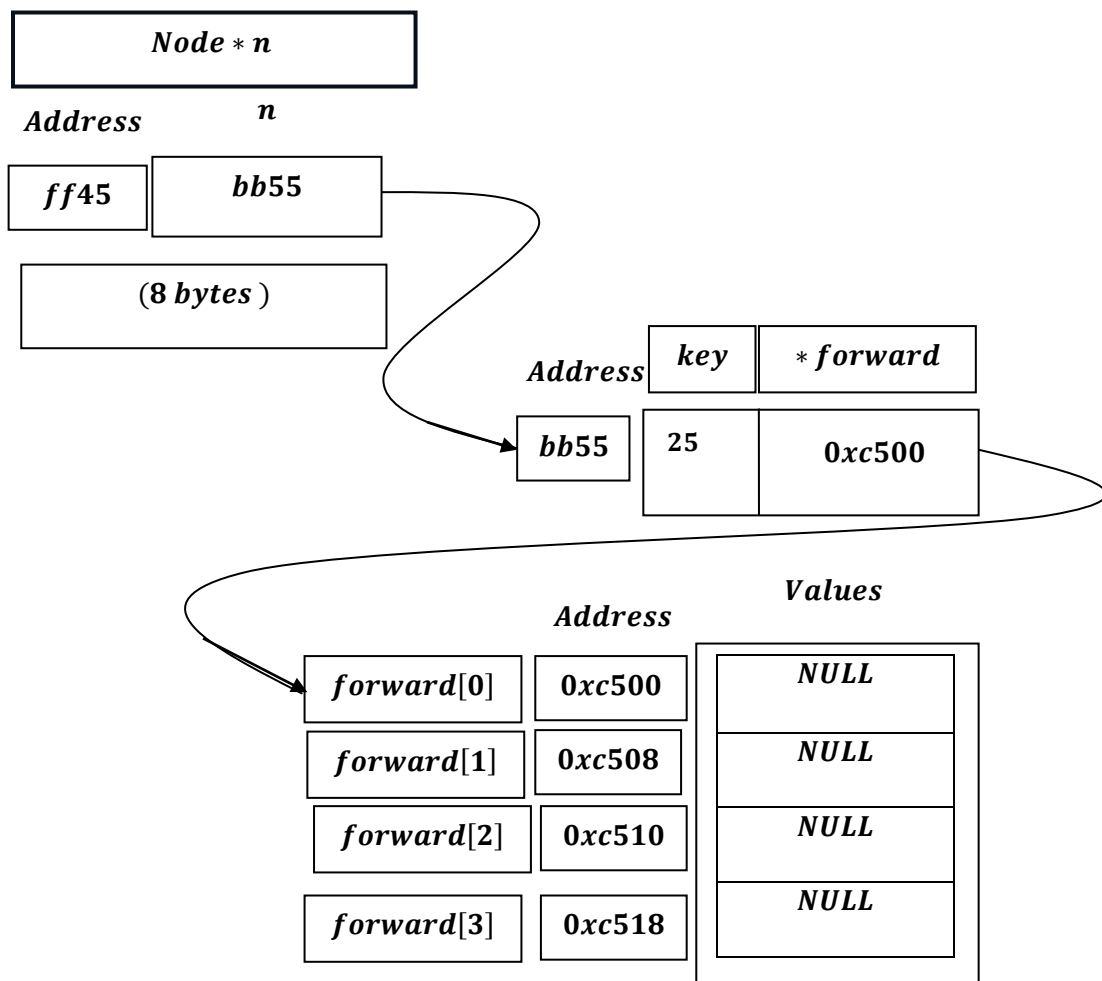
i. e. Return *n* = *bb55*

Actual structure

<i>Address at which value is stored</i>	<i>Value</i>	<i>offset used</i>
<i>bb55</i>	<i>key = 25</i>	<i>key[bb55 – bb58] 4 bytes for int</i>
		<i>padding → [bb59 – bb5c] 4 bytes for padding</i>
<i>bb5d</i>	<i>Address: 0xc500</i>	<i>* forward → [bb5d – bb64] 8 bytes for pointer variable</i>
<i>0xc500</i>	<i>NULL</i>	<i>forward[0] → [0xc500 – 0xc507] 8 bytes for pointer variable</i>
<i>0xc508</i>	<i>NULL</i>	<i>forward[1] → [0xc508 – 0xc50f] 8 bytes for pointer variable</i>
<i>0xc510</i>	<i>NULL</i>	<i>forward[2] → [0xc510 – 0xc517] 8 bytes for pointer variable</i>
<i>0xc518</i>	<i>NULL</i>	<i>forward[3] → [0xc518 – 0xc51f] 8 bytes for pointer variable</i>

struct Node * n = createNode(key = 25, rlevel = 3)

struct Node * n = bb55



```
for (int i = 0; i <= rlevel; i++) {
    n->forward[i] = update[i]->forward[i];
    update[i]->forward[i] = n;
}
```

$int\ i = 0; i \leq rlevel = 3$

$n \rightarrow forward[0] = update[0] \rightarrow forward[0] = e216$

$update[0] \rightarrow forward[0] = n = bb55$

$i = i + 1 = 0 + 1 = 1$

int i = 1; i ≤ rlevel = 3

n → forward[1] = update[1] → forward[1] = e216

update[1] → forward[1] = n = bb55

i = i + 1 = 1 + 1 = 2

int i = 2; i ≤ rlevel = 3

n → forward[2] = update[2] → forward[2] = NULL

update[2] → forward[2] = n = bb55

i = i + 1 = 2 + 1 = 3

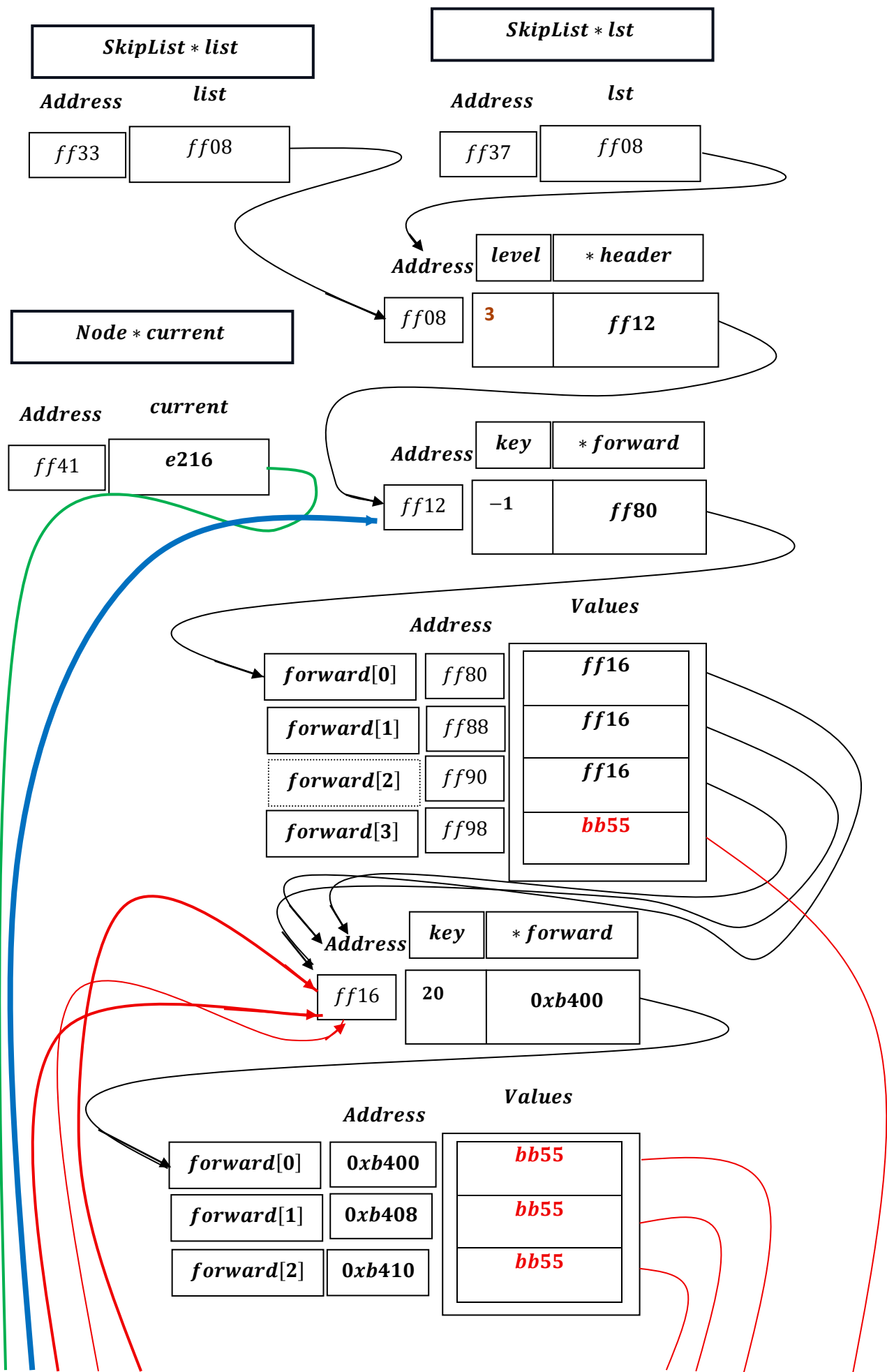
int i = 3; i ≤ rlevel = 3

n → forward[3] = update[3] → forward[3] = NULL

update[3] → forward[3] = n = bb55

i = i + 1 = 3 + 1 = 4

int i = 4; i ≤ rlevel = 3 $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$, *hence loop exits*



*Node * n*

Address *n*

ff45

bb55

Address

key

** forward*

(8 bytes)

bb55

25

0xc500

Address

Values

forward[0]

0xc500

forward[1]

0xc508

forward[2]

0xc510

forward[3]

0xc518

e216

e216

NULL

NULL

Address

key

** forward*

e216

30

0xd800

Address

Values

forward[0]

0xd800

forward[1]

0xd808

NULL

NULL

Address

Values

update[0]

0x7fff0000

ff16

update[1]

0x7fff0008

ff16

update[2]

0x7fff0010

ff16

update[3]

0x7fff0018

ff12

```
cout << "Successfully inserted key " << key << endl;
```

Successfully inserted key : 25 and function ends .

As , the function ends :

*SkipList * lst*

Address	lst
ff37	ff08

*Node * current*

Address	current
ff41	e216

	Address	Values
update[0]	0x7fff0000	ff16
update[1]	0x7fff0008	ff16
update[2]	0x7fff0010	ff16
update[3]	0x7fff0018	ff12

*Node * n*

Address	n
ff45	bb55

Gets Destroyed

*SkipList * list*

Address *list*

ff33 *ff08*

Address *level* ** header*

ff08 **3** *ff12*

Address *key* ** forward*

ff12 -1 *ff80*

Address

Values

<i>Address</i>	<i>Values</i>
<i>forward[0]</i> <i>ff80</i>	<i>ff16</i>
<i>forward[1]</i> <i>ff88</i>	<i>ff16</i>
<i>forward[2]</i> <i>ff90</i>	<i>ff16</i>
<i>forward[3]</i> <i>ff98</i>	<i>bb55</i>

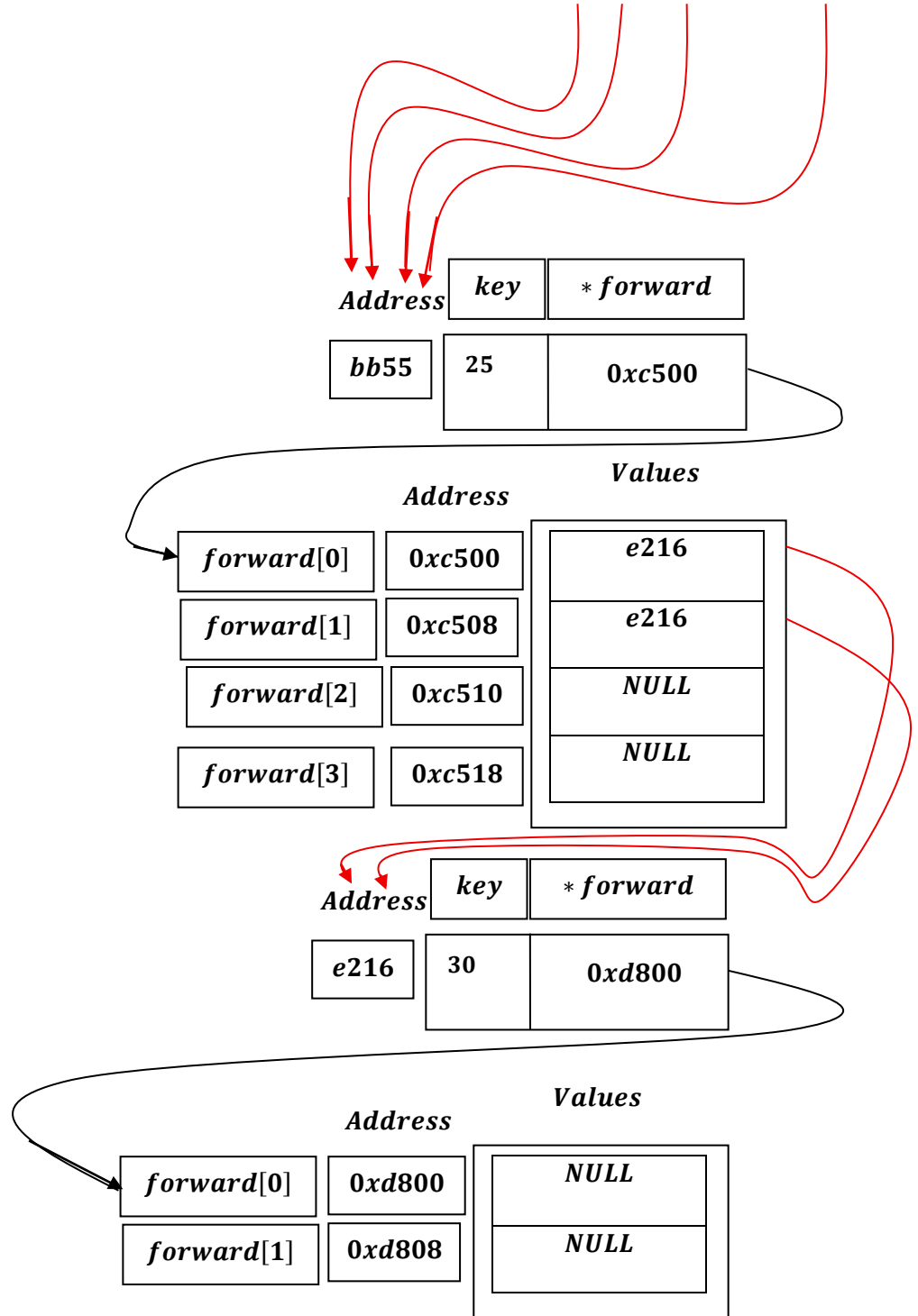
Address *key* ** forward*

ff16 20 *0xb400*

Address

Values

<i>Address</i>	<i>Values</i>
<i>forward[0]</i> <i>0xb400</i>	<i>bb55</i>
<i>forward[1]</i> <i>0xb408</i>	<i>bb55</i>
<i>forward[2]</i> <i>0xb410</i>	<i>bb55</i>



Skip List – Insert Element

–Program Analysis

–Insertion

(Attempting to insert a duplicate key)

*SkipList * list*

Address *list*

ff33 *ff08*

Address *level* ** header*

ff08 **3** *ff12*

Address *key* ** forward*

ff12 -1 *ff80*

Address *Values*

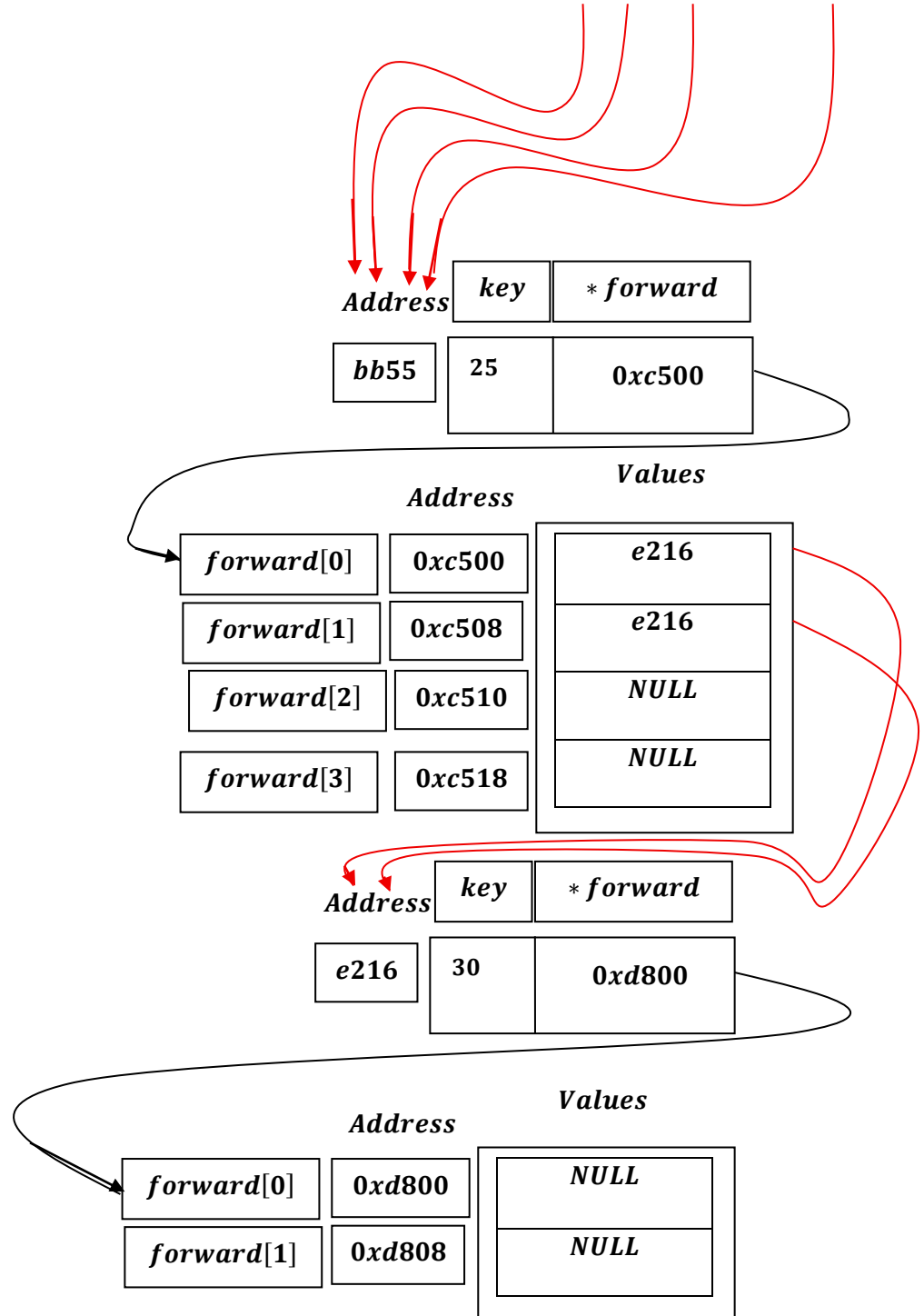
<i>forward</i>	<i>Address</i>	<i>Values</i>
<i>forward</i> [0]	<i>ff80</i>	<i>ff16</i>
<i>forward</i> [1]	<i>ff88</i>	<i>ff16</i>
<i>forward</i> [2]	<i>ff90</i>	<i>ff16</i>
<i>forward</i> [3]	<i>ff98</i>	<i>bb55</i>

Address *key* ** forward*

ff16 20 *0xb400*

Address *Values*

<i>forward</i>	<i>Address</i>	<i>Values</i>
<i>forward</i> [0]	<i>0xb400</i>	<i>bb55</i>
<i>forward</i> [1]	<i>0xb408</i>	<i>bb55</i>
<i>forward</i> [2]	<i>0xb410</i>	<i>bb55</i>



```
case 1:
    cout << "Enter key to insert: ";
    cin >> key;
    insertElement(list, key);
    break;
```

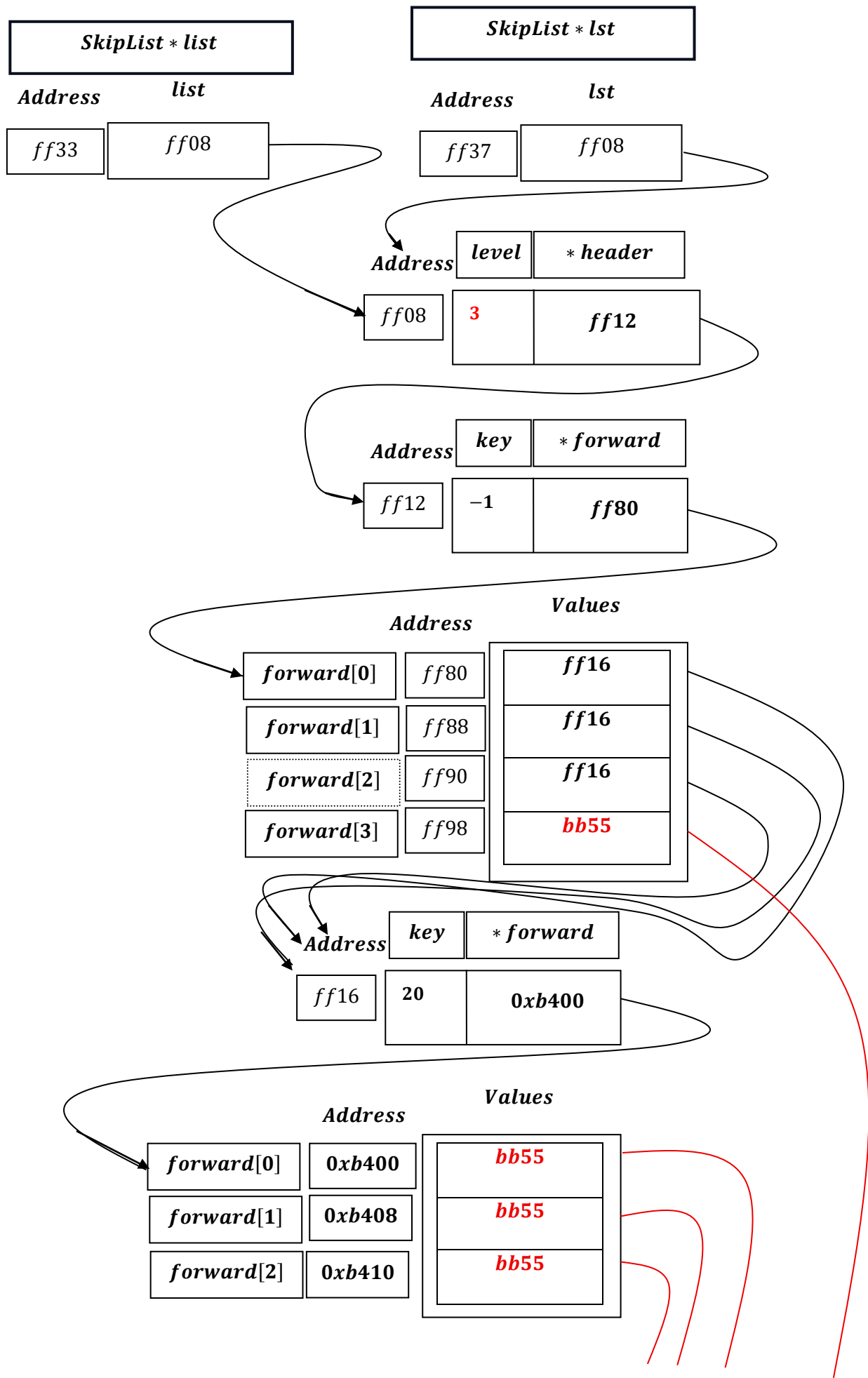
Entering : Key = 25 .

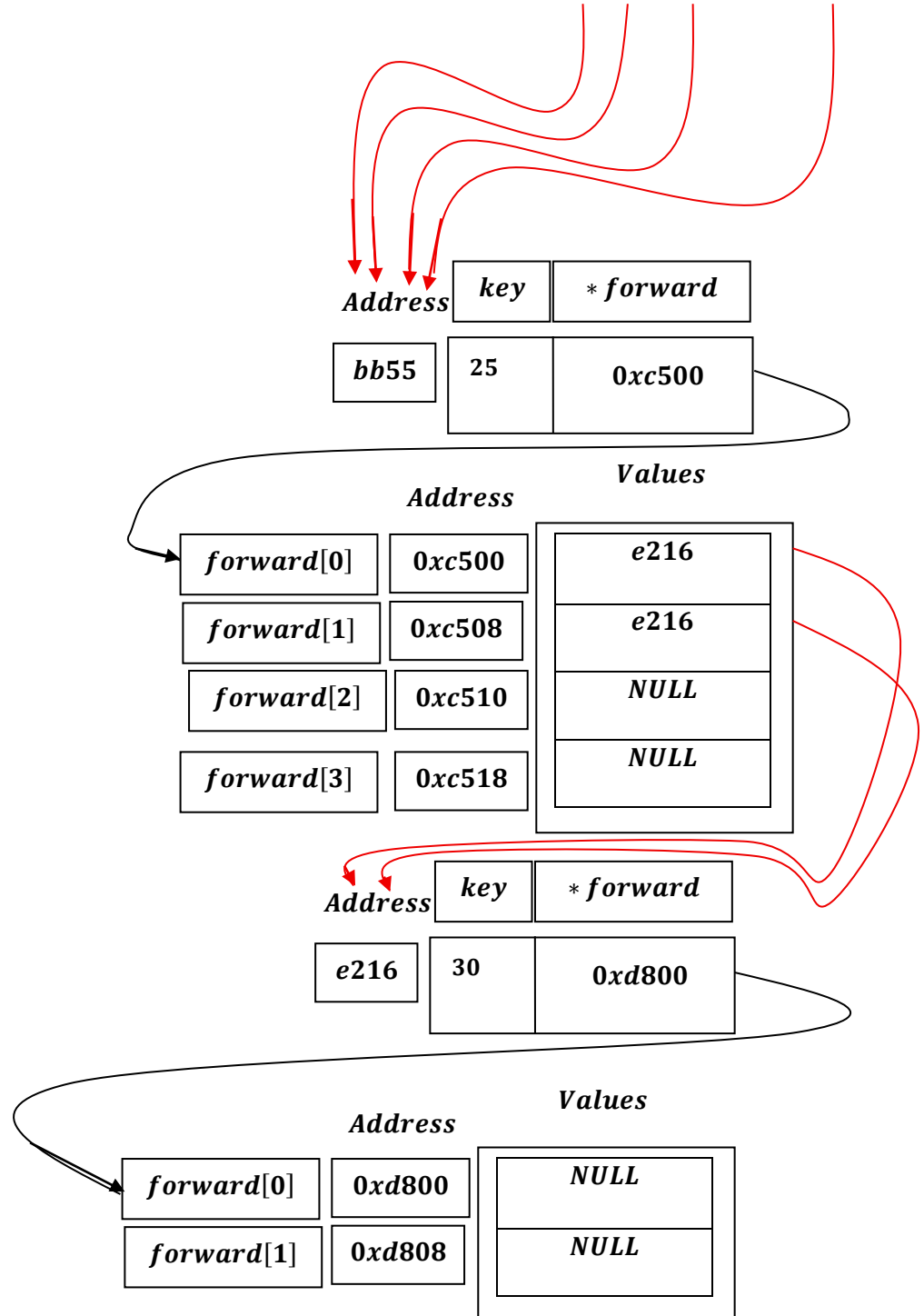
insertElement(list: ff08 ,key: 25); [25 is a duplicate key]

Now ,we have :

```
void insertElement(struct SkipList *lst, int key)
```

*list = struct SkipList * lst = ff08 and int key: 25.*





```
struct Node *current = lst->header;
```

current = *lst* → *header* : *ff12*.

```
struct Node *update[MAX_LEVEL + 1];
```

update[3 + 1 = 4],

MAX_LEVEL evaluates to 3. Consequently, the array size calculation is $\text{update}[3 + 1 = 4]$.

This means the array is allocated to hold exactly 4 elements.

Before memset call, the update array may look like :

<i>Array Element</i>	<i>Hypothetical Stack Address</i>	<i>Value (Garbage Data)</i>	<i>Pointer Status</i>
<i>update[0]</i>	<i>0x7fff0000 to 0x7fff0007</i>	<i>0x4a3f8921bce45099</i>	<i>Wild Pointer (Points to a random, invalid address)</i>
<i>update[1]</i>	<i>0x7fff0008 to 0x7fff000f</i>	<i>0x0000000000000002</i>	<i>Wild Pointer</i>
<i>update[2]</i>	<i>0x7fff0010 to 0x7fff0017</i>	<i>0x7ffee1245a00881c</i>	<i>Wild Pointer</i>
<i>update[3]</i>	<i>0x7fff0018 to 0x7fff001f</i>	<i>0x0000000000000000</i>	<i>Accidentally NULL (Just by chance)</i>

More simpler way:

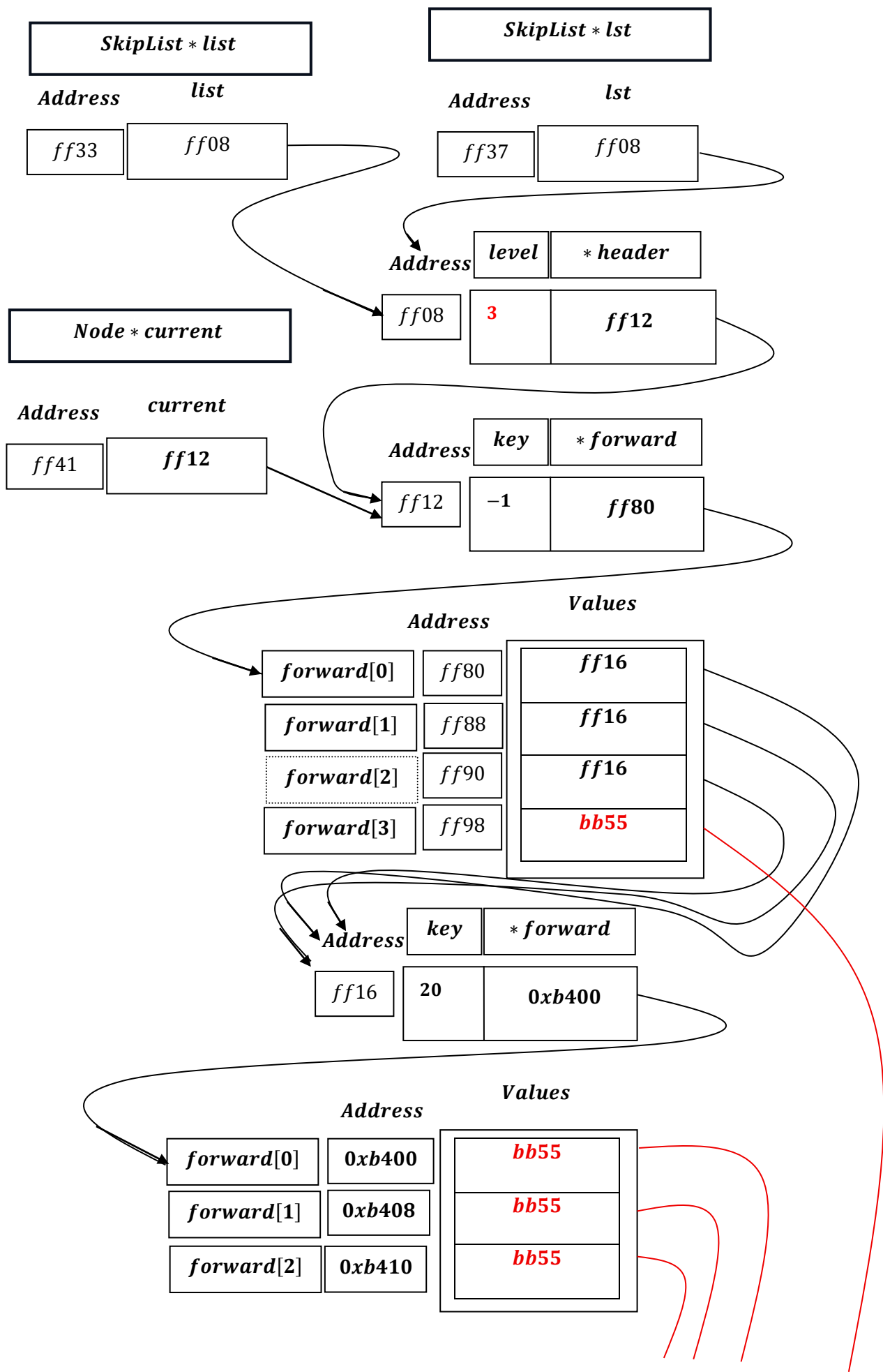
	<i>Address</i>	<i>Values</i>	
<i>update[0]</i>	<i>0x7fff0000</i>	<i>0x4a3f8921bce45099</i>	<i>[8 bytes]</i>
<i>update[1]</i>	<i>0x7fff0008</i>	<i>0x0000000000000002</i>	<i>[8 bytes]</i>
<i>update[2]</i>	<i>0x7fff0010</i>	<i>0x7ffee1245a00881c</i>	<i>[8 bytes]</i>
<i>update[3]</i>	<i>0x7fff0018</i>	<i>0x0000000000000000</i>	<i>[8 bytes]</i>

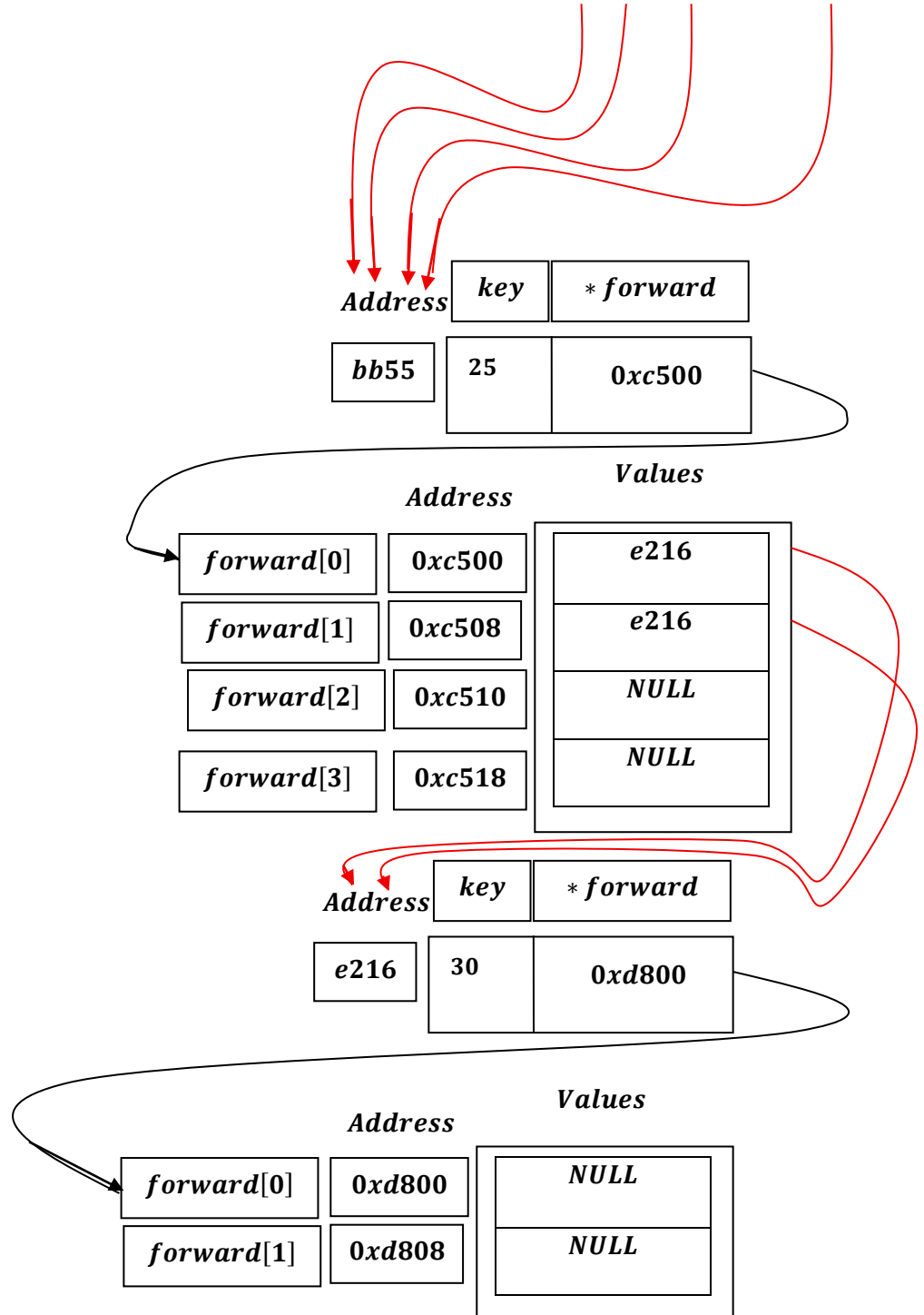
```
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

“memset” stands for Memory Set. It takes a block of memory and rapidly fills it with a specific value. Here `0` or NULL.

	Address	Values
update[0]	0x7fff0000	NULL
update[1]	0x7fff0008	NULL
update[2]	0x7fff0010	NULL
update[3]	0x7fff0018	NULL

Here we kept addresses same like: 0x7fff0000 , just for an example , but it may not be same , in reality.





	Address	Values
update[0]	0x7fff0000	NULL
update[1]	0x7fff0008	NULL
update[2]	0x7fff0010	NULL
update[3]	0x7fff0018	NULL

```

for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}

```

For : $i = lst \rightarrow level = 3 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

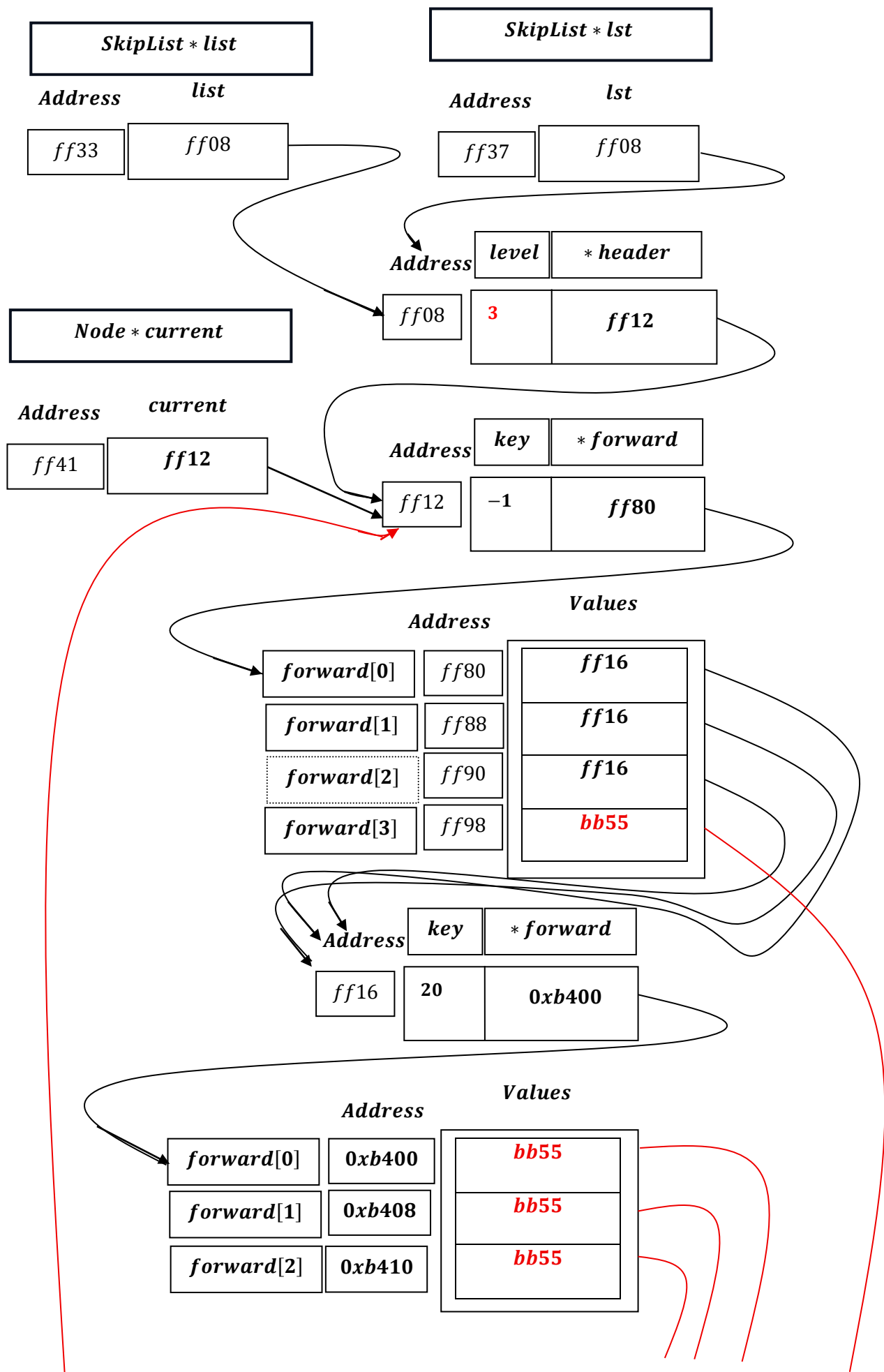
while: $current \rightarrow forward[3] = bb55 \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$ And

$current \rightarrow forward[3] \rightarrow key: 25 < key: 25$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$

[while loop exits]

update[3] = current = ff12.

$i = i - 1 = 3 - 1 = 2.$



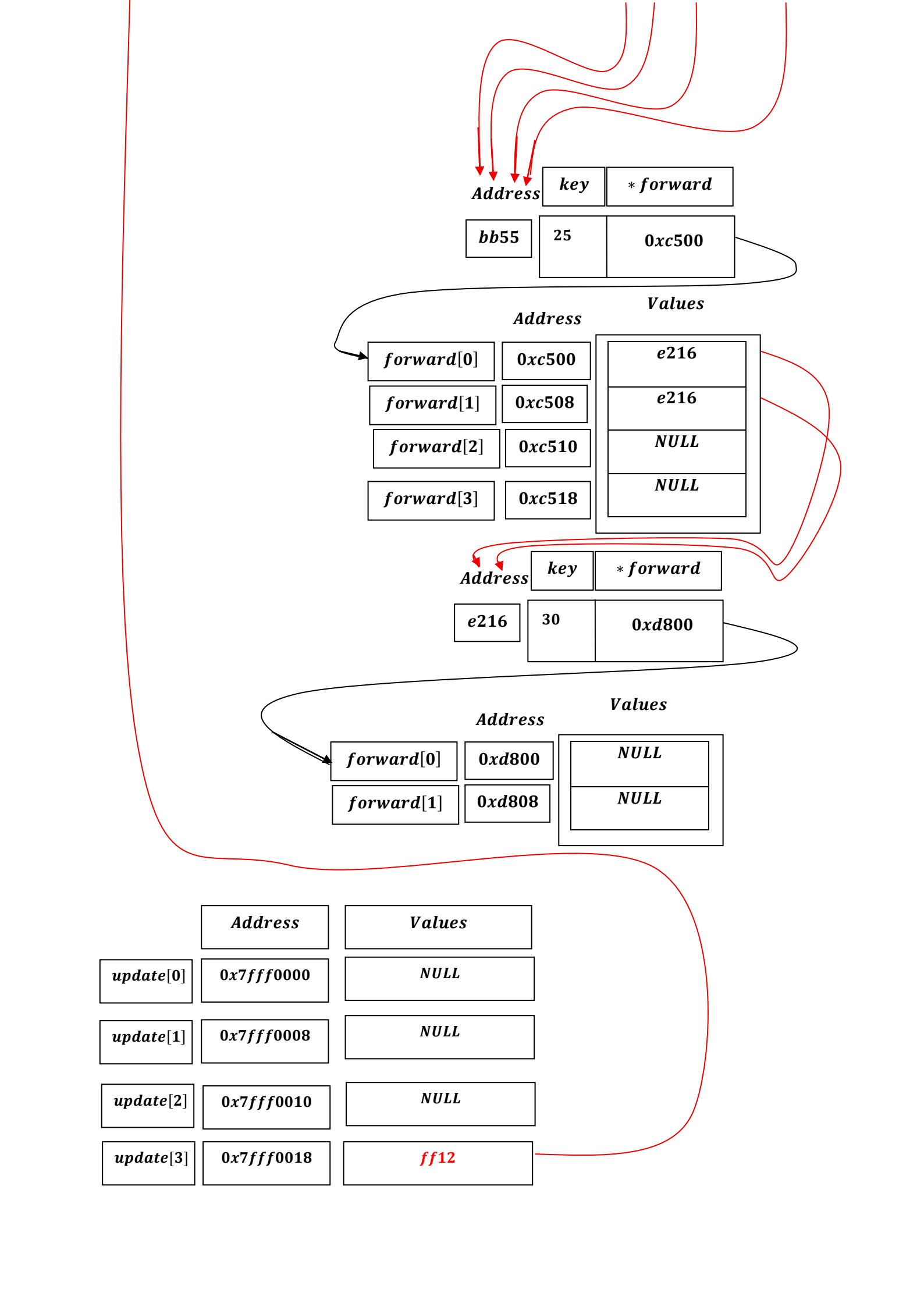
<i>Address</i>	<i>key</i>	<i>* forward</i>
<i>bb55</i>	25	0xc500

<i>Address</i>		<i>Values</i>
<i>forward[0]</i>	0xc500	e216
<i>forward[1]</i>	0xc508	e216
<i>forward[2]</i>	0xc510	NULL
<i>forward[3]</i>	0xc518	NULL

<i>Address</i>	<i>key</i>	<i>* forward</i>
<i>e216</i>	30	0xd800

<i>Address</i>		<i>Values</i>
<i>forward[0]</i>	0xd800	NULL
<i>forward[1]</i>	0xd808	NULL

	<i>Address</i>	<i>Values</i>
<i>update[0]</i>	0x7fff0000	NULL
<i>update[1]</i>	0x7fff0008	NULL
<i>update[2]</i>	0x7fff0010	NULL
<i>update[3]</i>	0x7fff0018	<i>ff12</i>



For : $i = 2 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

while: $current[ff12] \rightarrow forward[2] = ff16 \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$ And

$current[ff12] \rightarrow forward[2] \rightarrow key: 20 < key: 25$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

$current = current \rightarrow forward[2] = ff16$

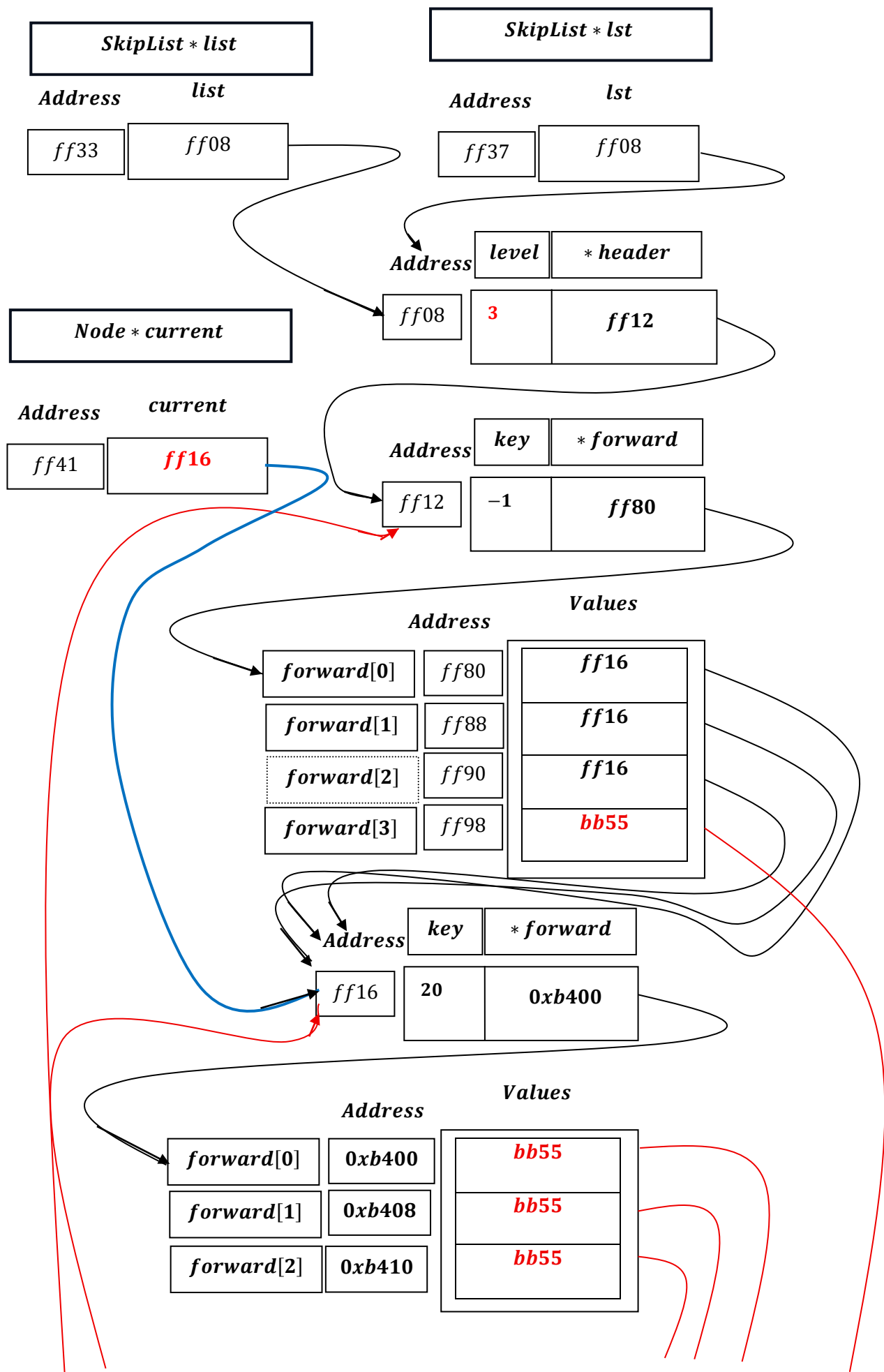
while: $current[ff16] \rightarrow forward[2] = bb55 \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$ And

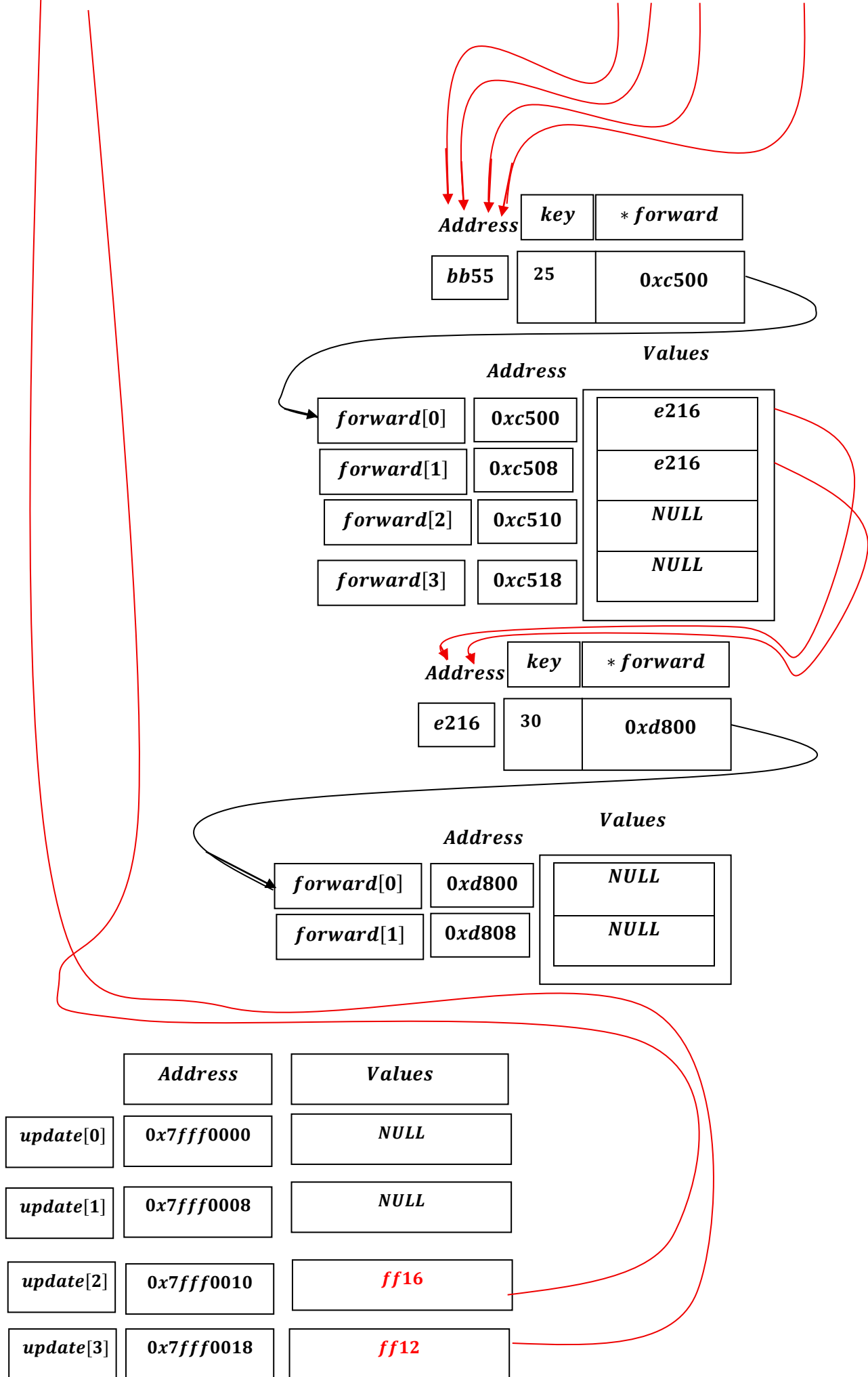
$current[ff16] \rightarrow forward[2] \rightarrow key: 25 < key: 25$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$

[while loop exits]

$update[2] = current = ff16.$

$i = i - 1 = 2 - 1 = 1.$





For : $i = 1 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

while: $current[ff16] \rightarrow forward[1] = bb55 \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$ And

$current[ff16] \rightarrow forward[1] \rightarrow key: 25 < key: 25$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$

[while loop exits]

update[1] = current = ff16.

$i = i - 1 = 1 - 1 = 0$.

For : $i = 0 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

while: $current[ff16] \rightarrow forward[0] = bb55 \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$ And

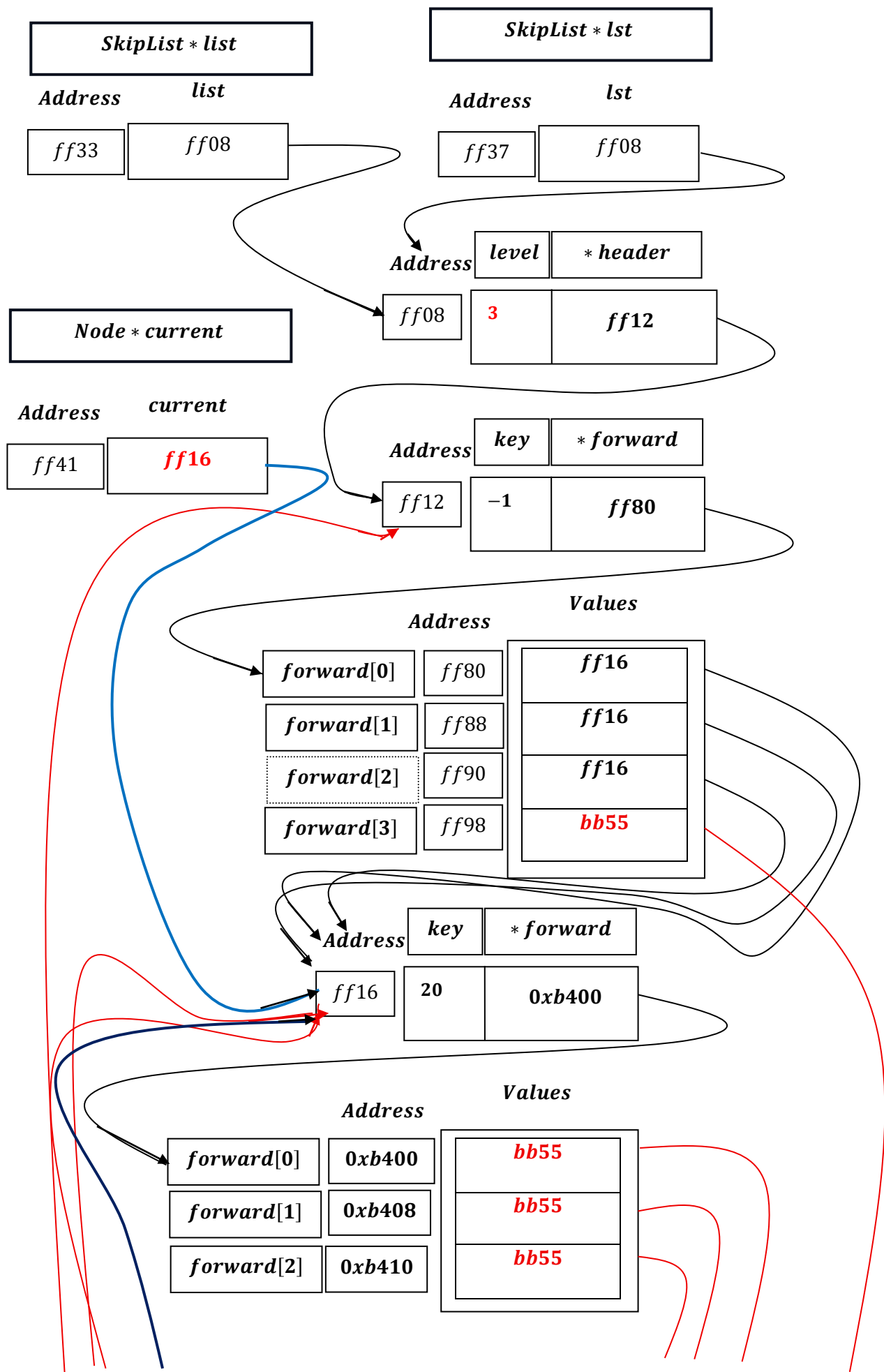
$current[ff16] \rightarrow forward[0] \rightarrow key: 25 < key: 25$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$

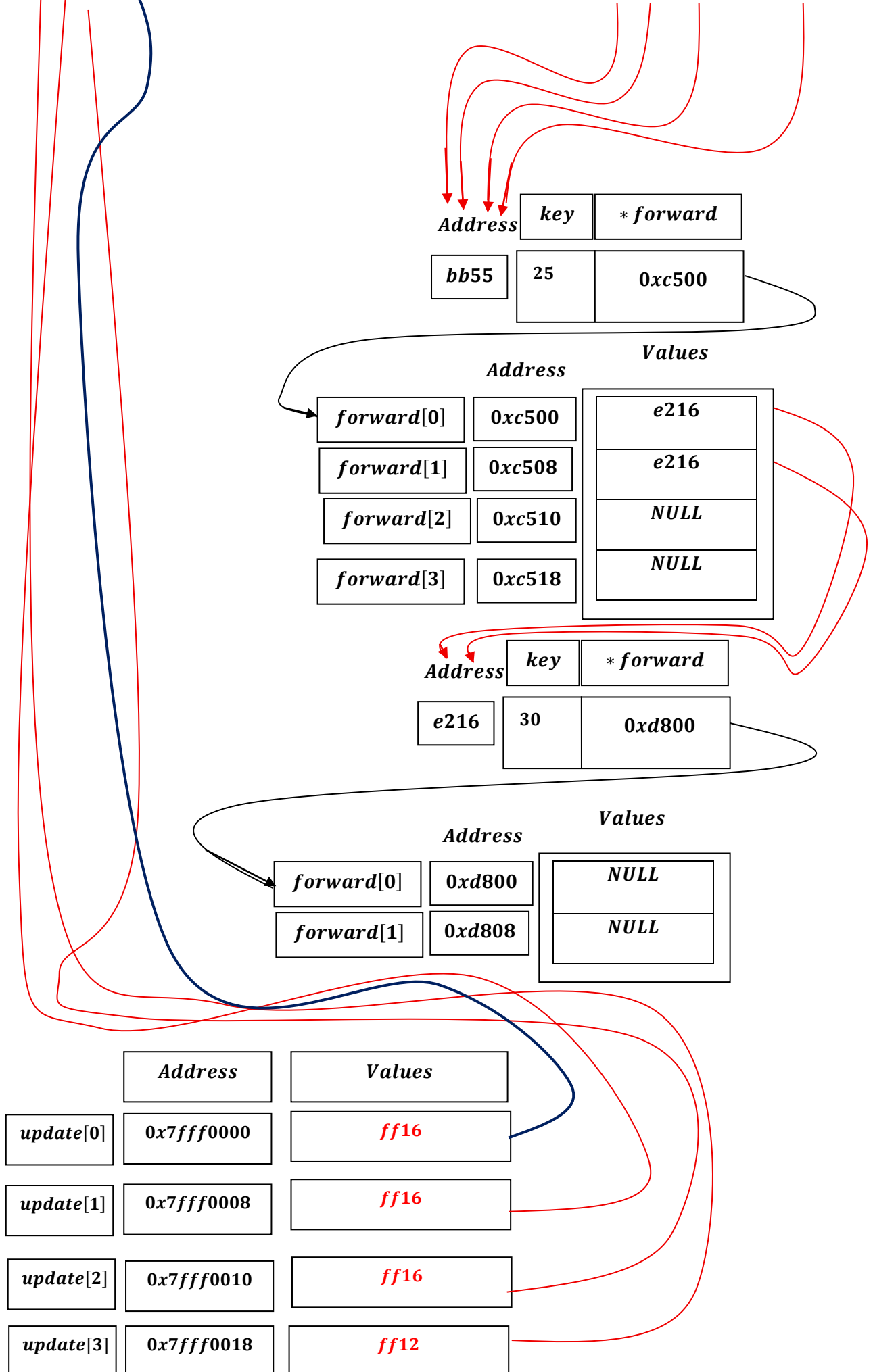
[while loop exits]

update[0] = current = ff16.

$i = i - 1 = 0 - 1 = -1$.

For : $i = -1 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$ $\rightarrow$ For loop exits.



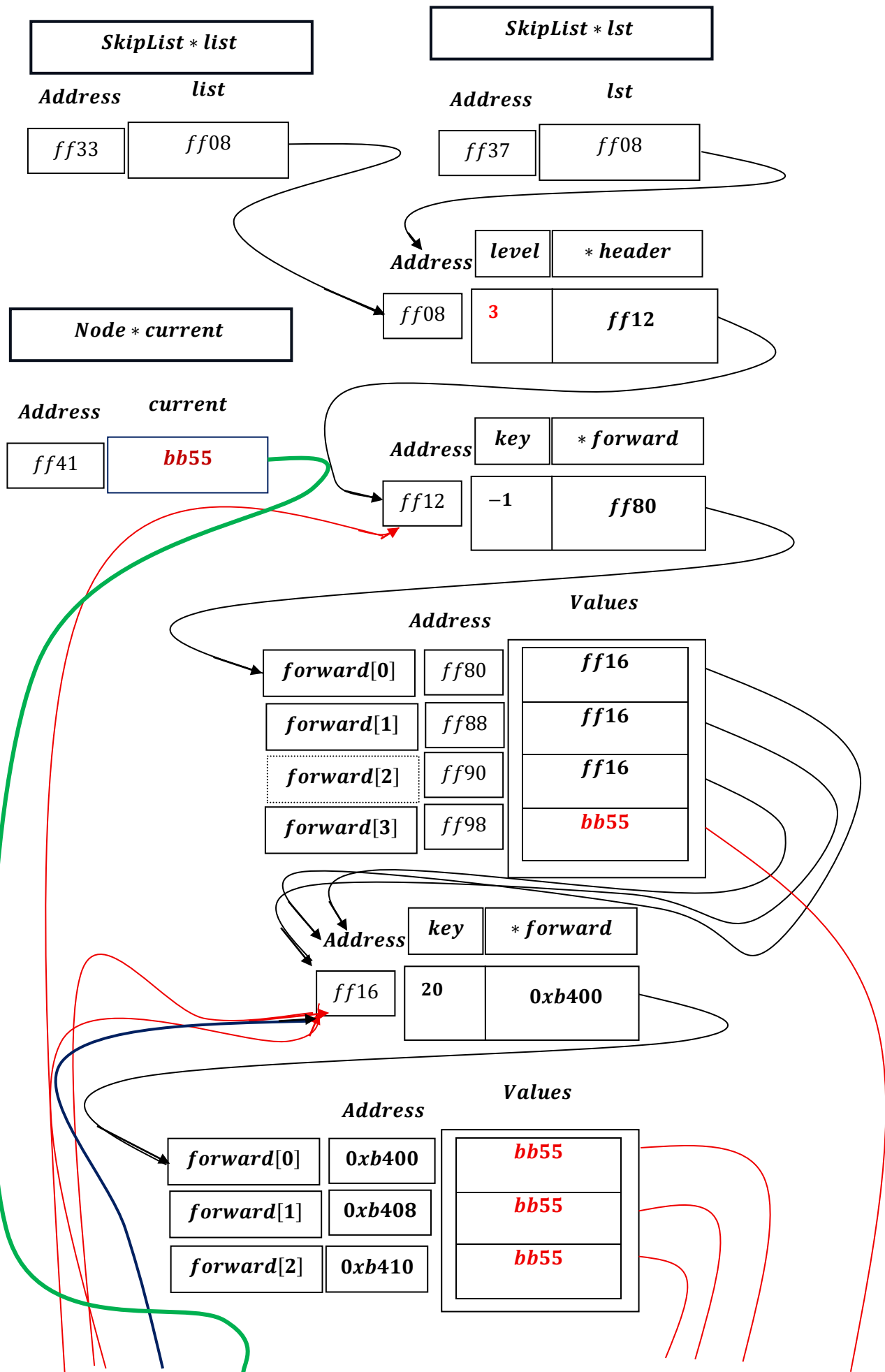


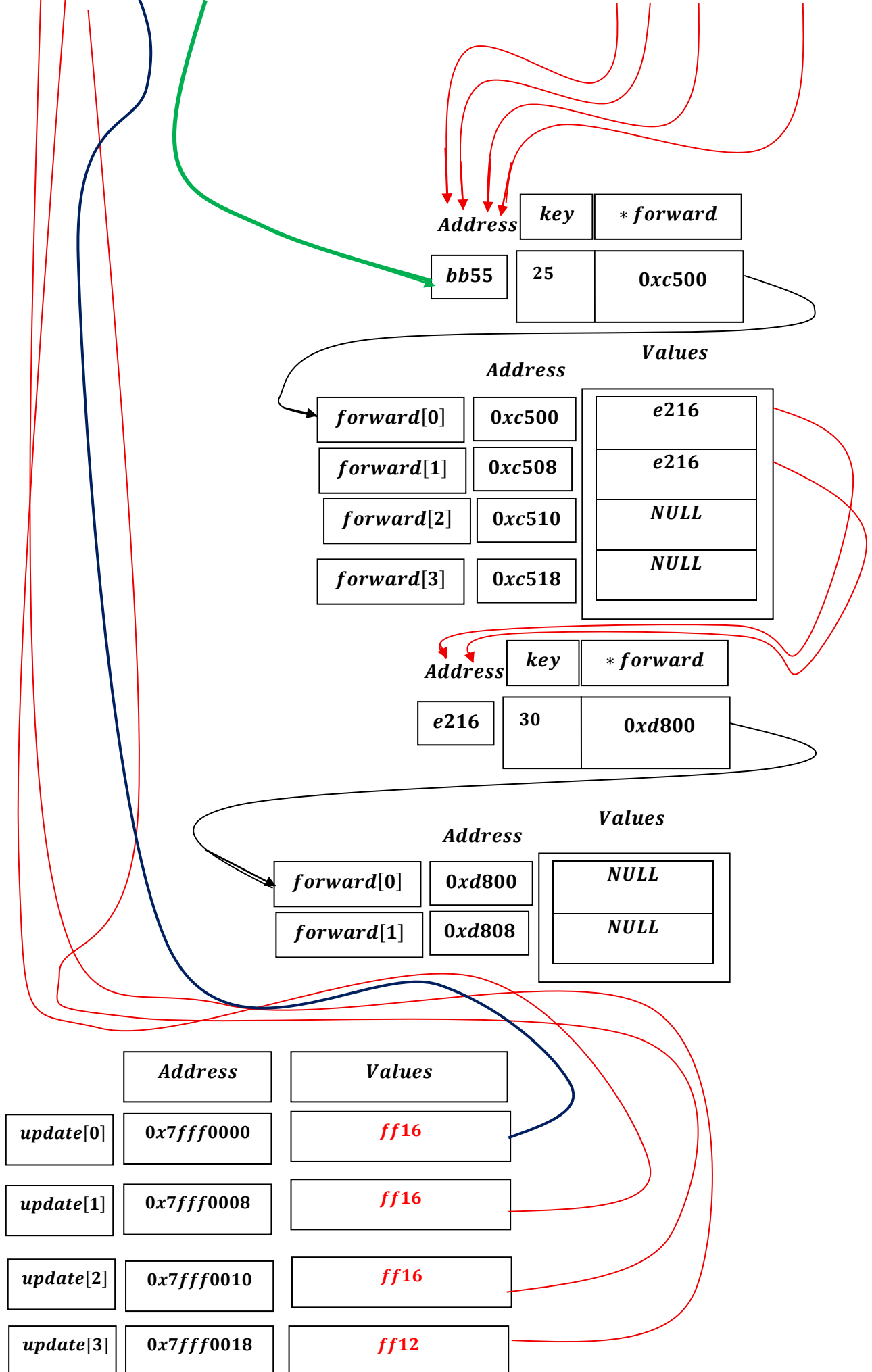

```
current = current->forward[0];
```

current = current → forward[0] = bb55

```
if (current == NULL || current->key != key) {...}
```

*current = ff16 ≠ NULL (false) OR current → key = 25 ≠ key = 25 (False),
hence if block terminates.*





*As we attempt to insert duplicate key it fails, hence ,
here in this program ,no duplicate keys are allowed.*

Can Skip Lists ever have duplicates?

As a general computer science data structure, a Skip List can be programmed to allow duplicates if the developer wants them to. To do that, we would just remove that:

`!= key`

check and allow the new node to be inserted right next to the existing one.

*However, standard sets and dictionaries (which skip lists are often used to build)
usually require unique keys, which is why the provided program is an implementation that
strictly blocks duplicates!*

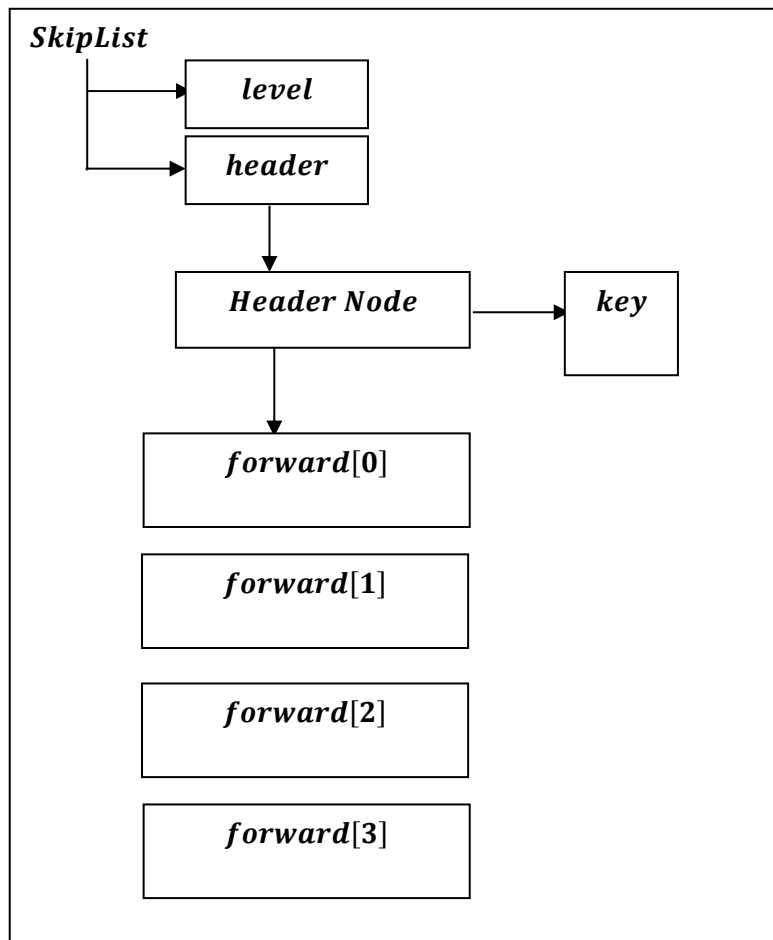
Skip List – Insert Element

–Insert Module Summary

```
void insertElement(struct SkipList *lst, int key)
```

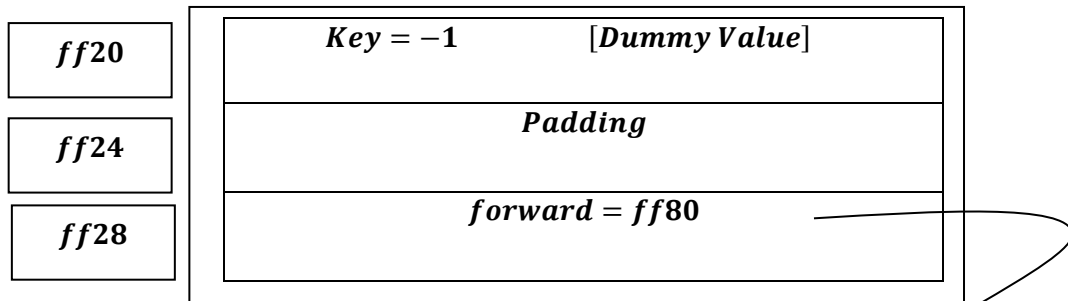
It receives two things:

- *lst* → *pointer to the Skip List.*
- *key* → *value that we want to insert.*

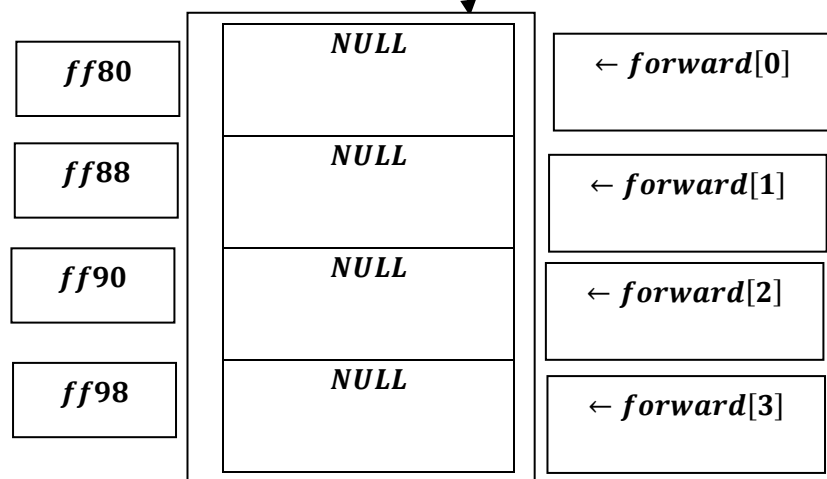


More Precisely:

Header Node or Dummy Node [address: ff20]

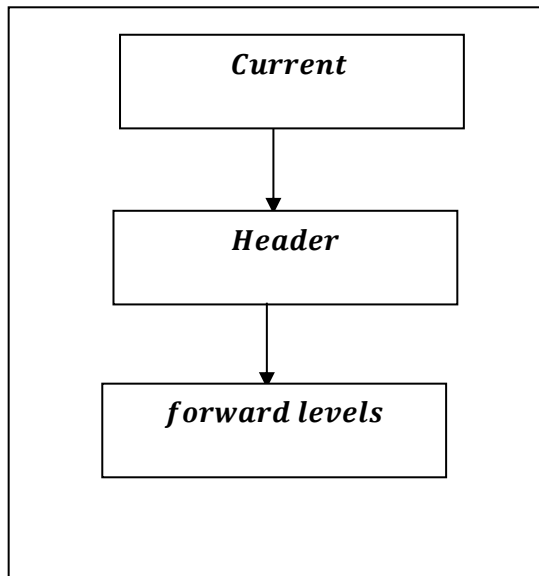


***Forward pointer array starts
at address ff80***



II. Initialize Current

```
struct Node *current = lst->header;
```



Why we need current ?

Because the algorithm must walk through the Skip List to find where the new key belongs.

- **To Initialize the Search:**

The current pointer acts as the active navigator or cursor for the list, explicitly starting its journey at the header node of the skip list (`struct Node *current = lst->header;`).

- **To Traverse and Find the Insertion Point:**

It is used within a while loop to physically move horizontally across the nodes at each level. The program compares the next node's key against the target key being inserted (`current->forward[i] -> key < key`). If the next key is smaller, the current pointer steps forward to that node (`current = current->forward[i]`).

- **To Record the Pointer Path:**
Once current cannot move any further right on a given level, its exact address is saved into the update array (`update[i] = current`). This creates a map of the path taken, ensuring the program knows precisely which previous nodes must be rewired to point to the newly inserted node later on.
- **To Prevent Duplicates:**
After dropping down through all the levels, the program moves current one final step forward on level 0 (`current = current → forward[0]`). This lands current exactly where the new key is supposed to go. The pointer is then used to validate whether that key already exists in the list (`if (current == NULL || current → key != key)`) to enforce unique keys and prevent duplicate insertions.

III. Initialization and the update Array

Before inserting a new element, the program must track where the new node will link into the existing list.

- The function initializes an `update` array to keep track of the path taken and the nodes that will need their pointers updated.
- The size of this array is calculated as `MAX_LEVEL + 1`.
- Because `MAX_LEVEL` is 3, the array holds exactly 4 elements.
- When initially declared, the array holds random garbage data (wild pointers) on the stack.
- To prevent undefined behavior, the `memset` function is used to rapidly fill this entire memory block with 0 or NULL values.

Why we need Update ?

→ To Track Pointer Updates:

Its primary purpose is to keep track of the specific path taken during the search, logging the exact nodes that will need their pointers updated.

→ To Mark the "Drop" Points:

As the **current** pointer searches for the correct insertion spot, it moves rightwards and then drops down level by level. At each level **i**, right before dropping down, the program saves the current node's address into the array (**update[i] = current;**). This essentially records the "predecessor" node for the new element at every single level.

→ To Handle New Maximum Levels:

If the new node generates a random level (**rlevel**) that is taller than the current list, the **update** array is used to link those brand – new upper levels directly to the list's header:
(**update[i] = lst->header;**).

→ To Execute the Final Rewiring:

Once the new node (**n**) is created in memory, the program relies entirely on the **update** array to connect it. It uses the saved predecessor nodes to point the new node forward

(**n → forward[i] = update[i] → forward[i]**), and then rewires those predecessor nodes to point directly to the new node

(**update[i] → forward[i] = n**).

IV. The core search and mapping engine for the Skip List.

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

- *for (int i = lst → level; i ≥ 0; i --): This initiates a top – down search.*

Instead of searching one by one from the bottom,

it starts at the highest current level of the list (lst → level)

and drops down step – by – step to level 0 (i – –). This is what gives a

skip list its speed, allowing it to use the upper levels as express lanes

to skip over large chunks of data.

- *while (current → forward[i] != NULL
&& current → forward[i] → key < key):*

At each level, the program looks ahead to the next node.

It checks two critical things before moving:

1. *!= NULL: Does a next node actually exist, or have we reached the end of the list?*
2. *→ key < key: Is the next node's key smaller than the one we are trying to insert?*

- *`current = current → forward[i];` If the next node exists and has a smaller key, the current pointer moves horizontally to the right. This while loop continues moving the pointer rightward until it hits a node that is larger, or hits the end of the list.*
- *`update[i] = current;` This is the most crucial part for the actual insertion. When the `current` pointer can no longer move forward on level `i`, it stops.*

The program then records this exact node into the `update` array (`update[i] = current`) before the `for` loop forces the pointer to drop down to the next level.

V. Duplicate Key Validation

Skip lists often act as sets or dictionaries, meaning they require unique keys.

- *After navigating down to level 0, the program steps forward one final time using `current = current → forward[0]` to inspect the exact spot where the new key belongs.*
- *The program validates uniqueness using the condition*
`if (current == NULL || current->key != key).`
- *If `current` matches the key we are trying to insert, the condition fails, the insertion block is skipped, and the program prints that the key already exists.*
- *While this implementation strictly blocks duplicates, standard computer science skip lists could theoretically allow them by removing the `!= key` check.*

VI. Random Level Generation and Header Adjustment

If the key is unique, the program prepares to build the new node.

- *A random level (**rlevel**) is generated for the new node via the **randomLevel()** function.*
- *The program must account for cases where the generated **rlevel** is higher than any node currently in the list (**rlevel** > **lst** → **level**).*
- *If **rlevel** is greater than the current **level** a **for loop** updates the new highest slots in the **update** array to point directly to the list's **header**.*
- *The list's overall maximum level tracker (**lst** → **level**) is then permanently updated to match this new **rlevel**.*

*If the randomly generated level (**rlevel**) for the new node is strictly greater than the list's current level (**lst** → **level**), the program executes a specific if block to safely increase the **height of the skip list**.*

Here is exactly what happens in that scenario:

- **Anchoring to the Header:** *A **for** loop iterates from the first new level (**lst** → **level** + 1) up to the new **rlevel**. Because these newly generated upper levels are completely empty and have no existing predecessor nodes, the program must set the **update** array for these levels to point directly to the list's **header** node (**update[i]** = **lst** → **header**).*

```
for (int i = lst->level + 1; i <= rlevel; i++) {
    update[i] = lst->header;
}
```

- **Updating the Global Tracker:** After initializing the **update** array for the new levels, the program updates the skip list's internal property that tracks its height. It permanently sets **lst → level = rlevel**, ensuring that all future searches and insertions know the list has grown taller.

```
lst->level = rlevel;
```

Exceeding the Current Max Level (lst → level): If the new node's randomly generated level (**rlevel**) is higher than the tallest node currently existing in the list, the program gracefully adapts. Instead of crashing, it safely anchors those brand – new upper levels to the list's **header** and securely updates the list's internal height tracker (**lst → level = rlevel**).

```
for (int i = lst->level + 1; i <= rlevel; i++) {
    update[i] = lst->header;
}
lst->level = rlevel;
```

The Absolute Hard Limit (MAX_LEVEL): The program will never actually exceed the absolute hard cap defined by **MAX_LEVEL** (which evaluates to 3 in this program). The **randomLevel()** function includes a strict built – in safety check:

while ((rand() % 2) == 0 && lvl < MAX_LEVEL). This condition guarantees that the loop generating the random level immediately stops incrementing if it hits the **MAX_LEVEL** cap.

```

int randomLevel() {
    int lvl = 0;
    // 50% probability to increase the level
    while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {
        lvl++;
    }
    return lvl;
}

```

VII. Node Creation and Memory Allocation

```

    struct Node *n = createNode(key, rlevel);
struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
elements)
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}

```

Once the level is finalized, memory must be explicitly allocated for the new data.

- The **createNode(key, rlevel)** function uses **malloc** to reserve memory space.
- It allocates 8 bytes to hold the primary **Node** structure
(which stores the integer key itself).
- It dynamically allocates memory for the **forward** array based on the level, taking **(level + 1) * 8 bytes**.

- A **for loop** inside **createNode** ensures that all freshly allocated forward pointers are initially set to **NULL** before returning the new node **n**.

VII. Pointer Rewiring (Insertion)

```
for (int i = 0; i <= rlevel; i++) {
    n->forward[i] = update[i]->forward[i];
    update[i]->forward[i] = n;
}
cout << "Successfully inserted key " << key << endl;
```

The final step stitches the newly created node into the fabric of the existing list.

- A loop iterates from level 0 up to the node's **rlevel** (**i <= rlevel**).
- First, the new node's forward pointers take on the destinations previously held by the **update** array:
n->forward[i] = update[i]->forward[i].
- Next, the nodes stored in the **update** array drop their old connections and point directly to the new node:
update[i]->forward[i] = n.
- Once rewiring is complete, the program outputs a **success message and the insertion concludes**.

The Skip List's ``Skip`` mechanism in insertion

The skip in a Skip List happens during the very first phase of the insertion process: the search phase.

Before a new node can be inserted, the program must figure out exactly where it belongs. Instead of checking every single node one by one (like a standard linked list), it uses the upper levels to "skip" over large sections of the data.

Here is exactly where and how this skipping occurs in the provided program:

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

- **Starting in the Express Lane:** The skip is initiated by the loop `for (int i = lst->level; i ≥ 0; i--)`. The program explicitly states it will "Start from the highest current level and move down". By starting at `lst->level` rather than `level 0`, the pointer begins its search on the tallest available structural layer.
- **Skipping Nodes Horizontally:** Inside that for loop is the while loop: `while (current->forward[i] != NULL && current->forward[i]->key < key)`. When `i` is a high number (like 2 or 3), the command `current = current->forward[i]` moves the pointer to the next node on that specific upper level.

- **The Actual Skip:** Because nodes at higher levels are spread further apart, a single horizontal jump at **level 2 or 3** might physically bypass (or skip) multiple smaller nodes that only exist down on **level 0**.
- **Dropping Down to Fine – Tune:** When the pointer can no longer skip forward on the current high level (because the next node is either **NULL** or its **key** is larger than the **target key**), the **while** loop exits. The **for** loop then triggers **i – –**, causing the pointer to drop down to the next level to continue a more fine – grained search.

By repeatedly skipping horizontally on high levels and dropping down when necessary, the program rapidly homes in on the correct insertion spot without having to evaluate every single key in the list.

Note: –

```
int randomLevel() {
    int lvl = 0;
    // 50% probability to increase the level
    while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {
        lvl++;
    }
    return lvl;
}
```

Here, **MAX_LEVEL = 3**, according to the program:

say 1st, rand() = 42

1st loop run:

while $\left((rand() = 42 \% 2) = 0 == 0 \left[\frac{42}{2} = 0[Remainder] \right] [True] \right)$:
AND lvl = 0 < MAX_LEVEL = 3 [TRUE]

lvl = 0 + 1 = 1

2nd, rand() = 998

2nd loop run:

while $\left((rand() = 998 \% 2) = 0 == 0 \left[\frac{998}{2} = 0[Remainder] \right] [True] \right)$:
AND lvl = 1 < MAX_LEVEL = 3 [TRUE]

lvl = 1 + 1 = 2

3rd, rand() = 8470

3rd loop run:

while $\left((rand() = 8470 \% 2) = 0 == 0 \left[\frac{8470}{2} = 0[Remainder] \right] [TRUE] \right)$:
AND lvl = 2 < MAX_LEVEL = 3 [TRUE]

lvl = 2 + 1 = 3

4th, rand() = 1070

3rd loop run:

while $\left(\begin{array}{l} (rand() = 1070 \% 2) = 0 == 0 \left[\frac{1070}{2} = 0[Remainder] \right] [TRUE] \\ \text{AND } lvl = 3 < MAX_LEVEL = 3 [False] \end{array} \right)$

[While Loop Exits]

return, lvl = 3

∴ Maximum lvl or level or rlevel, can be `3` as in program, it is:

```
#define MAX_LEVEL 3
```

Skip List – Insert Element

–Time Complexity Analysis

Program:

```
void insertElement(struct SkipList *lst, int key) {
    struct Node *current = lst->header;

    // Array to keep track of the path we took (nodes needing pointer
    updates)
    struct Node *update[MAX_LEVEL + 1];
    memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));

    // Start from the highest current level and move down
    for (int i = lst->level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key <
key) {
            current = current->forward[i];
        }
        update[i] = current;
    }

    // Move to level 0 and look at the next node
    current = current->forward[0];

    // If the key doesn't already exist, insert it
    if (current == NULL || current->key != key) {
        int rlevel = randomLevel();

        // If the random level is greater than the list's current max
        level,
        // initialize the update array for the new levels to point to
        the header.
        if (rlevel > lst->level) {
            for (int i = lst->level + 1; i <= rlevel; i++) {
                update[i] = lst->header;
            }
            lst->level = rlevel;
        }
    }
}
```

```

    // Create the new node and rewire pointers
    struct Node *n = createNode(key, rlevel);
    for (int i = 0; i <= rlevel; i++) {
        n->forward[i] = update[i]->forward[i];
        update[i]->forward[i] = n;
    }
    cout << "Successfully inserted key " << key << endl;
} else {
    cout << "Key " << key << " already exists in the list." << endl;
}
}
int main() {
    // Seed random number generator for random levels
    srand((unsigned)time(0));
    struct SkipList *list = createSkipList();
    int choice, key;

    while (true) {

        cout << "\n==== Skip List Menu =====" << endl;
        cout << "1. Insert" << endl;
        cout << "Enter your choice: ";

        if (!(cin >> choice)) {
            cout << "Invalid input. Exiting." << endl;
            break;
        }

        switch (choice) {

        case 1:
            cout << "Enter key to insert: ";
            cin >> key;
            insertElement(list, key);
            break;
        }
    }
    return 0;
}

```

Time Complexity Analysis

1. Best Case Time Complexity: $O(1)$

The best – case scenario occurs when the element to be inserted is smaller than the very first element in the list (right after the header), or when the list is completely empty.

- *The traversal loop begins at the highest level but immediately hits NULL or a key larger than the target at every single level.*
- *The search halts immediately without horizontally traversing any nodes.*
- *Time Complexity: $O(1)$.*

True Absolute Best Case Scenario

For this to happen in a Skip List insertion, two specific conditions must be met simultaneously:

1. ***The insertion point is at the very front:*** *The list is either completely empty, or the key you are inserting is smaller than the very first element in the list.*
2. ***The coin flip fails immediately:*** *The `randomLevel()` function generates an `rlevel` of 0 on its very first try.*

1. Initialization

```
struct Node *current = lst->header;
struct Node *update[MAX_LEVEL + 1];
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

Behavior: This section always executes normally. The **update** array is allocated on the stack and zeroed out. Because **MAX_LEVEL** is a small constant, this takes a tiny, constant amount of time (**$O(1)$**).

2. The Search Phase (Ducking the Horizontal Traversal)

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

Recalling the **for loop** behavior:

$\text{for (int } i = \text{lst} \rightarrow \text{level}; i \geq 0; i--)$: This initiates a top – down search.

Instead of searching one by one from the bottom, it starts at the highest current level of the list (**$\text{lst} \rightarrow \text{level}$**) and drops down step – by – step to level 0 (**$i--$**).

This is what gives a skip list its speed, allowing it to use the upper levels as "express lanes" to skip over large chunks of data.

1st → Level 3

2nd → Level 2

3rd → Level 1

4th → Level 0

Goes top – down vertically.

while (current-> forward[i] != NULL && current-> forward[i]-> key < key):

At each level, the program looks ahead to the next node. It checks two critical things before moving:

1. *!= NULL: Does a next node actually exist, or have we reached the end of the list?*

2. *→ key < key: Is the next node's key smaller than the one we are trying to insert?*

- *current = current → forward[i]; If the next node exists and has a smaller key, the current pointer moves horizontally to the right. This while loop continues moving the pointer rightward until it hits a node that is larger, or hits the end of the list.*

- *update[i] = current; This is the most crucial part for the actual insertion. When the current pointer can no longer move forward on level i, it stops. The program then records this exact node into the update array (update[i] = current) before the for loop forces the pointer to drop down to the next level.*

Hence, while (current-> forward[i] != NULL && current-> forward[i]-> key < key): moves horizontally.

Now coming back to the Time Complexity:

Behavior: The **outer for** loop must still run to drop down from the **top level to level 0**. However, the **inner while loop** is completely ducked.

- Because the new key belongs at the very front of the list, the condition **$current \rightarrow forward[i] \neq NULL \ \&\& \ current \rightarrow forward[i] \rightarrow key < key$** evaluates to **False** on the very first check at every single level.
- The pointer **current** never moves horizontally. Instead, the **update** array at every level **i** is simply populated with the address of the **header** node.

The Inner while Loop is 100% Ducked: This is the horizontal traversal loop : **$while (current \rightarrow forward[i] \neq NULL \ \&\& \ current \rightarrow forward[i] \rightarrow key < key)$** . Because the next node is either **NULL** (empty list) or its key is larger than our target key, this condition evaluates to **False** on the very first check at every single level. The program entirely bypasses the inside of this loop, meaning zero horizontal steps are taken.

If inserting 20 before 50: The telescope sees the node with 50. Because 50 is not less than 20, the condition fails immediately. **$current \rightarrow forward[i] \rightarrow key$** is the Next Node's Key and **Key** is our target key, eg: Next Node's Key : 50 < Key : 20 , the condition sets to false.

The Outer for Loop MUST Still Run: This is the vertical drop-down loop: **$for (int i = lst \rightarrow level; i \geq 0; i--)$** .

The program cannot duck this loop. Even though no horizontal movement is needed, the program must still drop down level by level to populate the **update** array with the header node's address. It has to tell the update array, "Hey, at level 3, the predecessor is the header. At level 2, the predecessor is the header," and so on, so that the new node can be wired correctly.

Note: *MAX_LEVEL* is constant and *lst* $\rightarrow$ *level* is also a constant and

lst $\rightarrow$ *level* $\leq$ *MAX_LEVEL* ≤ 3 , hence for loop runs from *lst* $\rightarrow$ *level* to 0 takes constant amount of time, *O(c)* or *O(1)*.

3. Duplicate Key Validation

```
// Move to level 0 and look at the next node
current = current->forward[0];

// If the key doesn't already exist, insert it
if (current == NULL || current->key != key) {
```

Behavior: *current* moves one step forward on level 0.

Because the list is empty (so *current* is *NULL*) or the new key is smaller than the first key (so *current* $\rightarrow$ *key* $\neq$ *key*), this if condition evaluates to *True*, and the program proceeds to insert.

4. Level Generation

```
int rlevel = randomLevel();
```

Behavior: In this absolute best – case scenario, the random coin flip inside *randomLevel()* fails on its very first try. The function immediately breaks its internal loop and returns an *rlevel* of 0. This takes instantaneous *O(1)* time.

```
int randomLevel() {
    int lvl = 0;
    // 50% probability to increase the level
    while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {
        lvl++;
    }
    return lvl;
}
```

rand() $\% 2 \neq 0$, hence, while loop condition fails and return *lvl* = 0.
hence, *rlevel* = *randomLevel()* = 0.

5. Height Adjustment (Completely Skipped)

```
if (rlevel > lst->level) {
    for (int i = lst->level + 1; i <= rlevel; i++) {
        update[i] = lst->header;
    }
    lst->level = rlevel;
}
```

- **Behavior:** Because our lucky *rlevel* is 0, and the list's current level (*lst->level*) is at least 0, the condition (*0 > lst->level*) evaluates to False.
- This entire block of code – including the *for* loop and the reassignment of *lst->level* – is completely skipped, saving execution time.

6. Node Creation and Rewiring

```
struct Node *n = createNode(key, rlevel);
for (int i = 0; i <= rlevel; i++) {
    n->forward[i] = update[i]->forward[i];
    update[i]->forward[i] = n;
}
```

- **Behavior:** *createNode* allocates memory for the new node with an array size supporting just *level 0*.
- Because *rlevel* is 0, the rewiring *for* loop runs exactly one time (*for i = 0*).
- It takes the *update[0]* address (which we know is the *header*), points the new node's level 0 pointer to whatever the header was pointing to, and then points the header's level 0 pointer to the new node.

The loop then terminates instantly.

In the true absolute best – case scenario, the program effortlessly bypasses the horizontal while loop entirely, skips the height adjustment if block completely, and executes the final rewiring for loop exactly once. The entire insertion collapses into an ultra – efficient sequence of constant – time $O(1)$ operations with the absolute bare minimum number of CPU instructions.

2. Worst Case Time Complexity

The worst – case scenario is a structural failure of the skip list's probabilistic balancing.

Imagine a situation where the random number generator consistently flips tails(fails to promote)for every single node inserted, meaning every node in the list only exists at level 0.

- *The skip list devolves into a standard, **standard singly linked list**.*
- *If we want to insert a key at the very end of this list, the search algorithm must start at the header and visit every single node one by one at level 0.*
- **Math:** *If we insert an element that belongs at the very end of this list, the algorithm has no upper levels to skip across. It must evaluate every single node one by one. For n elements, this requires n comparisons.*
- **Result:** *The complexity degrades to $O(n)$.*

Worst Case Analysis

1. Initialization

```
struct Node *current = lst->header;
struct Node *update[MAX_LEVEL + 1];
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

Behavior: This setup phase behaves exactly as it normally would.

The **update** array is allocated to hold 4 elements (since **MAX_LEVEL** is 3) and is wiped clean using `memset` to ensure no wild pointers exist.

2. The Search Phase (The $O(n)$ Bottleneck)

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

- **Behavior:** This is where the disaster happens. Because every previously inserted node had a level of 0, the skip list's overall maximum height (`lst->level`) never grew. It remains at 0.
- Consequently, the `for` loop executes exactly once (`for i = 0`). There are no "express lanes" to drop down through.
- The **inner while** loop must now do all the heavy lifting. The condition `current->forward[0] != NULL` ensures the pointer moves until it hits the end of the list.

- Because the new target **key** belongs at the very end, the condition **current → forward[0] → key < key** remains **True** for every single node currently in the list.
- The command **current = current->forward[0]** executes **n** times, forcing the pointer to physically step through all **n** nodes **one by one**. This single line is what degrades the time complexity to **O(n)**.
- Finally, the **loop halts** when it hits **NULL**, and **update[0]** records the very last node in the list.

3. Validation

```
current = current->forward[0];

// If the key doesn't already exist, insert it
if (current == NULL || current->key != key) {
```

- **Behavior:** After traversing the entire list, stepping forward one final time makes **current** step off the **last node** and land exactly on **NULL**.
- The **if** condition evaluates (**current == NULL**). Since this is **True**, the program validates that the key does not already exist and proceeds to insert it.

4. Level Generation and Height Adjustment

```
int rlevel = randomLevel();

// If the random level is greater than the list's current max
level,
// initialize the update array for the new levels to point to
the header.
if (rlevel > lst->level) {
    for (int i = lst->level + 1; i <= rlevel; i++) {
        update[i] = lst->header;
    }
    lst->level = rlevel;
}
```

- **Behavior:** The program rolls the dice for the new node.
- If this new node finally gets lucky and generates an **rlevel** greater than 0 (e.g., **rlevel = 2**), the if block executes.
- The **update** array slots for **levels 1 and 2** are anchored directly to the **lst → header**, and the global list height is finally raised, effectively fixing the "**flat list**" problem for future insertions!

5. Node Creation and Rewiring

```
struct Node *n = createNode(key, rlevel);
for (int i = 0; i <= rlevel; i++) {
    n->forward[i] = update[i]->forward[i];
    update[i]->forward[i] = n;
}
```

- **Behavior:** The new node is created in memory.
- The rewiring loop executes. For $i = 0$, it takes the pointer from `update[0]` (which we recorded earlier as the former last node of the list) and points it directly to our newly created node. The new node is successfully appended to the absolute tail end of the list.

For new $rlevel$ (eg: $rlevel = 2$), it goes like this : –

1. Declaring the New Height

In the first block you shared, the program realizes that the new node's random level ($rlevel = 2$) is bigger than the list's current maximum level ($lst \rightarrow level = 0$).

- It executes $lst \rightarrow level = rlevel$; which officially upgrades the entire skip list from a flat 1 – story road to a 3 – story building (Levels 0, 1, and 2).

2. Setting Up the "Starting Line"

Because Levels 1 and 2 are brand new, there are no existing nodes on these "upper floors" to connect to.

- The *if* block handles this by setting `update[1] = lst->header` and `update[2] = lst->header`.
- This tells the program: "For Levels 1 and 2, the only thing that exists to connect to so far is the starting gate (the header node)."

3. Building the Tall Node

The program calls `struct Node *n = createNode(key,rlevel);`.

- This creates a physical node in memory that is tall enough to hold three forward pointers (one for Level 0, one for Level 1, and one for Level 2).

4. Stringing the Pointers (The Magic)

The final *for* loop is where the new levels are actually wired into existence:

- At Level 0 (*i* = 0): The new node links up to the old "tail end" of the list, just like a normal linked list addition.
- At Level 1 (*i* = 1): The program looks at `update[1]`, which we explicitly set to the *header*. It takes the header's Level 1 pointer and strings it all the way across the list, skipping over all the short, flat nodes, and attaches it directly to our newly created tall node.

Dynamic Node Sizes: Unlike a standard linked list where every single node is the exact same size, nodes in a Skip List are dynamically sized based on their randomly assigned level (*rlevel*).

A Short Node (Level 0): If a node rolls an *rlevel* of 0, the *createNode* function allocates an array with only 1 pointer slot (*forward[0]*). Because it only has one connection pointing horizontally along the bottom floor, we can visualize this node as being "short" or "flat."

A Tall Node (Level 2 or 3): If a node gets lucky and rolls an *rlevel* of 2, the *createNode* function allocates an array with 3 pointer slots (*forward[0]*, *forward[1]*, and *forward[2]*). Because this node has to hold connections for the *ground floor* and the *upper express lanes*, it physically takes up more memory.

- **At Level 2 ($i = 2$):** It does the exact same thing. It takes the header's **Level 2 pointer** and strings a massive bridge directly to the new node.

By the time this loop finishes, Level 1 and Level 2 officially exist!

They act as giant express lanes bridging directly from the starting header to our newly inserted node at the very end of the list.

"Because every node is generated with a height of 0, the structural advantage of the skip list is completely lost. The array of forward pointers inside every node contains only a single valid link (at level 0) pointing to the immediate next node. The algorithm is forced to traverse a single, flat chain of nodes one by one."

Alternative Worst Case

The "All Heads" Scenario (The Express Lane Traffic Jam)

The exact opposite of the all tails scenario can also destroy the efficiency of a Skip List.

- *Imagine the random number generator is completely broken in the other direction, consistently flipping **heads** and promoting every single node to the absolute maximum level (e.g., **MAX_LEVEL = 3**).*
- *In this scenario, every single node in the list has a pointer at **level 3**.*

- When we search for an item, the algorithm starts at the highest level (*1st → level, which is 3*). Because every node exists on **level 3**, the algorithm cannot skip over anything. It is forced to traverse **horizontally** across every single node on the top floor.
- The **express lane** is completely clogged, turning the top level into a **standard singly linked list**. The performance degrades to **$O(n)$** , proving that some arrangements of randomly generated levels will give poor execution times.

The above alternative worst case explanation as follows:

The "Express Train" Analogy

How a Skip List SHOULD Work:

- **Level 0 is the Local Train.** It stops at every single station (every node) on the track. It is slow, but it guarantees we don't miss your stop.
- **Level 3 is the Express Train.** It only stops at major city hubs. When we ride the Express Train, we bypass (or skip) dozens of tiny local stations, covering massive distances in a single jump.

How the All Heads Scenario Ruins It: Now, imagine a scenario where the random number generator promoted every single node to Level 3.

- In our analogy, this means the city planners accidentally upgraded every tiny local station into a major city hub.
- Because every single node is now a Level 3 station, the Express Train is forced to stop at Node 1, then Node 2, then Node 3, and so on.

The Code Mechanics Behind the Traffic Jam

Let's look at what the pointers are actually doing when this happens.

When you start your search, the for loop begins at the top floor: $i = 3$.

*The algorithm looks at the **current** $\rightarrow$ **forward[3]** pointer, hoping to make a massive jump across the list.*

- ***Normally:*** The **forward[3]** pointer of the header might point to the 8th node in the list. We just successfully skipped 7 nodes in a single step!
- ***The Worst Case:*** Because every node has a Level 3 pointer, the header's **forward[3]** pointer just points to Node 1. Node 1's **forward[3]** pointer points to Node 2. Node 2's **forward[3]** points to Node 3.

*The algorithm is trapped on the top floor, but the top floor has become exactly as crowded as the bottom floor. The search is forced to evaluate every single node one by one horizontally across Level 3, resulting in a painfully slow **$O(n)$** execution time.*

*Here trapping can mean **Dead Lock** situation, but its not:*

For example: –

1. Starting at the Top Floor ($i = 3$)

- The **for** loop starts at $i = 3$. The current pointer starts at the **header**.
- The **while** loop looks ahead at **Level 3**. It sees **Node 10**. Since **$10 < 100$** , it steps forward to **Node 10**.
- It looks ahead again on **Level 3**. It sees **Node 20**. Since **$20 < 100$** , it steps forward to **Node 20**.

- It looks ahead again on **Level 3**. It sees **Node 30**. Since **30 < 100**, it steps forward to **Node 30**.
- It looks ahead one last time on **Level 3**. It sees **NULL** (the end of the list). The **while** loop safely and successfully terminates.
- **Result:** The algorithm just did **O(n)** work crossing the entire list on the top floor.

2. Dropping Down (*i* = 2, 1, 0)

- Now the **for** loop drops down to **i = 2**.
- The **current** pointer is already sitting on **Node 30** at the very end of the list.
- The **while** loop looks ahead on **Level 2**. Because **Node 30** is the last node, **forward[2]** is **NULL**. The **while** loop terminates instantly with zero steps taken.
- This repeats for **Level 1** and **Level 0**. The **while** loop safely skips execution because we are already at the end of the line.

3. Finishing the Program

- The loop structure completes successfully.
- The program proceeds to create the new node for 100 and wires it into the list.

"Because every single node in the structure has been promoted to the maximum level, the forward pointers at the highest level can no longer 'skip' over anything. Instead of jumping across large sections of data, the top – level pointer of each node simply points to its immediate neighbor, forcing a linear O(n) traversal across the top floor."

Why It Is Not a Deadlock

A deadlock means the program physically cannot move to the next line of code. As the trace above shows, the algorithm never stops running. The **while** loop successfully processes **Node 10**, then **Node 20**, then **Node 30**, and then it correctly exits the loop. It just had to perform **n** horizontal comparisons on **Level 3** to get there.

Average Time Complexity

In a skip list, the total time to insert a new element is the sum of three distinct phases:

1. *The Search Phase: Finding the correct insertion point.*
2. *The Coin Flip Phase: Generating the random level for the new node.*
3. *The Splicing Phase: Rewiring the pointers.*

Phase 1: The Search Phase

Let n be the total number of elements in the skip list.

Let p be the probability that a node at level k is promoted to level $k + 1$ (typically, 50%, so, $p = 0.5$).

Step 1A: Calculate the Expected Maximum Height(H)

To know how far up we have to climb, we first need to determine the expected maximum height of the entire skip list.

- *The probability that any single node reaches level k is p^k .*
- *Let N_k be the number of nodes at level k . The expected number of nodes at level k across the whole list is:*

$$E[N_k] = n \times p^k$$

The maximum height **H** of the skip list is defined as the level where we expect to find exactly **1** node. Therefore, we set $E[N_k] = 1$:

$$n \times p^H = 1$$

Solving for **H**:

$$p^H = \frac{1}{n}$$

Putting $\log_p$ at both the side we get:

$$\log_p(p^H) = \log_p\left(\frac{1}{n}\right)$$

$$H = \log_p(n^{-1}) \left[\log_b x^y = y \times \log_b x, \log_b b = 1 \text{ and } \frac{1}{n} = n^{-1} \right]$$

$$H = -1 \times \log_p n \left[\log_b x^y = y \times \log_b x \right]$$

$$H = -\log_p n$$

To get rid of that negative sign, we use the Change of Base Formula

$\left(\log_b(x) = \frac{\log_c(x)}{\log_c(b)} \right)$ to change the base from **p** to $\frac{1}{p}$:

$$H = -\log_p(n) = -\left(\frac{\log_{\frac{1}{p}}(n)}{\log_{\frac{1}{p}}(p)} \right)$$

Look at the denominator : $\log_{\frac{1}{p}}(p)$. Because **p** is the exact

inverse of $\frac{1}{p}$ (meaning $p = \left(\frac{1}{p}\right)^{-1}$), this denominator evaluates

to **-1**.

$$H = -\log_p(n) = -\left(\frac{\log_{\frac{1}{p}}(n)}{\log_{\frac{1}{p}}\left(\frac{1}{p}\right)^{-1}} \right)$$

$$H = - \left(\frac{\log_{\frac{1}{p}}(n)}{-1 \times \log_{\frac{1}{p}}\left(\frac{1}{p}\right)} \right)$$

$$H = - \left(\frac{\log_{\frac{1}{p}}(n)}{-1} \right)$$

$$H = \log_{\frac{1}{p}}(n)$$

Alternative calculation:

$$p^H = \frac{1}{n}$$

$$\Rightarrow n = \frac{1}{p^H}$$

We can also write it as:

$$\Rightarrow n = \left(\frac{1}{p}\right)^H \quad [as, 1^H = 1]$$

*The fundamental definition of a logarithm is that if $b^y = x$, then $\log_b(x) = y$. Here our base $\left(\frac{1}{p}\right)$, our exponent is H , and our result is n . By simply rewriting it in **log** form, we instantly get:*

$$H = \log_{\frac{1}{p}}(n)$$

if $p = 0.5$, the expected maximum height is:

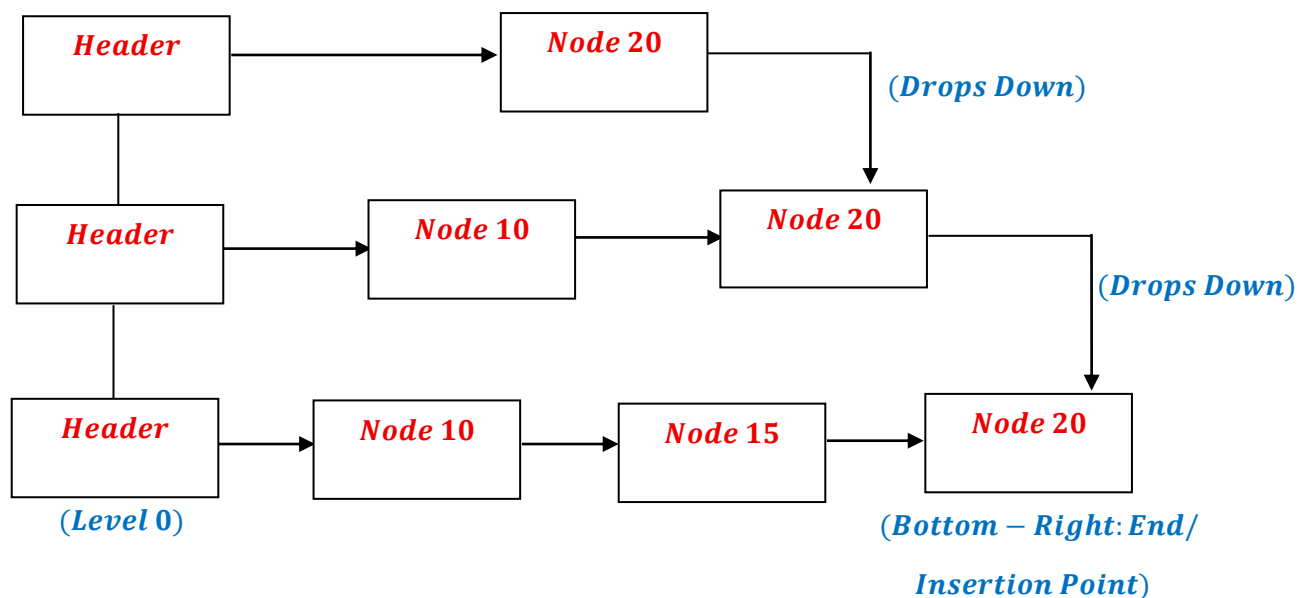
$$H = \log_{\frac{1}{0.5}}(n)$$

$$H = \log_{\frac{1}{\frac{1}{2}}}(n)$$

$$H = \log_2(n)$$

Step 1B : Calculate the expected Horizontal Steps per Level

Now, imagine walking backward from the bottom – right insertion point . At any given node on level `k`, there are only two ways the forward search could have reached it:



→ We place ourself at the final destination [Node 20] at Level 0.

- *Instead of moving Right, you move Left.*
- *Instead of dropping Down, you climb Up.*

The diagram illustrates the insertion of a new node into a B+ tree. The tree structure is as follows:

- Level 0 (Header):** Contains three **Header** nodes.
- Level 1:** Contains three nodes: **Node 10**, **Node 15**, and **Node 20**.
- Level 2:** Contains one node: **Node 20**.

The insertion process is shown in three stages:

- Searching:** The process starts at the **Header** node, moves down to **Node 10**, and then to **Node 15**. A green arrow labeled **Searching** indicates the path.
- Drops Down:** The process moves from **Node 15** to **Node 20**. A green arrow labeled **(Drops Down)** indicates the path.
- Moves Up:** The process moves from **Node 20** to **Node 15**. A green arrow labeled **Moves Up** indicates the path.

The final state shows the new node **Node 20** at the bottom right, labeled **(Bottom – Right: End/ Insertion Point)**. A green arrow labeled **Moves Up** indicates the path from the new node to the parent node **Node 15**.

- **Forward Search:** Moves RIGHT, then drops DOWN, then moves RIGHT, then drops DOWN...until it hits the bottom.
- **Reverse Analysis (Rewind):** Moves LEFT, then climbs UP, then moves LEFT, then climbs UP...until it hits the top – left start.

1. It dropped down from the above: This happens if the node exists at level $k + 1$. The probability of this p . In reverse, this means we move UP.

Scenario A: The Code Dropped Down (Reverse = Move UP)

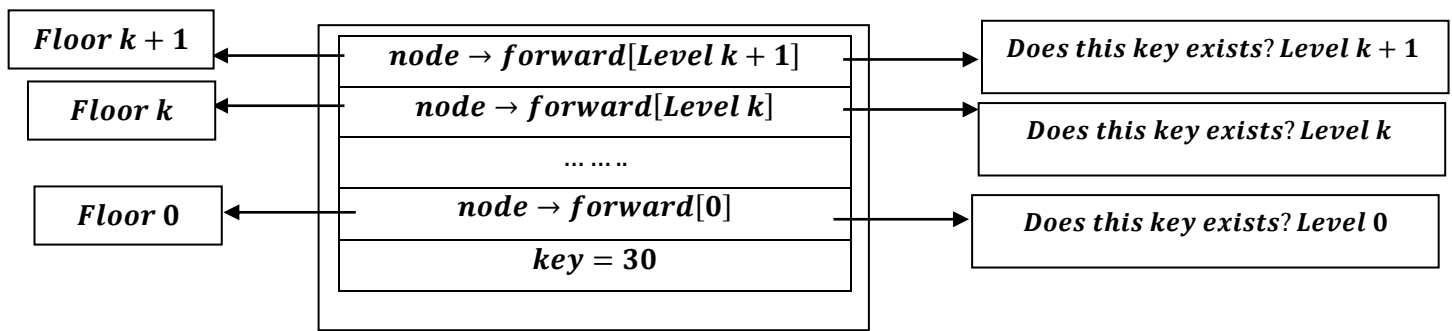
```
struct Node *current = lst->header;
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    ...
}
```

- **The Code:** Imagine the node we are standing on was lucky during `randomLevel()` and was built with a level of $k + 1$ or higher. Because this node physically exists on the floor above us, our for loop ($i--$) must have used this exact node to drop down from level $k + 1$ to level k .
- **The Reverse Math:** Since the forward code dropped DOWN at this node, our backward analysis climbs UP to level $k + 1$.
- **The Probability:** What are the odds that `randomLevel()` gave this node a height of at least $k + 1$? Exactly p (which is 50% in our code).

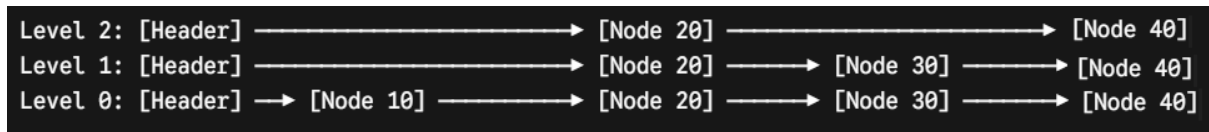
```
#define MAX_LEVEL 3
```

```
lst->header = createNode(-1, MAX_LEVEL);
```

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));
```



if



Then, **level 2** is here **level $k+1$** .

Normal Forward Search – Which is defined by Program

Step 1: Forward Code Execution

1. Iteration $i = 2$ (Level 2)

- **current starts at Header.**
- **while loop runs:**
 - **Header $\rightarrow$ forward[2] is Node 20 ($20 < 35$).current becomes Node 20 (Moved RIGHT).**
 - **Node 20 $\rightarrow$ forward[2] is Node 40 ($40 < 35$ is False).while loop stops.**
- **update[2] = Node 20.**

- The for loop executes $i--$:
 - i becomes 1.
 - Notice: **current** is still **Node 20**. The pointer did not move to another node; only the floor i changed from 2 to 1.
 - This is the vertical drop from (**Node 20, Level 2**) → (**Node 20, Level 1**).

2. Iteration $i = 1$ (Level 1)

- Entering with: **current** = **Node 20**, $i = 1$.
- while loop runs:
 - **Node 20** → **forward[1]** is **Node 30** ($30 < 35$). **current** becomes **Node 30** (**Moved RIGHT**).
 - **Node 30** → **forward[1]** is **Node 40** ($40 < 35$ is **False**). while loop stops.
- **update[1]** = **Node 30**.
- The for loop executes $i--$:
 - i becomes 0.
 - Notice: **current** is still **Node 30**.
 - This is the vertical drop from (**Node 30, Level 1**) → (**Node 30, Level 0**).

3. Iteration $i = 0$ (Level 0)

- Entering with: **current** = **Node 30**, $i = 0$.
- while loop runs:
 - **Node 30** → **forward[0]** is **Node 40** ($40 < 35$ is **False**). while loop stops immediately.
- **update[0]** = **Node 30**.

Step 2: Now Look at the Backward Path

Look at what the code recorded in **update[]**:

- **update[2]** = **Node 20**
- **update[1]** = **Node 30**
- **update[0]** = **Node 30**

*In program , it runs strictly forward search (Top – left to Bottom right),
program doesn't run from Bottom ,step – up i. e. Rewind or Backward Walk
, Backward Walk is only done for mathematical purpose.*

Let's rewind state – by – state starting from Level 0:

Rewind from (Node 30, Level 0):

- *The state immediately before (Node 30, $i = 0$) was (Node 30, $i = 1$).*
- *Rewinding: The same node at the previous level $\Rightarrow$
Step UP to (Node 30, Level 1).*

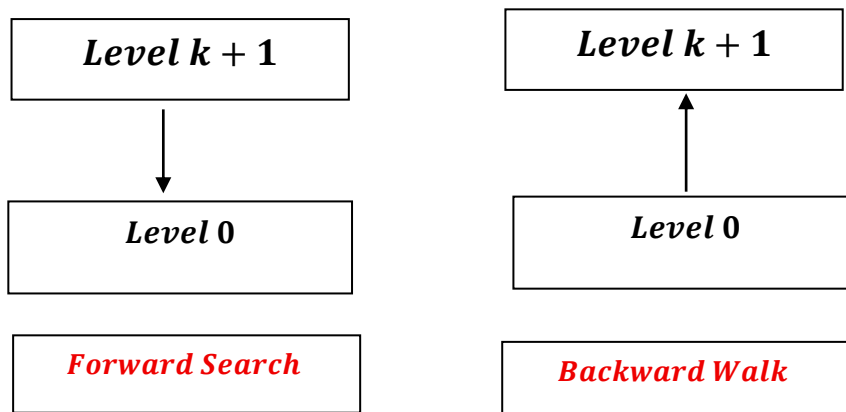
Rewind from (Node 30, Level 1):

- *The state immediately before was (Node 20, $i = 1$).*
- *Rewinding: A different node on the same level
 $\Rightarrow$ Step LEFT to (Node 20, Level 1).*

Rewind from (Node 20, Level 1):

- *Node 20 exists on Level 2 ($update[2] == \text{Node 20}$).*
- *Rewinding: $\Rightarrow$ Step UP to (Node 20, Level 2).*

*That is : **Backward Walk** ,also walks from Level 0 $\rightarrow$ Level $k + 1$.*



Phase 1: The Search Phase

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

Phase 2: The Coin Flip & Insertion Phase

```
if (current == NULL || current->key != key) {
    int rlevel = randomLevel(); // <--- 1. THE COIN FLIP HAPPENS
    HERE
    ...
    // 2. CREATE AND ATTACH POINTERS FOR EACH LEVEL REACHED
    struct Node *n = createNode(key, rlevel);
    for (int i = 0; i <= rlevel; i++) {
        n->forward[i] = update[i]->forward[i];
        update[i]->forward[i] = n;
    }
}
```

How randomLevel() works under the likelihood:

```
int randomLevel() {
    int lvl = 0;
    // 50% probability to increase the level
    while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {
        lvl++;
    }
    return lvl;
}
```

<i>Returned lvl</i>	<i>Formula</i>	<i>Reason</i>	<i>Probability</i>
<i>lvl = 0</i>	<i>$1 - p$</i>	<i>rand()%2 = 1, very first coin flipped failed.</i>	<i>50% = 0.5</i>
<i>lvl = 1</i>	<i>$p(1 - p)$</i>	<i>rand()%2 = 1, at 2nd time coin flipped failed.</i>	<i>50% × 50% = 0.5² = 0.25 = 0.25 × 100 = 25%</i>
<i>lvl = 2</i>	<i>$p^2(1 - p)$</i>	<i>rand()%2 = 1, at 3rd time coin flipped failed.</i>	<i>50% × 50% × 50% = 0.5³ = 0.125 = 0.125 × 100 = 12.5%</i>
<i>lvl = 3</i>	<i>$p^3(1 - p)$</i>	<i>rand()%2 = 1, at 4th time coin flipped failed.</i>	<i>50% × 50% × 50% × 50% = 0.5⁴ = 0.0625 = 0.0625 × 100 = 6.25%</i>

When **Coin** flipped failed , it generates **Tails**(**rand()%2 = 1**) i. e. **1 - p**

When **Coin** flipped passed , it generates **Head**(**rand()%2 = 0**) i. e. **p**

→ So, when returned level 0 , 1st flipped failed so : **1 - p**

→ When returned level 1 , 1st flipped passed generated `p` but 2nd flipped failed
generates `1 - p` ⇒ **$p(1 - p)$**

→ When returned level 2 , 2 times flipped passed generated `p` but 3rd flipped failed
generates `1 - p` ⇒ **$p^2(1 - p)$**

And so on

There fore ,we find out that:

→ *Probability of reaching Level 0: 100% [i. e. Reching Level 0 is guaranteed]*

→ *Probability of reaching Level 1: 50% (p)*

→ *Probability of reaching Level 2: 25% ($p \times p = p^2$)*

→ *Probability of reaching Level 3: 12.5% ($p \times p \times p = p^3$)*

Conditional Probability (When ALREADY standing on Level k)

During the search analysis,if we are already standing on a node at Level k .That means this node already had k successful coin flips in the past.

Now, : What is the chance this node also reaches Level $k + 1$?

Because coin flips are independent (memoryless), it only depends on one single next coin flip:

- *Next flip is Heads (50% or p): The node reaches Level $k + 1$.*

How p ? Now Mathematical terms:

$$\begin{aligned} \text{Pr}(\text{Reaches Level } k + 1 \mid \text{Already on Level } k) &= \frac{\text{Pr}(\text{Reaches Level } k + 1)}{\text{Pr}(\text{Reaches Level } k)} = \frac{p^{k+1}}{p^k} \\ &= p = 0.5 \end{aligned}$$

Where , p represents probability $\left(\text{i. e. } \frac{\text{Probability when reaches level } k + 1}{\text{Probability when we are Already standing at level } k} \right)$

$\therefore$ Reaching Level ' $k + 1$ ' ,while moving through horizontal step is: ' p '

2. It moved horizontally: This happens if the node doesnot exist at level ` $k + 1$ '. The probability of this ` $1 - p$ '. In reverse . this means we move LEFT .

This happens because , say,

→ we have **Level ` $k + 1$ `** , say **3** .

→ But **rand() %2** fails and gives us **Level ` $k$ `** , i. e. returns 2 ,
i. e. it will go at most **level 2** to insert our node.

Suppose **Node 30** was created , **randomLevel()** failed to promote it in **Level $k + 1$** (eg: **Level 3**).

As,

- Next flip is Tails (**50% or $1 - p$**): The node's tower stops at **Level k** .

Why **$1 - p$** ?

Due to calculation, when the probability that it stops at level k :

$$\text{Pr}(\text{Stops at Level } k \mid \text{Already on Level } k) = \frac{p^k(1 - p)}{p^k} = 1 - p = 0.5$$

Where , **pr** represents **probability** $\left(\text{i. e. } \frac{\text{Probability when stops at level } k}{\text{Probability when we are Already standing at level } k} \right)$

Why 50% or $1 - p$ on failing to reach Level $k + 1$ and why 50% or p on successful to reach Level $k + 1$ generates both 0.5 ?

Both generate 0.5 because standard Skip Lists are designed around a fair 50/50 coin toss, where p is chosen to be 0.5.

Here is the exact math and code reason:

1. The Rule of Total Probability

For any trial, there are only two possible outcomes that must add up to 1 (100%):

- *Success (Promoted to Level $k + 1$): Probability = p*
- *Failure (Stops at Level k): Probability = $1 - p$*

$$\text{Success} + \text{Failure} = p + (1 - p) = 1$$

2. Why Both Equal 0.5

Because we explicitly chose $p = 0.5$ (50% chance):

- *Success:*

$$p = 0.5 \quad (50\%)$$

- *Failure:*

$$1 - p = 1 - 0.5 = 0.5 \quad (50\%)$$

0.5 is the only number in mathematics where the probability of success equals the probability of failure ($p = 1 - p$).

3. In our Code (`rand() % 2`)

In the line:

```
while ((rand() % 2) == 0 && lvl < MAX_LEVEL)
```

- *Even numbers (0): `rand()` produces an even number half the time $\Rightarrow p = 0.5$ (Success / Move UP).*
- *Odd numbers (1): `rand()` produces an odd number half the time $\Rightarrow 1 - p = 0.5$ (Failure / Move LEFT).*

4. What If a Different Probability Was Used?

If we designed the Skip List with a 1 – in – 4 chance ($p = 0.25$):

- *Success (p): 0.25 (25%) $\rightarrow$ Move UP*
- *Failure ($1 - p$): $1 - 0.25 = 0.75$ (75%) $\rightarrow$ Move LEFT*

They are only both 0.5 when a standard fair 50/50 coin toss is used.

When we analyze **BACKWARD** (Reverse):

- *Whether a step goes UP or LEFT depends only on the coin flip*

(`randomLevel()`) of the node you are standing on:

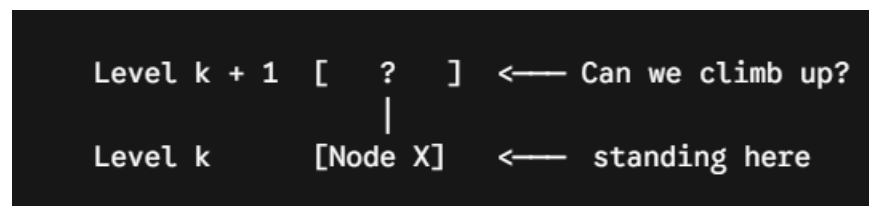
- *Promotion succeeded? $\rightarrow$ Move UP (probability p).*
- *Promotion failed? $\rightarrow$ Move LEFT (probability $1 - p$).*
- *It does not depend on the keys or the target at all.*
- *Every step is an independent coin toss, allowing us to directly use the simple Geometric Distribution formula $\left(\mathbb{E}[X] = \frac{1}{p}\right)$.*

Why Move Left ?

1. What we check while standing at Node X on Level k?

Imagine we are rewinding the search path and you are currently standing at Node X on Level k.

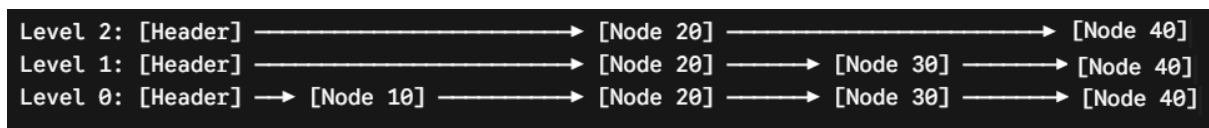
Now Question: "Is there a floor above me on Node X (Level k + 1)?"



2. If Promotion FAILED ($1 - p$):

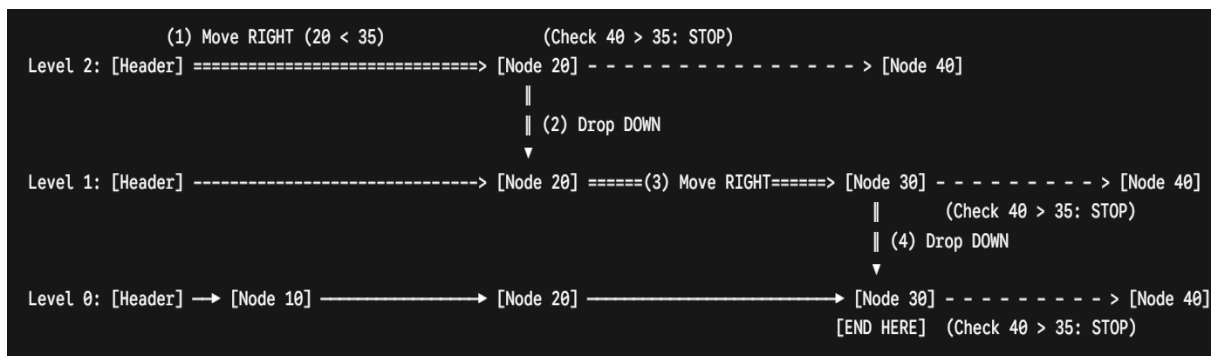
- *The fact: Node X was not promoted to Level k + 1.*
There is no floor above on Node X.
- *The deduction: Since there is no floor above on Node X, the forward search could not have dropped down from above into Node X.*
- *The only possibility: The forward search had to arrive by stepping RIGHT from its left neighbor (Node W) on Level k.*
- *The reverse action: To rewind that step, we must move LEFT to Node W.*

Lets take example of:



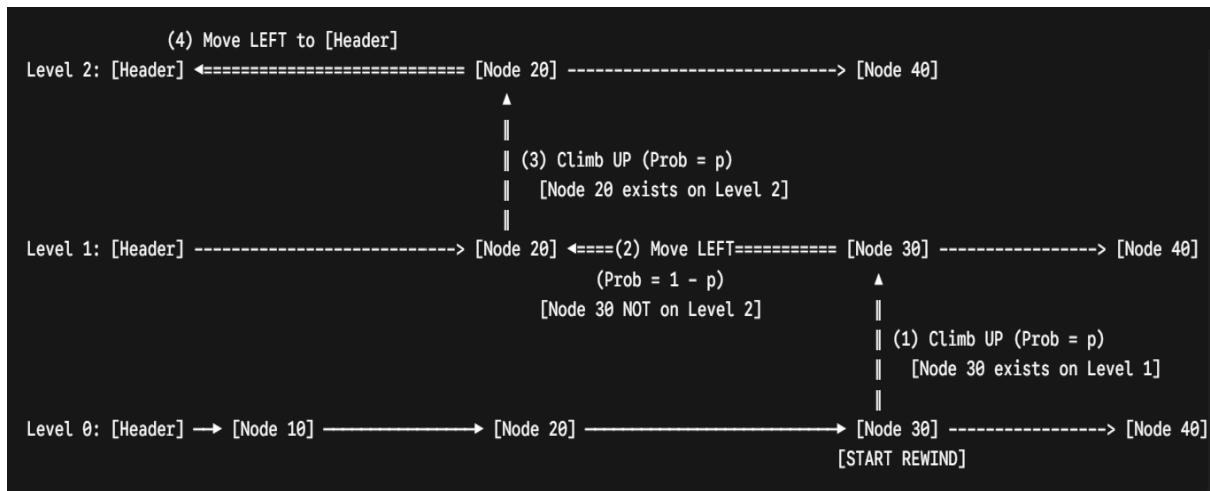
Part 1: The Forward Search (Start → End)

1. ***At Level 2:*** Start at [Header].
 - Move **RIGHT** to [Node 20] ($20 < 35$).
 - Look ahead to [Node 40]: $40 > 35$ (too big).
 - **Drop DOWN** at Node 20 to **Level 1**.
2. ***At Level 1:*** You are at [Node 20].
 - Move **RIGHT** to [Node 30] ($30 < 35$).
 - Look ahead to [Node 40]: $40 > 35$ (too big).
 - **Drop DOWN** at Node 30 to **Level 0**.
3. ***At Level 0:*** You are at [Node 30].
 - Look ahead to [Node 40]: $40 > 35$. Cannot move right.
 - **Search ends at (Node 30, Level 0).**



Part 2: The Backward Walk (End → Start)

Now we retrace this exact path in reverse, starting from (Node 30, Level 0):



Step 1: Standing at (Node 30, Level 0) ($k = 0$)

- **Ask:** Does Node 30 exist on Level 1 ($k + 1$)?
- **Look at the diagram:** YES, [Node 30] has a box on Level 1 (its coin flip succeeded with probability p).
- **Decision:** Climb UP $\Rightarrow$ Now at (Node 30, Level 1).

Step 2: Standing at (Node 30, Level 1) ($k = 1$)

- **Ask:** Does Node 30 exist on Level 2 ($k + 1$)?
- **Look at the diagram:** NO, look above Node 30 on Level 2 – there is no box there! (Its coin flip to Level 2 failed with probability $1 - p$).
- **Decision:** Since there is no floor above to climb, you must move **LEFT** on Level 1 $\Rightarrow$ Now at (Node 20, Level 1).

Step 3: Standing at (Node 20, Level 1) ($k = 1$)

- **Ask:** Does Node 20 exist on Level 2 ($k + 1$)?
- **Look at the diagram:** YES, [Node 20] has a box on Level 2 (its coin flip succeeded with probability p).
- **Decision:** Climb UP $\Rightarrow$ Now at (Node 20, Level 2).

Step 4: Standing at (Node 20, Level 2) ($k = 2$)

- Move **LEFT** to [Header] on Level 2.
- **Rewind complete!** We reached the top – left starting position.

Things we must remember of this backward walk:

*A **purely theoretical mathematical proof technique** on paper used to analyze and calculate expected search complexity .It is never an actual function or algorithm executed by the computer.*

RandomLevel() viewed in program code execution vs Average case math.

1. In the Program (Once per Node)

- When a node is created, `randomLevel()` is called **only once**.
- That single call flips the coins and sets the permanent height (`rlevel`) for that node.
- Once inserted, the node's height never changes.

2. In the Average – Case Analysis (The Math)

Because the program uses a randomized algorithm:

- Every node's height was determined by its own independent call to `randomLevel()`.
- When we calculate the **average (expected) search time** across all nodes in the list:
 - At any arbitrary node on Level k , the probability that its past `randomLevel()` call reached Level $k + 1$ is $p = 0.5$ (Move **UP**).
 - The probability that its `randomLevel()` stopped at Level k is $1 - p = 0.5$ (Move **LEFT**).

<i>Perspective</i>	<i>How randomLevel() is Viewed</i>
<i>Code Execution</i>	<i>Called once per node during <code>insertElement()</code> to allocate its pointers.</i>
<i>Average – Case Math</i>	<i>Because every node had independent coin flips, each backward step has a 50% chance to go UP and a 50% chance to go LEFT, yielding the Geometric Distribution ($\mathbb{E}[\text{steps}] = 1/p = 2$).</i>

When Node is Inserted



randomLevel() flips coins



Decides the Node's Height (How many boxes it gets)



During Backward Analysis:

- Does it have a box above? (Coin flip was Heads) → Go UP
- No box above? (Coin flip was Tails) → Go LEFT

*We want to find the expected number of leftward steps we must take on any given level before we are finally allowed to move UP. This is a classic **Geometric Distribution**(flipping a biased coin until we get our first `success`).*

*The expected number of trials **$E[X]$** to get the first success(moving UP) in a Geometric Distribution is:*

$$E[X] = \frac{1}{p}$$

How?

First way $\rightarrow$ Through Infinite $\left[\begin{array}{c} \text{Arithmetic Geometric} \\ \text{Progression,} \\ \text{[An Infinite Geometric Series]} \end{array} \right]$

Let X be the number of steps until the first success(moving UP with probability p).

Let $q = 1 - p$ be the probability of a failure (moving Left)

Earlier we have seen During Node Creation and Add Levels :

$\rightarrow$ When returned level 1 , 1st flipped passed generated ` $p$ ` but 2nd flipped failed generates ` $1 - p$ ` $\Rightarrow p(1 - p)$

$\rightarrow$ When returned level 2 , 2 times flipped passed generated ` $p$ ` but 3rd flipped failed generates ` $1 - p$ ` $\Rightarrow p^2(1 - p)$

.

.

$\rightarrow$ When returned level k , k times flipped passed generated ` $p$ ` but $k + 1$ flipped failed generates ` $1 - p$ ` $\Rightarrow p^k(1 - p)$

But during our Backward Search :

Finish in 1 step:

1^{st} node can go UP $\Rightarrow P$

Finish in 2 step:

1^{st} node went LEFT $(1 - p) \times 2^{nd}$ node goes UP $[p] \Rightarrow (1 - p)p$ or qp

Finish in 3 step:

1^{st} LEFT $(1 - p) \times 2^{nd}$ LEFT $(1 - p) \times 3^{rd}$ UP $[p] \Rightarrow (1 - p)^2 p$ or $q^2 p$

Finish in k step:

$(k - 1)$ LEFT moves by 1 UP move $\Rightarrow (1 - p)^{k-1} p$ or $q^{k-1} p$

<i>Process</i>	<i>What keeps it going?</i>	<i>What causes it to STOP?</i>	<i>Formula for step /level k</i>
1. randomLevel() (Node Creation)	Heads $(p) \rightarrow$ keep adding levels	Tails $(1 - p)$ $\rightarrow$ stop adding levels	$p^k(1 - p)$
2. Backward Search (Level Traversal)	Tails $(1 - p)$ $\rightarrow$ keep moving LEFT	Heads (p) $\rightarrow$ STOP and climb UP	$(1 - p)^{k-1} p$

What Happens When $p = 0.5$?

When $p = 0.5$, then $(1 - p) = 0.5$ as well:

- **For randomLevel() at lvl = 1:**

$$p(1 - p) = 0.5 \times 0.5 = 0.25 \quad (25\%)$$

- *For Backward Search at step 2:*

$$(1 - p)p = 0.5 \times 0.5 = 0.25 \quad (25\%)$$

Because $p = 1 - p = 0.5$, the numerical values match, but the roles of Heads and Tails are flipped between the two processes.

→ *Success on 1st step: Probability = p (Value = 1)*

→ *Success on 2nd step: Probability = $q \cdot p$ (Value = 2)*

→ *Success on 3rd step: Probability = $q^2 \cdot p$ (Value = 3)*

→ *Success on k – th step: Probability = $q^{k-1} \cdot p$ (Value = k)*

By the definition of Expected Value $\mathbb{E}[X]$:

$$\mathbb{E}[X] = \sum_{k=1}^{\infty} k \times \Pr(X = k) = p(1 + 2q + 3q^2 + 4q^3 + \dots)$$

Solving the infinite Sum S :

$$\text{Let } S = 1 + 2q + 3q^2 + 4q^3 + \dots$$

Multiply both sides by q :

$$qS = q + 2q^2 + 3q^3 + \dots$$

Subtract qS from S :

$$\begin{aligned} S - qS &= (1 - 0) + (2q - q) + (3q^2 - 2q^2) + (4q^3 - 3q^3) + \dots \\ &= 1 + q + q^2 + q^3 + \dots \end{aligned}$$

Now, general infinite geometric series is:

$$a_1 + a_1r + a_1r^2 + a_1r^3 + \dots$$

$$S_{\infty} = \frac{a_1}{1-r} ; |r| < 1$$

Now , we have :

$$S - qS = 1 + q + q^2 + q^3 + \dots$$

$$S - qS = 1 + 1 \times q + 1 \times q^2 + 1 \times q^3 + \dots$$

$$\text{hence , } a_1 = 1$$

$$\text{common ratio} = r = \frac{\text{any term}}{\text{previous term}}$$

$$r = \frac{q}{1} = q; r = \frac{q^2}{q} = q; r = \frac{q^3}{q^2} = q \therefore r = q.$$

$$S_{\infty} = \frac{1}{1-q}$$

$$\text{or, } S - qS = S_{\infty} = \frac{1}{1-q}$$

$$S(1-q) = \frac{1}{1-q}$$

We, know , $q = 1 - p$, then : $1 - (1 - p) = 1 - 1 + p = p$.

$$S \times p = \frac{1}{p}$$

$$\Rightarrow S = \frac{1}{p^2}$$

Now we know:

$$\mathbb{E}[X] = \sum_{k=1}^{\infty} k \times \Pr(X = k) = p(1 + 2q + 3q^2 + 4q^3 + \dots)$$

$$\text{or, } \mathbb{E}[X] = \sum_{k=1}^{\infty} k \times \Pr(X = k) = p \times S$$

Now, substitute S back into $\mathbb{E}[X]$:

$$\mathbb{E}[X] = \sum_{k=1}^{\infty} k \times \Pr(X = k) = p \times \frac{1}{p^2} = \frac{1}{p}.$$

$$\text{Hence, } \mathbb{E}[X] = \frac{1}{p}.$$

2nd Way → First Step Analysis(Intuitive)

We can also solve it by looking at just the very first step:

- 1. With probability p , the first step succeeds (we move UP), We took 1 step.**
- 2. With probability $1 - p$, the first step fails (we move LEFT), We took 1 step, and from this new node, we still need an Expected $\mathbb{E}[X]$ steps.**

$$\mathbb{E}[X] = \text{Pr}(\text{Success}) \times (\text{Steps if Success}) + \text{Pr}(\text{Failure}) \times (\text{Steps if failure})$$

Let $\mathbb{E}[X]$ be the average total steps needed to get our first success (moving UP).

When we take the 1st step, only two mutually exclusive outcomes can happen:

1. Scenario A: The 1st Step is a Success (Probability = p)

- *We flipped Heads and moved UP on our very first try.*
- *The search on this level is finished !*
- *Total steps taken: Exactly 1 step.*
- *Contribution to expectation:*

$$p \times (1)$$

2. Scenario B: The 1st Step is a Failure (Probability = $1 - p$)

- *We flipped Tails and moved Left to the neighbor node.*
- *We have already used up 1 step.*
- *From this new neighbor node, because the coin flips have no memory(memoryless property), our expected number of remaining steps to reach the top is still $\mathbb{E}[X]$.*
- *Total Steps taken in this scenario:*

$$\underbrace{1}_{\substack{\text{1st Step} \\ \text{already} \\ \text{taken}}} + \underbrace{\mathbb{E}[X]}_{\substack{\text{Expected Steps} \\ \text{from the} \\ \text{new node}}} = 1 + \mathbb{E}[X]$$

- *Contribution to expectation:*

$$(\mathbf{1} - \mathbf{p}) \times (\mathbf{1} + \mathbb{E}[X])$$

Set up the equation:

$$\mathbb{E}[X] = \mathbf{p}(\mathbf{1}) + (\mathbf{1} - \mathbf{p}) \times (\mathbf{1} + \mathbb{E}[X])$$

Expand:

$$\mathbb{E}[X] = \mathbf{p} + (\mathbf{1} - \mathbf{p}) + (\mathbf{1} - \mathbf{p}) \times \mathbb{E}[X]$$

$$\mathbb{E}[X] = \mathbf{1} + (\mathbf{1} - \mathbf{p}) \times \mathbb{E}[X]$$

Subtract $(\mathbf{1} - \mathbf{p}) \mathbb{E}[X]$ from both sides:

$$\mathbb{E}[X] - \{(\mathbf{1} - \mathbf{p}) \mathbb{E}[X]\} = \mathbf{1} + \{(\mathbf{1} - \mathbf{p}) \mathbb{E}[X]\} - \{(\mathbf{1} - \mathbf{p}) \mathbb{E}[X]\}$$

$$\mathbb{E}[X] - \{(\mathbf{1} - \mathbf{p}) \mathbb{E}[X]\} = \mathbf{1}$$

$$\mathbb{E}[X] - \{\mathbb{E}[X] - \mathbf{p} \times \mathbb{E}[X]\} = \mathbf{1}$$

$$\mathbb{E}[X] - \mathbb{E}[X] + \mathbf{p} \times \mathbb{E}[X] = \mathbf{1}$$

$$\mathbf{p} \times \mathbb{E}[X] = \mathbf{1}$$

$$\mathbb{E}[X] = \frac{\mathbf{1}}{\mathbf{p}}$$

If $p = 0.5$, we expect to take exactly 2 steps (comparisons) per level before moving up to the next one.

When $p = 0.5$:

$$\mathbb{E}[X] = \frac{1}{0.5} = \frac{1}{\frac{1}{2}} = 2 \text{ steps.}$$

Extra Calculation:

1. Expected Pointers per Node (Memory Complexity)

When a node is created, how many pointers (forward[]) does it have on average?

- *Every node has Level 0: 1 pointer (always, probability = 1).*
- *It also gets Level 1 with probability: p .*
- *It also gets Level 2 with probability: p^2 .*
- *It also gets Level 3 with probability: p^3 , and so on.*

The expected total number of pointers in a node is the sum of this infinite geometric series:

$$\mathbb{E}[\text{Pointers per Node}] = 1 + p + p^2 + p^3 + \dots = \sum_{i=0}^{\infty} p^i = \frac{1}{1-p}$$

$$\text{if } p = 0.5 (50\%) = \frac{1}{1-0.5} = \frac{1}{1-\frac{1}{2}} = \frac{1}{\frac{1}{2}} = 2 \text{ pointers per node on average.}$$

$$\text{if } p = 0.25(25\%) = \frac{1}{1 - 0.25} = \frac{1}{1 - \frac{1}{4}} = \frac{1}{\frac{3}{4}} = \frac{4}{3} \\ \approx 1.33 \text{ pointers per node on average.}$$

2. Calculation of Expected strictly horizontal (left) steps per level:

1. Problem Setup and Random Variable Definition

Let H be the random variable for the number of strictly horizontal(leftward) steps taken on a level before moving UP.

- Probability of moving UP(Success): P
- Probability of moving Left (Failure and Step Left): $q = 1 - p$

The possible values for H are 0, 1, 2, 3,:

- $H = 0$ (0 left steps): The current node goes UP immediately $\Rightarrow Pr(H = 0) = p$
- $H = 1$ (1 left step): 1 left move, then 1 UP move $\Rightarrow Pr(H = 1) = (1 - p)p$ or qp
- $H = 2$ (2 left steps): 2 left moves, then 1 UP move $\Rightarrow Pr(H = 2) = (1 - p)^2p$ or q^2p .
- .
- .
- .
- $H = (k \text{ left steps})$: k left moves, then 1 UP move $\Rightarrow Pr(H = k) = (1 - p)^k p$ or $q^k p$

Method 1: Direct Infinite Series (AGP) Derivation

By the definition of mathematical expectation:

$$\mathbb{E}[H] = \sum_{k=0}^{\infty} k \cdot Pr(H = k)$$

Since the term for $k = 0$ is $0 \cdot Pr(H = k) = 0$, we can start sum at $k = 1$.

$$\mathbb{E}[H] = \sum_{k=1}^{\infty} k \cdot \text{Pr}(H = k)$$

$$\Rightarrow \mathbb{E}[H] = \sum_{k=1}^{\infty} k \cdot (q^k p)$$

$$\Rightarrow \mathbb{E}[H] = p \sum_{k=1}^{\infty} k q^k$$

Factor out one q :

$$\Rightarrow \mathbb{E}[H] = p \cdot q \sum_{k=1}^{\infty} k q^{k-1}$$

$$\text{Let } S = \sum_{k=1}^{\infty} k q^{k-1} = 1 + 2q + 3q^2 + 4q^3 + \dots$$

Multiply both sides by q :

$$qS = q + 2q^2 + 3q^3 + \dots$$

Subtract qS from S :

$$\begin{aligned} S - qS &= (1 - 0) + (2q - q) + (3q^2 - 2q^2) + (4q^3 - 3q^3) + \dots \\ &= 1 + q + q^2 + q^3 + \dots \end{aligned}$$

Now, general infinite geometric series is:

$$a_1 + a_1 r + a_1 r^2 + a_1 r^3 + \dots$$

$$S_{\infty} = \frac{a_1}{1 - r} ; |r| < 1$$

Now, we have :

$$S - qS = 1 + q + q^2 + q^3 + \dots$$

$$S - qS = 1 + 1 \times q + 1 \times q^2 + 1 \times q^3 + \dots$$

$$\text{hence, } a_1 = 1$$

$$\text{common ratio} = r = \frac{\text{any term}}{\text{previous term}}$$

$$r = \frac{q}{1} = q; r = \frac{q^2}{q} = q; r = \frac{q^3}{q^2} = q \therefore r = q.$$

$$S_{\infty} = \frac{1}{1 - q}$$

$$\text{or, } S - qS = S_{\infty} = \frac{1}{1 - q}$$

$$S(1 - q) = \frac{1}{1 - q}$$

$$\text{We, know, } q = 1 - p, \text{ then : } 1 - (1 - p) = 1 - 1 + p = p.$$

$$S \times p = \frac{1}{p}$$

$$\Rightarrow S = \frac{1}{p^2}$$

$$\text{Substitute } S \text{ back into } \mathbb{E}[H] :$$

$$\mathbb{E}[H] = p \cdot q \sum_{k=1}^{\infty} kq^{k-1}$$

$$\Rightarrow \mathbb{E}[H] = p \cdot q \times S$$

$$\Rightarrow \mathbb{E}[H] = p \cdot q \times \frac{1}{p^2}$$

$$\Rightarrow \mathbb{E}[H] = \frac{q}{p}$$

$$\Rightarrow \mathbb{E}[H] = \frac{1-p}{p}$$

Method 2: Relationship to Total Steps($X = H + 1$)

On any given floor during the backward walk:

- *We take some number of LEFT steps (H).*
- *We finish the floor by taking exactly 1 UP step.*

Therefore, the Total Steps(X) we took on that floor is always:

Total Steps(X) = (Horizontal Steps H) + (1 Vertical Step UP)

$$**X = H + 1**$$

Rearranging the equation to find H :

$$**H = X - 1**$$

<i>What happens on the floor</i>	<i>Sequence of Moves</i>	<i>Total Steps (X)</i>	<i>Horizontal Steps (H)</i>	<i>Check (H = X - 1)</i>
0 Left moves, then 1 UP	[UP]	$X = 1$	$H = 0$	$0 = 1 - 1$
1 Left move, then 1 UP	[LEFT, UP]	$X = 2$	$H = 1$	$1 = 2 - 1$
2 Left moves, then 1 UP	[LEFT, LEFT, UP]	$X = 3$	$H = 2$	$2 = 3 - 1$
3 Left moves, then 1 UP	[LEFT, LEFT, LEFT, UP]	$X = 4$	$H = 3$	$3 = 4 - 1$

In every single case, the number of LEFT steps is simply the total steps minus 1.

$\mathbb{E}$ simply means ``the average value``.

if a quantity H is always $X - 1$, then its average $\mathbb{E}[H]$ must also equal the average of $X - 1$:

$$\mathbb{E}[H] = \mathbb{E}[X] - 1$$

*Now we know from **Backward Walk**:*

$$\mathbb{E}[X] = \sum_{k=1}^{\infty} k \times \text{Pr}(X = k) = p(1 + 2q + 3q^2 + 4q^3 + \dots) = \frac{1}{p}$$

Now substituting this into our formula, we get:

$$\mathbb{E}[H] = \frac{1}{p} - 1$$

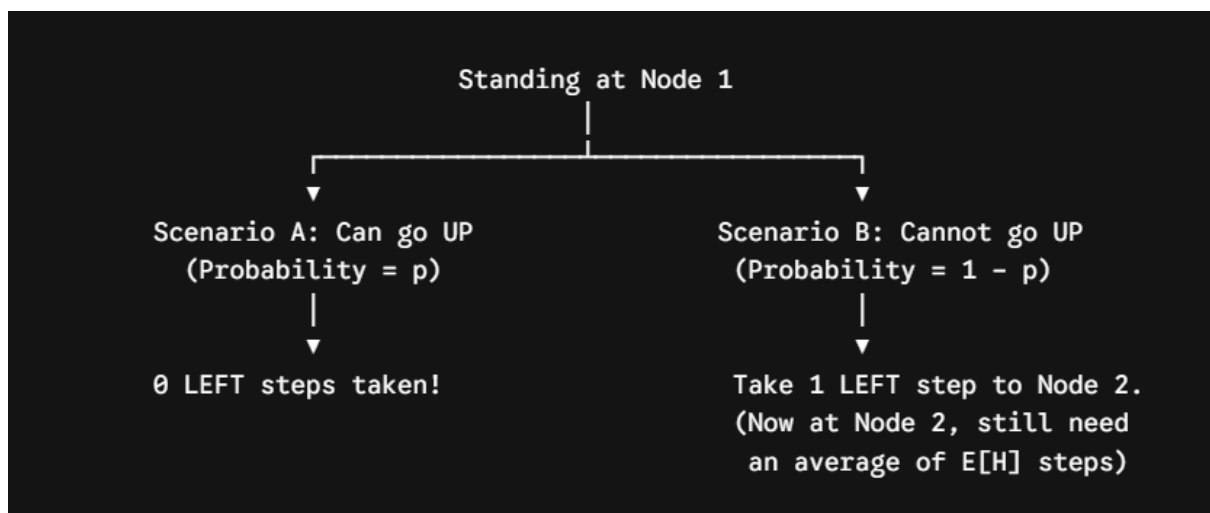
$$\mathbb{E}[H] = \frac{1}{p} - 1$$

$$\Rightarrow \mathbb{E}[H] = \frac{1-p}{p}$$

Method 3: First Step Analysis

Imagine → standing at the starting node on Level k.

if we look at this very first node, and only two mutually exclusive scenarios can happen:



Scenario A: The 1st node can go UP (Probability = p)

- *The node exists on the floor above .*
- *We climb UP immediately.*
- *How many LEFT steps did we take? Exactly 0 horizontal steps.*
- *Contribution to average:*

$$p \times (0)$$

Scenario B: The 1st node can go UP (Probability = $1 - p$)

- *The node does not exist on the floor above.*
- *We are forced to take 1 step LEFT to the neighbor node.*
- *Now we are standing at this new neighbor node. Because coin flips have no memory (memory less property), the expected number of remaining LEFT steps from this new node is still $\mathbb{E}(H)$.*
- *Total LEFT steps taken in this scenario:*

$$\begin{array}{ccc}
 1 & + & \mathbb{E}[H] \\
 \downarrow & & \downarrow \\
 \boxed{\begin{array}{c} \text{the 1 LEFT} \\ \text{step we just} \\ \text{took} \end{array}} & & \boxed{\begin{array}{c} \text{average LEFT} \\ \text{steps remaining} \\ \text{from new node.} \end{array}}
 \end{array}
 = 1 + \mathbb{E}[H]$$

- *Contribution to average:*

$$(1 - p) \times (1 + \mathbb{E}[H])$$

By the Law of Total Expectation, the overall average $\mathbb{E}[H]$ is the weighted sum of both scenarios:

$$\mathbb{E}[H] = \underbrace{p \cdot (0)}_{\text{Scenario A (Climb UP)}} + \underbrace{(1-p) \cdot (1 + \mathbb{E}[H])}_{\text{Scenario B (Step left)}}$$

$$\begin{aligned}\mathbb{E}[H] &= p \cdot (0) + (1-p) \cdot (1 + \mathbb{E}[H]) \\ \Rightarrow \mathbb{E}[H] &= 0 + (1-p) \cdot (1 + \mathbb{E}[H]) \\ \Rightarrow \mathbb{E}[H] &= (1-p) + \mathbb{E}[H](1-p) \\ \Rightarrow \mathbb{E}[H] &= (1-p) + \mathbb{E}[H] - p \cdot \mathbb{E}[H]\end{aligned}$$

Adding $p \cdot \mathbb{E}[H]$ on both the sides we get:

$$\begin{aligned}\Rightarrow \mathbb{E}[H] + p \cdot \mathbb{E}[H] &= (1-p) + \mathbb{E}[H] - p \cdot \mathbb{E}[H] + p \cdot \mathbb{E}[H] \\ \Rightarrow \mathbb{E}[H] + p \cdot \mathbb{E}[H] &= (1-p) + \mathbb{E}[H]\end{aligned}$$

Subtracting $\mathbb{E}[H]$ on both the sides we get:

$$\begin{aligned}\Rightarrow \mathbb{E}[H] + p \cdot \mathbb{E}[H] - \mathbb{E}[H] &= (1-p) + \mathbb{E}[H] - \mathbb{E}[H] \\ \Rightarrow p \cdot \mathbb{E}[H] &= (1-p) \\ \Rightarrow \mathbb{E}[H] &= \frac{1-p}{p}\end{aligned}$$

- *Total steps on average:* $\mathbb{E}[X] = \frac{1}{0.5} = \frac{1}{\frac{1}{2}} = 2$ *total steps* ([1 LEFT + 1 UP])
- *Horizontal Only Steps :* $\mathbb{E}[H] = \frac{1-0.5}{0.5} = \frac{1-\frac{1}{2}}{\frac{1}{2}} = \frac{\frac{1}{2}}{\frac{1}{2}}$
 $= 1$ *horizontal step on average per level* [1 LEFT]

Now coming to back to the calculation of Average Case Complexity:

Step 1C: Combine for Total Search Cost

The total expected number of steps in the search path is the number of levels multiplied by the expected steps taken on each level:

$$\text{Expected Search Cost} = H \cdot \mathbb{E}[X]$$

$$\text{Expected Search Cost} = \left(\log_{\frac{1}{p}} n \right) \cdot \left(\frac{1}{p} \right)$$

Evaluating for $p = 0.5$:

$$\begin{aligned} &= \left(\log_{\frac{1}{0.5}} n \right) \cdot \left(\frac{1}{0.5} \right) \\ &= \left(\log_{\frac{1}{\frac{1}{2}}} n \right) \cdot \left(\frac{1}{\frac{1}{2}} \right) \\ &= (\log_2 n) \cdot (2) \end{aligned}$$

Because $\frac{1}{p}$ is a constant factor, we drop it for asymptotic notation, leaving us with a search time complexity of $O(\log_2 n)$.

Therefore, for Phase 1 : Time Complexity is $O(\log_2 n)$.

Phase 2: The Coin Flip Phase (Level Generation)

Once the search is complete, the algorithm generates a random level for the newly inserted node using a while loop that flips a coin.

- The probability of stopping the loop (getting a "tails") is $1 - p$.
- This is another Geometric Distribution.

Recall:

```
int randomLevel() {  
    int lvl = 0;  
    // 50% probability to increase the level  
    while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {  
        lvl++;  
    }  
    return lvl;  
}
```

Returned lvl	Formula	Reason	Probability
$lvl = 0$	$1 - p$	$rand() \% 2 = 1$, very first coin flipped failed.	$50\% = 0.5$
$lvl = 1$	$p(1 - p)$	$rand() \% 2 = 1$, at 2nd time coin flipped failed.	$50\% \times 50\%$ $= 0.5^2 = 0.25$ $= 0.25 \times 100$ $= 25\%$
$lvl = 2$	$p^2(1 - p)$	$rand() \% 2 = 1$, at 3rd time coin flipped failed.	$50\% \times 50\% \times 50\%$ $= 0.5^3 = 0.125$ $= 0.125 \times 100$ $= 12.5\%$
$lvl = 3$	$p^3(1 - p)$	$rand() \% 2 = 1$, at 4th time coin flipped failed.	$50\% \times 50\%$ $\times 50\% \times 50\%$ $= 0.5^4 = 0.0625$ $= 0.0625 \times 100$ $= 6.25\%$

- $N = 1$ flip(stops immediately on 1st flip):

$$\Pr(N = 1) = (1 - p)$$

- $N = 2$ flips(1 Heads, then 1 Tails):

$$\Pr(N = 2) = p(1 - p)$$

- $N = 3$ flips(2 Heads, then 1 Tails):

$$\Pr(N = 3) = p^2(1 - p)$$

- $N = k$ flips($k - 1$ Heads, then 1 Tails):

$$\Pr(N = k) = p^{k-1}(1 - p)$$

By the definition of Expected Value $\mathbb{E}[N] = \sum_{k=1}^{\infty} k \cdot \Pr(N = k)$:

$$\Rightarrow \mathbb{E}[N] = 1 \cdot (1 - p) + 2 \cdot p(1 - p) + 3 \cdot p^2(1 - p) + 4 \cdot p^3(1 - p) + \dots$$

$$\Rightarrow \mathbb{E}[N] = (1 - p) \sum_{k=1}^{\infty} k \times p^{k-1}$$

$$\Rightarrow \mathbb{E}[N] = (1 - p)(1 + 2p + 3p^2 + 4p^3 + \dots)$$

$$\text{Let, } S = \sum_{k=1}^{\infty} k \cdot p^{k-1} = 1 + 2p + 3p^2 + 4p^3 + \dots$$

$$\therefore \mathbb{E}[N] = (1 - p) \times S$$

Now ,Evaluating the Infinite Series `S`:

$$\text{Let } S = \sum_{k=1}^{\infty} kp^{k-1} = 1 + 2p + 3p^2 + 4p^3 + \dots$$

Multiply both sides by p:

$$pS = p + 2p^2 + 3p^3 + \dots$$

Subtract pS from S:

$$\begin{aligned} S - pS &= (1 - 0) + (2p - p) + (3p^2 - 2p^2) + (4p^3 - 3p^3) + \dots \\ &= 1 + p + p^2 + p^3 + \dots \end{aligned}$$

Now, general infinite geometric series is:

$$a_1 + a_1r + a_1r^2 + a_1r^3 + \dots$$

$$S_{\infty} = \frac{a_1}{1-r} ; |r| < 1$$

Now ,we have :

$$S - pS = 1 + p + p^2 + p^3 + \dots$$

$$S - pS = 1 + 1 \times p + 1 \times p^2 + 1 \times p^3 + \dots$$

$$\text{hence , } a_1 = 1$$

$$\text{common ratio} = r = \frac{\text{any term}}{\text{previous term}}$$

$$r = \frac{p}{1} = p; r = \frac{p^2}{p} = p; r = \frac{p^3}{p^2} = p \therefore r = p.$$

$$S_{\infty} = \frac{1}{1-p}$$

$$\text{or, } S - pS = S_{\infty} = \frac{1}{1-p}$$

$$S(1-p) = \frac{1}{1-p}$$

$$\Rightarrow S = \frac{1}{(1-p)^2}$$

Final Substitution:

$$\begin{aligned} \mathbb{E}[N] &= (1-p) \times S \\ \Rightarrow \mathbb{E}[N] &= (1-p) \times \frac{1}{(1-p)^2} \end{aligned}$$

Cancel $(1-p)$ from numerator and denominator:

$$\mathbb{E}[N] = \frac{1}{1-p}$$

$$\Rightarrow \mathbb{E}[\text{flips}] = \frac{1}{1-p}$$

Even if we calculate from the Table:

$$\mathbb{E}[\text{Coin Flips}] = \sum (\text{Flips}) \times \sum (\text{Probability from the table})$$

$$\mathbb{E}[\text{Coin Flips}] = \underbrace{1 \cdot (0.5)}_{\text{for lvl} = 0} + \underbrace{2 \cdot (0.25)}_{\text{for lvl} = 1} + \underbrace{3 \cdot (0.125)}_{\text{for lvl} = 2} + \underbrace{4 \cdot (0.0625)}_{\text{for lvl} = 3} + \dots$$

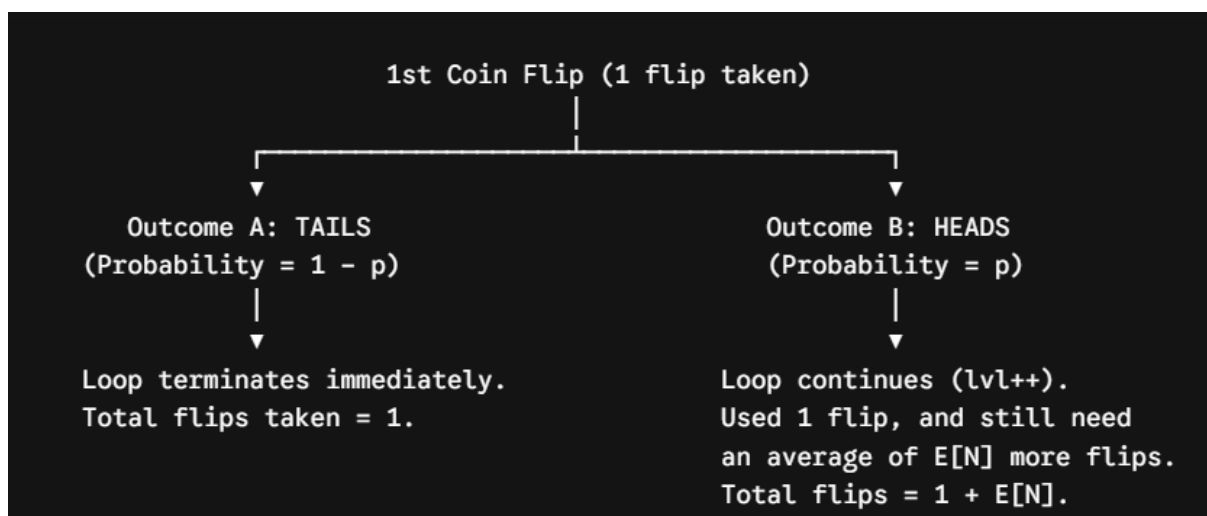
$$\mathbb{E}[\text{Coin Flips}] = 0.5 + 0.5 + 0.375 + 0.25 + \dots = 2 \text{ flips.}$$

This matches the closed – form formula:

$$\Rightarrow \mathbb{E}[\text{flips}] = \frac{1}{1-p} = \frac{1}{1-0.5} = 2 \text{ flips.}$$

Through Method 2: First Step Analysis

Let $\mathbb{E}[N]$ be the expected total number of coin flips inside `randomLevel()` until the loop terminates.



Outcome A: The 1st flip is TAILS (Probability = $1 - p$)

- *The while condition is false. The loop exists immediately.*
- *Total flips taken: Exactly 1 flip.*
- *Contribution to expectation:*

$$(1 - p) \cdot (1)$$

Outcome B: The 1st flip is Heads (Probability = p)

- *The while condition is true, `lvl++` executes, and the loop prepares for the next flip.*
- *We have spent 1 flip.*
- *By the memoryless property of independent coin tosses, the expected number of remaining flips needed from this point onward is still $\mathbb{E}[N]$.*
- *Total flips taken in this scenario:*

$$1 + \mathbb{E}[N]$$

- *Contribution to expectation:*

$$p \cdot (1 + \mathbb{E}[N])$$

By the law of Total Expectation:

$$\mathbb{E}[N] = \underbrace{(1-p) \cdot (1)}_{\text{If 1st flip is Tails}} + \underbrace{p \cdot (1 + \mathbb{E}[N])}_{\text{If 1st flip is Heads}}$$

$$\Rightarrow \mathbb{E}[N] = 1 - p + p + p \cdot \mathbb{E}[N]$$

$$\Rightarrow \mathbb{E}[N] = 1 + p \cdot \mathbb{E}[N]$$

Subtract $p \cdot \mathbb{E}[N]$ from both sides to group $\mathbb{E}[N]$ on the left:

$$\Rightarrow \mathbb{E}[N] - p \cdot \mathbb{E}[N] = 1$$

$$\Rightarrow \mathbb{E}[N](1 - p) = 1 \text{ [Factor out } \mathbb{E}[N] \text{]}$$

Divide both sides by $(1 - p)$:

$$\Rightarrow \mathbb{E}[N] = \frac{1}{1 - p}$$

$$\Rightarrow \mathbb{E}[flips] = \frac{1}{1 - p}$$

Results for $p = 0.5$

$$\Rightarrow \mathbb{E}[flips] = \frac{1}{1 - 0.5}$$

$$\Rightarrow \mathbb{E}[flips] = \frac{1}{1 - \frac{1}{2}}$$

$$\Rightarrow \mathbb{E}[flips] = \frac{1}{\frac{1}{2}}$$

$$\Rightarrow \mathbb{E}[flips] = 2$$

If $p = 0.5$, the algorithm expects to flip the coin exactly 2 times. Because this is a strict mathematical constant regardless of how large n grows, this phase takes $O(1)$ expected time.

Phase 3: The Splicing Phase (Pointer Rewiring)

1. Mechanism: The algorithm loops from level 0 up to $rlevel$, using the saved `update[ ]` array to splice the new node into each linked list:

```
for (int i = 0; i <= rlevel; i++) {
    n->forward[i] = update[i]->forward[i];
    update[i]->forward[i] = n;
}
```

Average Cost: The number of levels rewired is **$rlevel + 1$** .

As proved in **Phase 2**, the expected number of pointers per node is a constant:

$$\mathbb{E}[Pointers] = \frac{1}{1 - p} = 2 \text{ (for } p = 0.5 \text{)}$$

Thus, pointer rewiring runs in $O(1)$ expected time.

2. Worst – Case Upper Bound $\rightarrow O(\log n)$

1. The Maximum Level Cap:

- If a node gets exceptionally lucky and flips consecutive Heads, $rlevel$ can grow up to the maximum height of the list:

$$MAX_LEVEL = \log_{\frac{1}{p}} n$$

$$p = 0.5, \text{ then } MAX_LEVEL = \log_2 n \quad \left[\begin{array}{l} \text{Already proved} \\ \text{from Phase 1} \end{array} \right]$$

- In that worst – case scenario, the for loop will run $\log_2 n$ times, performing $O(\log_2 n)$ operations.

2. List Level Extension:

- If $rlevel > lst \rightarrow level$, the code runs an extra loop to point new levels to the Header:

```
if (rlevel > lst->level) {
    for (int i = lst->level + 1; i <= rlevel; i++) {
        update[i] = lst->header;
    }
    lst->level = rlevel;
}
```

This is also strictly bounded by $MAX_LEVEL = \log_{\frac{1}{p}} n$

$$p = 0.5, \text{ then } MAX_LEVEL = \log_2 n \quad \left[\begin{array}{l} \text{Already proved} \\ \text{from Phase 1} \end{array} \right]$$

- In that worst – case scenario, the for loop will run $\log_2 n$ times, performing $O(\log_2 n)$ operations.

Therefore, the splicing phase requires an average of $O(1)$ operations, strictly bounded by a worst – case of $O(\log_2 n)$ operations.

*Since $O(1) \subseteq O(\log n)$, hence we keep the upper bound $O(\log_2 n)$ for **Phase3 : Splicing Phase(Pointer Rewiring)**.*

Final Conclusion

To find the average case time complexity for the entire insertion module, we sum the mathematical bounds of the three phases:

$$\text{Search: } O(\log_2 n) + \text{Coin Flip: } O(1) + \text{Splicing: } O(\log_2 n) = O(\log_2 n)$$

$$\text{Total Time} = O(\log_2 n)$$

Through geometric probability distributions and expected value derivations, the mathematics prove that the average case time complexity for inserting an element into a skip list is strictly $O(\log_2 n)$.

And we get $O(\log_2 n)$, when $p = 0.5$ falls in $O(\log n)$ class.

And both notations represent the exact same complexity class.

$$O(\log_2 n) \approx O(\log n).$$

*Skip List – Insert Element – Time Complexity According to William Pugh
–Type B*

Average Case – Time Complexity by William Pugh

This explanation is flawless, rigorous, and beautifully structured. It captures the exact classical recurrence relation formulated by William Pugh (the inventor of Skip Lists) in his original 1990 paper.

$C(k)$

- *$C(k)$ is the average total number of steps needed to climb up `k` levels during the backward walk.*

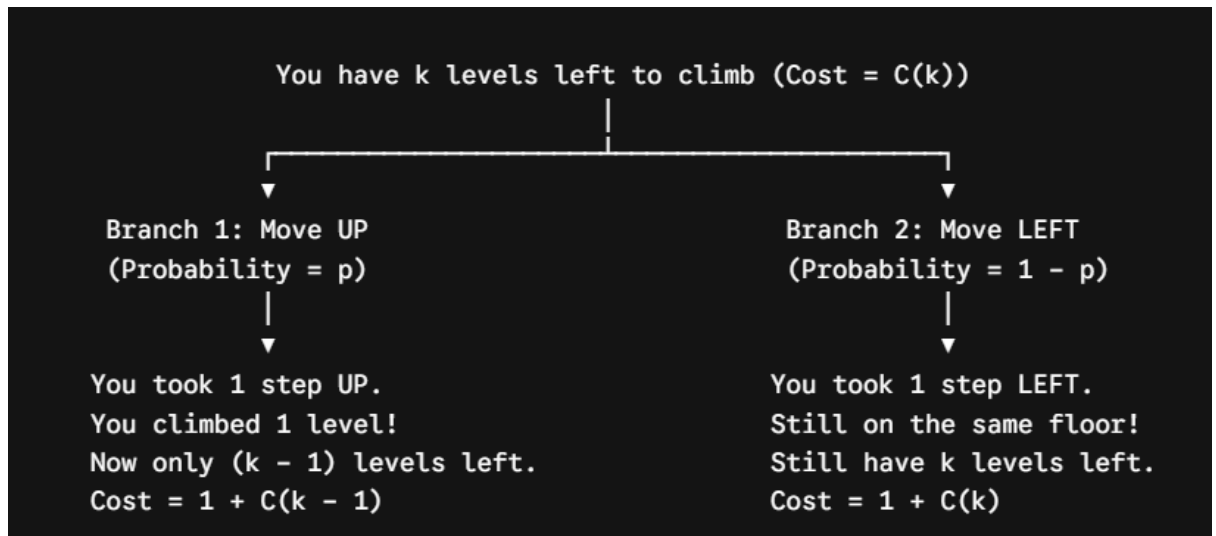
*To know what is `Backward Walk`, you have to go through **Type A** Average Case of Time Complexity of previous document.*

- *if we need to climb 5 levels, the cost is **$C(5)$** .*
- *if we need to climb 4 levels, the cost is **$C(4)$** .*
- *if we need to climb 0 levels, the cost is **$C(0) = 0$** (We are already at the top.)*

We are currently standing at a node and we have k levels left to climb.

We must take 1 step.

There are only two possible outcomes:



Explaining Each Branch

Branch 1: UP Branch $\rightarrow p(1 + C(k - 1))$

- **Probability (p)**: The current node exists on the level above.
- **The Step: 1 step UP**
- **Since we just climbed 1 level, we now only have $(k - 1)$ levels left to climb.**
- **Remaining Cost** : By definition , climbing $(k - 1)$ levels costs $C(k - 1)$.
- **Total Cost for this branch:**

$$\begin{array}{c}
 \text{1} \\
 \text{1 step UP}
 \end{array}
 +
 \begin{array}{c}
 C(k - 1) \\
 \text{Remaining levels}
 \end{array}
 = 1 + C(k - 1)$$

- *Weighted by probability p :*

$$p \cdot (1 + C(k - 1))$$

Branch 2: The Left Branch $\rightarrow (1 - p)(1 + C(k))$

- **Probability $(1 - p)$:** *The current node doesn't exist on the level above.*
- **The Step:** *We are blocked from going UP, so we take 1 step LEFT to the previous node.*
- **What remains?** *We did not climb any levels – we are still on the exact same floor! Therefore, we still have all the **k levels** left to climb.*
- **Remaining Cost:** *Climbing k levels from the new node still costs $C(k)$.*
- **Total Cost for this branch:**

$$\begin{array}{c}
 \text{1} \\
 \underbrace{\hspace{1cm}} \\
 \boxed{\text{1 step Left}}
 \end{array}
 +
 \begin{array}{c}
 C(k) \\
 \underbrace{\hspace{1cm}} \\
 \boxed{\text{still need } k \text{ levels}}
 \end{array}
 = 1 + C(k)$$

- *Weighted by probability p :*

$$(1 - p) \cdot (1 + C(k))$$

Now combining both the branches (Law of Total Expectation)

The total expected cost $C(k)$ is simply the weighted sum of both branches:

$$\text{Total Cost } C(k) = \text{Pr}(\text{LEFT}) \times (\text{Cost if LEFT}) + \text{Pr}(\text{UP}) \times (\text{Cost if UP})$$

- ***where `Pr` represents Probability.***

$$\begin{aligned}\therefore C(k) &= (1 - p)(1 + C(k)) + p(1 + C(k - 1)) \\ \Rightarrow C(k) &= (1 - p) + (1 - p) \cdot C(k) + p + p \cdot C(k - 1) \\ \Rightarrow C(k) &= (1 - p) + p + (1 - p) \cdot C(k) + p \cdot C(k - 1) \\ \Rightarrow C(k) &= 1 + (1 - p) \cdot C(k) + p \cdot C(k - 1)\end{aligned}$$

Subtracting $(1 - p) \cdot C(k)$ from both sides we get:

$$\begin{aligned}\Rightarrow C(k) - (1 - p) \cdot C(k) &= 1 + (1 - p) \cdot C(k) - (1 - p) \cdot C(k) + p \cdot C(k - 1) \\ \Rightarrow C(k) - (1 - p) \cdot C(k) &= 1 + p \cdot C(k - 1) \\ \Rightarrow C(k) - (C(k) - p \cdot C(k)) &= 1 + p \cdot C(k - 1) \\ \Rightarrow C(k) - C(k) + p \cdot C(k) &= 1 + p \cdot C(k - 1) \\ \Rightarrow p \cdot C(k) &= 1 + p \cdot C(k - 1)\end{aligned}$$

Dividing entire equation by p , we get:

$$\frac{p \cdot C(k)}{p} = \frac{1}{p} + \frac{p \cdot C(k - 1)}{p}$$

$$\Rightarrow C(k) = \frac{1}{p} + C(k - 1)$$

The formula : $C(k) = \frac{1}{p} + C(k-1)$ says:

*The cost to climb `k` levels is the cost of climbing **1** level $\left(\frac{1}{p}\right)$ plus the cost of climbing the remaining **k - 1** levels.*

Unrolling it all the way down to $C(0) = 0$, the topmost level:

By substitution method:

$$C(k) = \frac{1}{p} + C(k-1) \rightarrow k \text{ levels}$$

$$\text{When } C(k-1) = \frac{1}{p} + C(k-2)$$

Substituting $C(k-1) = \frac{1}{p} + C(k-2)$ in $C(k)$ we get:

$$C(k) = \frac{1}{p} + \underbrace{\left(\frac{1}{p} + C(k-2)\right)}_{\boxed{C(k-1)}}$$

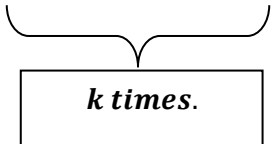
$$\text{When } C(k-2) = \frac{1}{p} + C(k-3)$$

Substituting $C(k-2) = \frac{1}{p} + C(k-3)$ in $C(k)$ we get:

$$C(k) = \frac{1}{p} + \left(\frac{1}{p} + \left(\frac{1}{p} + C(k-3) \right) \right)$$

Thus, repeating all way down to $C(0)$:

$$C(k) = \frac{1}{p} + \frac{1}{p} + \frac{1}{p} + \cdots + \frac{1}{p} + C(0)$$



*As from **Type A** Average Case of Time Complexity, previous document, we have seen there are ' $k+1$ ' level.*

We know, $C(0) = 0$, applying we get:

$$\Rightarrow C(k) = \frac{1 + 1 + 1 + \cdots k \text{ times}}{p} + 0$$

$$\Rightarrow C(k) = \frac{1 \times k}{p}$$

$$\Rightarrow C(k) = \frac{k}{p}$$

*We know from, **Type A** Average Case of Time Complexity Analysis, that*

$$MAX_LEVEL = \log_{\frac{1}{p}} n$$

Where **MAX_LEVEL** is the maximum height of the list.

When climbing all $k = \log_{\frac{1}{p}} n$ levels of the Skip List:

$$C(k) = \frac{k}{p}$$
$$\Rightarrow C\left(\log_{\frac{1}{p}} n\right) = \frac{\log_{\frac{1}{p}} n}{p}$$

For $p = 0.5$:

$$\Rightarrow C\left(\log_{\frac{1}{0.5}} n\right) = \frac{\log_{\frac{1}{0.5}} n}{0.5}$$

$$\Rightarrow C\left(\log_{\frac{1}{\frac{1}{2}}} n\right) = \frac{\log_{\frac{1}{\frac{1}{2}}} n}{\frac{1}{2}}$$
$$\Rightarrow C(\log_2 n) = 2 \times \log_2 n$$

And we get $O(\log_2 n)$, when $p = 0.5$ falls in $O(\log n)$ class.

And both notations represent the exact same complexity class.

$$O(\log_2 n) \approx O(\log n).$$

Skip List – Insert Element –Space Complexity and Algorithm

Program:

```
void insertElement(struct SkipList *lst, int key) {
    struct Node *current = lst->header;

    // Array to keep track of the path we took (nodes needing pointer
    updates)
    struct Node *update[MAX_LEVEL + 1];
    memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));

    // Start from the highest current level and move down
    for (int i = lst->level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key <
key) {
            current = current->forward[i];
        }
        update[i] = current;
    }

    // Move to level 0 and look at the next node
    current = current->forward[0];

    // If the key doesn't already exist, insert it
    if (current == NULL || current->key != key) {
        int rlevel = randomLevel();

        // If the random level is greater than the list's current max
        level,
        // initialize the update array for the new levels to point to
        the header.
        if (rlevel > lst->level) {
            for (int i = lst->level + 1; i <= rlevel; i++) {
                update[i] = lst->header;
            }
            lst->level = rlevel;
        }
    }
}
```

```

    // Create the new node and rewire pointers
    struct Node *n = createNode(key, rlevel);
    for (int i = 0; i <= rlevel; i++) {
        n->forward[i] = update[i]->forward[i];
        update[i]->forward[i] = n;
    }
    cout << "Successfully inserted key " << key << endl;
} else {
    cout << "Key " << key << " already exists in the list." << endl;
}
}
int main() {
    // Seed random number generator for random levels
    srand((unsigned)time(0));
    struct SkipList *list = createSkipList();
    int choice, key;

    while (true) {

        cout << "\n==== Skip List Menu =====" << endl;
        cout << "1. Insert" << endl;
        cout << "Enter your choice: ";

        if (!(cin >> choice)) {
            cout << "Invalid input. Exiting." << endl;
            break;
        }

        switch (choice) {

        case 1:
            cout << "Enter key to insert: ";
            cin >> key;
            insertElement(list, key);
            break;
        }
    }
    return 0;
}

```

Space Complexity Analysis

A. Input Space

The input space is evaluated from two different perspectives: the exact arguments passed to the function, and the overall memory footprint of the entire data structure.

A.1) Parametric Input Space: $O(1)$

The parametric input space strictly measures the memory required to hold the arguments passed directly into the `insertElement` function.

- *The function signature is `void insertElement(struct SkipList * lst, int key)`.*
- ***lst**: A standard memory pointer addressing the existing skip list structure. This takes a small, constant amount of memory (typically 8 bytes on a 64 – bit system).*
- ***key**: A standard integer variable representing the value to be inserted. This also takes a small, constant amount of memory (typically 4 bytes).*
- *Because the size of these parameters never scales with the number of elements in the list (n), the parametric input space is strictly $O(1)$.*

A.2) Full Data Holistic Input Space: $O(n)$

The holistic input space evaluates the total memory consumed by the entire Skip List data structure containing n elements.

- *A standard linked list requires exactly one forward pointer per node. Skip lists add probabilistic express lanes, meaning nodes require arrays of multiple forward pointers.*
- *However, the probability distribution ensures that the number of pointers drops off exponentially at higher levels.*

- *As established in the theoretical analysis, a skip list can be easily configured to require remarkably little extra space – for instance, an average of 1.33 pointers per element when $p = \frac{1}{4}$, or 2 pointers per element when $p = \frac{1}{2}$.*
- *Because the expected maximum number of pointers per node remains a small mathematical constant, the total memory footprint to store n nodes scales directly and linearly with the number of elements i. e. n .*
- *Therefore, the full data holistic input space is $O(n)$.*

B) Auxiliary Space

Auxiliary space measures the temporary, extra memory the algorithm requests specifically to complete the insertion process, which is then destroyed when the function finishes.

- *Inside the insertion module, the primary consumer of auxiliary memory is the update array: `struct Node * update[MAXLEVEL + 1]`.*
- *This array is used to temporarily store the memory addresses of the predecessor nodes at each level of the search path.*

- The array size calculation is explicitly tied to MAX_{LEVEL} .
- In the provided C++ program, MAX_LEVEL is hardcoded to 3, meaning the array allocates exactly 4 slots (32 bytes). In this heavily restricted micro – scenario, the auxiliary space is technically $O(1)$.
- However, in a generic, scalable Skip List mathematically designed to hold n elements, the maximum level is dynamically capped at $H = \log_{\frac{1}{p}} n$. Consequently, the size of the *update* array must scale logarithmically to match the height of the list.

As we have observed in time complexity:

Let n be the total number of elements in the skip list.

Let p be the probability that a node at level k is promoted to level $k + 1$ (typically, 50%, so, $p = 0.5$).

Calculate the Expected Maximum Height(H)

To know how far up we have to climb, we first need to determine the expected maximum height of the entire skip list.

- The probability that any single node reaches level k is p^k .
- Let N_k be the number of nodes at level k . The expected number of nodes at level k across the whole list is:

$$E[N_k] = n \times p^k$$

The maximum height H of the skip list is defined as the level where we expect to find exactly 1 node. Therefore, we set $E[N_k] = 1$:

$$n \times p^H = 1$$

Solving for H :

$$p^H = \frac{1}{n}$$

Putting $\log_p$ at both the side we get:

$$\log_p(p^H) = \log_p\left(\frac{1}{n}\right)$$

$$H = \log_p(n^{-1}) \left[\log_b x^y = y \times \log_b x, \log_b b = 1 \text{ and } \frac{1}{n} = n^{-1} \right]$$

$$H = -1 \times \log_p n \left[\log_b x^y = y \times \log_b x \right]$$

$$H = -\log_p n$$

To get rid of that negative sign, we use the Change of Base Formula $\left(\log_b(x) = \frac{\log_c(x)}{\log_c(b)} \right)$ to change the base from p to $\frac{1}{p}$:

$$H = -\log_p(n) = -\left(\frac{\log_{\frac{1}{p}}(n)}{\log_{\frac{1}{p}}(p)} \right)$$

Look at the denominator : $\log_{\frac{1}{p}}(p)$. Because p is the exact

inverse of $\frac{1}{p}$ (meaning $p = \left(\frac{1}{p}\right)^{-1}$), this denominator evaluates

to -1 .

$$H = -\log_p(n) = -\left(\frac{\log_{\frac{1}{p}}(n)}{\log_{\frac{1}{p}}\left(\frac{1}{p}\right)^{-1}}\right)$$

$$H = -\left(\frac{\log_{\frac{1}{p}}(n)}{-1 \times \log_{\frac{1}{p}}\left(\frac{1}{p}\right)}\right)$$

$$H = -\left(\frac{\log_{\frac{1}{p}}(n)}{-1}\right)$$

$$H = \log_{\frac{1}{p}}(n)$$

Alternative calculation:

$$p^H = \frac{1}{n}$$

$$\Rightarrow n = \frac{1}{p^H}$$

We can also write it as:

$$\Rightarrow n = \left(\frac{1}{p}\right)^H \quad [as, 1^H = 1]$$

The fundamental definition of a logarithm is that if $b^y = x$, then $\log_b(x) = y$. Here our base $\left(\frac{1}{p}\right)$, our exponent is H , and our result is n . By simply rewriting it in **log form, we instantly get:**

$$H = \log_{\frac{1}{p}}(n)$$

And

if $p = 0.5$, the expected maximum height is:

$$H = \log_{\frac{1}{0.5}}(n)$$

$$H = \log_{\frac{1}{\frac{1}{2}}}(n)$$

$$H = \log_2(n)$$

Also, in pointer rewiring (Splicing) phase, we have found:

1. Mechanism: The algorithm loops from level 0 up to rlevel, using the saved update[] array to splice the new node into each linked list:

```
for (int i = 0; i <= rlevel; i++) {  
    n->forward[i] = update[i]->forward[i];  
    update[i]->forward[i] = n;  
}
```

A. The Maximum Level Cap:

- *If a node gets exceptionally lucky and flips consecutive Heads, rlevel can grow up to the maximum height of the list:*

$$MAX_LEVEL = \log_{\frac{1}{p}} n$$

$$p = 0.5, \text{ then } MAX_LEVEL = \log_2 n$$

- *In that worst – case scenario, the for loop will run $\log_2 n$ times, performing $O(\log_2 n)$ operations.*

B. List Level Extension:

- If $rlevel > lst \rightarrow level$, the code runs an extra loop to point new levels to the Header:

```
if (rlevel > lst->level) {  
    for (int i = lst->level + 1; i <= rlevel; i++) {  
        update[i] = lst->header;  
    }  
    lst->level = rlevel;  
}
```

This is also strictly bounded by $MAX_LEVEL = \log_{\frac{1}{p}} n$

$p = 0.5$, then $MAX_LEVEL = \log_2 n$

There fore ,

Auxiliary Space	Complexity
1.Theoretically	$O(\log_2 n)$
2. Practical Scenario	$O(1)$ $\left[\begin{array}{l} \text{As, Array Size is} \\ \text{Explicitly tied to} \\ \text{Max_Level ,} \\ \text{Which is a} \\ \text{Constant.} \\ \text{(Here , in the program} \\ \text{Max_Level = 3)} \end{array} \right]$

∴ Total Space Complexity : Full Data Holistic Input + Auxiliary Space

In Theoretical Scenario :

$$= O(n) + O(\log_2 n) = O(n) \text{ [As } n \text{ dominates over } \log_2 n \text{]}$$

In practical Scenario:

$$= O(n) + O(1) = O(n) \text{ [As } n \text{ dominates over } 1 \text{]}$$

Now, this : $O(n)$ is considered as Average or Expected Case of Total Space Complexity of the Skip List .

Alternatively Average or Expected

Total Space Complexity of the Skip List , can be calculated as :

Expected Pointers per Node (Memory Complexity) [Vertical Pointers per Node Approach – Sums Average pointers inside a single node.]

When a node is created, how many pointers (forward[]) does it have on average?

- ***Every node has Level 0: 1 pointer (always, probability = 1).***
- ***It also gets Level 1 ,node flips a coin to see if it gets a second pointer with probability: p .***
- ***It also gets Level 2, It flips again for a third pointer with probability: p^2 .***

- It also gets Level 3,, It flips again for a fourth pointer with probability: p^3 , and so on.

The expected total number of pointers in a node is the sum of this infinite geometric series:

$$\mathbb{E}[\text{Pointers per Node}] = 1 + p + p^2 + p^3 + \dots = \sum_{i=0}^{\infty} p^i = \frac{1}{1-p}$$

$$\text{if } p = 0.5 (50\%) = \frac{1}{1-0.5} = \frac{1}{1-\frac{1}{2}} = \frac{1}{\frac{1}{2}} = 2 \text{ pointers per node on average.}$$

And n is the total number of elements in the skip list.

Therefore, n number of elements need : $2n$ total pointers.

Hence, **Average or Expected Case of Total Space Complexity of the Skip List** : $O(2n) = O(n)$.

Alternative Calculation of Average or Expected

Total Space Complexity of the Skip List, can be calculated as :

Elements per Layer $\left(\begin{array}{c} \text{Horizontal Approach} \\ \text{Sum of Total Nodes on Every floor} \end{array} \right)$

→ **Level 0**: Must contain every single element. Total pointers = n .

→ **Level 1**: Because of the 50% coin flip ($p = 0.5$), only half of the elements

from Level 0 are expected to be promoted. Total expected pointers = $\frac{n}{2}$.

→ **Level 2**: Only half of Level 1 is expected to be promoted. Total expected pointers = $\frac{n}{4}$.

→ **Level 3**: Half of Level 2. Total expected pointers = $\frac{n}{8}$.

$$TotalSpace = n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots$$

factor out the n:

$$TotalSpace = n \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots \right)$$

We know, through geometric infinite series : $s_{\infty} = \frac{a_1}{1-r}$

$$a_1 = 1 \text{ (First Term) and } r = \frac{\text{Any Term}}{\text{Previous Term}} \Rightarrow \frac{1}{2} = \frac{1}{2} ; \frac{1/4}{1/2} = \frac{1}{2} \text{ etc.}$$

$$\therefore S_{\infty} = \frac{a_1}{1-r} = \frac{1}{1-\frac{1}{2}} = \frac{1}{\frac{1}{2}} = 2.$$

$$TotalSpace = n \times S_{\infty} = n \times 2 = 2n.$$

Alternatively, Algebraic Shifting Method:

$$\text{Let, } S = 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots$$

Now, Multiplying $\frac{1}{2}$ on both the sides we get:

$$\Rightarrow \frac{1}{2}S = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots$$

Now subtracting we get:

$$\Rightarrow S - \frac{1}{2}S = (1 - 0) + \left(\frac{1}{2} - \frac{1}{2}\right) + \left(\frac{1}{4} - \frac{1}{4}\right) + \left(\frac{1}{8} - \frac{1}{8}\right) + \dots$$

$$\Rightarrow S - \frac{1}{2}S = 1$$

$$\Rightarrow S \left(1 - \frac{1}{2}\right) = 1$$

$$\Rightarrow S \left(\frac{1}{2}\right) = 1$$

$$\Rightarrow S = 2$$

$$\text{TotalSpace} = n \times S = n \times 2 = 2n.$$

***Worst Case of
Total Space Complexity
of the Skip List(Bounded Height)***

Let `n` be the total number of elements in the skip list.

Calculate the Expected Maximum Height(H)

To know how far up we have to climb, we first need to determine the expected maximum height of the entire skip list.

- *The probability that any single node reaches level `**k**` is **p^k** .*
- *Let `**N<sub>k</sub>**` be the number of nodes at level `**k**'. The expected number of nodes at level `**k**` accross the whole list is:*

$$E[N_k] = n \times p^k$$

The maximum height **H** of the skip list is defined as the level where we expect to find exactly **1** node. Therefore, we set $E[N_k] = 1$:

$$n \times p^H = 1$$

Solving for **H**:

$$p^H = \frac{1}{n}$$

Putting $\log_p$ at both the side we get:

$$\log_p(p^H) = \log_p\left(\frac{1}{n}\right)$$

$$H = \log_p(n^{-1}) \left[\log_b x^y = y \times \log_b x, \log_b b = 1 \text{ and } \frac{1}{n} = n^{-1} \right]$$

$$H = -1 \times \log_p n \left[\log_b x^y = y \times \log_b x \right]$$

$$H = -\log_p n$$

To get rid of that negative sign, we use the Change of Base Formula

$\left(\log_b(x) = \frac{\log_c(x)}{\log_c(b)} \right)$ to change the base from **p** to $\frac{1}{p}$:

$$H = -\log_p(n) = -\left(\frac{\log_{\frac{1}{p}}(n)}{\log_{\frac{1}{p}}(p)} \right)$$

Look at the denominator : $\log_{\frac{1}{p}}(p)$. Because **p** is the exact

inverse of $\frac{1}{p}$ (meaning $p = \left(\frac{1}{p}\right)^{-1}$), this denominator evaluates

to **-1**.

$$H = -\log_p(n) = -\left(\frac{\log_{\frac{1}{p}}(n)}{\log_{\frac{1}{p}}\left(\frac{1}{p}\right)^{-1}}\right)$$

$$H = -\left(\frac{\log_{\frac{1}{p}}(n)}{-1 \times \log_{\frac{1}{p}}\left(\frac{1}{p}\right)}\right)$$

$$H = -\left(\frac{\log_{\frac{1}{p}}(n)}{-1}\right)$$

$$H = \log_{\frac{1}{p}}(n)$$

Alternative calculation:

$$p^H = \frac{1}{n}$$

$$\Rightarrow n = \frac{1}{p^H}$$

We can also write it as:

$$\Rightarrow n = \left(\frac{1}{p}\right)^H \quad [as, 1^H = 1]$$

The fundamental definition of a logarithm is that if $b^y = x$, then $\log_b(x) = y$. Here our base $\left(\frac{1}{p}\right)$, our exponent is H , and our result is n . By simply rewriting it in **log form, we instantly get:**

$$H = \log_{\frac{1}{p}}(n)$$

if $p = 0.5$, the expected maximum height is:

$$H = \log_{\frac{1}{0.5}}(n)$$

$$H = \log_{\frac{1}{\frac{1}{2}}}(n)$$

$$H = \log_2(n)$$

Also, in pointer rewiring (Splicing) phase, we have found:

1. Mechanism: The algorithm loops from level 0 up to rlevel, using the saved update[] array to splice the new node into each linked list:

```
for (int i = 0; i <= rlevel; i++) {  
    n->forward[i] = update[i]->forward[i];  
    update[i]->forward[i] = n;  
}
```

C. The Maximum Level Cap:

- *If a node gets exceptionally lucky and flips consecutive Heads, rlevel can grow up to the maximum height of the list:*

$$MAX_LEVEL = \log_{\frac{1}{p}} n$$

$$p = 0.5, \text{ then } MAX_LEVEL = \log_2 n$$

- *In that worst – case scenario, the for loop will run $\log_2 n$ times, performing $O(\log_2 n)$ operations.*

D. List Level Extension:

- If $rlevel > lst \rightarrow level$, the code runs an extra loop to point new levels to the Header:

```
if (rlevel > lst->level) {  
    for (int i = lst->level + 1; i <= rlevel; i++) {  
        update[i] = lst->header;  
    }  
    lst->level = rlevel;  
}
```

This is also strictly bounded by $MAX_LEVEL = \log_{\frac{1}{p}} n$

$p = 0.5$, then $MAX_LEVEL = \log_2 n$

Even if a node gets exceptionally lucky and flips consecutive Heads, its level is not allowed to grow infinitely; it is capped at the maximum height of the list ($MAX_{LEVEL} = \log_2 n$)

Hence, Worst Case of Total Space Complexity

of the Skip List(Bounded Height) = $n \times \log_2 n = n \log_2 n = O(n \log_2 n)$ or $O(n \log n)$,

Note: Here number of pointers = $n \log_2 n$, where $\log_2 n$ is number of levels, n is number of elements and number of pointers = $n \log_2 n$.

→ Every single node in the list keeps getting promoted to higher and higher levels.

→ To stop the list from taking up infinite memory, the algorithm artificially stops the nodes from growing once they hit the MAX_LEVEL cap,

which mathematically is $\log_2 n$.

→ In this disaster scenario, we do not have a mix of short and tall nodes.

Every single one of our n nodes is perfectly tied at the maximum height of $\log_2 n$.

→ If we have n nodes, and every single one holds $\log_2 n$ pointers, we get $n \times \log_2 n$ total pointers.

*Example: Suppose There are 1024 nodes and $\log_2 1024 = 10$ levels.
 $1024 \times 10 = 10240$ pointers exists.*

And $n < n \log_2 n$ or $n \log_2 n$ dominates over n , Hence, $O(n \log_2 n)$.

Worst Case of Total Space Complexity of the Skip List(UnBounded Height)

If there is absolutely no artificial ceiling programmed into the algorithm, a theoretical coin could flip heads infinitely. A single node would attempt to generate an infinite number of pointers, exhausting all available system memory.

Leads to : $O(\infty)$

This is precisely why all real – world and academic algorithms must enforce

a maximum level cap.

Note : The algorithm requires an **update** array to track the pointer path, which scales with the expected maximum height of the list ($\log_2 n$). However, in practical implementations that enforce a constant upper bound on the maximum level (e. g., 32), the auxiliary space becomes $O(1)$ since the array size is strictly capped to a constant size in real word scenario of a skip list. That is Only the **auxiliary space** used by the **update[]** array becomes $O(1)$. The skip – list nodes themselves still require space that grows with n elements (**Full Data Holistic Input Space**).

In fact, the total expected space for the skip – list structure is typically $O(n)$."

Algorithm: SkipListInsert(LST, KEY)

1. [Initialize Pointers and Arrays]

1. a. **SET CURRENT := HEADER[LST].**

1. b. **Declare an array UPDATE of size MAX_LEVEL + 1 and initialize all elements to NULL.**

2. [Traverse List to Record Path]

2. a. **Repeat for $i = \text{LEVEL}[LST]$ down to 0:**

2. b. **Repeat while $\text{FORWARD}[i][\text{CURRENT}] \neq \text{NULL AND INFO}[\text{FORWARD}[i][\text{CURRENT}]] < \text{KEY}$:**

2. b. i. **SET CURRENT := FORWARD[i][CURRENT].**

[End of while Loop]

2. a. i. **SET UPDATE[i] := CURRENT.**

[End of for Loop]

3. [Move to Target Node]

SET CURRENT := FORWARD[0][CURRENT].

4. [Check for Duplicates]

*If CURRENT = NULL OR INFO[CURRENT] ≠ KEY, Then proceed to Step 5.
Else, proceed to Step 9.*

5. [Generate Random Level]

SET RLEVEL := CALL randomLevel().

6. [Adjust List Max Level]

If RLEVEL > LEVEL[LST], Then:

Repeat for i = LEVEL[LST] + 1 to RLEVEL:

SET UPDATE[i] := HEADER[LST].

[End of for loop]

SET LEVEL[LST] := RLEVEL.

[End of If Structure]

7. [Create Node and Rewire Pointers]

SET NEWPTR := CALL createNode(KEY, RLEVEL).

Repeat for i = 0 to RLEVEL:

SET FORWARD[i][NEWPTR] := FORWARD[i][UPDATE[i]].

SET FORWARD[i][UPDATE[i]] := NEWPTR.

[End of for Loop]

8. [Success Message]

Write: "Successfully inserted key ", KEY.

Proceed to Step 10.

9. [Handle Duplicate Key]

Write: "Key ", KEY, " already exists in the list."

10. [Exit]

END OF ALGORITHM.

Skip – List
–Search Element
–Program Analysis

Program Analysis:

```
void searchElement(struct SkipList *lst, int key) {
    struct Node *current = lst->header;

    // Traverse from highest level to the lowest
    for (int i = lst->level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key <
key) {
            current = current->forward[i];
        }
    }

    // Step into the exact match at level 0
    current = current->forward[0];

    if (current != NULL && current->key == key) {
        cout << "Found key " << key << " in the Skip List." << endl;
    } else {
        cout << "Key " << key << " not found." << endl;
    }
}

...
case 3:
    cout << "Enter key to search: ";
    cin >> key;
    searchElement(list, key);
    break;
```


Program Analysis

There are three cases:

1st, when Current = NULL [*List is Empty*]

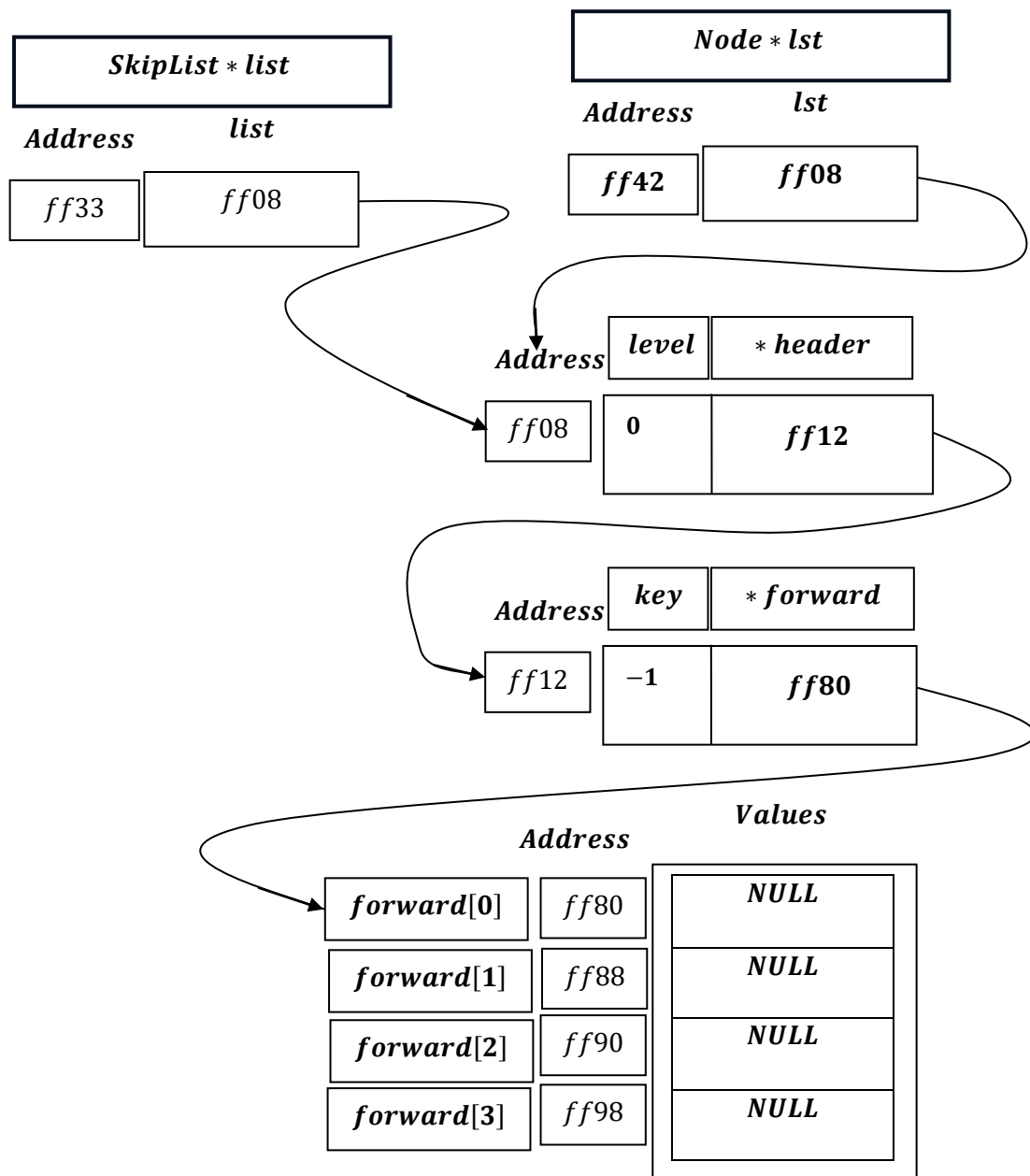
```
searchElement(list, key);
```

searchElement(list: ff08, key: 20) → passed.

And,

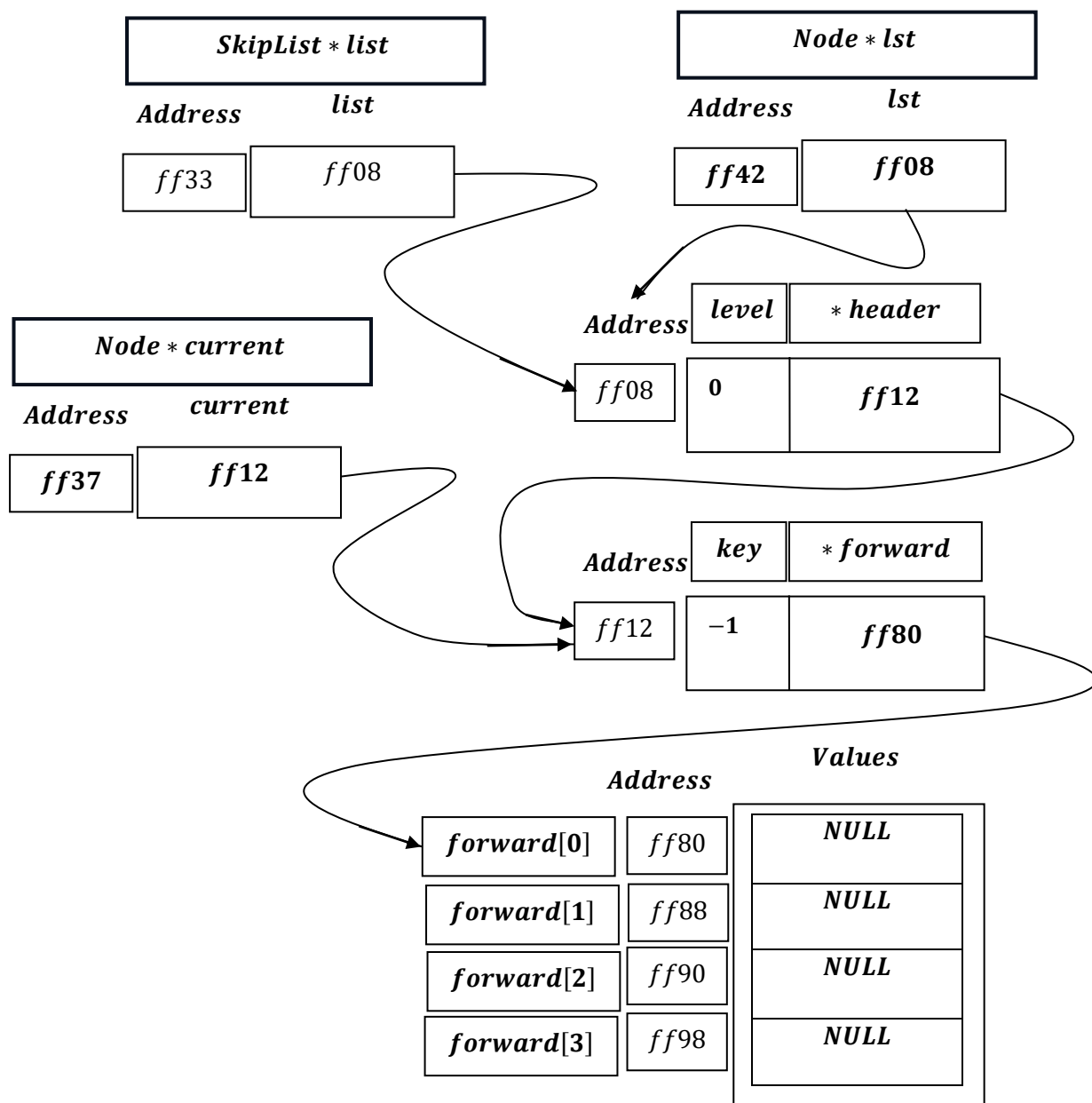
```
void searchElement(struct SkipList *lst, int key)
```

searchElement(list := lst: ff08, key := key: 20) → Assigned



```
struct Node *current = lst->header;
```

current = lst → header = ff12.



```
for (int i = lst->level; i >= 0; i--)
```

$i = \text{lst} \rightarrow \text{level} = 0$ and $i \geq 0$ [Condition is True]

```
while (current->forward[i] != NULL && current->forward[i]->key < key)
```

$\text{current} \rightarrow \text{forward}[0]$ i.e. $\text{ff80} = \text{NULL}$., hence condition is false .

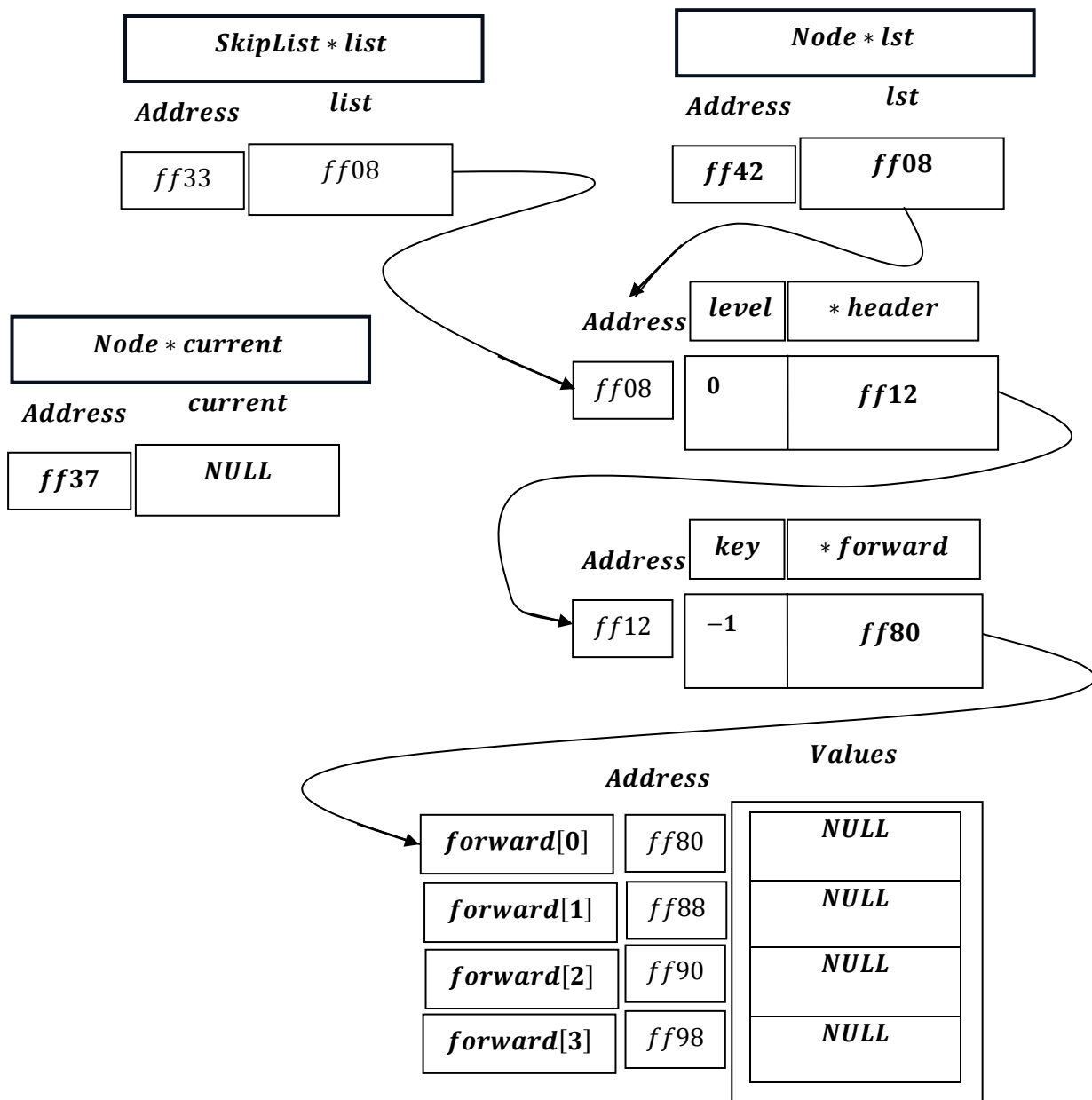
$i-- \Rightarrow i = i - 1 \Rightarrow i = 0 - 1 = -1$.

```
for (int i = lst->level; i >= 0; i--)
```

$i = -1$ and $i \geq 0$ $\left[\begin{array}{c} \text{Condition} \\ \text{Fails} \\ \text{and} \\ \text{Loop Exits} \end{array} \right]$

```
current = current->forward[0];
```

$\text{current} = \text{current} \rightarrow \text{forward}[0]$ i.e. $\text{ff80} = \text{NULL}$



```
if (current != NULL && current->key == key)
```

Current ≠ *NULL* [*Condition Fails , hence if exits*].

```
else {
    cout << "Key " << key << " not found." << endl;
}
```

Hence, now it prints ``Key 20 not found.`` and function exits.

2nd, when Current $\neq$ NULL $\left[\begin{array}{l} \text{List} \\ \text{is not} \\ \text{Empty} \end{array} \right]$

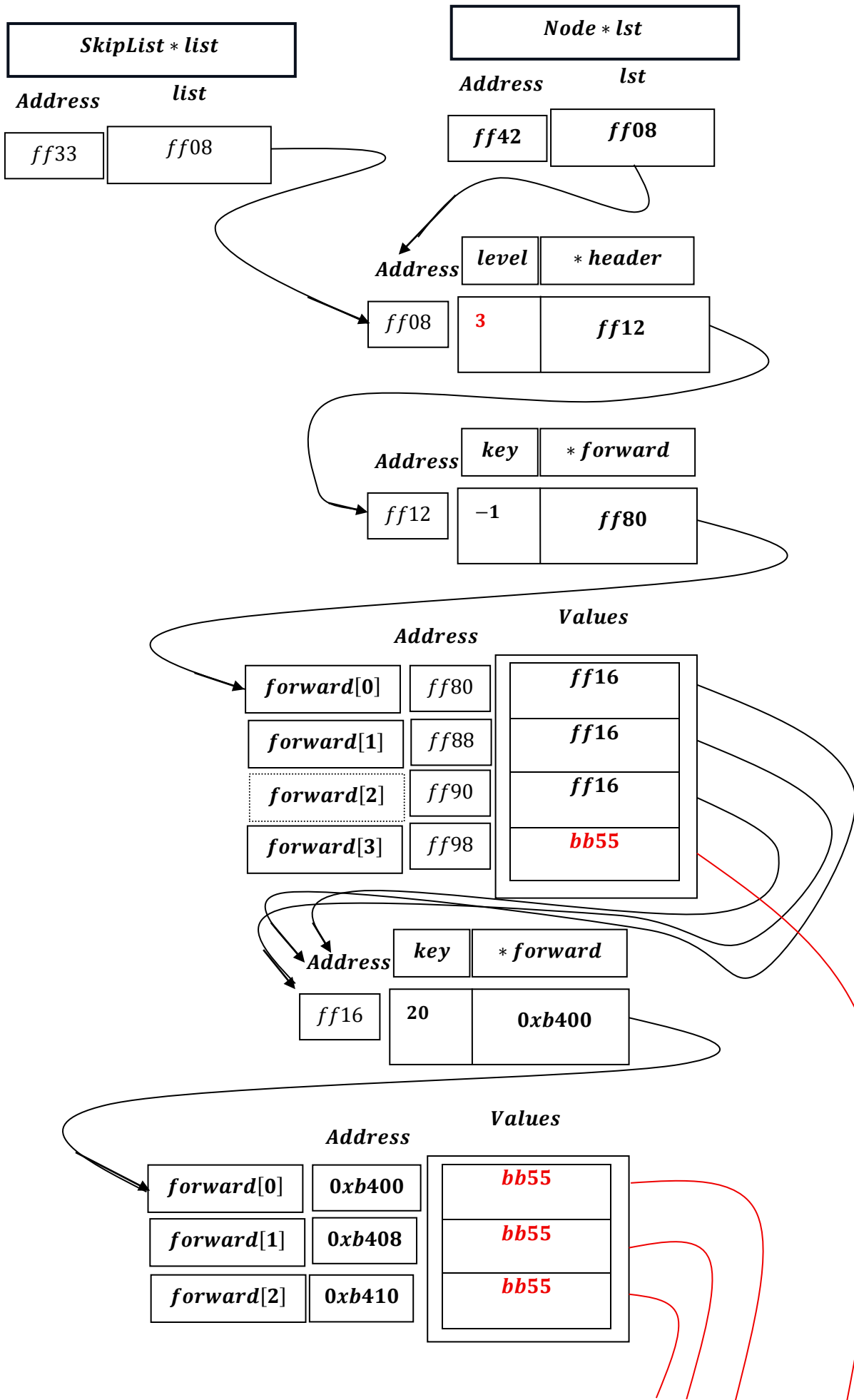
```
searchElement(list, key);
```

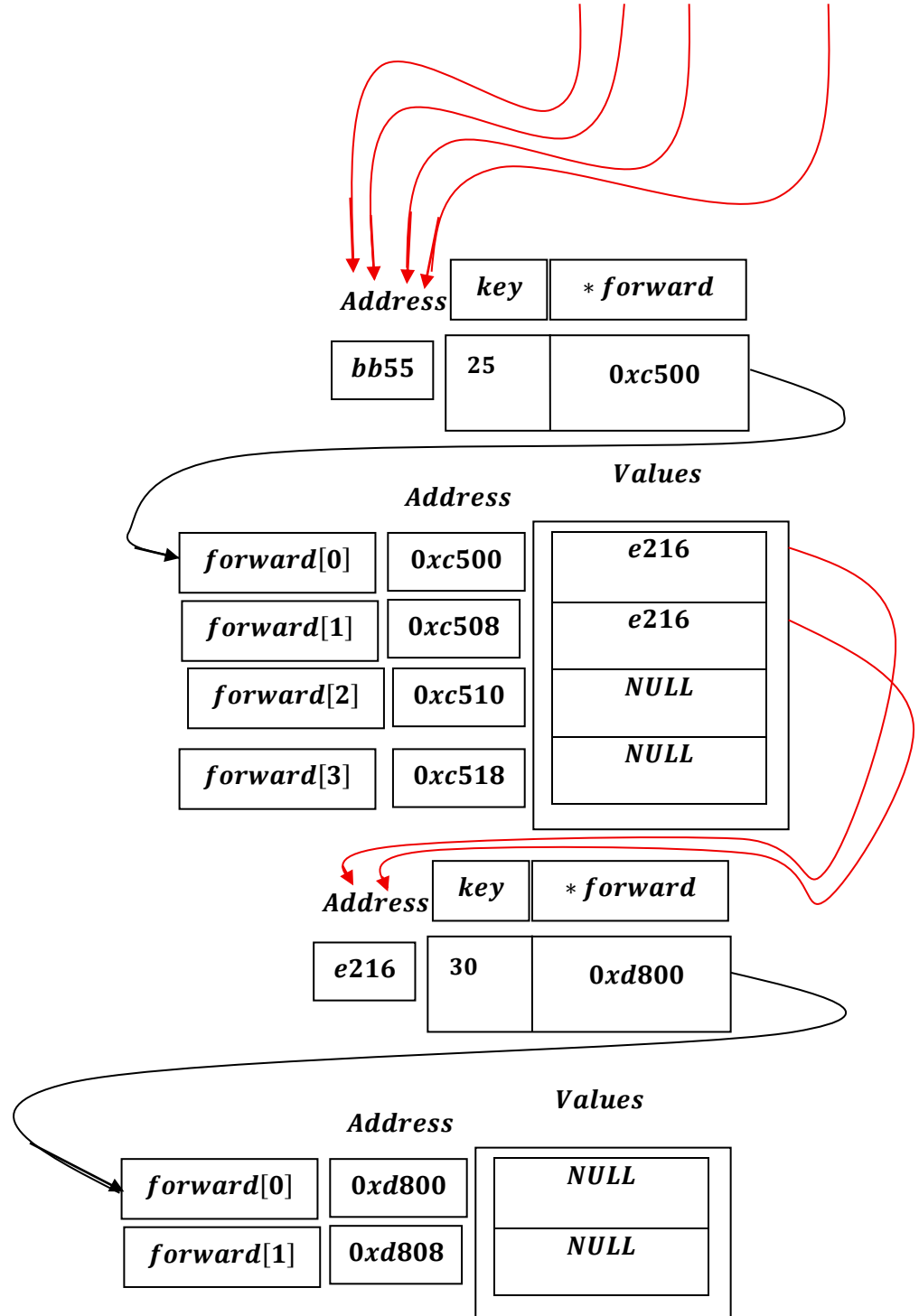
searchElement(list: ff08 ,key: 30) $\rightarrow$ passed .

And,

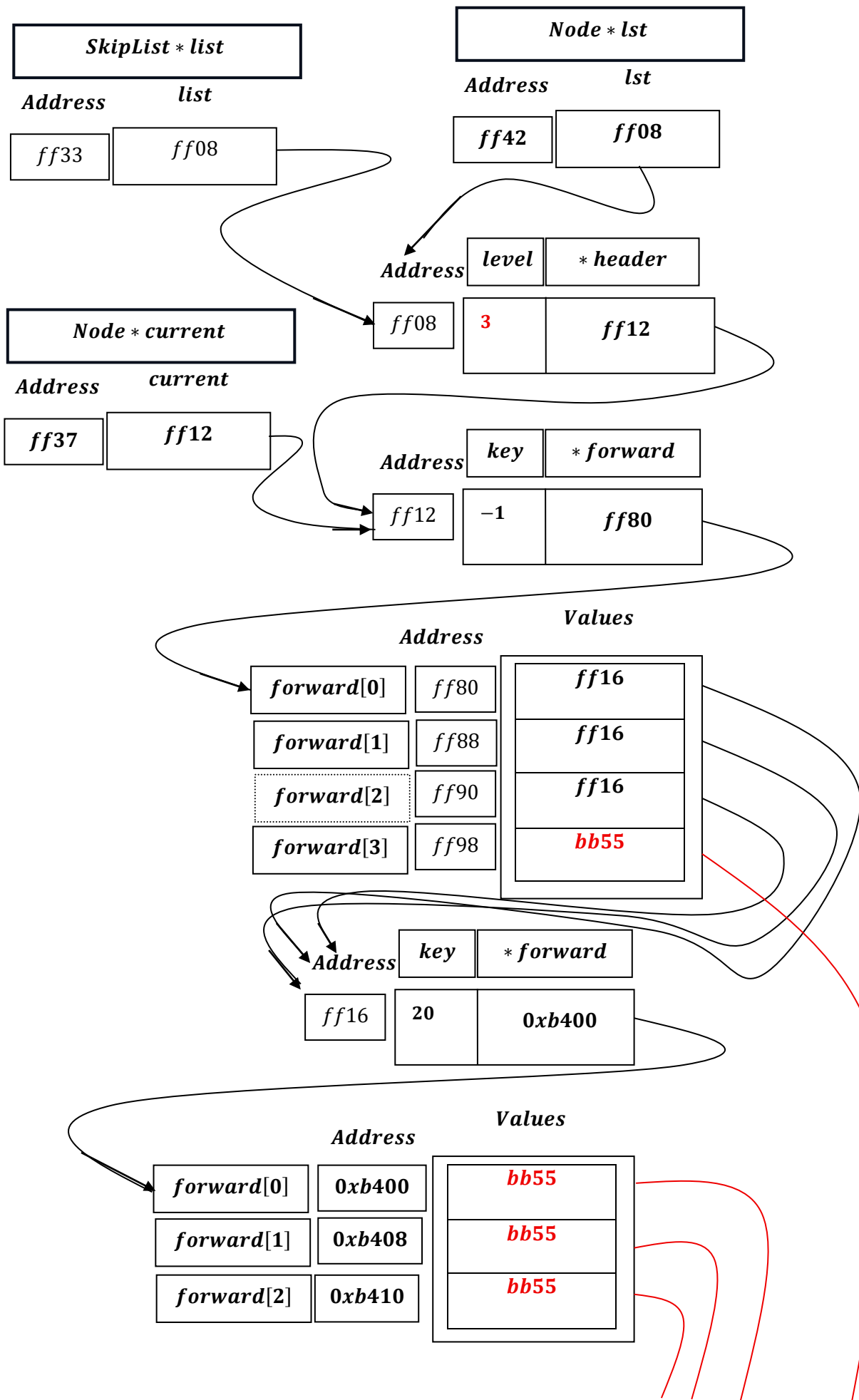
```
void searchElement(struct SkipList *lst, int key)
```

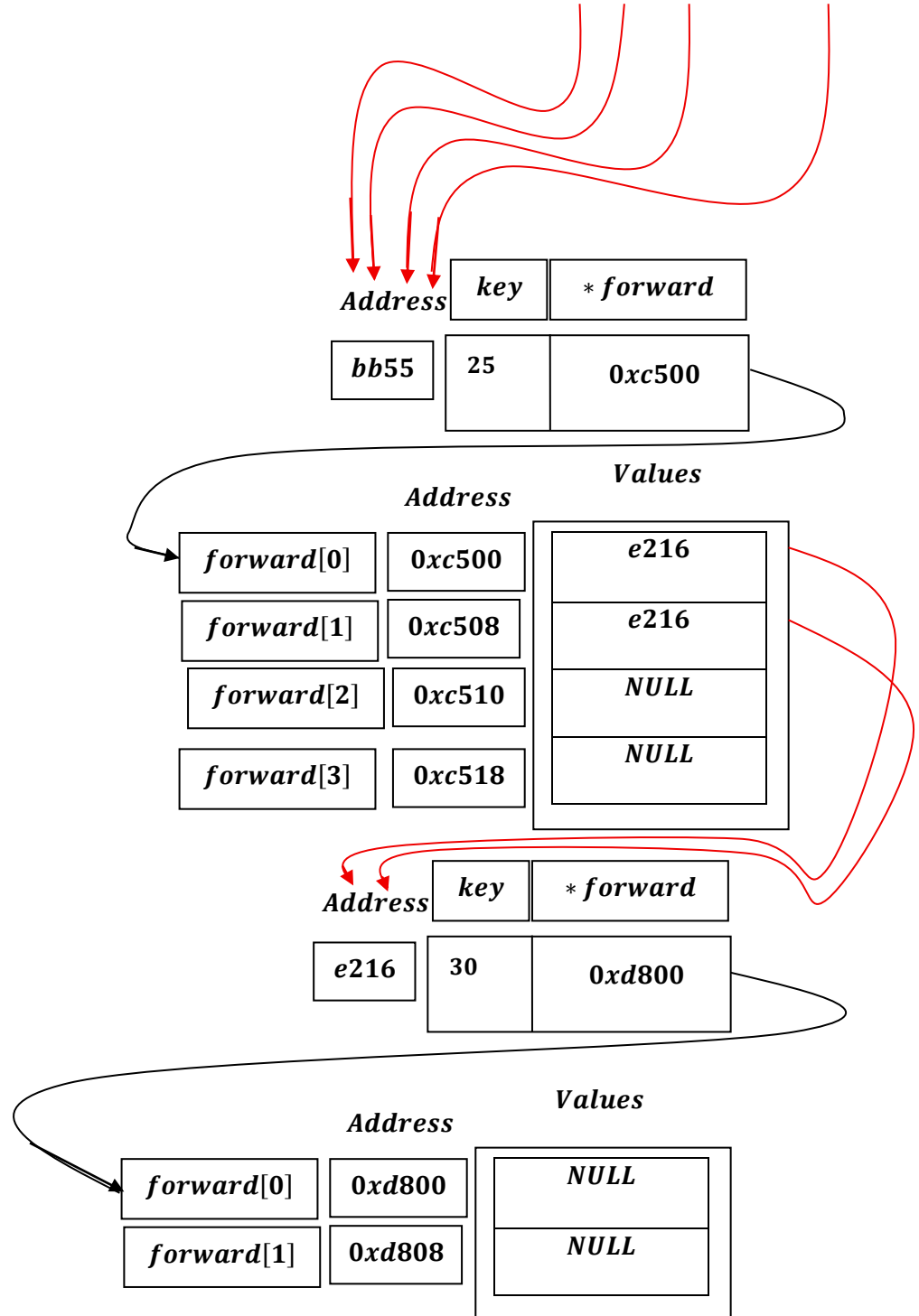
searchElement(list := lst: ff08 ,key := key: 30) $\rightarrow$ Assigned





```
struct Node *current = lst->header;
current = lst → header = ff12.
```





```

for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
}

```

lst → *level* = 3

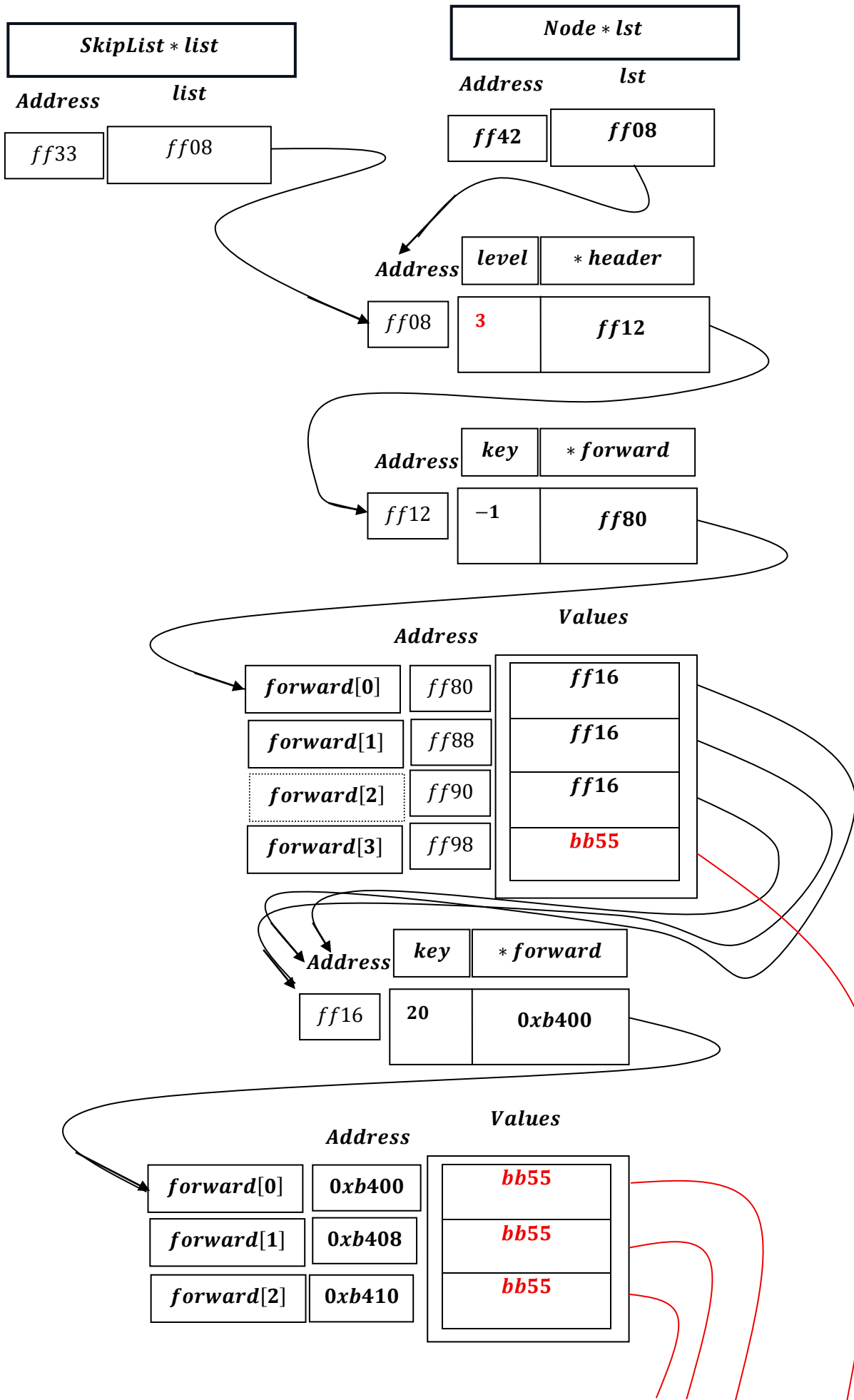
$i = lst \rightarrow level = 3; i \geq 0$

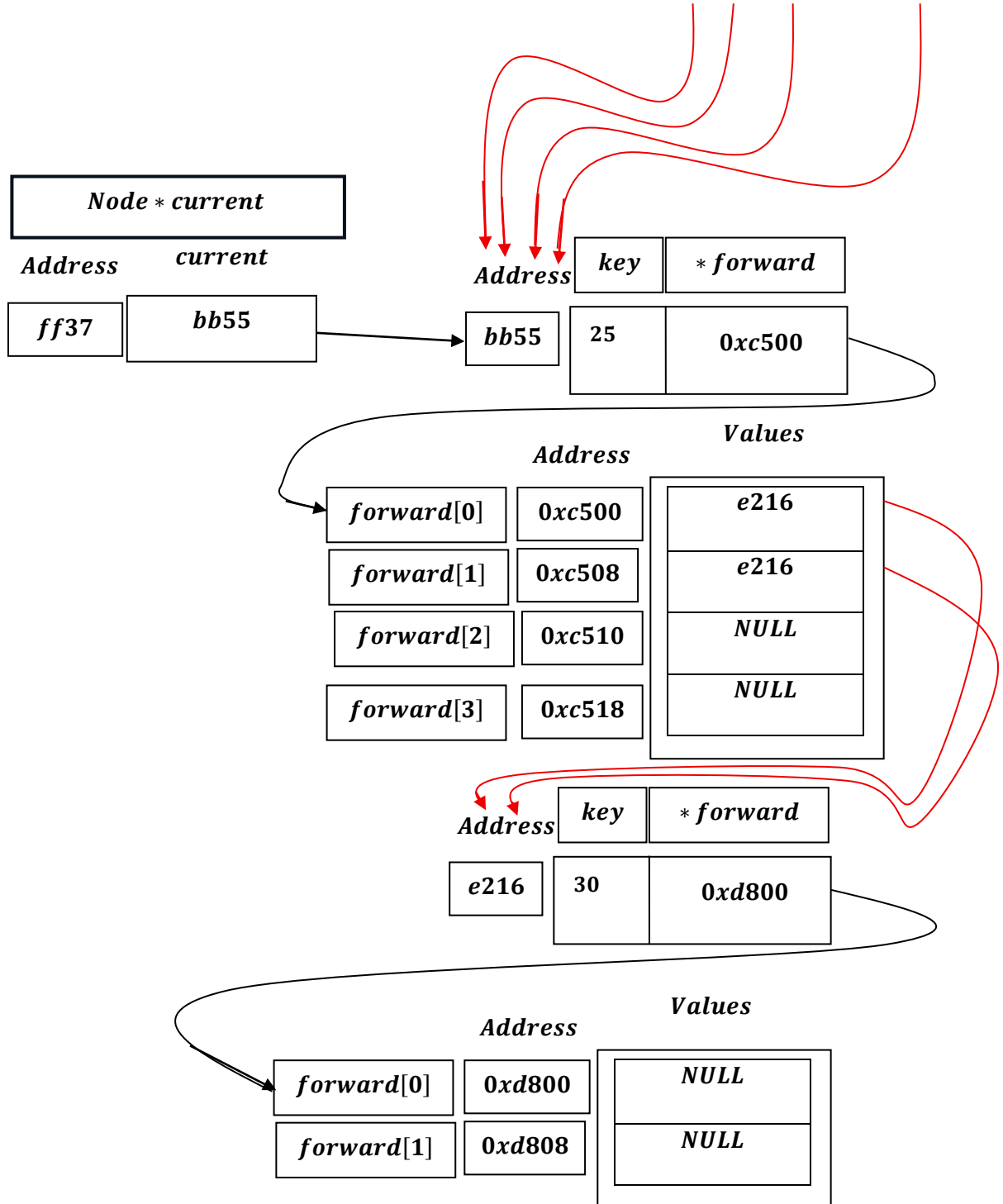
while: current: ff12 $\rightarrow$ forward[3]: ff98 $\rightarrow$ bb55 $\neq$ NULL [True] **AND**

current: ff12 $\rightarrow$ forward[3]: ff98[bb55] $\rightarrow$ key: 25 $<$ key: 30 [True]:

current = current: ff12 $\rightarrow$ forward[3]: ff98 = bb55

while: current: bb55 $\rightarrow$ forward[3]: 0xc518: NULL $\neq$ NULL [False]





$$i \longrightarrow i = i - 1 = 3 - 1 = 2$$

$i = 2; i \geq 0$

$\text{while: current: } bb55 \rightarrow \text{forward}[2]: 0xc518: NULL \neq NULL \text{ [False]}$

$i -- \Rightarrow i = i - 1 = 2 - 1 = 1$

$i = 1; i \geq 0$

$\text{while: current: } bb55 \rightarrow \text{forward}[1]: 0xc508 \rightarrow e216 \neq NULL \text{ [True]}$

$\text{current: } bb55 \rightarrow \text{forward}[1]: 0xc508[e216] \rightarrow \text{key: } 30 < \text{key: } 30 \text{ [False]}$

$i -- \Rightarrow i = i - 1 = 1 - 1 = 0$

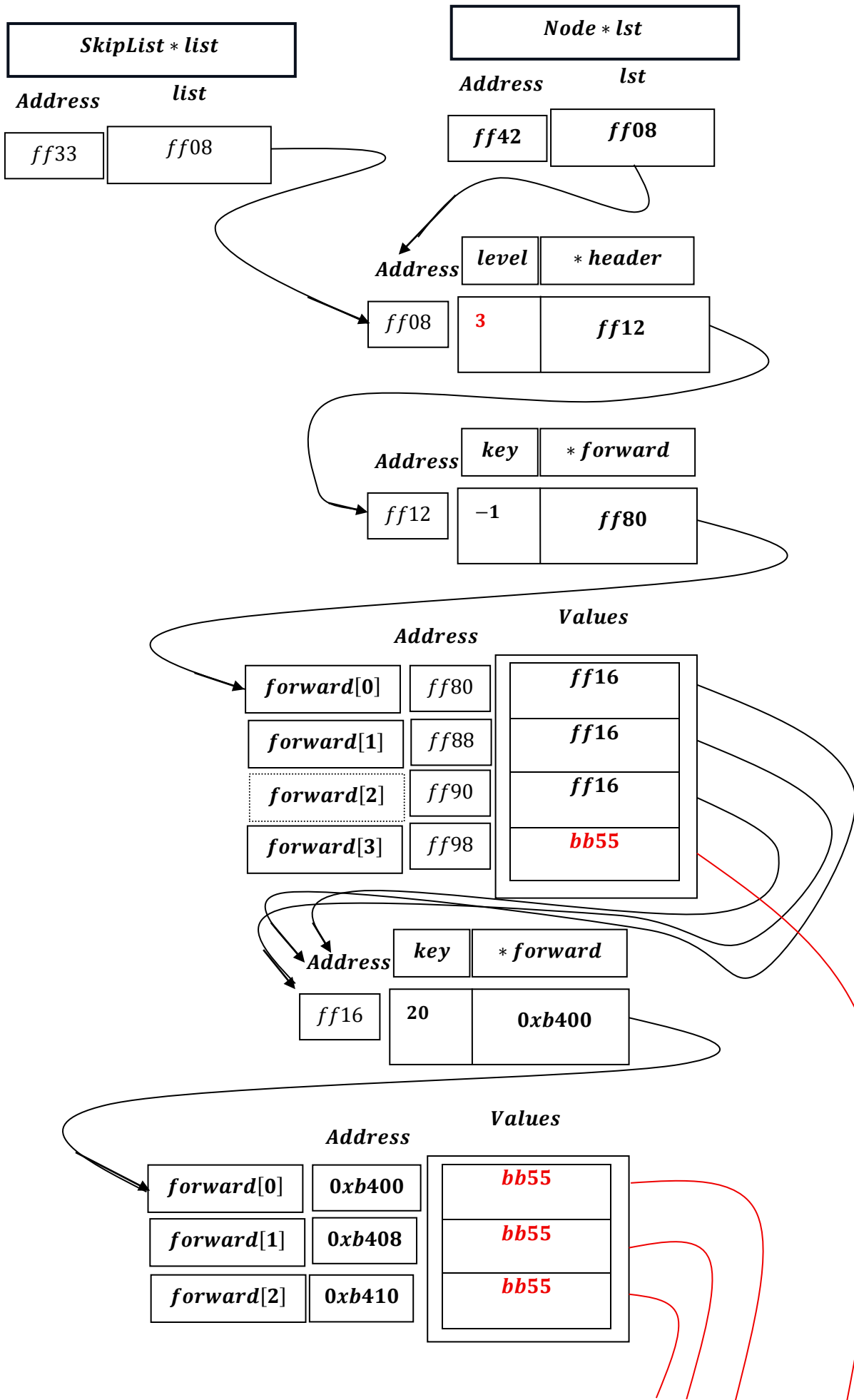
$i = 0; i \geq 0$

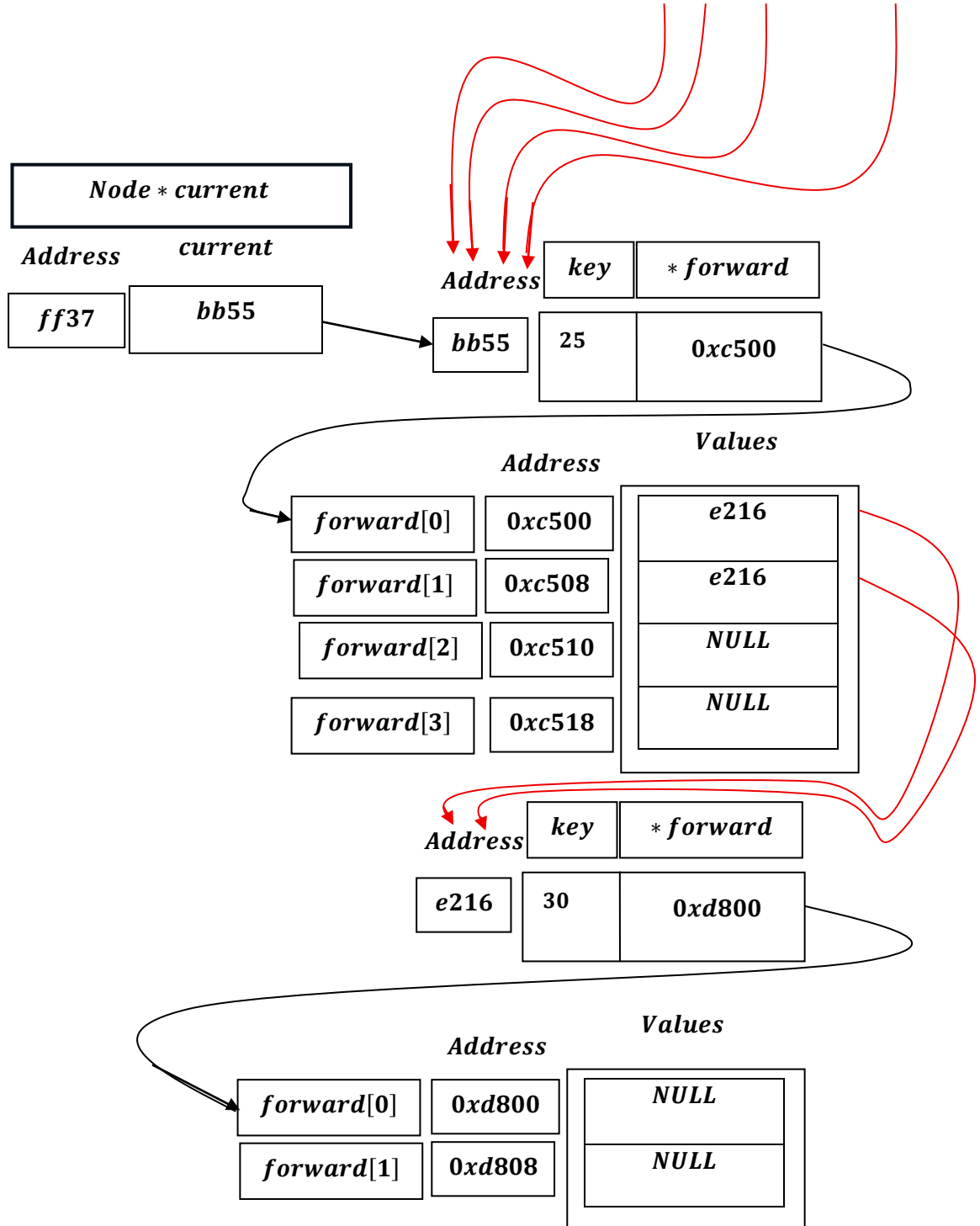
$\text{while: current: } bb55 \rightarrow \text{forward}[0]: 0xc500 \rightarrow e216 \neq NULL \text{ [True]}$

$\text{current: } bb55 \rightarrow \text{forward}[0]: 0xc500[e216] \rightarrow \text{key: } 30 < \text{key: } 30 \text{ [False]}$

$i -- \Rightarrow i = i - 1 = 0 - 1 = -1$

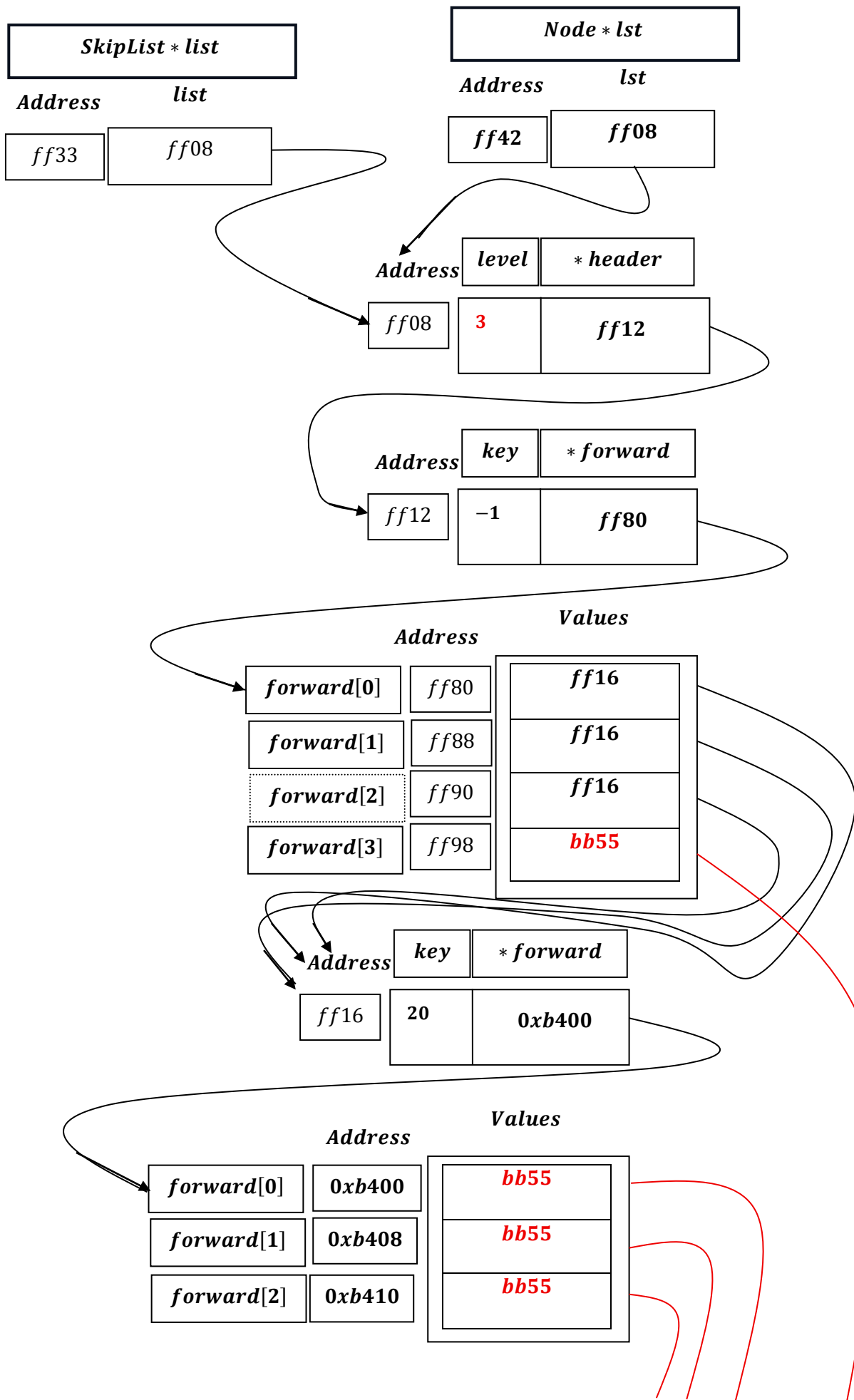
$i = -1; i \geq 0 \left[\begin{array}{l} \text{For loop condition false,} \\ \text{and Loop exits.} \end{array} \right]$

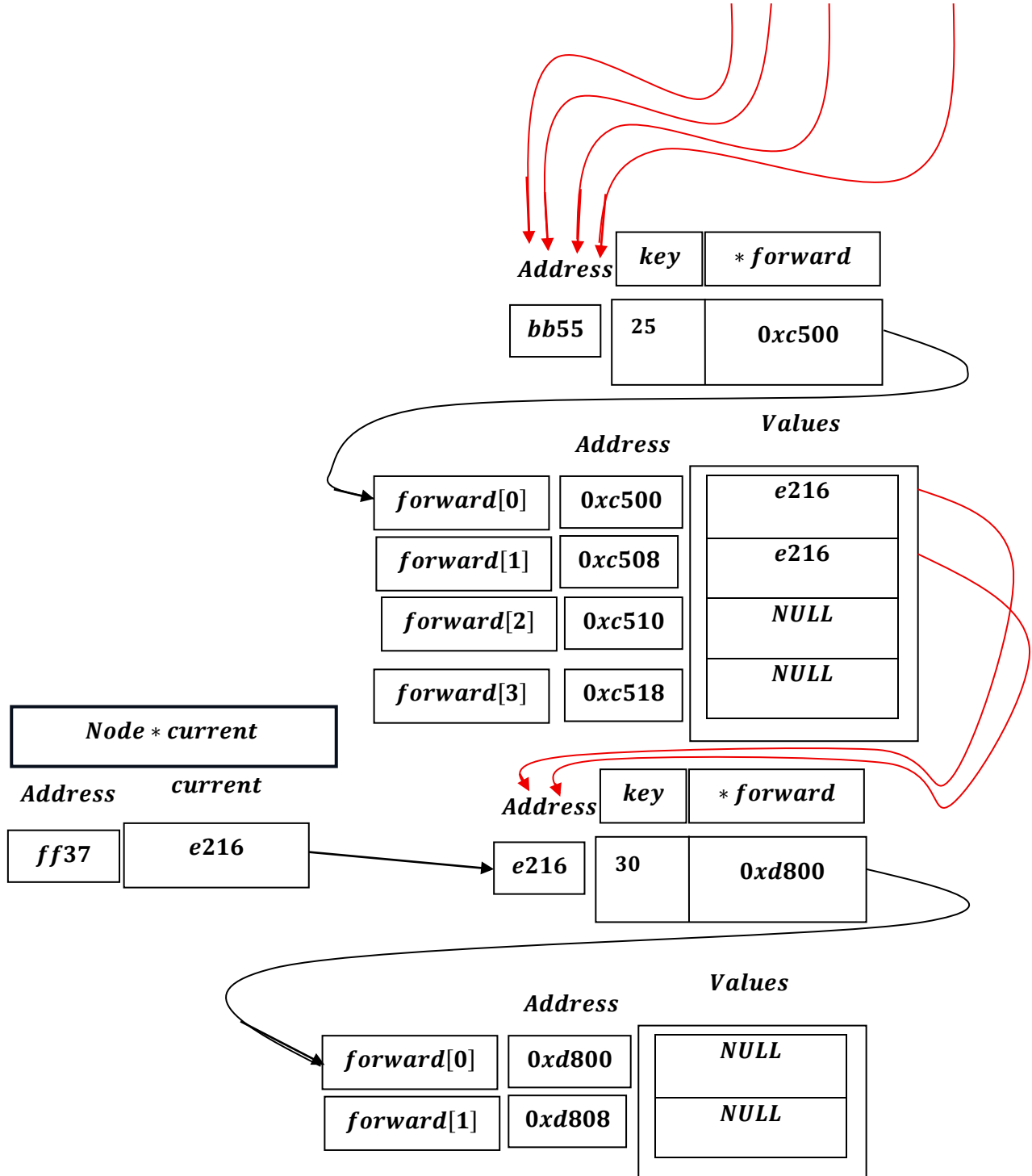




```
// Step into the exact match at level 0
current = current->forward[0];
```

current = *current*: bb55 → forward[0]: 0xc500 → e216.





```
if (current != NULL && current->key == key) {
    cout << "Found key " << key << " in the Skip List." << endl;
}
```

current: e216 ≠ NULL [True] AND current: e216 → key: 30 = key: 30 [True]:
Print "Found key 30 in the Skip List."

3rd, when $current \rightarrow key \neq key$ $\begin{bmatrix} \text{Element} \\ \text{Not} \\ \text{Found} \end{bmatrix}$

```
searchElement(list, key);
```

searchElement(list: ff08, key: 15) → passed .

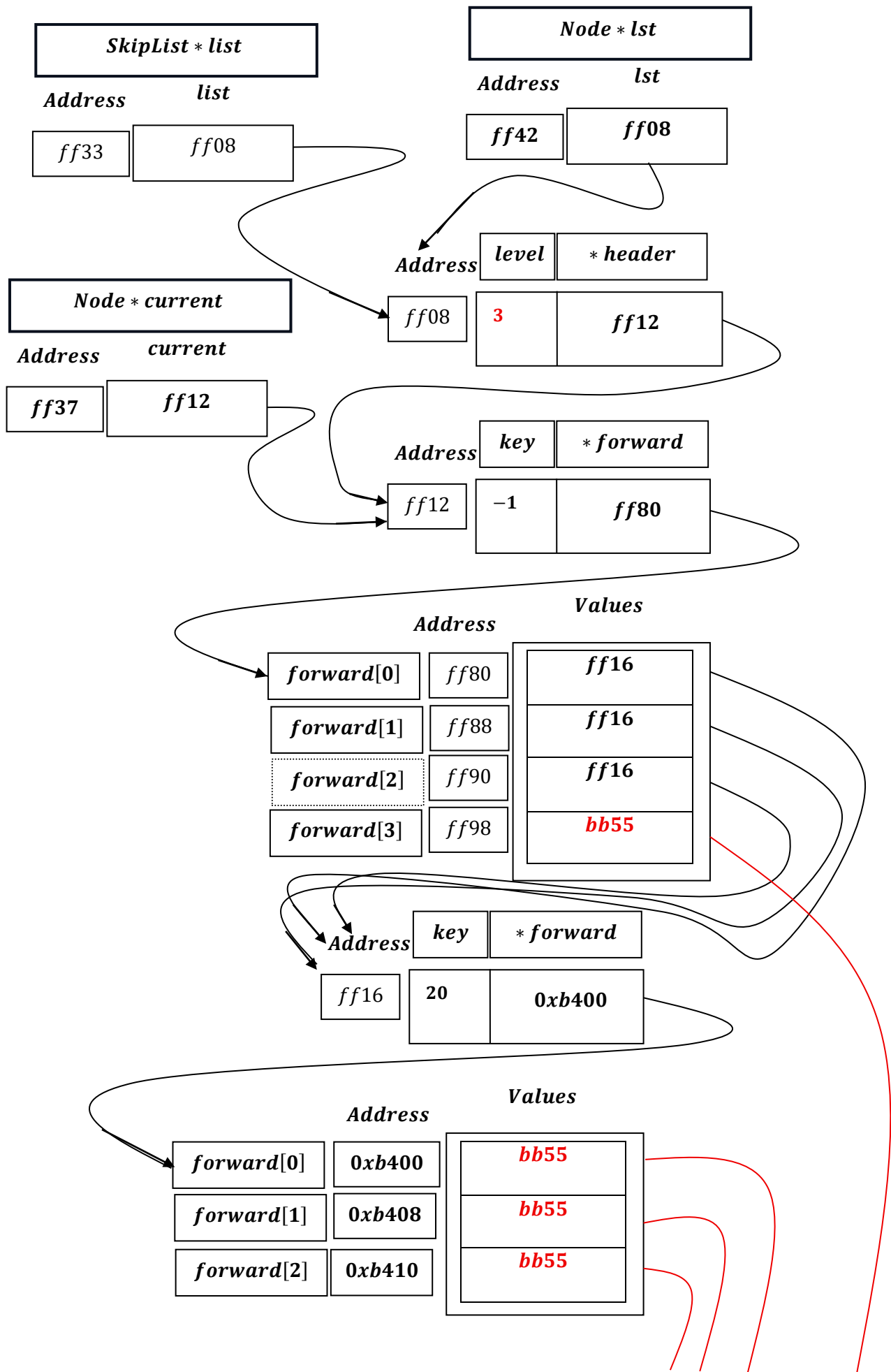
And,

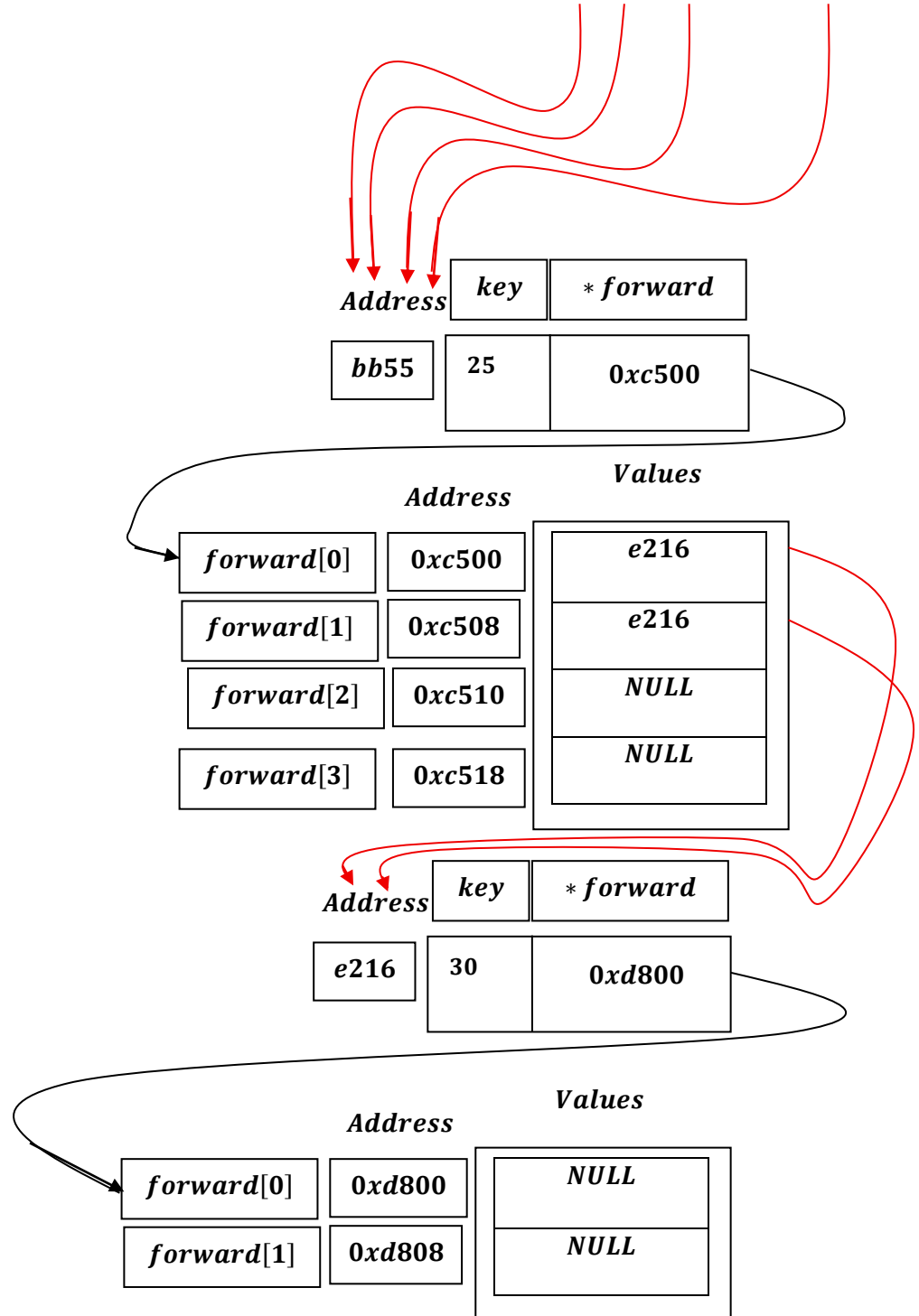
```
void searchElement(struct SkipList *lst, int key)
```

searchElement(list := lst: ff08, key := key: 15) → Assigned

```
struct Node *current = lst->header;
```

current = lst → header = ff12.





```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
}
```

lst → *level* = 3

$i = lst \rightarrow level = 3; i \geq 0$

while: $current: ff12 \rightarrow forward[3]: ff98 \rightarrow bb55 \neq NULL$ [True] **AND**

$current: ff12 \rightarrow forward[3]: ff98[bb55] \rightarrow key: 25 < key: 15$

Condition
False
Loop
Exit

$i -- \Rightarrow i = i - 1 = 3 - 1 = 2$

$i = 2; i \geq 0$

while: $current: ff12 \rightarrow forward[2]: ff90 \rightarrow ff16 \neq NULL$ [True] **AND**

$current: ff12 \rightarrow forward[3]: ff90[ff16] \rightarrow key: 20 < key: 15$

Condition
False
Loop
Exit

$i -- \Rightarrow i = i - 1 = 2 - 1 = 1$

while: $current: ff12 \rightarrow forward[1]: ff88 \rightarrow ff16 \neq NULL$ [True] **AND**

$current: ff12 \rightarrow forward[1]: ff88[ff16] \rightarrow key: 20 < key: 15$

Condition
False
Loop
Exit

$i -- \Rightarrow i = i - 1 = 1 - 1 = 0$

while: $current: ff12 \rightarrow forward[0]: ff80 \rightarrow ff16 \neq NULL$ [True] **AND**

$current: ff12 \rightarrow forward[0]: ff80[ff16] \rightarrow key: 20 < key: 15$

Condition
False
Loop
Exit

$i -- \Rightarrow i = i - 1 = 0 - 1 = -1$

$i = -1; i \geq 0$

For loop condition false, and Loop exits.
--

```
// Step into the exact match at level 0
current = current->forward[0];
```

current = current: ff12 → forward[0]: ff80 → ff16.

```
if (current != NULL && current->key == key) {
    cout << "Found key " << key << " in the Skip List." << endl;
} else {
    cout << "Key " << key << " not found." << endl;
}
```

current: ff16 ≠ NULL [TRUE] AND current: ff16 → key: 20 ≠ key: 15 $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

Else

Print "Key 15 not found."

And , function ends.

Skip – List
–Search Element
–Program Analysis

Program Analysis:

```
void searchElement(struct SkipList *lst, int key) {
    struct Node *current = lst->header;

    // Traverse from highest level to the lowest
    for (int i = lst->level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key <
key) {
            current = current->forward[i];
        }
    }

    // Step into the exact match at level 0
    current = current->forward[0];

    if (current != NULL && current->key == key) {
        cout << "Found key " << key << " in the Skip List." << endl;
    } else {
        cout << "Key " << key << " not found." << endl;
    }
}

...
case 3:
    cout << "Enter key to search: ";
    cin >> key;
    searchElement(list, key);
    break;
```

Time Complexity, Space Complexity and Algorithm

A)Time Complexity

Note: –This searching part of the program, already discussed during insertion of element lets make it short:

1. Best Case Time Complexity: $O(1)$

The absolute best – case scenario occurs when the search halts immediately without horizontally traversing any nodes.

- ***The Condition:** This happens if the **list is completely empty**, or **if the target key is smaller than the very first element in the list.***
- ***The Execution:** The horizontal **while** loop condition **$(current \rightarrow forward[i] \rightarrow key < key)$** evaluates to **False** on the very first check at every single level.*
- ***The Result:** The algorithm completely ducks the horizontal traversal. The outer for loop simply drops down vertically, which takes a tiny, constant amount of time because the maximum level is a hard fixed constant.*

2. Worst Case Time Complexity: $O(n)$

The worst – case scenario occurs when the probabilistic balancing of the Skip List suffers a complete structural failure. This degrades the structure into a standard singly linked list, forcing a linear scan.

We can summarize this using either of the two extreme scenarios:

- ***Scenario A (The All Tails Flat List):*** *Every node generated a level of 0, meaning no express lanes exist. The algorithm is forced to traverse every single node one by one on Level 0.*
- ***Scenario B (The All Heads Traffic Jam):*** *Every node generated the maximum possible level (e. g. , Level 3). Because every node exists on the top floor, the pointer cannot skip over anything and is forced to perform a linear $O(n)$ traversal across the crowded top level.*

This we have discussed in Insert Element.

3. Average Case Time Complexity: $O(\log n)$

To summarize the average case without rewriting all the math, we only need to reference William Pugh's "Backward Walk" analysis.

- **The Expected Steps:** Because every node's height is determined by an independent coin flip with probability p , a geometric distribution dictates that the expected number of horizontal steps taken on any level before moving up is $\frac{1}{p}$.
- **The Expected Height:** The expected maximum height of the list, $L(n)$, where we expect to find exactly 1 node, is $\log_{\frac{1}{p}}(n)$.
- **The Total Cost:** The total expected search cost is the number of levels multiplied by the expected steps per level.

$$E[\text{search cost}] = \frac{L(n)}{p} + \frac{p}{1-p}$$

[This we have found out from William Pugh's Explanation and its adaptation to our – 0 based index program.]

- **The Conclusion:** When evaluated for a fair coin toss ($p = 0.5$), the constants drop away, leaving a strict average time complexity of $O(\log_2 n)$.

$O(\log_2 n)$ falls to : $O(\log n)$ category as $\frac{1}{p}$ and $\frac{p}{1-p}$ are constants

and $L(n) = \log_{\frac{1}{p}}(n)$.

B) Space Complexity

1. Input Space Complexity

The input space for the Search module is completely identical to the Insertion module because both functions act on the exact same data structure using the exact same arguments.

A. Parametric Input Space: $O(1)$

- The search function takes the same parameters: a memory pointer addressing the existing skip list structure (**lst**) and a standard integer variable (**key**).*
- Because these take a small, constant amount of memory (typically 8 bytes for **lst** pointer variable in 64 – bit architecture) and 4 bytes for **key** integer variable.*
*that never scales with the number of elements (**n**),*
*the parametric input space is strictly **$O(1)$** .*

B. Full Data Holistic Input Space: $O(n)$

- *The function operates on the entire pre – existing Skip List data structure.*
- *Because the expected maximum number of pointers per node remains a small mathematical constant, the total memory footprint to store n nodes scales directly and linearly with the number of elements.*
- *Therefore, the full data holistic input space is $O(n)$.*

2. Auxiliary Space Complexity: $O(1)$

Here is where the Search module becomes drastically simpler to write about than the Insertion module!

- ***The Insertion Contrast:*** *In insertion analysis, the primary consumer of auxiliary memory was the update array, which temporarily stored the memory addresses of predecessor nodes. Because that array had to scale with the maximum height of the list, it introduced a theoretical auxiliary space of $O(\log_2 n)$ (even though it was practically capped at $O(1)$).*

***[This we have already discussed in Insertion Element's
Space Complexity]***

- ***The Search Reality:*** *The `searchElement` module does not use an update array. It does not need to remember its path to rewire pointers later.*

- ***The Conclusion:*** *To execute a search, the algorithm only requires a single temporary pointer (**current**) and a loop counter variable (**i**). Because it allocates zero arrays, makes zero recursive calls, and requests no extra memory, the auxiliary space complexity for searching is an absolute, strict **O(1)** in both theoretical and practical scenarios.*

C) Algorithm

Algorithm: SkipListSearch(LST, KEY)

1. [Initialize Pointer]

SET CURRENT := HEADER[LST].

2. [Traverse List Vertically and Horizontally]

Repeat for $i = \text{LEVEL}[LST]$ down to 0:

Repeat while $\text{FORWARD}[i][\text{CURRENT}] \neq \text{NULL AND INFO}[\text{FORWARD}[i][\text{CURRENT}]] < \text{KEY}$:

SET CURRENT := FORWARD[i][CURRENT].

[End of while Loop]

[End of for Loop]

3. [Step into Exact Match]

SET CURRENT := FORWARD[0][CURRENT].

4. [Verify Key Existence]

If CURRENT $\neq$ NULL AND INFO[CURRENT] = KEY, Then:

Write: "Found key ", KEY, " in the Skip List."

Else:

Write: "Key ", KEY, " not found."

[End of If Structure]

5. [Exit]

END OF ALGORITHM.

Skip – List
–Delete Element
–Program Analysis – Type A

Program Analysis:

```
void deleteElement(struct SkipList *lst, int key) {
    struct Node *current = lst->header;
    struct Node *update[MAX_LEVEL + 1];
    memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));

    // Find the node, keeping track of pointers that point to it
    for (int i = lst->level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key <
key) {
            current = current->forward[i];
        }
        update[i] = current;
    }

    current = current->forward[0];

    // If the target node is found
    if (current != NULL && current->key == key) {
        // Rewire the pointers to bypass the deleted node
        for (int i = 0; i <= lst->level; i++) {
            if (update[i]->forward[i] != current) {
                break; // If a level doesn't point to current, higher levels
won't
                // either
            }
            update[i]->forward[i] = current->forward[i];
        }

        // Free memory using free()
        free(current->forward);
        free(current);

        // Reduce the maximum level of the list if layers become empty
        while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
```

```
    lst->level--;  
    }  
    cout << "Successfully deleted key " << key << endl;  
} else {  
    cout << "Key " << key << " not found. Cannot delete." << endl;  
}  
}  
  
... .  
case 2:  
    cout << "Enter key to delete: ";  
    cin >> key;  
    deleteElement(list, key);  
    break;
```


Program Analysis

```
deleteElement(list, key);
```

deleteElement(list: ff08, key: 30) → passed .

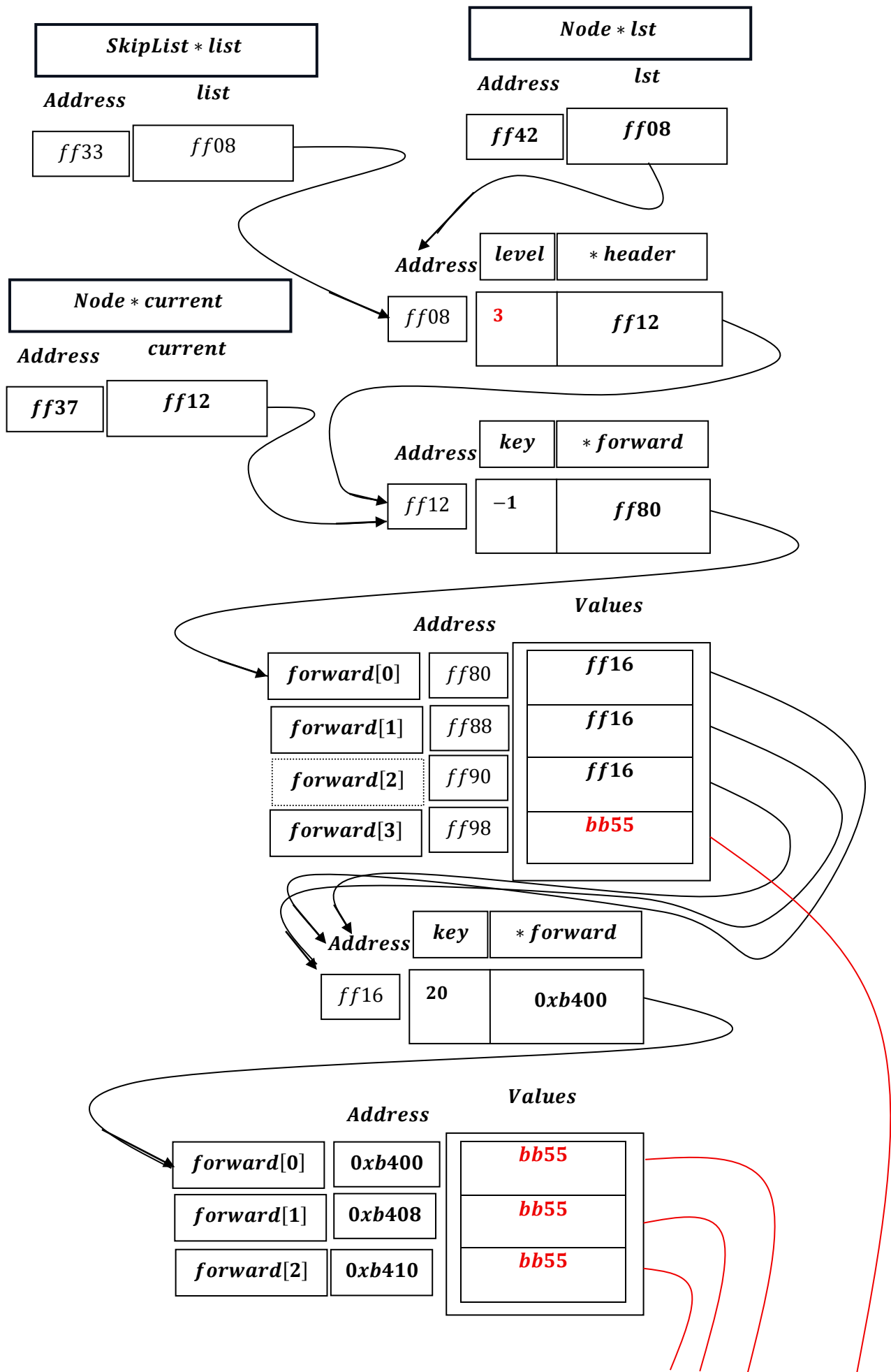
And,

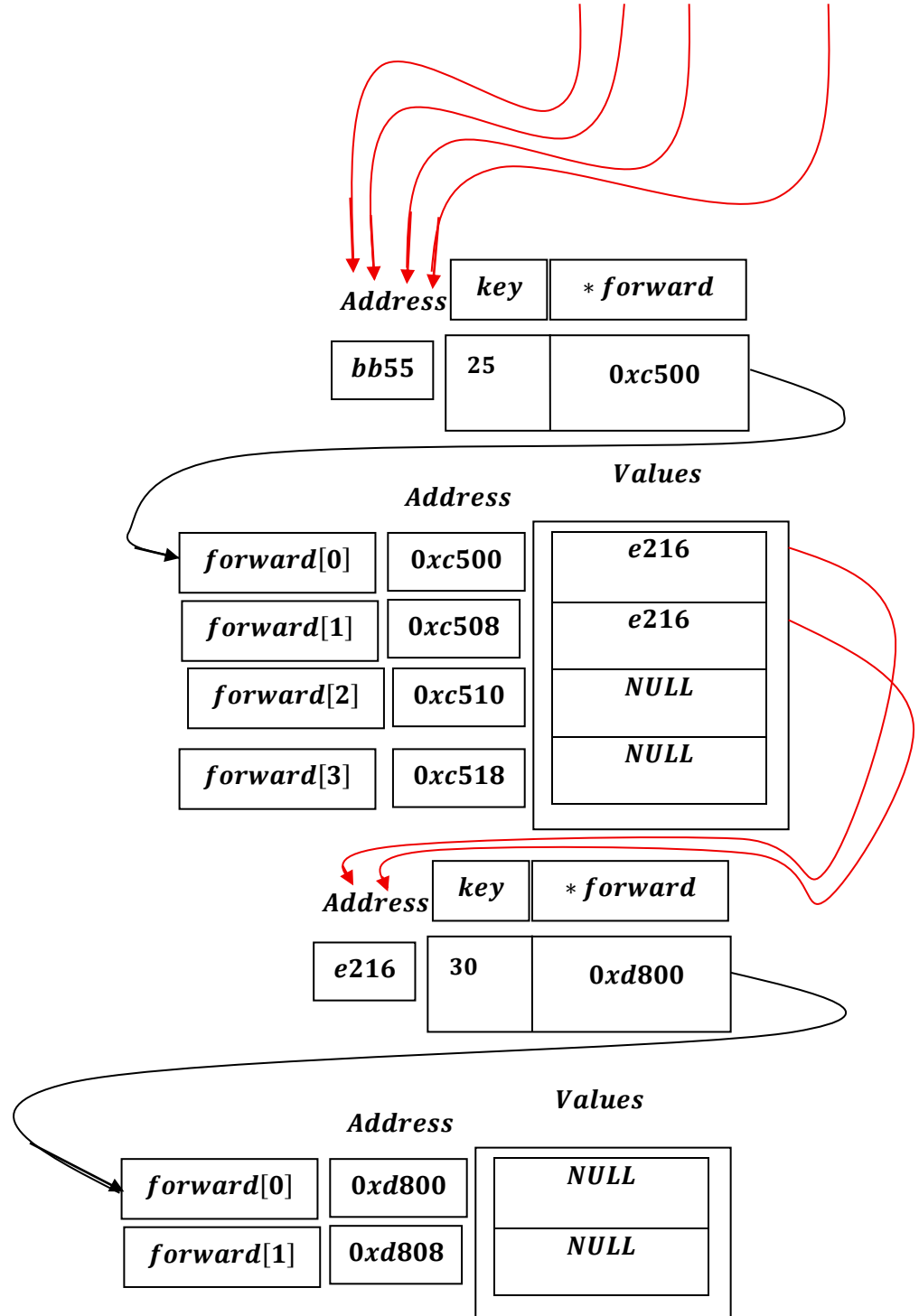
```
void deleteElement(struct SkipList *lst, int key)
```

deleteElement(list := lst: ff08, key := key: 15) → Assigned

```
    struct Node *current = lst->header;
```

current = lst → header = ff12.





```
struct Node *update[MAX_LEVEL + 1];
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

```
struct Node *update[MAX_LEVEL + 1];
```

update[3 + 1 = 4],

MAX_LEVEL evaluates to 3. Consequently, the array size calculation is update[3 + 1 = 4].

This means the array is allocated to hold exactly 4 elements.

Before memset call, the update array may look like :

<i>Array Element</i>	<i>Hypothetical Stack Address</i>	<i>Value (Garbage Data)</i>	<i>Pointer Status</i>
<i>update[0]</i>	<i>0x7fff0000 to 0x7fff0007</i>	<i>0x4a3f8921bce45099</i>	<i>Wild Pointer (Points to a random, invalid address)</i>
<i>update[1]</i>	<i>0x7fff0008 to 0x7fff000f</i>	<i>0x0000000000000002</i>	<i>Wild Pointer</i>
<i>update[2]</i>	<i>0x7fff0010 to 0x7fff0017</i>	<i>0x7ffee1245a00881c</i>	<i>Wild Pointer</i>
<i>update[3]</i>	<i>0x7fff0018 to 0x7fff001f</i>	<i>0x0000000000000000</i>	<i>Accidentally NULL (Just by chance)</i>

More simpler way:

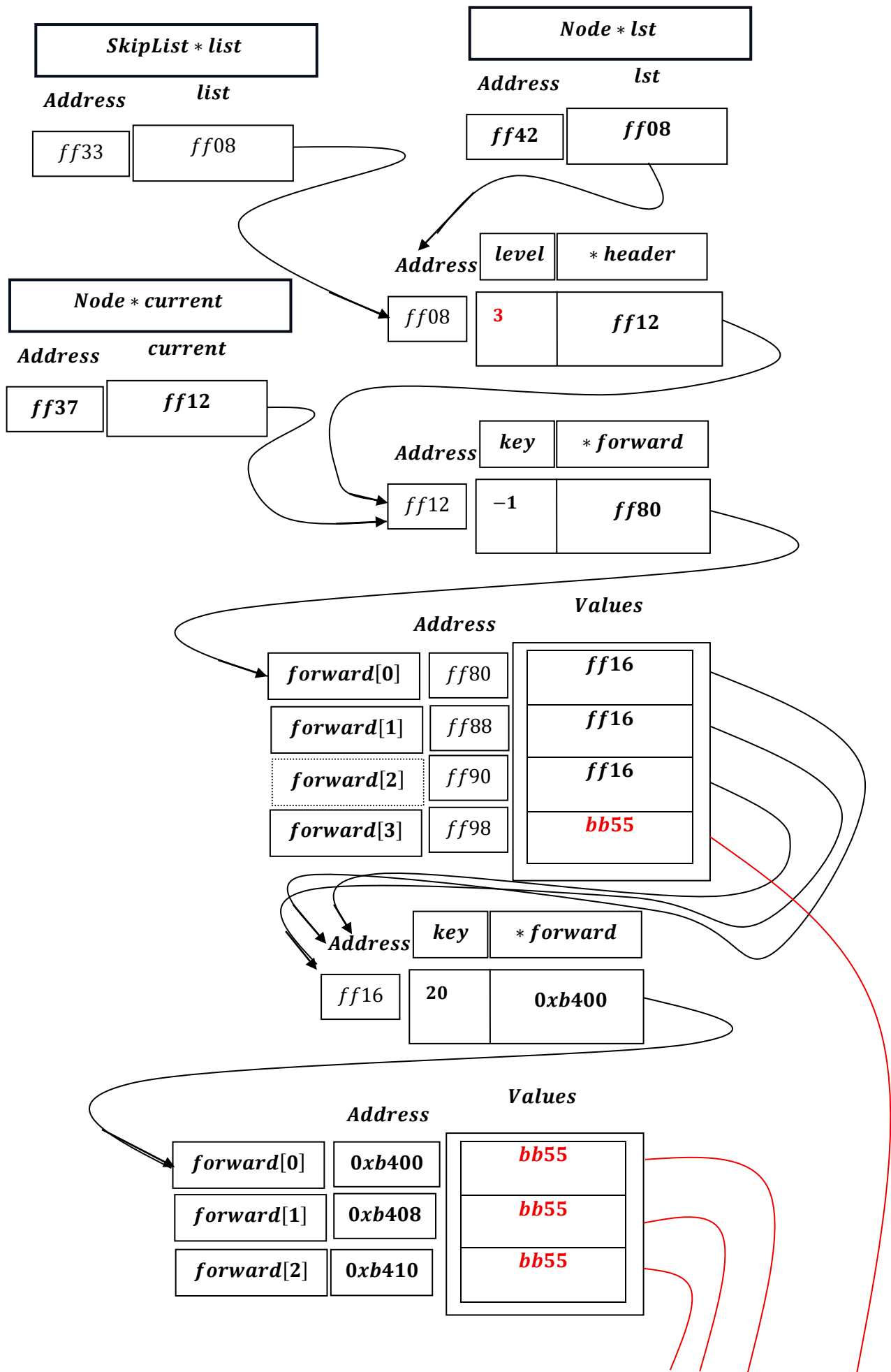
	<i>Address</i>	<i>Values</i>	
<i>update[0]</i>	<i>0x7fff0000</i>	<i>0x4a3f8921bce45099</i>	<i>[8 bytes]</i>
<i>update[1]</i>	<i>0x7fff0008</i>	<i>0x0000000000000002</i>	<i>[8 bytes]</i>
<i>update[2]</i>	<i>0x7fff0010</i>	<i>0x7ffee1245a00881c</i>	<i>[8 bytes]</i>
<i>update[3]</i>	<i>0x7fff0018</i>	<i>0x0000000000000000</i>	<i>[8 bytes]</i>

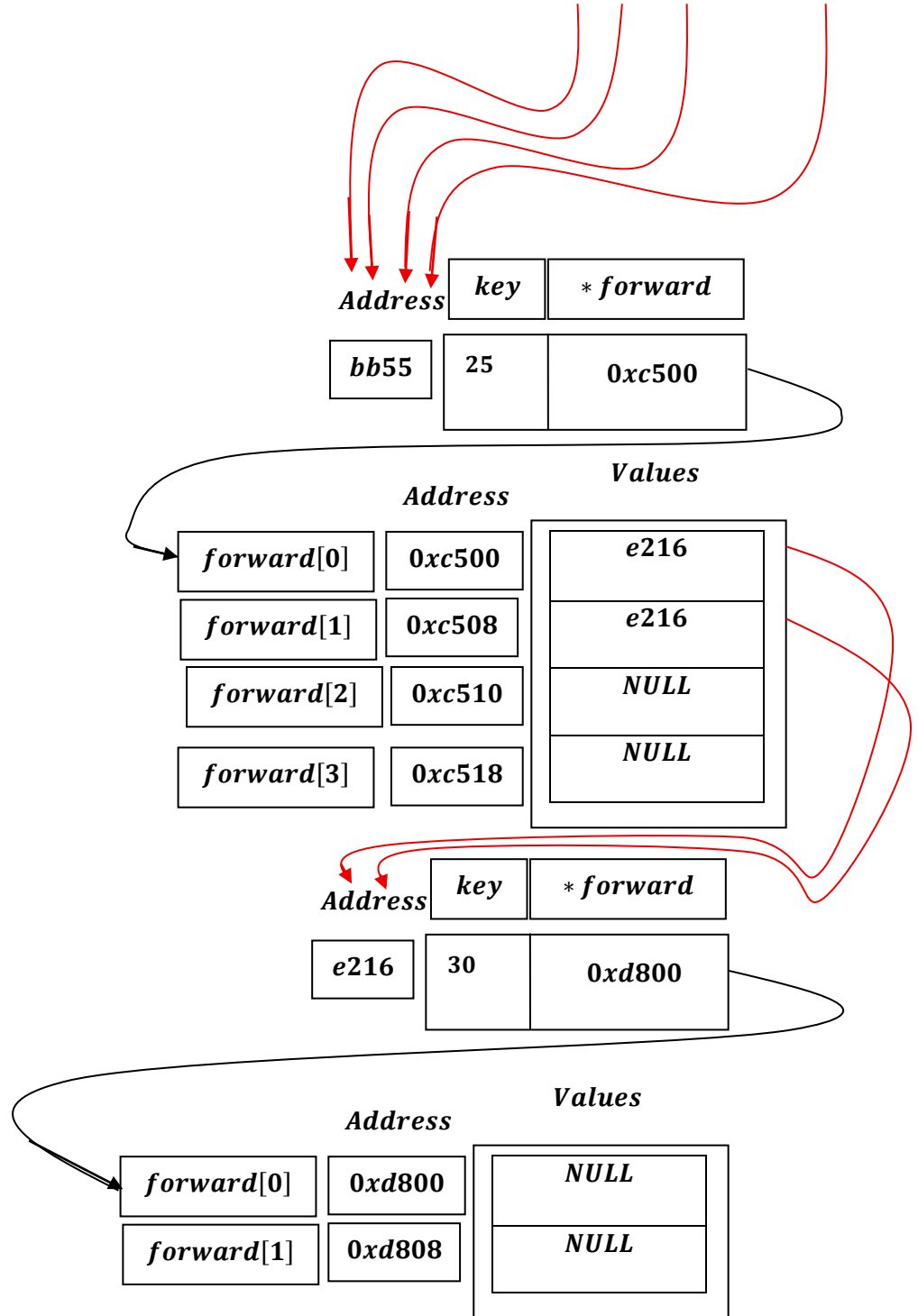
```
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

“memset” stands for Memory Set. It takes a block of memory and rapidly fills it with a specific value. Here `0` or NULL.

	<i>Address</i>	<i>Values</i>
<i>update[0]</i>	<i>0x7fff0000</i>	<i>NULL</i>
<i>update[1]</i>	<i>0x7fff0008</i>	<i>NULL</i>
<i>update[2]</i>	<i>0x7fff0010</i>	<i>NULL</i>
<i>update[3]</i>	<i>0x7fff0018</i>	<i>NULL</i>

*Here we kept addresses same like: 0x7fff0000
, just for an example , but it may not be same ,
in reality.*





	Address	Values
update[0]	0x7fff0000	NULL
update[1]	0x7fff0008	NULL
update[2]	0x7fff0010	NULL
update[3]	0x7fff0018	NULL

```

for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}

```

$i = lst \rightarrow level = 3; i \geq 0 :$

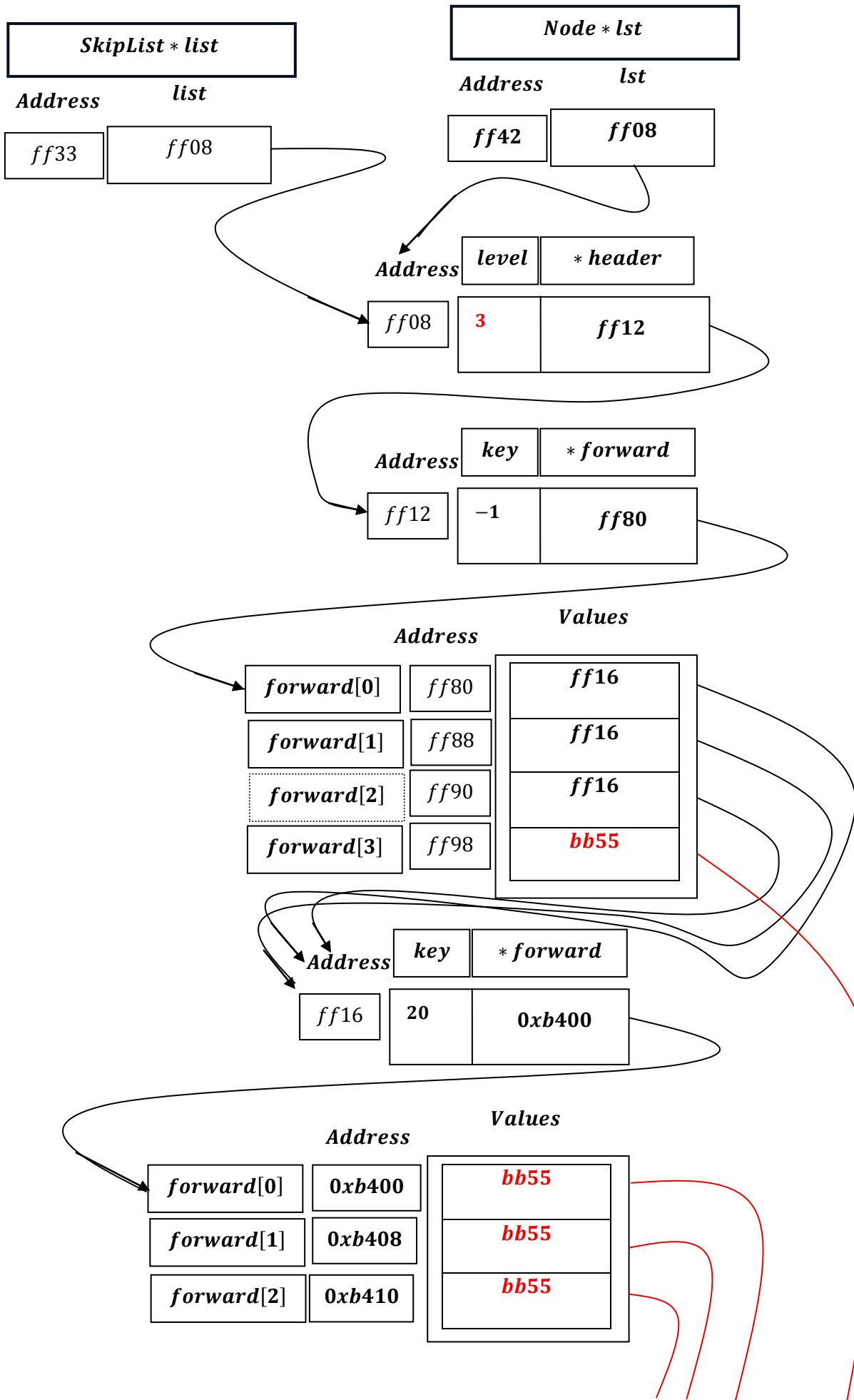
$current: ff12 \rightarrow forward[3]: ff98 \rightarrow bb55 \neq NULL[True] \text{ AND}$

$current: ff12 \rightarrow forward[3]: bb55 \rightarrow key: 25 < key: 30[True]$

$current = current: ff12 \rightarrow forward[3]: bb55$

$current: bb55 \rightarrow forward[3]: 0xc518 \rightarrow NULL \neq NULL[False]$

$update[3] = current: bb55.$



*Node * current*

Address

current

ff37

bb55

Address

key

** forward*

bb55

25

0xc500

Address

Values

forward[0]

0xc500

forward[1]

0xc508

forward[2]

0xc510

forward[3]

0xc518

e216

e216

NULL

NULL

Address

key

** forward*

e216

30

0xd800

Address

Values

forward[0]

0xd800

forward[1]

0xd808

NULL

NULL

Address

Values

update[0]

0x7fff0000

NULL

update[1]

0x7fff0008

NULL

update[2]

0x7fff0010

NULL

update[3]

0x7fff0018

bb55

$$i = i - - = i - 1 = 3 - 1 = 2$$

$i = 2; i \geq 0 :$

$current: bb55 \rightarrow forward[2]: 0xc510 \rightarrow NULL \neq NULL[False]$

$update[2] = current: bb55.$

$$i = i - - = i - 1 = 2 - 1 = 1$$

$i = 1; i \geq 0 :$

$current: bb55 \rightarrow forward[1]: 0xc508 \rightarrow e216 \neq NULL[True] AND$

$current: bb55 \rightarrow forward[1]: e216 \rightarrow key: 30 < key: 30[False]$

$update[1] = current: bb55.$

$$i = i - - = i - 1 = 1 - 1 = 0$$

$i = 0; i \geq 0 :$

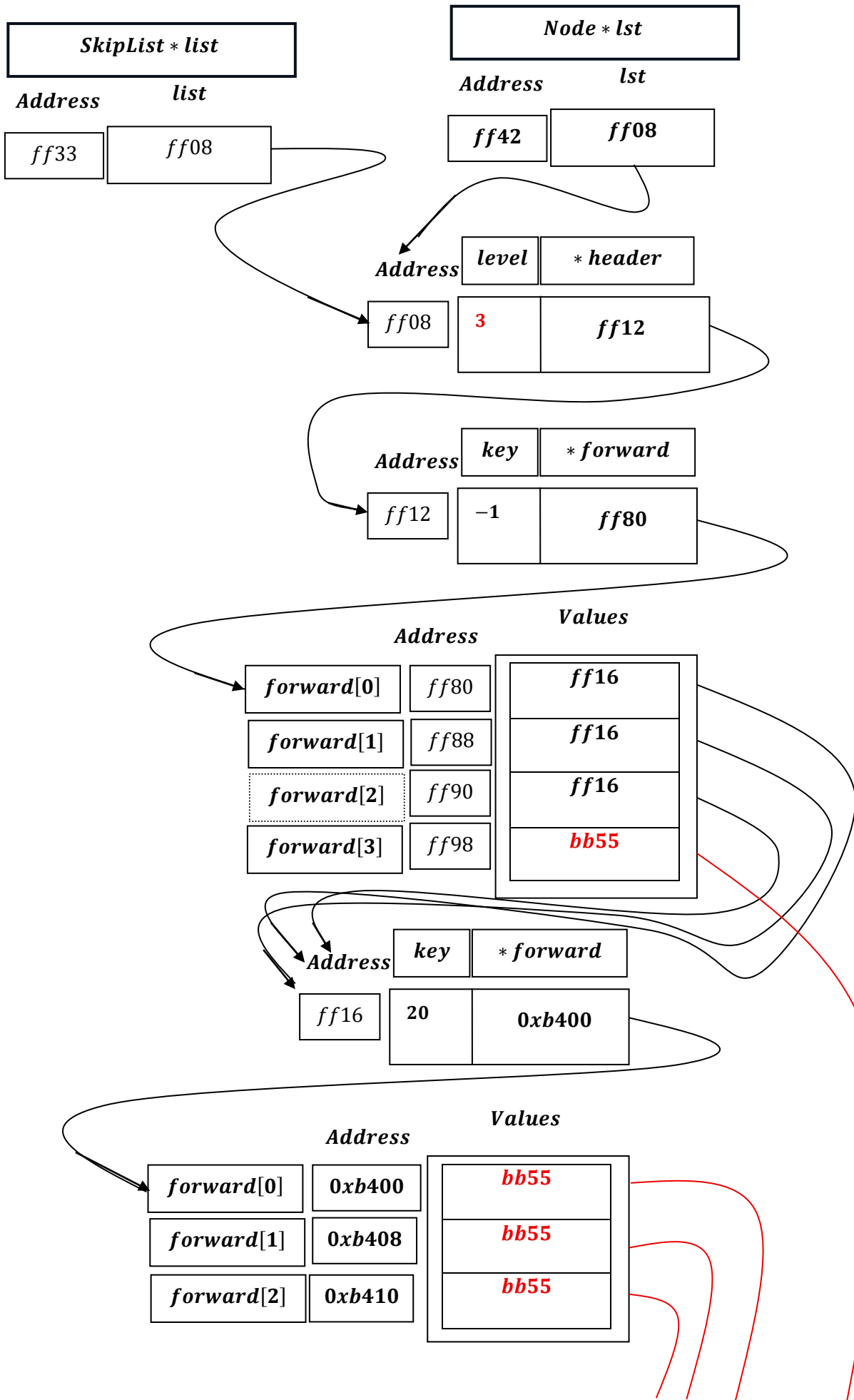
$current: bb55 \rightarrow forward[0]: 0xc500 \rightarrow e216 \neq NULL[True] AND$

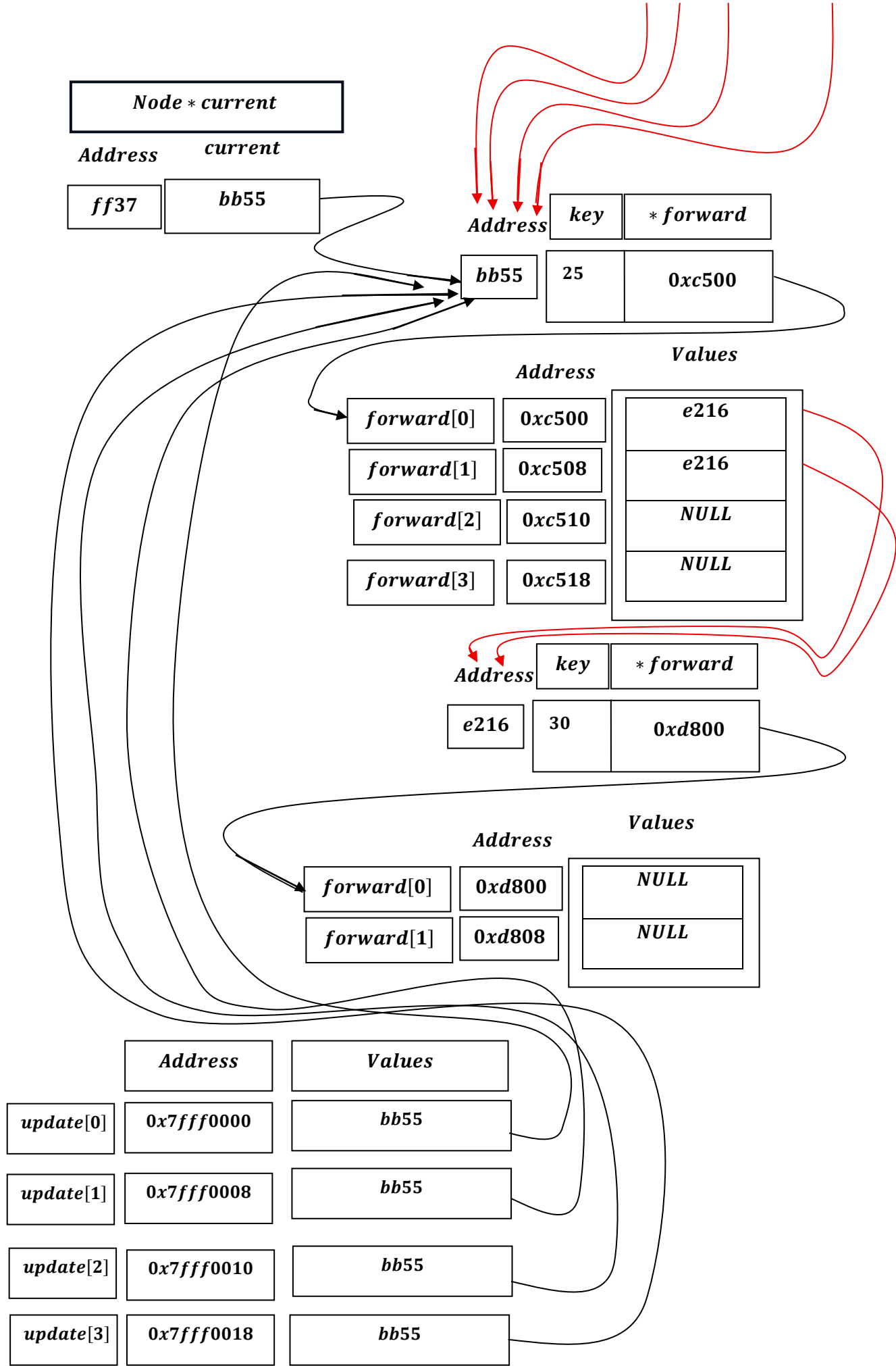
$current: bb55 \rightarrow forward[0]: e216 \rightarrow key: 30 < key: 30[False]$

$update[0] = current: bb55.$

$$i = i - - = i - 1 = 0 - 1 = -1$$

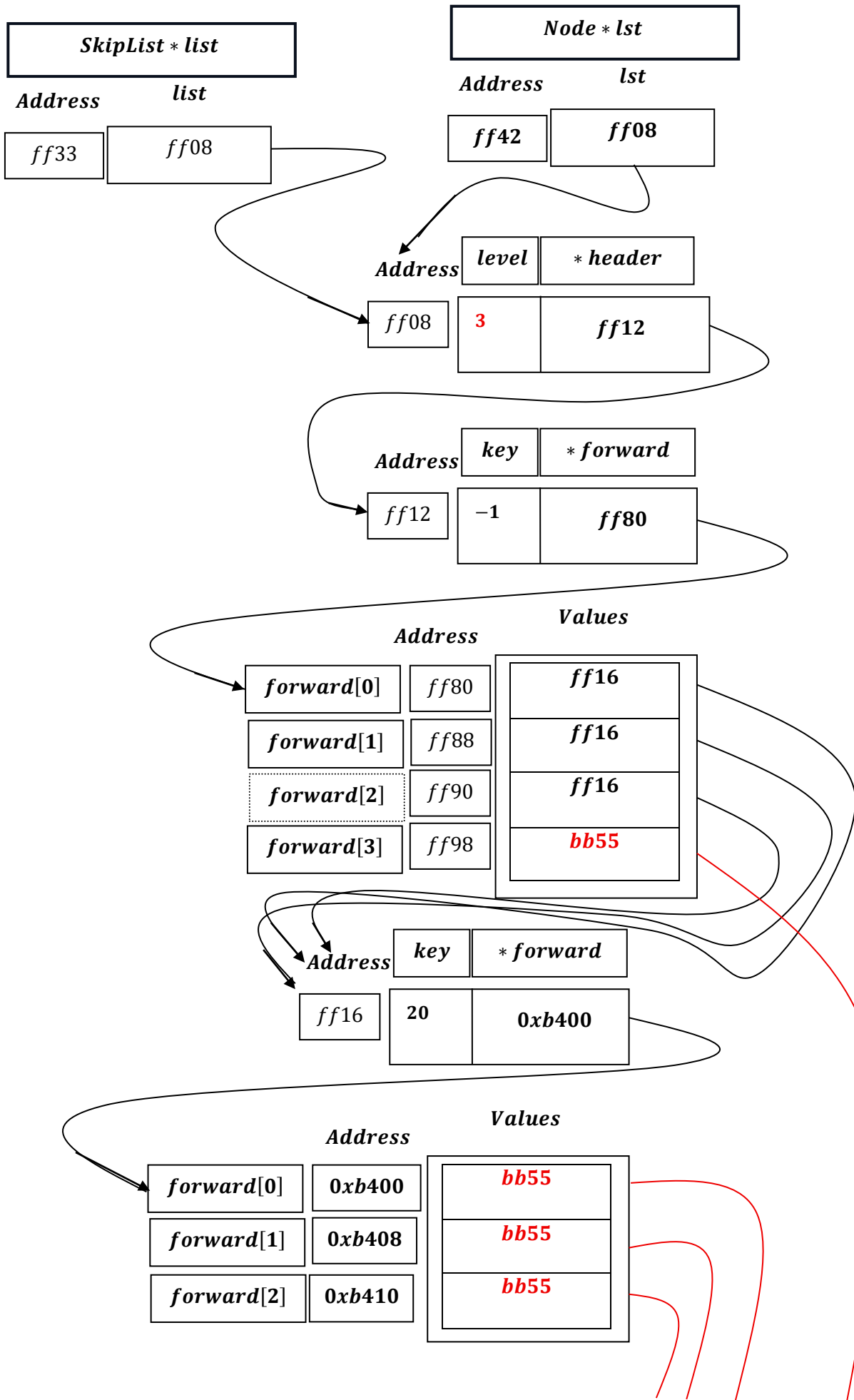
$i = -1; i \geq 0 [Condition is False]$





```
current = current->forward[0];
```

current = *current*: *bb55* → *forward*[0]: *0xc500* → *e216*.



*Node * current*

Address *current*

<i>ff37</i>	<i>e216</i>
-------------	-------------

Address

key

** forward*

bb55

25

0xc500

Address

Values

forward[0]

0xc500

forward[1]

0xc508

forward[2]

0xc510

forward[3]

0xc518

e216

e216

NULL

NULL

Address

key

** forward*

e216

30

0xd800

Address

Values

forward[0]

0xd800

forward[1]

0xd808

NULL

NULL

Address

Values

update[0]

0x7fff0000

bb55

update[1]

0x7fff0008

bb55

update[2]

0x7fff0010

bb55

update[3]

0x7fff0018

bb55


```

if (current != NULL && current->key == key) {
    // Rewire the pointers to bypass the deleted node
    for (int i = 0; i <= lst->level; i++) {
        if (update[i]->forward[i] != current) {
            break; // If a level doesn't point to current, higher levels
won't
                // either
        }
        update[i]->forward[i] = current->forward[i];
    }

    // Free memory using free()
    free(current->forward);
    free(current);

    // Reduce the maximum level of the list if layers become empty
    while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
        lst->level--;
    }
    cout << "Successfully deleted key " << key << endl;
} else {
    cout << "Key " << key << " not found. Cannot delete." << endl;
}
}

```

if(current: e216 $\neq$ NULL is True and

current: e216 $\rightarrow$ key: 30 == 30 is True) :

for : int i = 0; i $\leq$ lst $\rightarrow$ level: 3 :

if(update[0]: bb55 $\rightarrow$ forward[0]: e216 $\neq$ current: e216) [false]

update[0]: bb55 $\rightarrow$ forward[0] = current: e216 $\rightarrow$ forward[0]
: NULL

i = i + 1 = 0 + 1 = 1.

for : int i = 1; i ≤ lst → level: 3 :

if(update[1]: bb55 → forward[1]: e216 ≠ current: e216) [false]

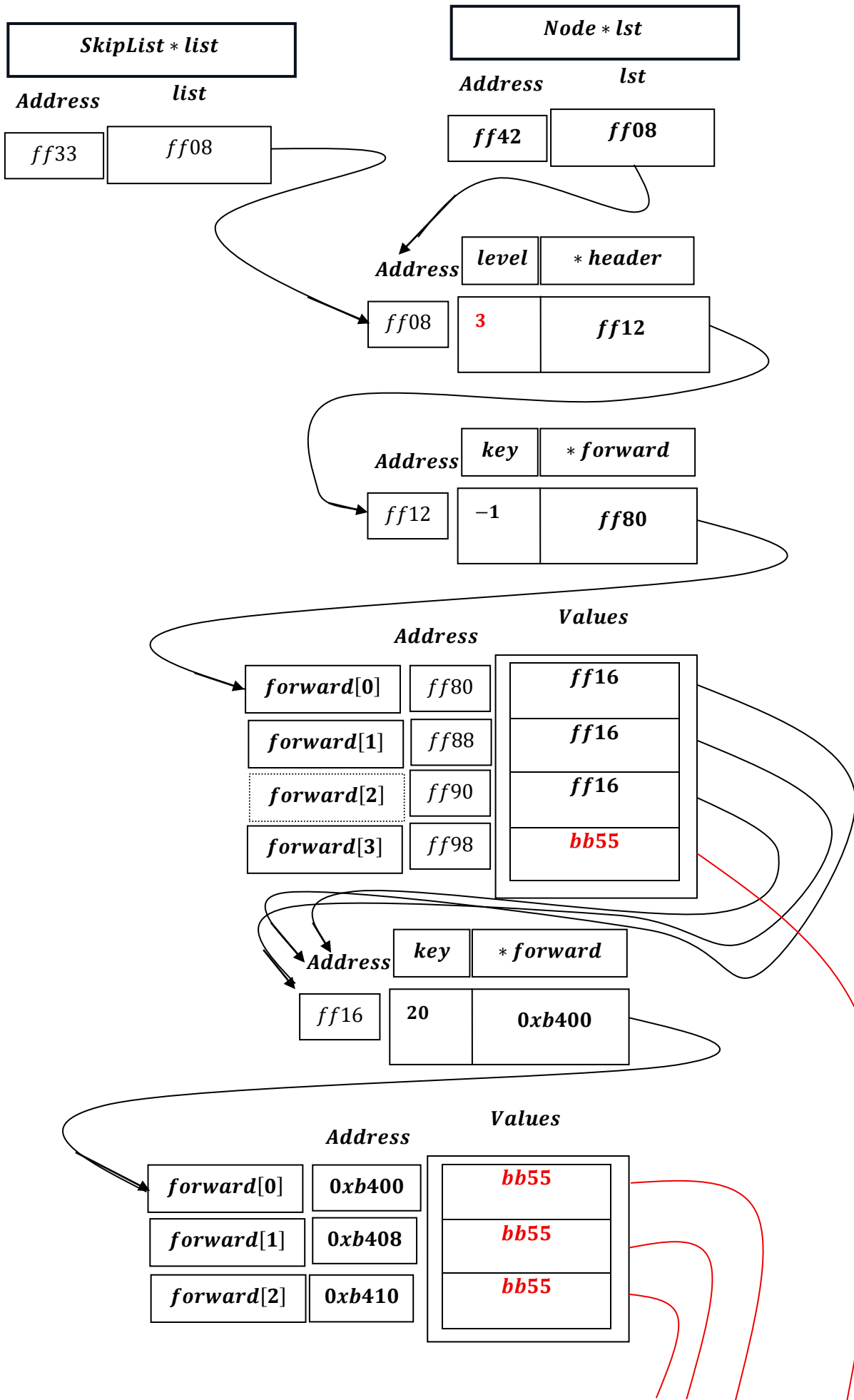
*update[1]: bb55 → forward[1] = current: e216 → forward[1]
: NULL*

i = i + 1 = 1 + 1 = 2.

for : int i = 2; i ≤ lst → level: 3 :

if(update[2]: bb55 → forward[2]: NULL ≠ current: e216) [True]

*break; The control flow statement break;
Terminates loop immediately terminates
here.*



*Node * current*

Address *current*

<i>ff37</i>	<i>e216</i>
-------------	-------------

Address

key

** forward*

bb55

25

0xc500

Address

Values

forward[0]

0xc500

NULL

forward[1]

0xc508

NULL

forward[2]

0xc510

NULL

forward[3]

0xc518

NULL

Address

key

** forward*

e216

30

0xd800

Address

Values

forward[0]

0xd800

NULL

forward[1]

0xd808

NULL

Address

Values

update[0]

0x7fff0000

bb55

update[1]

0x7fff0008

bb55

update[2]

0x7fff0010

bb55

update[3]

0x7fff0018

bb55

```
free(current->forward);  
free(current);
```

Now,

```
free(current->forward);
```

*The pointer **current** → **forward** holds the starting address of this chunk, which is **0xd800**. This is where **forward[0]** lives.*

***The Offset:** The address **0xd808** is simply the location of the second element in that array (**forward[1]**). It is exactly 8 bytes after the starting address, sitting inside the same block of memory.*

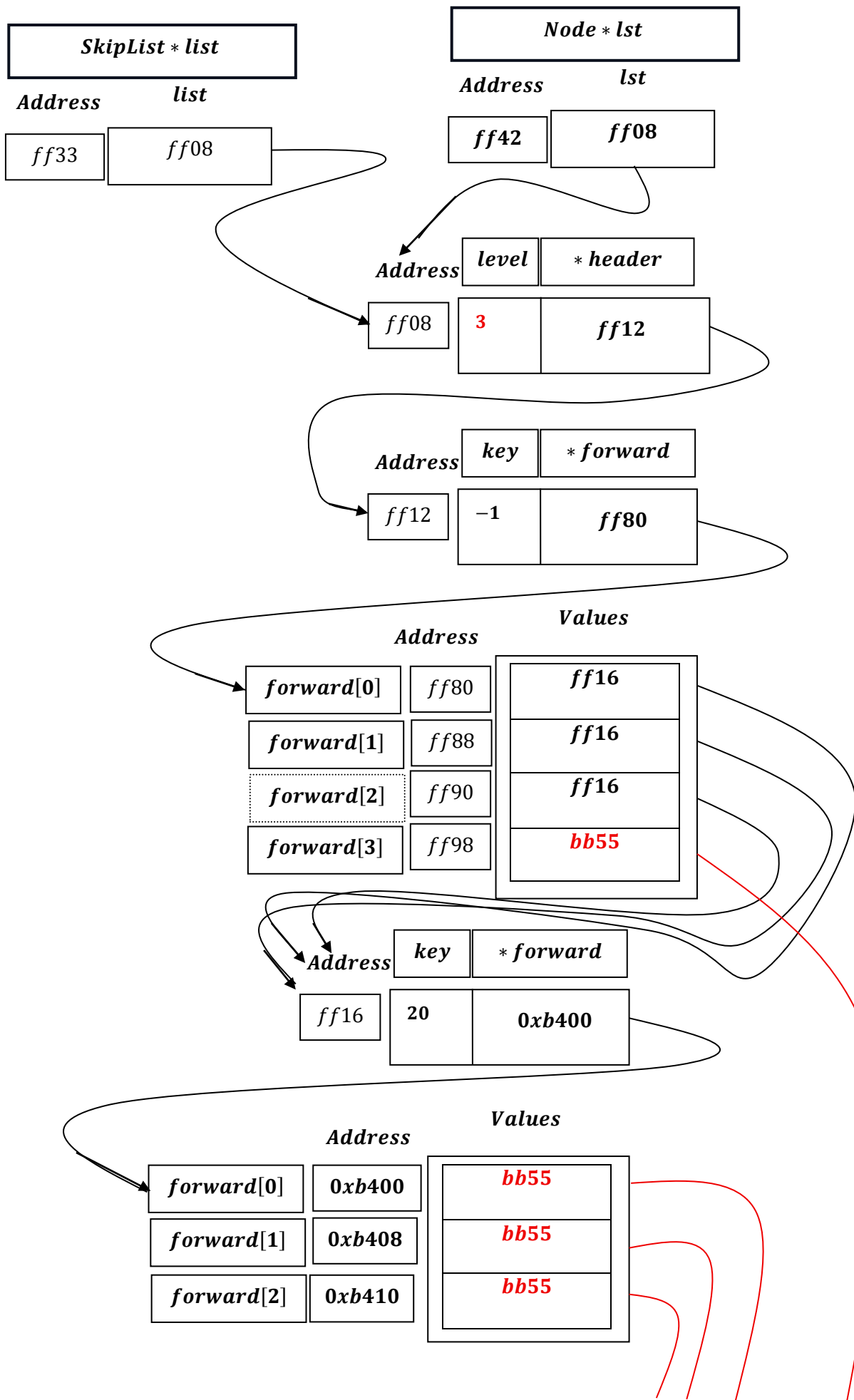
*When the program executes **free(current → forward)**, it tells the system to deallocate the entire chunk of memory that begins at **0xd800**. Because **0xd808** is just a part of that continuous chunk, it gets completely freed at the exact same time.*

It will not add any offsets and delete each block of memory but all at once.

And ,

```
free(current);
```

Will free the memory block that current pointer pointed to.



*Node * current*

Address *current*

<i>ff37</i>	<i>e216</i>
-------------	-------------

Address

key

** forward*

bb55

25

0xc500

Address

Values

forward[0]

0xc500

NULL

forward[1]

0xc508

NULL

forward[2]

0xc510

NULL

forward[3]

0xc518

NULL

Address

Value

e216

Address

Values

0xd800

0xd808

Address

Values

update[0]

0x7fff0000

bb55

update[1]

0x7fff0008

bb55

update[2]

0x7fff0010

bb55

update[3]

0x7fff0018

bb55

```

// Reduce the maximum level of the list if layers become empty
while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
    lst->level--;
}
cout << "Successfully deleted key " << key << endl;
}

```

lst → *level* = 3 > 0 [True] AND *lst* → *header: ff12* → *forward*[*lst* →
level: 3] = *bb55* == *NULL* [False] [**Condition is**
False]

Print `` Successfully deleted key : 30``.

Skip – List
–Delete Element
–Program Analysis – Type B

Program Analysis:

```
void deleteElement(struct SkipList *lst, int key) {
    struct Node *current = lst->header;
    struct Node *update[MAX_LEVEL + 1];
    memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));

    // Find the node, keeping track of pointers that point to it
    for (int i = lst->level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key <
key) {
            current = current->forward[i];
        }
        update[i] = current;
    }

    current = current->forward[0];

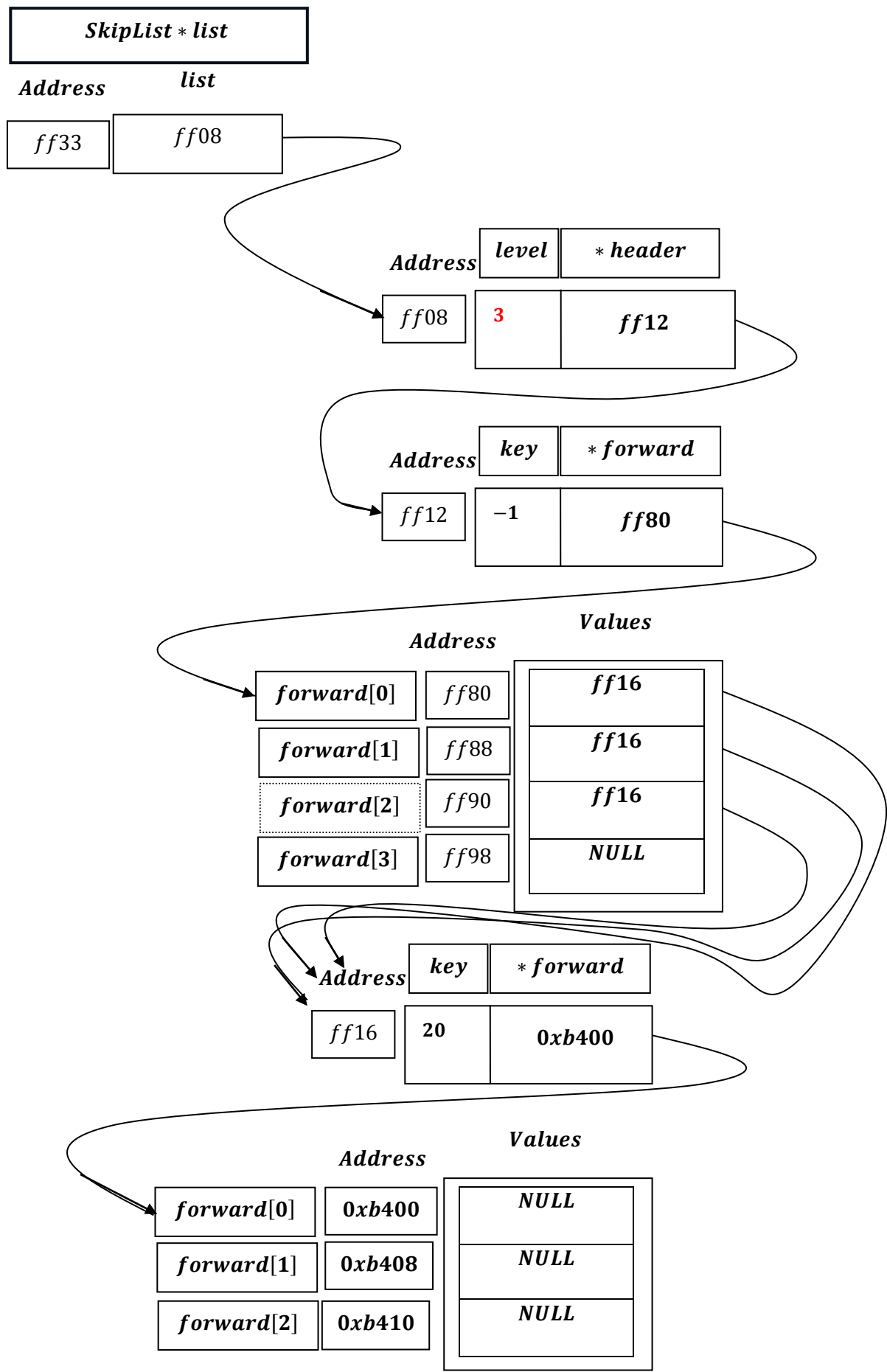
    // If the target node is found
    if (current != NULL && current->key == key) {
        // Rewire the pointers to bypass the deleted node
        for (int i = 0; i <= lst->level; i++) {
            if (update[i]->forward[i] != current) {
                break; // If a level doesn't point to current, higher levels
won't
                // either
            }
            update[i]->forward[i] = current->forward[i];
        }

        // Free memory using free()
        free(current->forward);
        free(current);

        // Reduce the maximum level of the list if layers become empty
        while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
```

```
        lst->level--;  
    }  
    cout << "Successfully deleted key " << key << endl;  
} else {  
    cout << "Key " << key << " not found. Cannot delete." << endl;  
}  
}  
  
... .  
case 2:  
    cout << "Enter key to delete: ";  
    cin >> key;  
    deleteElement(list, key);  
    break;
```

B. Now Say ,We have single Node left ,after deletion.



```
deleteElement(list, key);
```

deleteElement(list: ff08, key: 20) → passed .

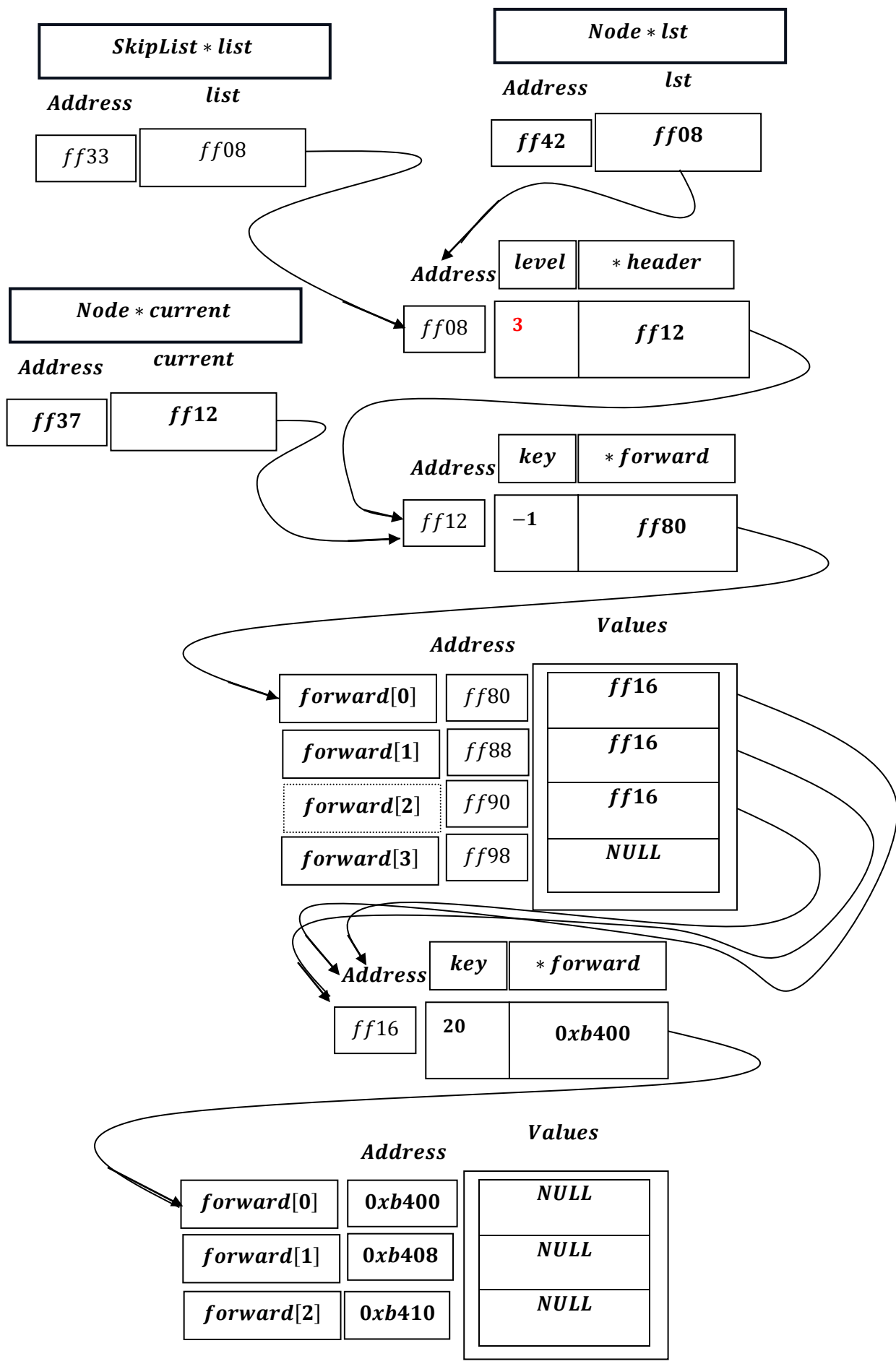
And,

```
void deleteElement(struct SkipList *lst, int key)
```

deleteElement(list := lst: ff08, key := key: 20) → Assigned

```
struct Node *current = lst->header;
```

current = lst → header = ff12.



```
struct Node *update[MAX_LEVEL + 1];
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

```
struct Node *update[MAX_LEVEL + 1];
```

update[3 + 1 = 4],

MAX_LEVEL evaluates to 3. Consequently, the array size calculation is update[3 + 1 = 4].

This means the array is allocated to hold exactly 4 elements.

Before memset call, the update array may look like :

Array Element	Hypothetical Stack Address	Value (Garbage Data)	Pointer Status
update[0]	0x7fff0000 to 0x7fff0007	0x4a3f8921bce45099	Wild Pointer (Points to a random, invalid address)
update[1]	0x7fff0008 to 0x7fff000f	0x0000000000000002	Wild Pointer
update[2]	0x7fff0010 to 0x7fff0017	0x7ffee1245a00881c	Wild Pointer
update[3]	0x7fff0018 to 0x7fff001f	0x0000000000000000	Accidentally NULL (Just by chance)

More simpler way:

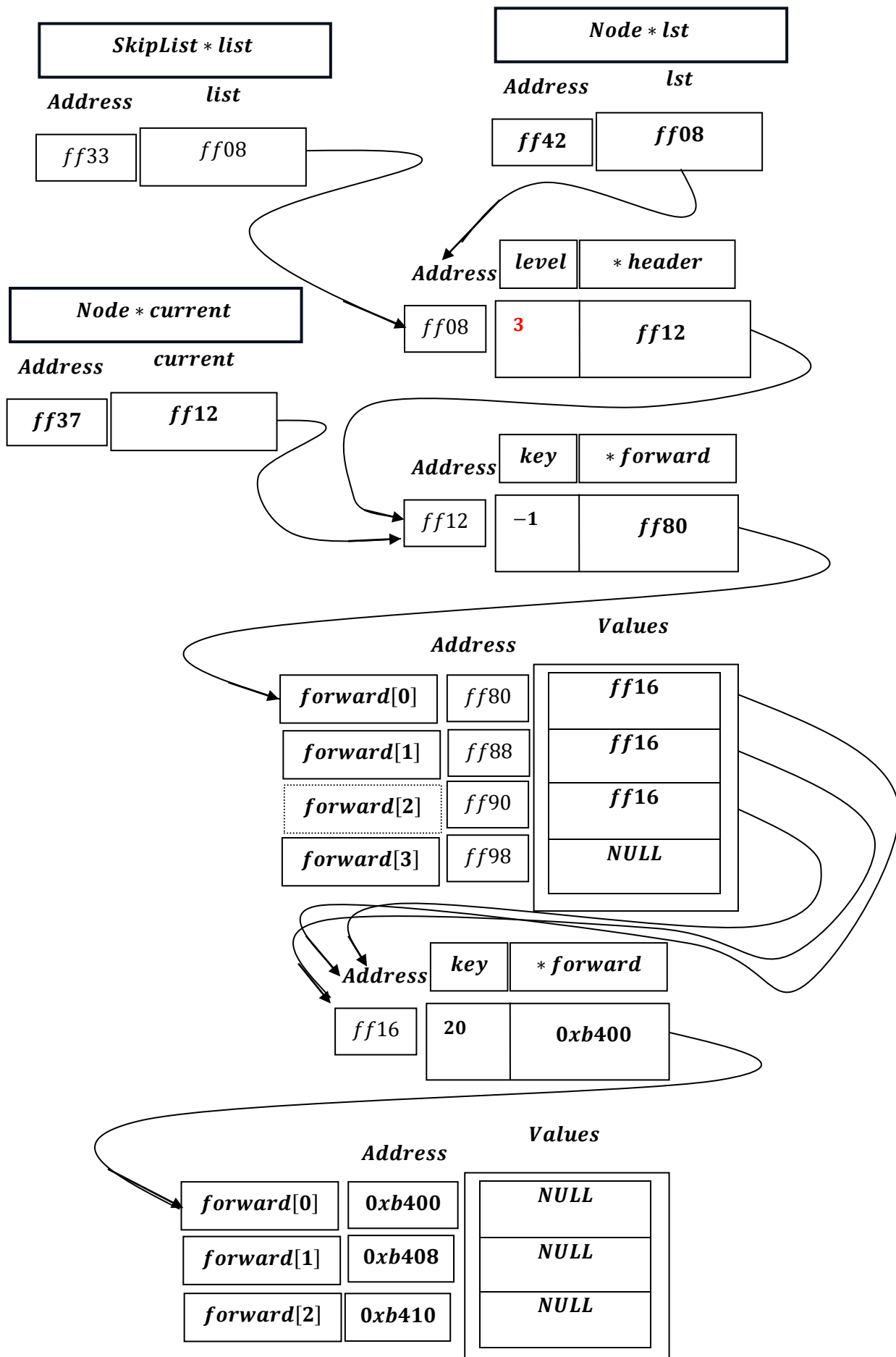
	Address	Values	
update[0]	0x7fff0000	0x4a3f8921bce45099	[8 bytes]
update[1]	0x7fff0008	0x0000000000000002	[8 bytes]
update[2]	0x7fff0010	0x7ffee1245a00881c	[8 bytes]
update[3]	0x7fff0018	0x0000000000000000	[8 bytes]

```
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

``memset`` stands for Memory Set. It takes a block of memory and rapidly fills it with a specific value. Here `0` or NULL.

	Address	Values
update[0]	0x7fff0000	NULL
update[1]	0x7fff0008	NULL
update[2]	0x7fff0010	NULL
update[3]	0x7fff0018	NULL

Here we kept addresses same like: 0x7fff0000 , just for an example , but it may not be same , in reality.



	<i>Address</i>	<i>Values</i>
<i>update</i> [0]	0x7fff0000	<i>NULL</i>
<i>update</i> [1]	0x7fff0008	<i>NULL</i>
<i>update</i> [2]	0x7fff0010	<i>NULL</i>
<i>update</i> [3]	0x7fff0018	<i>NULL</i>

```

for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}

```

for: int i = lst → level: 3; i ≥ 0;

while: current: ff12 → forward[3]: NULL ≠ NULL $\left[\begin{array}{l} \text{False,} \\ \text{Loop Ends} \end{array} \right]$

update[3] = current = ff12;

i = i − 1 = 3 − 1 = 2.

for: int i = 2; i ≥ 0;

while: current: ff12 → forward[2]: ff16 ≠ NULL[True] AND

current: ff12 → forward[2]: ff16 → key: 20 < key: 20 $\left[\begin{array}{l} \text{False,} \\ \text{Loop Ends} \end{array} \right]$

update[2] = current = ff12;

i = i − 1 = 2 − 1 = 1.

for: int i = 1; i ≥ 0;

while: current: ff12 → forward[1]: ff16 ≠ NULL[True] AND

current: ff12 → forward[1]: ff16 → key: 20 < key: 20 $\left[\begin{array}{l} \text{False,} \\ \text{Loop Ends} \end{array} \right]$

update[1] = current = ff12;

i = i - 1 = 1 - 1 = 0.

for: int i = 0; i ≥ 0;

while: current: ff12 → forward[0]: ff16 ≠ NULL[True] AND

current: ff12 → forward[0]: ff16 → key: 20 < key: 20 $\left[\begin{array}{l} \text{False,} \\ \text{Loop Ends} \end{array} \right]$

update[0] = current = ff12;

i = i - 1 = 0 - 1 = -1.

for: int i = -1; i ≥ 0; $\left[\begin{array}{l} \text{False,} \\ \text{Loop Ends} \end{array} \right]$

*SkipList * list*

Address list

ff33 ff08

*Node * lst*

Address lst

ff42 ff08

*Node * current*

Address current

ff37 ff12

Address level * header

ff08 3 ff12

Address key * forward

ff12 -1 ff80

Address Values

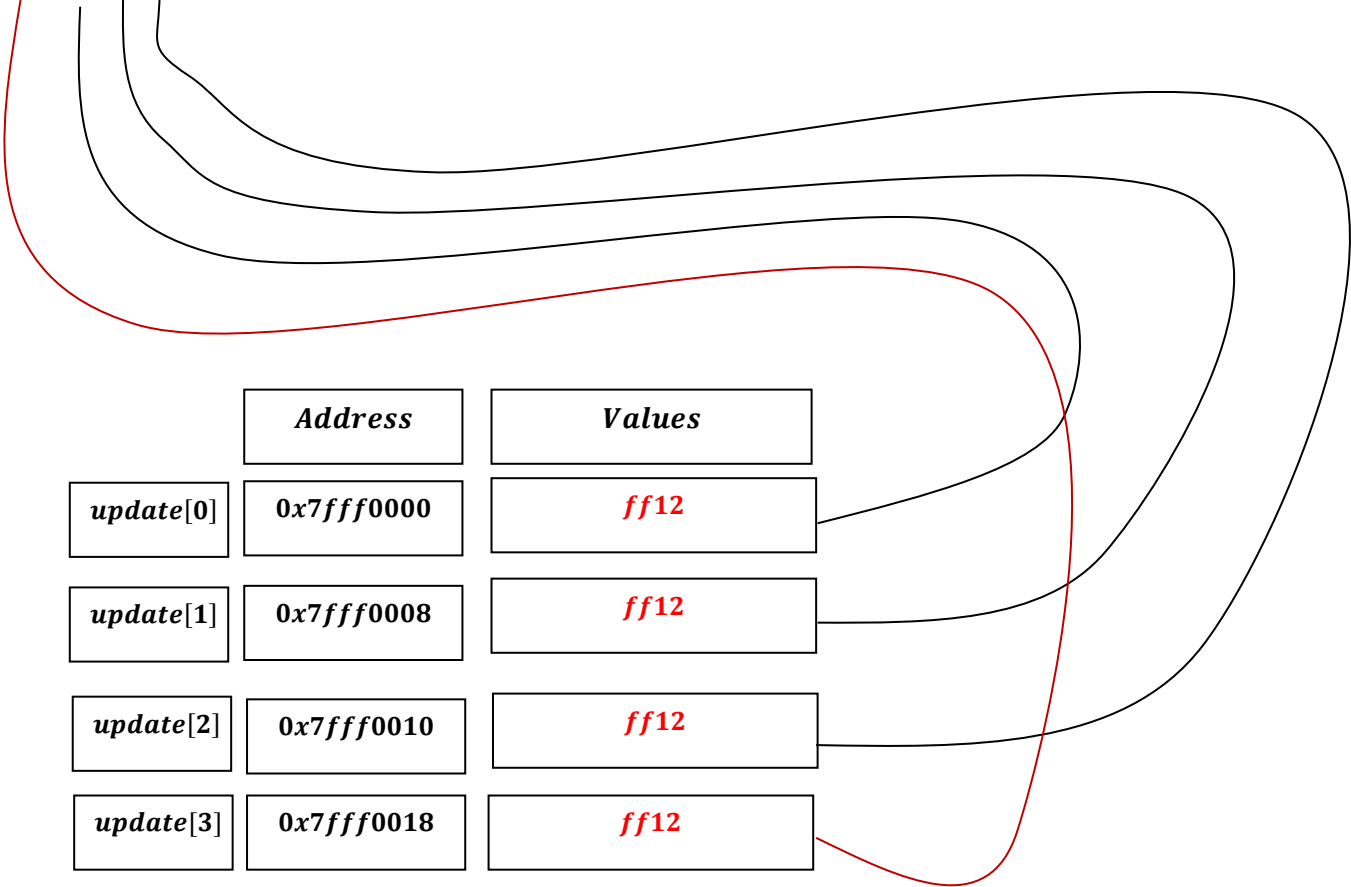
Address	Values
forward[0] ff80	ff16
forward[1] ff88	ff16
forward[2] ff90	ff16
forward[3] ff98	NULL

Address key * forward

ff16 20 0xb400

Address Values

Address	Values
forward[0] 0xb400	NULL
forward[1] 0xb408	NULL
forward[2] 0xb410	NULL



	<i>Address</i>	<i>Values</i>
<i>update[0]</i>	0x7fff0000	<i>ff12</i>
<i>update[1]</i>	0x7fff0008	<i>ff12</i>
<i>update[2]</i>	0x7fff0010	<i>ff12</i>
<i>update[3]</i>	0x7fff0018	<i>ff12</i>

```
current = current->forward[0];
```

current = *current: ff12* → *forward[0]: ff16*

*SkipList * list*

Address list

ff33 ff08

*Node * lst*

Address lst

ff42 ff08

*Node * current*

Address current

ff37 ff16

Address level * header

ff08 3 ff12

Address key * forward

ff12 -1 ff80

Address

Values

forward[0]	ff80	ff16
forward[1]	ff88	ff16
forward[2]	ff90	ff16
forward[3]	ff98	NULL

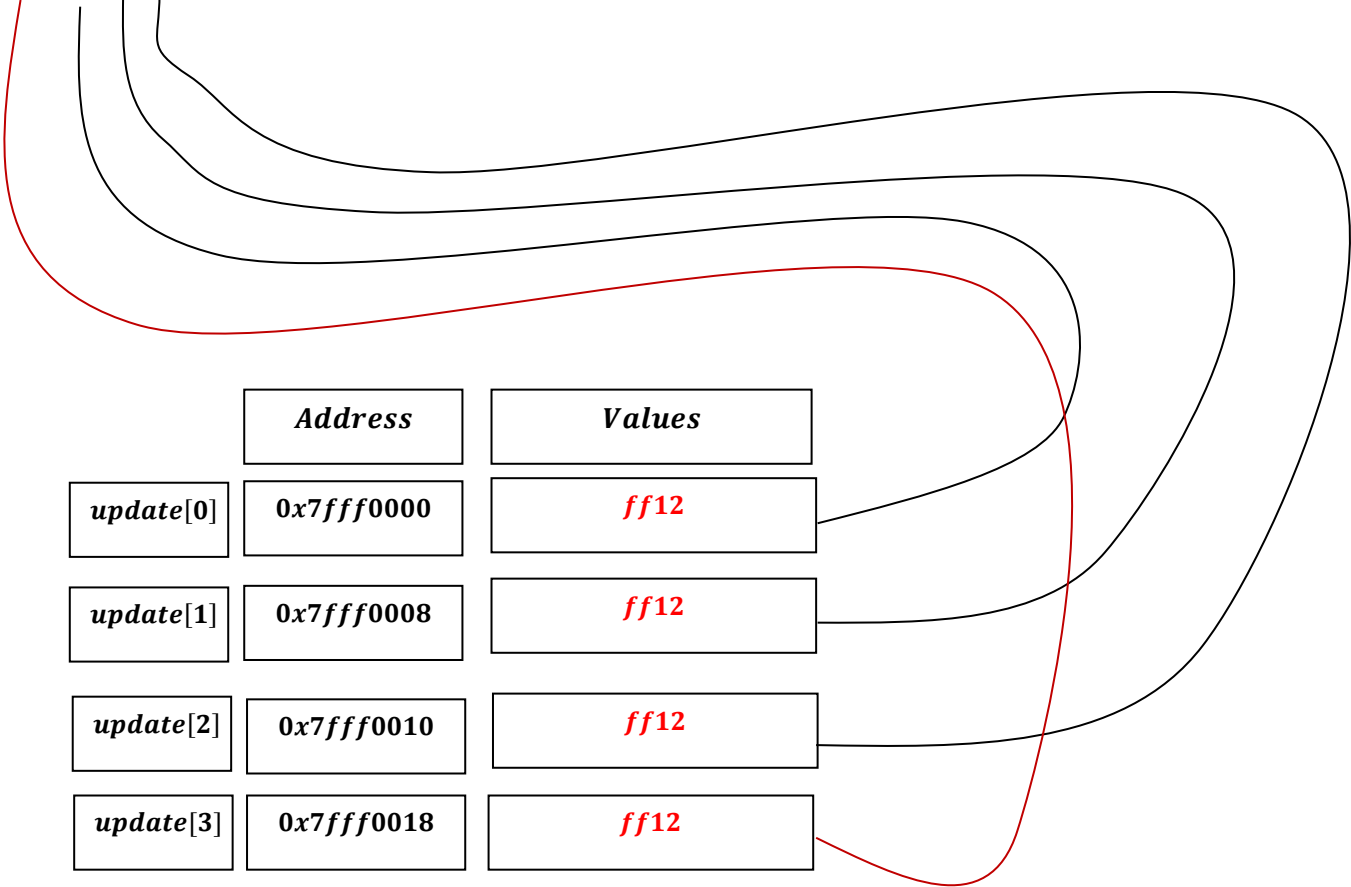
Address key * forward

ff16 20 0xb400

Address

Values

forward[0]	0xb400	NULL
forward[1]	0xb408	NULL
forward[2]	0xb410	NULL



	Address	Values
update[0]	0x7fff0000	ff12
update[1]	0x7fff0008	ff12
update[2]	0x7fff0010	ff12
update[3]	0x7fff0018	ff12

```
if (current != NULL && current->key == key) {
```

current: ff16 $\neq$ NULL AND current: ff16 $\rightarrow$ key: 20 == 20:

```
for (int i = 0; i <= lst->level; i++) {
    if (update[i]->forward[i] != current) {
        break; // If a level doesn't point to current, higher levels
won't
        // either
    }
    update[i]->forward[i] = current->forward[i];
}
```

int i = 0 ; i $\leq$ lst $\rightarrow$ level: 3

if(update[0]: ff12 $\rightarrow$ forward[0]: ff16 $\neq$ current: ff16) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

update[0] $\rightarrow$ forward[0] = current: ff16 $\rightarrow$ forward[0]: NULL

i = i + 1 = 0 + 1 = 1.

int i = 1 ; i ≤ lst → level: 3

if(update[1]: ff12 → forward[1]: ff16 ≠ current: ff16) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

update[1] → forward[1] = current: ff16 → forward[1]: NULL

i = i + 1 = 1 + 1 = 2.

int i = 2 ; i ≤ lst → level: 3

if(update[2]: ff12 → forward[2]: ff16 ≠ current: ff16) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

update[2] → forward[2] = current: ff16 → forward[2]: NULL

i = i + 1 = 2 + 1 = 3.

int i = 3 ; i ≤ lst → level: 3

if(update[3]: ff12 → forward[3]: NULL ≠ current: ff16) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

break; $\left[\begin{array}{c} \text{The control flow statement break;} \\ \text{Terminates loop immediately terminates} \\ \text{here.} \end{array} \right]$

*SkipList * list*

Address list

ff33 ff08

*Node * lst*

Address lst

ff42 ff08

*Node * current*

Address current

ff37 ff16

Address level * header

ff08 3 ff12

Address key * forward

ff12 -1 ff80

Address Values

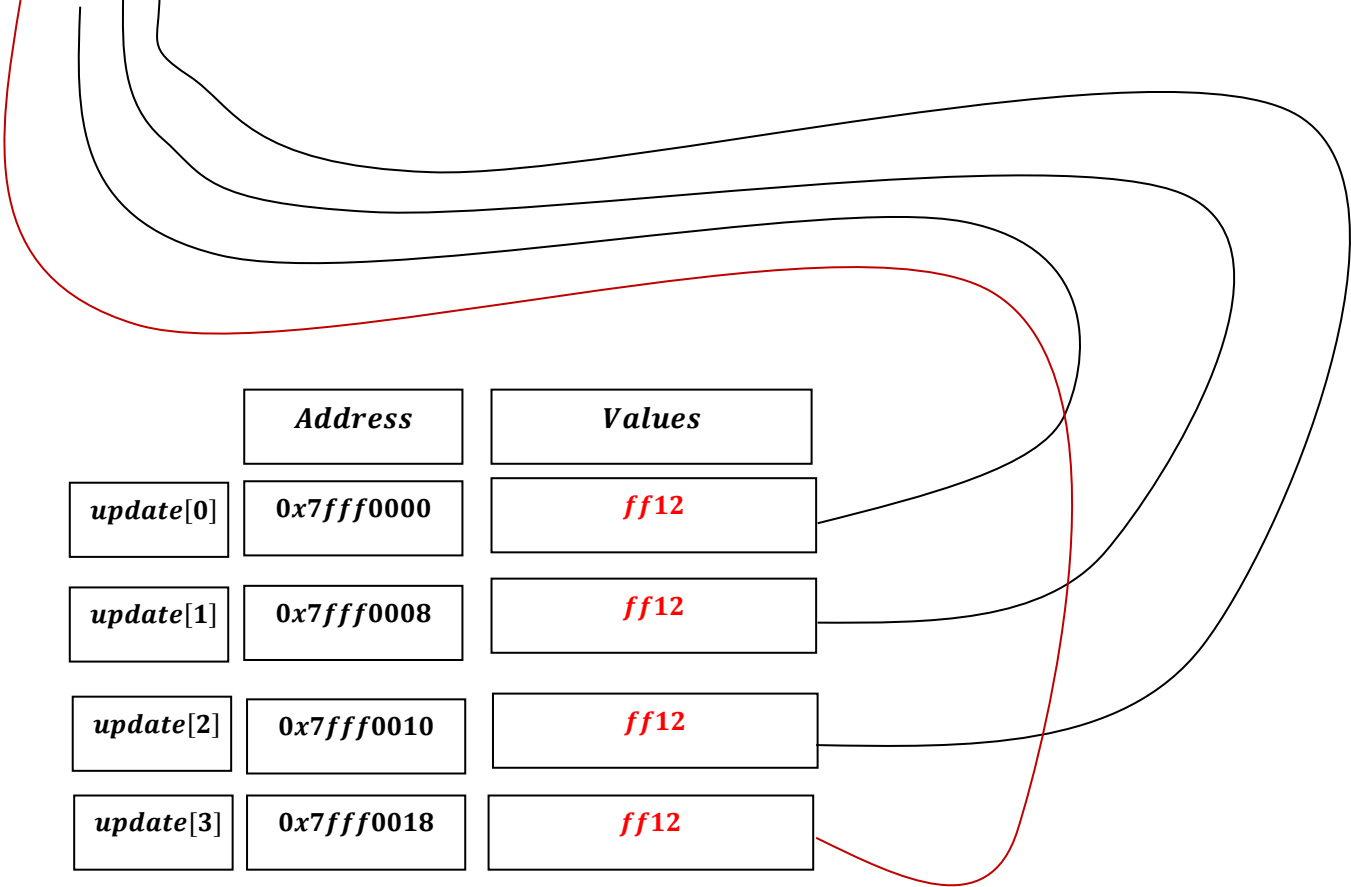
forward[0]	ff80	NULL
forward[1]	ff88	NULL
forward[2]	ff90	NULL
forward[3]	ff98	NULL

Address key * forward

ff16 20 0xb400

Address Values

forward[0]	0xb400	NULL
forward[1]	0xb408	NULL
forward[2]	0xb410	NULL



	Address	Values
update[0]	0x7fff0000	ff12
update[1]	0x7fff0008	ff12
update[2]	0x7fff0010	ff12
update[3]	0x7fff0018	ff12

```
free(current->forward);
free(current);
```

free(current: ff16 → forward)

The pointer ***current → forward*** holds the starting address of this chunk, which is **0xb400**. This is where ***forward[0]*** lives.

The Offset: The address **0xb408** is simply the location of the second element in that array (***forward[1]***). It is exactly 8 bytes after the starting address, sitting inside the same block of memory.

When the program executes ***free(current → forward)***, it tells the system to deallocate the entire chunk of memory that begins at **0xb400**. Because **0xb408** is just a part of that continuous chunk, it gets completely freed at the exact same time.

It will not add any offsets and delete each block of memory but all at once.

And ,

```
free(current);
```

Will free the memory block that current pointer pointed to.

*SkipList * list*

Address list

ff33 ff08

*Node * lst*

Address lst

ff42 ff08

*Node * current*

Address current

ff37 ff16

Address

*level * header*

ff08 3 ff12

Address

*key * forward*

ff12 -1 ff80

Address

Values

<i>forward[0]</i>	<i>ff80</i>	<i>NULL</i>
<i>forward[1]</i>	<i>ff88</i>	<i>NULL</i>
<i>forward[2]</i>	<i>ff90</i>	<i>NULL</i>
<i>forward[3]</i>	<i>ff98</i>	<i>NULL</i>

Address

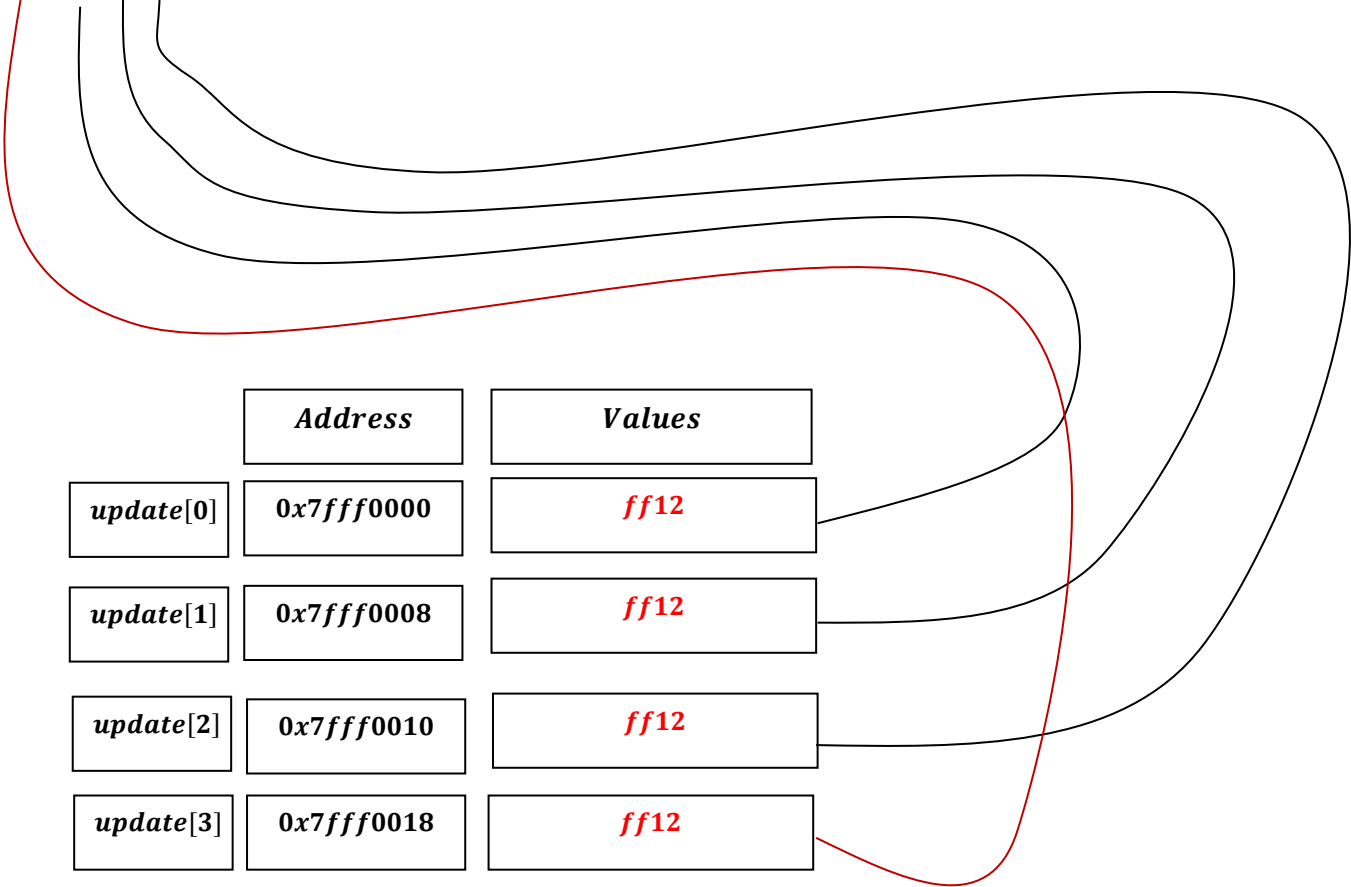
Value

ff16

Address

Values

<i>0xb400</i>	
<i>0xb408</i>	
<i>0xb410</i>	



	Address	Values
update[0]	0x7fff0000	ff12
update[1]	0x7fff0008	ff12
update[2]	0x7fff0010	ff12
update[3]	0x7fff0018	ff12

```

while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
    lst->level--;
}
cout << "Successfully deleted key " << key << endl;
}

```

Now layers becomed **Empty**.

$lst \rightarrow level: 3 > 0$ AND $lst \rightarrow header: ff12 \rightarrow forward[lst \rightarrow level: 3] NULL == NULL$:
 $lst \rightarrow level = lst \rightarrow level - 1 = 3 - 1 = 2$

$lst \rightarrow level: 2 > 0$ AND $lst \rightarrow header: ff12 \rightarrow forward[lst \rightarrow level: 2] NULL == NULL$:
 $lst \rightarrow level = lst \rightarrow level - 1 = 2 - 1 = 1$

$lst \rightarrow level: 1 > 0$ AND $lst \rightarrow header: ff12 \rightarrow forward[lst \rightarrow level: 1] NULL == NULL$:
 $lst \rightarrow level = lst \rightarrow level - 1 = 1 - 1 = 0$

$lst \rightarrow level: 0 > 0$ Condition
becomes
False

*SkipList * list*

Address list

ff33 ff08

*Node * lst*

Address lst

ff42 ff08

*Node * current*

Address current

ff37 ff16

Address

level * header

ff08 0 ff12

Address

key * forward

ff12 -1 ff80

Address

Values

forward[0]	ff80	NULL
forward[1]	ff88	NULL
forward[2]	ff90	NULL
forward[3]	ff98	NULL

Address

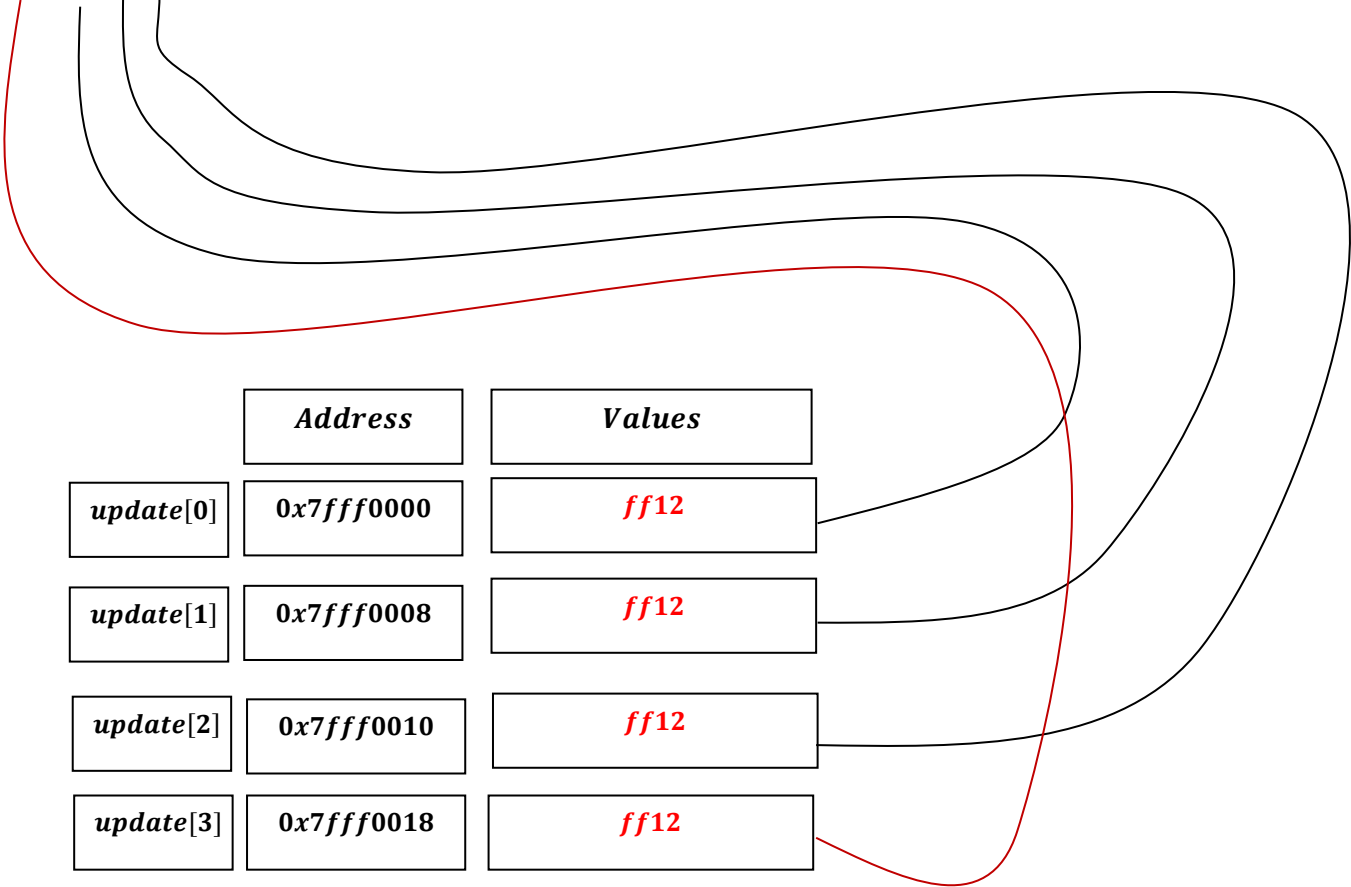
Value

ff16

Address

Values

0xb400	
0xb408	
0xb410	



	<i>Address</i>	<i>Values</i>
<i>update[0]</i>	0x7fff0000	<i>ff12</i>
<i>update[1]</i>	0x7fff0008	<i>ff12</i>
<i>update[2]</i>	0x7fff0010	<i>ff12</i>
<i>update[3]</i>	0x7fff0018	<i>ff12</i>

```
cout << "Successfully deleted key " << key << endl;
```

Now , it prints ``Successfully deleted key : 20.``

After function finishes , temporary variables get destroyed:

Node * current

Address current

ff37 ff16

Destroyed

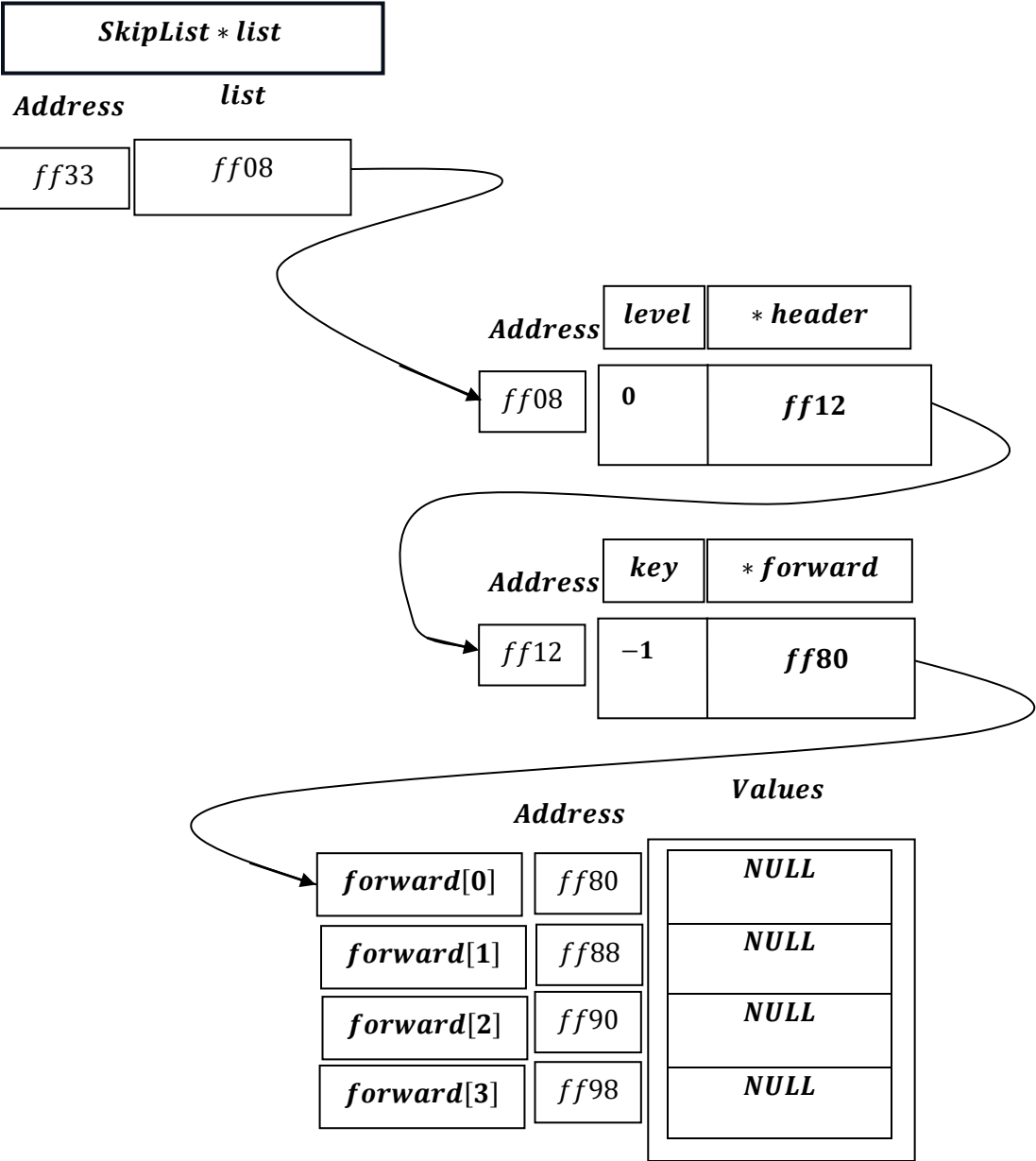
	Address	Values
update[0]	0x7fff0000	ff12
update[1]	0x7fff0008	ff12
update[2]	0x7fff0010	ff12
update[3]	0x7fff0018	ff12

Node * lst

Address lst

ff42 ff08

And we are left with:



Type C [*When Key is*
Not Found]

This we can interpret the process. By your own.

Skip – List
–Delete Element
–Time Complexity Analysis

Program :

```
void deleteElement(struct SkipList *lst, int key) {
    struct Node *current = lst->header;
    struct Node *update[MAX_LEVEL + 1];
    memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));

    // Find the node, keeping track of pointers that point to it
    for (int i = lst->level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key <
key) {
            current = current->forward[i];
        }
        update[i] = current;
    }

    current = current->forward[0];

    // If the target node is found
    if (current != NULL && current->key == key) {
        // Rewire the pointers to bypass the deleted node
        for (int i = 0; i <= lst->level; i++) {
            if (update[i]->forward[i] != current) {
                break; // If a level doesn't point to current, higher levels
won't
                // either
            }
            update[i]->forward[i] = current->forward[i];
        }

        // Free memory using free()
        free(current->forward);
        free(current);

        // Reduce the maximum level of the list if layers become empty
```

```

    while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
        lst->level--;
    }
    cout << "Successfully deleted key " << key << endl;
} else {
    cout << "Key " << key << " not found. Cannot delete." << endl;
}
}
...
case 2:
    cout << "Enter key to delete: ";
    cin >> key;
    deleteElement(list, key);
    break;

```

Time Complexity Analysis

1. Best Case Time Complexity: $O(1)$

The absolute best – case scenario occurs when the element to be deleted is exactly at the front of the list, or the list is empty or the key is smaller than the first element.

The Search Phase

```

for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}

```

- **The Search Phase:** The horizontal **while** loop condition (**current** → **forward[i]** → **key < key**) fails immediately. The program ducks the horizontal traversal and simply drops straight down the **for** loop to Level 0. Because **lst** → **level** is capped by **MAX_LEVEL** (a hardcoded constant), this vertical drop takes **O(1)** time.

1. "The list is empty"

If we try to delete an element from a Skip List that has no nodes, the header's pointers just point to **NULL**.

- **The Code:** The **while** loop checks **current** → **forward[i]** ≠ **NULL**.
- **The Result:** Because the next node is **NULL**, the condition instantly fails on the very first check at every single level. The pointer never takes a step forward.

2. "The key is smaller than the first element"

Imagine the first actual node in the list has a key of 50, but you ask the program to delete 20.

- **The Code:** The **while** loop checks **current** → **forward[i]** → **key < key**. It compares **50 < 20**.
 - ☐ Level 3: header → forward[3] points to 50. (Checks: 50 < 20 is False)
 - ☐ Level 2: header → forward[2] points to 50. (Checks: 50 < 20 is False)
 - ☐ Level 1: header → forward[1] points to 50. (Checks: 50 < 20 is False)
 - ☐ Level 0: header → forward[0] points to 50. (Checks: 50 < 20 is False)
- **The Result:** Because 50 is not less than 20, the condition instantly fails on the very first check. The program realizes the target number should be to its left, meaning it isn't in the list at all. It never takes a step forward.

3. "The element is exactly at the front"

Imagine the very first node in the list has a key of 30, and we ask the program to delete 30.

- **The Code:** Again, the while loop only moves forward if $current \rightarrow forward[i] \rightarrow key < key$. It compares $30 < 30$.
- **The Result:** Because 30 is exactly equal to 30, the strict less than condition fails! The search pointer realizes it is already looking directly at the target node, so it refuses to move to the right.

The Rewiring Phase

$[lst \rightarrow level \leq MAX_LEVEL]$

```
for (int i = 0; i <= lst->level; i++) {
    if (update[i]->forward[i] != current) {
        break; // If a level doesn't point to current, higher levels
won't
        // either
    }
    update[i]->forward[i] = current->forward[i];
}
```

- **The Rewiring Phase:** If the target is found, the algorithm loops to bypass the deleted node using $update[i] \rightarrow forward[i] = current \rightarrow forward[i]$. Because a single node's height is probabilistically bounded by a small constant, and the loop includes a break statement to exit early when higher levels don't point to **current**, this takes $O(1)$ time.

The Height Adjustment Phase

```
while (lst->level > 0 && lst->header->forward[lst->level] ==  
NULL) {  
    lst->level--;  
}
```

- **The Height Adjustment Phase:** The algorithm uses a while loop to decrement the list's overall level ($lst \rightarrow level --$) if the top layers become empty after deletion. Because the level is strictly capped, this takes $O(1)$ time.
- **Result:** $O(1)$ Search + $O(1)$ Rewiring + $O(1)$ Adjustment = $O(1)$.

Worst Case Scenario

The Root Cause: The Inherited Structure

The worst – case scenario mathematically devolves to $O(n)$ due to one of two severe structural failures that occurred when the list was built:

The All Tails Scenario (The Flat List): Every node previously inserted into the list flipped Tails on its first try and stopped at Level 0. The list has zero upper – level express lanes and physically resembles a standard singly linked list.

The All Heads Scenario (The Traffic Jam): Every node previously inserted flipped Heads repeatedly, promoting every single node to the absolute maximum level. The top floor becomes just as crowded as the ground floor, clogging the express lanes.

[This was discussed during time – complexity description of
Inserting Element.]

If the algorithm attempts to delete a node located at the very end of either of these broken structures, here is exactly how the three phases of the deletion code perform.

Phase 1: The Search Phase (The $O(n)$ Bottleneck)

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

To delete a node, the algorithm must first execute a normal search to find it. This normal search relies on upper – level express lanes to skip over nodes. However, if the physical structure of the Skip List is severely unbalanced, the search loses its skipping ability and degrades to $O(n)$.

This structural failure happens in two physical layouts:

- ***Structural Failure 1 (The Flat List):*** *Imagine the Skip List was built entirely of short nodes that only exist at Level 0. Because the maximum level of the list is 0, the vertical for loop executes exactly once. The inner while loop is forced to do all the heavy lifting. If the target node is at the very end of the list, the pointer must physically step through all n nodes one by one on the ground floor.*
- ***Structural Failure 2 (The Clogged Top Floor):*** *Imagine the list was built entirely of tall nodes, meaning every single node exists at the maximum level (e. g., Level 3). The normal search correctly starts at the top floor. However, because every node has a pointer on Level 3, the pointer cannot skip over anything. The while loop is forced to evaluate every single node horizontally across the crowded top floor.*

Result: *In either degenerate physical structure, the normal search algorithm is forced to perform n sequential horizontal comparisons to reach the end of the list. This phase degrades to $O(n)$.*

Phase 2: Pointer Rewiring & Deallocation

$[lst \rightarrow level \leq MAX_LEVEL]$

```
for (int i = 0; i <= lst->level; i++) {  
    if (update[i]->forward[i] != current) {  
        break; // If a level doesn't point to current, higher levels  
won't  
        // either  
    }  
    update[i]->forward[i] = current->forward[i];  
}
```

Once the $O(n)$ search finally reaches the target node, the algorithm must rewire the pointers to bypass it.

- **The Code:** The algorithm uses a for loop running from 0 to $lst \rightarrow level$ to bridge the gaps: $update[i] \rightarrow forward[i] = current \rightarrow forward[i]$.
- **In the All Tails Flat List:** The deleted node only exists at Level 0. On the very next iteration ($i = 1$), the condition $if (update[i] \rightarrow forward[i] \neq current)$ evaluates to True, the break statement fires, and the loop terminates immediately.
- **In the All Heads Traffic Jam:** The deleted node exists on every level up to MAX_LEVEL . The for loop must run MAX_LEVEL times to rewire every single floor.
- **Result:** Because MAX_LEVEL is strictly bounded by a mathematical cap (e.g., a hardcoded 3, or logarithmically capped at $\log_2 n$), therefore this loop can never scale to n . The pointer rewiring, along with the $free(current \rightarrow forward)$ and $free(current)$ commands, executes in a maximum of $O(\log n)$ or $O(1)$ time.

Phase 3: Global Height Adjustment

```
while (lst->level > 0 && lst->header->forward[lst->level] ==  
NULL) {  
    lst->level--;  
}
```

Finally, the algorithm cleans up the list's global height ceiling.

- **The Code:** A **while** loop checks $lst \rightarrow header \rightarrow forward[lst \rightarrow level] == NULL$ and shrinks the list via $lst \rightarrow level --$ if a floor becomes completely empty.
- **The Execution:** Even if deleting the final node leaves multiple top floors completely empty (such as deleting the final tall node in an "All Heads" list), the **while** loop can only drop the level down from **MAX_LEVEL** to 0.
- **Result:** Because it is bounded by the **MAX_LEVEL** cap, this phase takes a maximum of **$O(\log n)$** or **$O(1)$** time.

Final Calculation

To calculate the absolute Worst Case Time Complexity, we sum the mathematical bounds of the three phases:

- Phase 1 (Search): **$O(n)$**
- Phase 2 (Rewiring): **$O(\log n)$ or $O(1)$**
- Phase 3 (Adjustment): **$O(\log n)$ or $O(1)$**

Because the linear **n** term massively dominates the constant and logarithmic terms as the list grows towards infinity, the smaller terms are dropped.

Total Worst Case Time Complexity = $O(n)$

3. Average Case Time Complexity: $O(\log n)$

The expected time complexity is determined by breaking the *deleteElement* algorithm down into its three sequential phases.

Phase 1: The Search Phase

- The function uses a vertical for loop and a horizontal **while** loop to populate the **update** array with the memory addresses of predecessor nodes.
- As established in William Pugh's backward walk derivations, perfectly balanced coin flips ($p = 0.5$) allow the pointer to systematically skip halves of the list. The expected time to locate the node to be deleted is $O(\log n)$.

Phase 2: Pointer Rewiring & Deallocation

- To physically detach the node, the code iterates upward from 0 to $lst \rightarrow level$.
- Inside the loop, the algorithm checks **if** ($update[i] \rightarrow forward[i] \neq current$) and breaks early if higher levels do not point to the deleted node. This highly optimized loop only runs for the specific height of the deleted node, not the maximum height of the entire list.
- Because the expected height of any single node is a tiny constant (e. g., an average of 2 pointers when $p = 0.5$), the rewiring loop and subsequent **free()** commands execute in $O(1)$ expected time.

Phase 3: Global Height Adjustment

- After the node is removed from memory, a while loop checks $lst \rightarrow header \rightarrow forward[lst \rightarrow level] == NULL$. If the deleted node was the only tall node keeping the list's maximum level high, the code safely shrinks the global list height ($lst \rightarrow level--$).
- Because deleting a single node will rarely shrink the list by more than 1 or 2 levels, this cleanup loop executes in $O(1)$ expected time.

Final Calculation:

To find the average case, we sum the mathematical bounds of the three phases:

- *Search: $O(\log n)$ + Rewiring: $O(1)$ + Adjustment: $O(1)$*
 - *Total Expected Time Complexity = $O(\log n)$*
-

Skip – List
–Delete Element
–Space Complexity Analysis

Program :

```
void deleteElement(struct SkipList *lst, int key) {
    struct Node *current = lst->header;
    struct Node *update[MAX_LEVEL + 1];
    memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));

    // Find the node, keeping track of pointers that point to it
    for (int i = lst->level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key <
key) {
            current = current->forward[i];
        }
        update[i] = current;
    }

    current = current->forward[0];

    // If the target node is found
    if (current != NULL && current->key == key) {
        // Rewire the pointers to bypass the deleted node
        for (int i = 0; i <= lst->level; i++) {
            if (update[i]->forward[i] != current) {
                break; // If a level doesn't point to current, higher levels
won't
                // either
            }
            update[i]->forward[i] = current->forward[i];
        }

        // Free memory using free()
        free(current->forward);
        free(current);

        // Reduce the maximum level of the list if layers become empty
```

```

        while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
            lst->level--;
        }
        cout << "Successfully deleted key " << key << endl;
    } else {
        cout << "Key " << key << " not found. Cannot delete." << endl;
    }
}

...
case 2:
    cout << "Enter key to delete: ";
    cin >> key;
    deleteElement(list, key);
    break;

```

Space Complexity Analysis

1. Input Space Complexity

A. Parametric Input Space: $O(1)$

- The ***deleteElement*** function signature requires exactly two arguments: a pointer to the Skip List (***lst***) and the integer to be deleted (***key***).
- Because these take a fixed, tiny amount of memory (***typically 8 bytes and 4 bytes***) regardless of how many millions of nodes exist in the list, the parametric input space is strictly ***$O(1)$*** .

B. Full Data Holistic Input Space: $O(n)$

- *The function operates on an already existing Skip List containing n nodes in memory.*
- *Because the probabilistic coin flip ensures the average number of pointers per node remains a small constant (e.g., 2 pointers when $p = 0.5$), the total footprint of the list scales linearly with the number of inserted elements.*
- *Therefore, the full data holistic input space is $O(n)$.*

2. Auxiliary Space Complexity: $O(1)$ Practical

/ $O(\log n)$ Theoretical

- ***The Array Allocation:*** *To remember this path, the function explicitly declares an array: `struct Node * update[MAX_LEVEL + 1];`.*
- ***Practical Scenario ($O(1)$):*** *In your specific C++ program, `MAX_LEVEL` is a hardcoded constant (e.g., 3). Because the array is always strictly capped at 4 slots (32 bytes) no matter how large the list grows, the practical auxiliary space is exactly $O(1)$.*

In a real – world software implementation, `MAX_LEVEL` is not calculated dynamically based on n while the program runs.

It is defined as a fixed hard – coded limit based on the maximum anticipated capacity of the data structure (e.g., `Max_Level = 32 or 64`).

- *Theoretical Scenario ($O(\log n)$): In a purely mathematical, infinitely scaling Skip List, the maximum level H dynamically scales to $\log_{1/p}(n)$. Because the *update* array must have enough slots to hold a pointer for every possible floor, the size of the array scales logarithmically. Therefore, the theoretical auxiliary space is $O(\log n)$.*

Conclusion:

When asked in a standard academic or interview setting, the auxiliary space for a Skip List deletion algorithm is generally cited as $O(\log n)$ because it requires tracking the logarithmic height of the search path to safely rewire the pointers.

Algorithm: SkipListDelete(LST, KEY)

1. [Initialize Pointers and Arrays]

SET CURRENT := HEADER[LST].

Declare an array UPDATE of size MAX_LEVEL

+ 1 and initialize all elements to NULL (or 0).

2. [Traverse List to Record Path]

Repeat for $i = \text{LEVEL}[LST]$ down to 0:

*Repeat while FORWARD[i][CURRENT] $\neq$ NULL AND
INFO[FORWARD[i][CURRENT]] < KEY:*

SET CURRENT := FORWARD[i][CURRENT].

[End of while Loop]

SET UPDATE[i] := CURRENT.

[End of for Loop]

3. [Move to Target Node]

SET CURRENT := FORWARD[0][CURRENT].

4. [Verify Key Existence]

If CURRENT $\neq$ NULL AND INFO[CURRENT] = KEY, Then:

Proceed to Step 5.

Else:

Proceed to Step 9.

5. [Rewire Pointers to Bypass Node]

Repeat for $i = 0$ to LEVEL[LST]:

If FORWARD[i][UPDATE[i]] $\neq$ CURRENT, Then:

*BREAK loop (If a level doesn't point to current,
higher levels won't either).*

[End of If Structure]

SET FORWARD[i][UPDATE[i]] := FORWARD[i][CURRENT].

[End of for Loop]

6. [Free Memory]

CALL free(FORWARD[CURRENT]).

CALL free(CURRENT).

7. [Adjust Global List Level]

*Repeat while LEVEL[LST] > 0 AND FORWARD[LEVEL[LST]][HEADER[LST]]
= NULL:*

SET LEVEL[LST] := LEVEL[LST] - 1.

[End of while Loop]

8. [Success Message]

Write: "Successfully deleted key ", KEY.

Proceed to Step 10.

9. [Handle Key Not Found]

Write: "Key ", KEY, " not found. Cannot delete."

10. [Exit]

END OF ALGORITHM.

Skip List
–Display List
–Program Analysis

Program:

```
// Display the skip list layer by layer
void displayList(struct SkipList *lst) {
    cout << "\n--- Skip List ---" << endl;
    for (int i = 0; i <= lst->level; i++) {
        struct Node *node = lst->header->forward[i];
        cout << "Level " << i << ": ";
        while (node != NULL) {
            cout << node->key << " ";
            node = node->forward[i];
        }
        cout << endl;
    }
    cout << "-----" << endl;
}

case 4:
    displayList(list);
    break;
```

Program Analysis

```
displayList (list);
```

displayList(list: ff08) → passed .

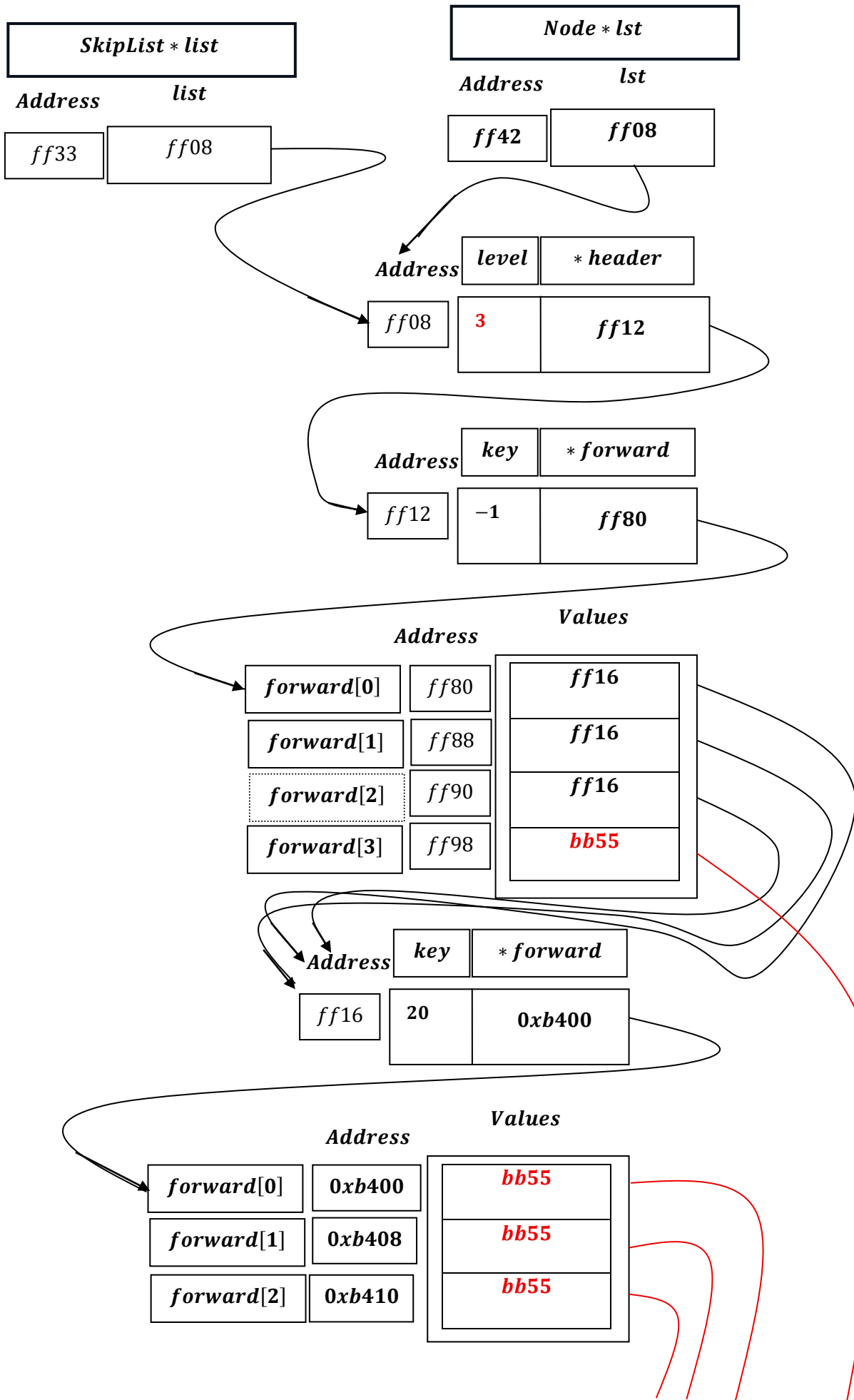
And,

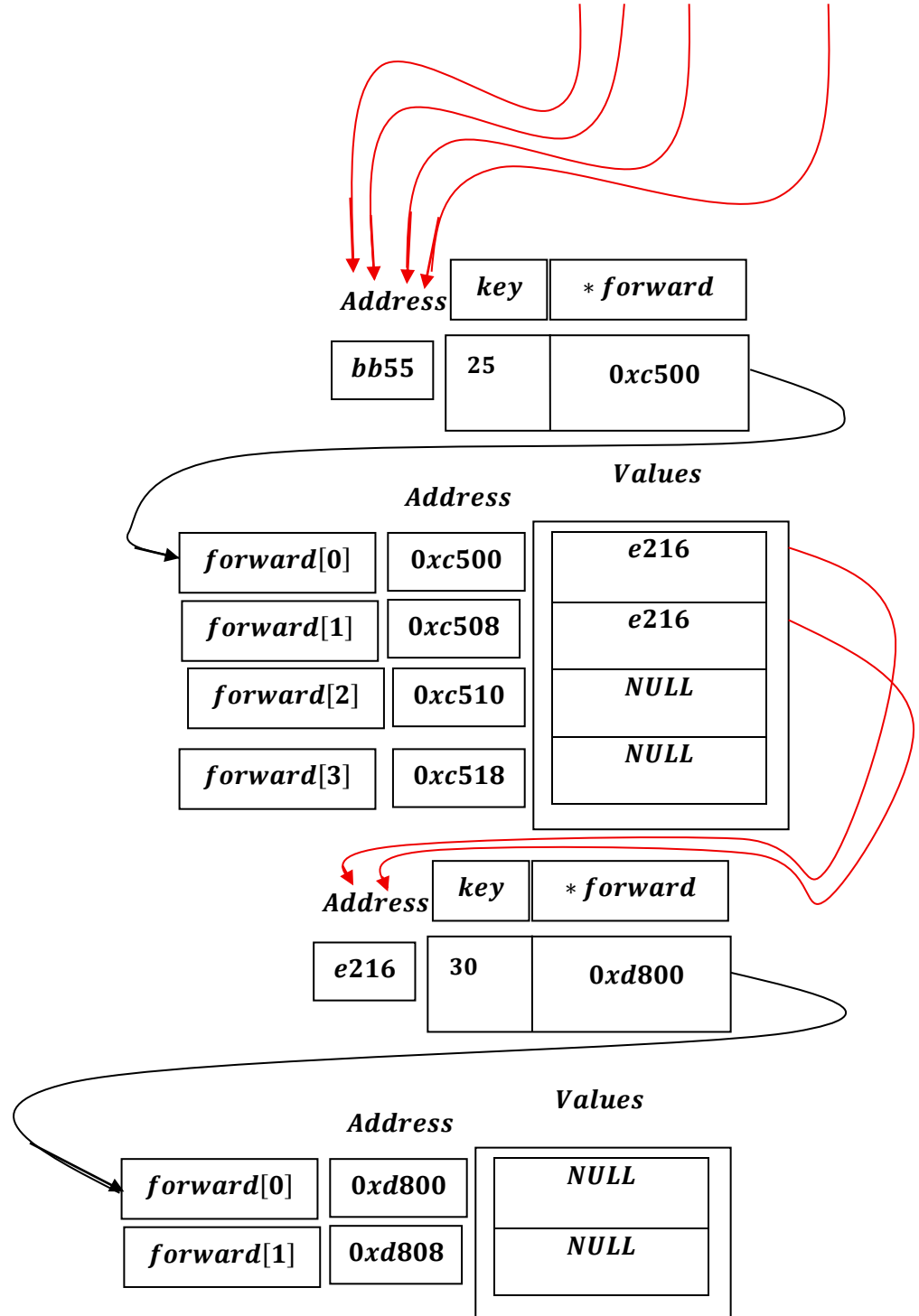
```
void displayList(struct SkipList *lst)
```

displayList(list := lst: ff08) → Assigned

```
cout << "\n--- Skip List ---" << endl;
```

Print: ---Skip List---





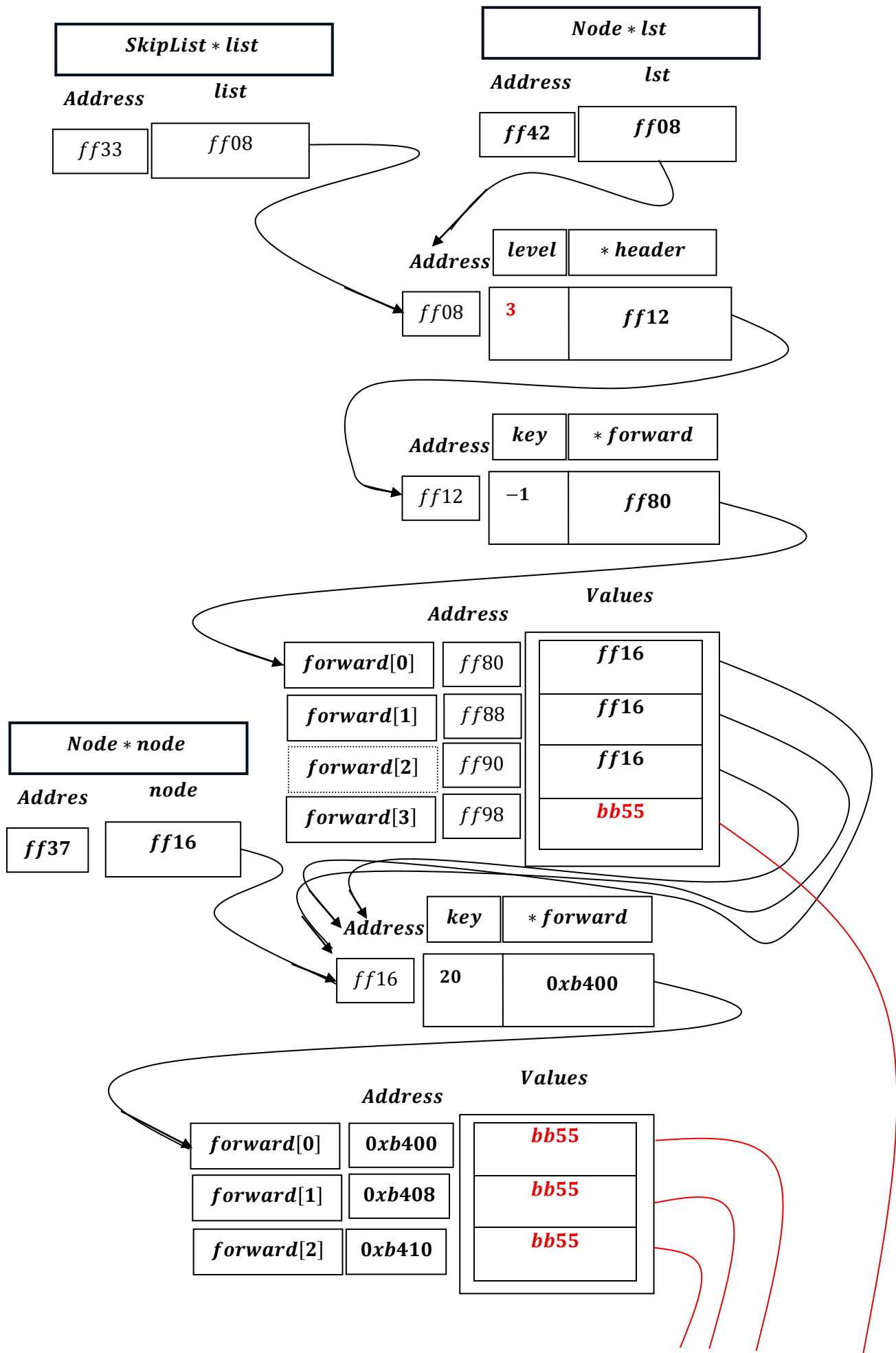
```

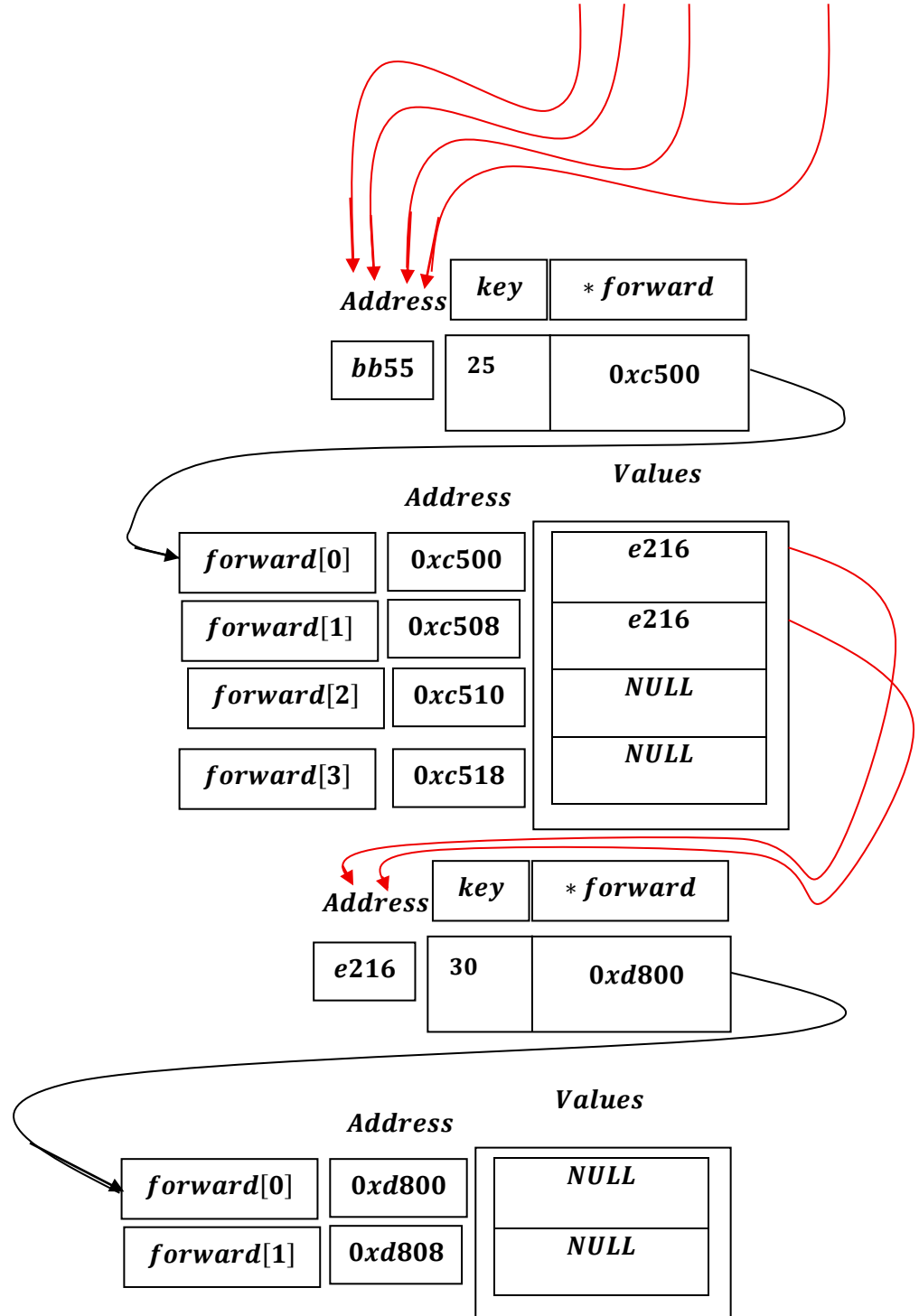
for (int i = 0; i <= lst->level; i++) {
    struct Node *node = lst->header->forward[i];
    cout << "Level " << i << ": ";
    while (node != NULL) {
        cout << node->key << " ";
        node = node->forward[i];
    }
    cout << endl;
}
cout << "-----" << endl;
}

```

for: int i = 0; i ≤ lst → level: 3[True]

```
struct Node * node = lst:ff08 → header:ff12 → forward[0]:ff16;
```





```
for: int i = 0; i ≤ lst → level: 3[True]
    struct Node * node = lst: ff08 → header: ff12 → forward[0]: ff16;
```

Print : `` Level ``0 ``:

```
while (node != NULL) {
    cout << node->key << " ";
    node = node->forward[i];
}
```

node ≠ NULL[True]

Print: ``node: ff16 → key" : 20

node = node: ff16 → forward[0]: bb55

node ≠ NULL[True]

Print: ``node: bb55 → key" : 25

node = node: bb55 → forward[0]: e216

node ≠ NULL[True]

Print: ``node: e216 → key" : 30

node = node: e216 → forward[0]: NULL

node = NULL[False]

Final Print:

Level 0 : 20 25 30

```
cout << endl;
```

Next Line of code

$i++ = i + 1 = 0 + 1 = 1$

for: int i = 1; i ≤ lst → level: 3[True]

struct Node * node = lst: ff08 → header: ff12 → forward[1]: ff16;

Print : `` Level ``1 ``:

```
while (node != NULL) {  
    cout << node->key << " ";  
    node = node->forward[i];  
}
```

node ≠ NULL[True]

Print: ``node: ff16 → key" : 20

node = node: ff16 → forward[1]: bb55

node ≠ NULL[True]

Print: ``node: bb55 → key" : 25

node = node: bb55 → forward[1]: e216

node ≠ NULL[True]

Print: ``node: e216 → key" : 30

node = node: e216 → forward[1]: NULL

node = NULL[False]

Final Print:

Level 0 : 20 25 30

Level 1 : 20 25 30

```
cout << endl;
```

Next Line of code

$i++ = i + 1 = 1 + 1 = 2$

for: int i = 2; i ≤ lst → level: 3[True]

struct Node * node = lst: ff08 → header: ff12 → forward[2]: ff16;

Print : `` Level ``2 ``:``

```
while (node != NULL) {  
    cout << node->key << " ";  
    node = node->forward[i];  
}
```

node ≠ NULL[True]

Print: ``node: ff16 → key" : 20

node = node: ff16 → forward[2]: bb55

node ≠ NULL[True]

Print: ``node: bb55 → key" : 25

node = node: bb55 → forward[2]: NULL

node = NULL[False]

Final Print:

Level 0 : 20 25 30

Level 1 : 20 25 30

Level 2 : 20 25

```
cout << endl;
```

Next Line of code

$i++ = i + 1 = 2 + 1 = 3$

for: int i = 3; i ≤ lst → level: 3[True]

struct Node * node = lst: ff08 → header: ff12 → forward[3]: bb55;

Print : `` Level ``3 ``:

```
while (node != NULL) {  
    cout << node->key << " ";  
    node = node->forward[i];  
}
```

node ≠ NULL[True]

Print: ``node: bb55 → key" : 30

node = node: bb55 → forward[3]: NULL

node = NULL[False]

Final Print:

Level 0 : 20 25 30

Level 1 : 20 25 30

Level 2 : 20 25

Level 3 : 30

```
cout << endl;
```

Next Line of code

i ++ = i + 1 = 3 + 1 = 4

for: int i = 3; i ≤ lst → level: 3 ***False***
for
Loop
Exit

∴ Final Print:

Level 0 : 20 25 30

Level 1 : 20 25 30

Level 2 : 20 25

Level 3 : 30

Skip List

–Display List

–Time Complexity Analysis

Program:

```
// Display the skip list layer by layer
void displayList(struct SkipList *lst) {
    cout << "\n--- Skip List ---" << endl;
    for (int i = 0; i <= lst->level; i++) {
        struct Node *node = lst->header->forward[i];
        cout << "Level " << i << ": ";
        while (node != NULL) {
            cout << node->key << " ";
            node = node->forward[i];
        }
        cout << endl;
    }
    cout << "-----" << endl;
}

case 4:
    displayList(list);
    break;
```

Time Complexity Analysis

1. Best Case Time Complexity: $O(n)$ or $O(1)$

- **Populated List ($O(n)$):** Even if the Skip List is an "All Tails" completely flat list (maximum level is 0), the `displayList` function still has to print every single element. The outer loop runs once ($i = 0$), and the inner loop traverses all n nodes on the ground floor. Therefore, if there are n elements to print, the absolute fastest it can run is linear $O(n)$ time.
- **Empty List ($O(1)$):** The only true constant – time scenario occurs if the Skip List contains exactly zero elements. The outer loop initializes, sees that the `header → forward[0]` is NULL, skips the inner `while` loop completely, and immediately exits. This evaluates in instantaneous $O(1)$ time.

2. Worst Case Time Complexity: $O(n \log n)$ or $O(n)$

The worst – case scenario happens if the insertion phase suffered a catastrophic All Heads statistical anomaly, promoting every single node to the absolute maximum level.

- **Theoretical Worst Case ($O(n \log n)$):** If the list has no hard cap [fixed constant] and the maximum level is allowed to scale to $\log n$, then all n nodes will exist on all $\log n$ levels.

The `displayList` function will be forced to print n nodes horizontally across $\log n$ separate floors.

This results in $O(n \log n)$ time.

- **Practical Worst Case ($O(n)$):** If the program enforces a strict, constant ceiling (e.g., `MAX_LEVEL = 3`), the outer loop can only ever run a maximum of 4 times. Even if all n nodes are clogged on all 4 floors, the loop runs $4n$ times. Because 4 is a constant, it drops away, leaving a practical worst – case complexity of $O(n)$.

Average Time Complexity

1. The Execution Mechanics

The *displayList* algorithm consists of two nested loops:

- **Vertical Traversal:** The outer *for* loop dictates the current floor, starting at *i = 0* and climbing to *lst → level*.
- **Horizontal Traversal:** The inner *while* loop scans across that specific floor, printing *node → key* until it hits *NULL*.

Instead of multiplying the maximum height by the maximum width (which would incorrectly imply an $O(n \log n)$ average), the algorithm simply touches every single valid *forward* pointer exactly once to print its value. Therefore, the total execution time is perfectly equal to the total number of pointers in the entire structure.

2. The Expected Number of Pointers

In a well – balanced Skip List (using a fair coin toss where $p = 0.5$), the probabilistic balancing ensures that the number of nodes halves on every subsequent floor.

In the specific *randomLevel()* code used in this program, Heads(a successful coin flip) corresponds to 0.

Here is exactly how the program simulates the coin toss using the modulo operator:

- **Heads (Success = 0):** The program evaluates $(rand() \% 2) == 0$. If the random number generator produces an even number, the result is 0. The program treats this as a successful “Heads,” increases the node's level (*lvl + +*), and loops to flip the coin again.
- **Tails (Failure = 1):** If the random number generator produces an odd number, the result of $rand() \% 2$ is 1. The program treats this as a failed Tails, the *while* loop condition instantly becomes False, and the node stops growing.

- **Level 0:** Contains all n nodes.

Level 0 (The Ground Floor): Every single element inserted into the list must have a pointer on the ground floor. There is a **100%** probability of reaching Level 0, meaning all n elements exist here.

- **Level 1:** Contains an expected $\frac{n}{2}$ nodes.

Level 1: To grow to Level 1, each of those n nodes flips a coin. Because the chance of getting Heads is exactly **50%**, we mathematically expect only half of the elements from **Level 0** to be promoted. Therefore, the total expected pointers on this floor is $\frac{n}{2}$.

- **Level 2:** Contains an expected $\frac{n}{4}$ nodes.

Level 2: Now, only those $\frac{n}{2}$ nodes that made it to Level 1 get to flip the coin again. They have a **50%** chance to win a second time. Only half of **Level 1** is expected to be promoted. Half of $\frac{n}{2}$ gives us $\frac{n}{4}$ expected pointers.

- **Level 3:** Contains an expected $\frac{n}{8}$ nodes.

Level 3: The surviving $\frac{n}{4}$ nodes flip the coin a third time. Again, we expect half of them to win and half to lose. Half of $\frac{n}{4}$ gives us $\frac{n}{8}$ expected pointers.

To find the total work the algorithm must perform, we sum the nodes across all levels. This forms an infinite geometric series:

$$Total = n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots$$

factor out the n:

$$TotalSpace = n \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots \right)$$

We know ,through geometric infinite series : $s_{\infty} = \frac{a_1}{1-r}$

$$a_1 = 1 \text{ (First Term) and } r = \frac{\text{Any Term}}{\text{Previous Term}} \Rightarrow \frac{1}{2} = \frac{1}{2} ; \frac{1/4}{1/2} = \frac{1}{2} \text{ etc.}$$

$$\therefore S_{\infty} = \frac{a_1}{1-r} = \frac{1}{1-\frac{1}{2}} = \frac{1}{\frac{1}{2}} = 2.$$

$$TotalSpace = n \times S_{\infty} = n \times 2 = 2n.$$

Alternatively ,Algebraic Shifting Method:

$$\text{Let , } S = 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots$$

Now, Multiplying $\frac{1}{2}$ on both the sides we get:

$$\Rightarrow \frac{1}{2}S = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots$$

Now subtracting we get:

$$\Rightarrow S - \frac{1}{2}S = (1 - 0) + \left(\frac{1}{2} - \frac{1}{2}\right) + \left(\frac{1}{4} - \frac{1}{4}\right) + \left(\frac{1}{8} - \frac{1}{8}\right) + \dots$$

$$\Rightarrow S - \frac{1}{2}S = 1$$

$$\Rightarrow S \left(1 - \frac{1}{2}\right) = 1$$

$$\Rightarrow S \left(\frac{1}{2}\right) = 1$$

$$\Rightarrow S = 2$$

$$\text{TotalSpace} = n \times S = n \times 2 = 2n.$$

*Because the nested loops must execute exactly once for every valid pointer, they will perform an average of **2n** operations to print the entire list. In Big – O notation, constant multipliers are discarded to focus on the scaling trend.*

*Leaving out constant : **2** , leaves a strict average time complexity of **O(n)**.*

Hence, Average Time Complexity : O(n)

Skip List
–Display List
–Space Complexity Analysis

Program:

```
// Display the skip list layer by layer
void displayList(struct SkipList *lst) {
    cout << "\n--- Skip List ---" << endl;
    for (int i = 0; i <= lst->level; i++) {
        struct Node *node = lst->header->forward[i];
        cout << "Level " << i << ": ";
        while (node != NULL) {
            cout << node->key << " ";
            node = node->forward[i];
        }
        cout << endl;
    }
    cout << "-----" << endl;
}

case 4:
    displayList(list);
    break;
```

Space Complexity Analysis

1. Input Space Complexity

The input space is evaluated by looking at the specific arguments passed into the function and the total size of the data structure it operates on.

A. Parametric Input Space: $O(1)$

- ***The Parameter:*** *The function signature is `void displayList(struct SkipList * lst)`.*

It only takes a single argument: a memory pointer addressing the existing Skip List structure.

- ***The Math:*** *A standard memory pointer takes a fixed, tiny amount of memory (typically 8 bytes on a 64 – bit system).*
- ***The Conclusion:*** *Because the size of this pointer never changes regardless of how many millions of elements are inside the list, the parametric input space is strictly $O(1)$.*

B. Full Data Holistic Input Space: $O(n)$

- ***The Structure:*** *The function is tasked with displaying the entire, fully – built Skip List.*

- **The Math:** As proven in the Space Complexity document, a Skip List using a 50% coin flip probability ($p = 0.5$) generates an average of 2 pointers per node. Therefore, a list of n elements will contain an expected $2n$ total pointers.

Method 1: The Vertical Approach (Expected Pointers per Node)

Instead of looking at the whole list, this method asks:

When a single node is created, how many pointers

does it have on average?

Because the node's height is decided by a 50% coin flip ($p = 0.5$), we calculate the probability of it getting a pointer on each specific level:

- Level 0: It ALWAYS gets a pointer here. (**Probability = 1**).
- Level 1: It gets a pointer here if the first coin flip is Heads. (**Probability = p**).
- Level 2: It gets a pointer here if it flips Heads twice in a row. (**Probability = p^2**).
- Level 3: It gets a pointer here if it flips Heads three times. (**Probability = p^3**).

To find the total expected pointers for one single node,

we sum this infinite geometric series:

$$\mathbb{E}[\text{Pointers per Node}] = 1 + p + p^2 + p^3 + \dots = S_{\infty} = \frac{1}{1 - p}$$

Now, substitute $p = 0.5$ (for our 50% coin flip):

$$\frac{1}{1 - 0.5} = \frac{1}{0.5} = 2$$

- **The Result:** A single node has an average of exactly **2** pointers.
- **Total Calculation:** If **1** node has **2** pointers, then n elements require exactly **$2n$** total pointers.

Method 2: The Horizontal Approach (Elements per Layer)

This method looks at the entire Skip List horizontally, summing up the total number of nodes expected to exist on every single floor.

- **Level 0:** Must contain every single element. (Total pointers = n).
- **Level 1:** Because of the 50% coin flip, only half of Level 0 is expected to be promoted. (Total expected pointers = $\frac{n}{2}$).
- **Level 2:** Only half of Level 1 is expected to be promoted. (Total expected pointers = $\frac{n}{4}$).
- **Level 3:** Half of Level 2. (Total expected pointers = $\frac{n}{8}$).

To find the total memory space, we add all the floors together:

$$\text{Total Space} = n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots$$

If we factor out the n , it looks like this:

$$\text{Total Space} = n \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots \right)$$

The sum of that infinite fraction series inside the parentheses

*$\left(1 + \frac{1}{2} + \frac{1}{4} \dots \right)$ mathematically resolves to exactly **2**.*

$$\text{Total Space} = n \times 2 = 2n$$

Both methods perfectly mathematically align to prove that a

*Skip List with n elements will generate exactly **$2n$** pointers on average!*

2. Auxiliary Space Complexity: $O(1)$

Auxiliary space measures the temporary, extra memory the algorithm requests specifically to do its job, which is then destroyed when the function finishes.

- **The Variables:** Inside the `displayList` function, the code only declares two temporary variables: an integer `i` for the `for` loop, and a traversal pointer `struct Node * node` to walk across the levels.
- **No Arrays Needed:** Unlike the Insertion and Deletion modules, the Display module does not need to remember its previous path. Therefore, it does not allocate the `update[MAX_LEVEL + 1]` array.
- **The Conclusion:** Because the function allocates zero arrays, makes zero recursive calls, and only uses two simple temporary variables, its auxiliary space overhead is an absolute, strict $O(1)$.

Final Total Space Complexity

To find the total space complexity of the module, we add the dominant input space to the auxiliary space:

- **Total Space Complexity** = Full Data Holistic Space: $(O(n))$ + Auxiliary Space: $(O(1)) = O(n)$.
-

Algorithm: SkipListDisplay(LST)

- [illegible]

Skip List
–Destroy List
–Program Analysis

Program:

```
void destroyList(struct SkipList *lst) {
    // Start at the header
    struct Node *current = lst->header;
    struct Node *next;

    // We only need to traverse level 0 (the "local train" level)
    // because it is the only level guaranteed to contain EVERY node.
    while (current != NULL) {
        // 1. Save the pointer to the next node
        next = current->forward[0];

        // 2. Free the dynamically allocated array of pointers
        free(current->forward);

        // 3. Free the node itself
        free(current);

        // 4. Move to the next node
        current = next;
    }

    // Finally, free the main Skip List structure
    free(lst);
    cout << "Memory successfully freed. Skip List destroyed." << endl;
}

...
case 5:
    cout << "Destroying the current list..." << endl;
    destroyList(list);
    // CRITICAL: We must give the program a fresh, empty list
    // so it doesn't crash if the user tries to insert again.
    break;
```

Program Analysis

```
destroyList(list);
```

destroyList(list: ff08) → passed .

And,

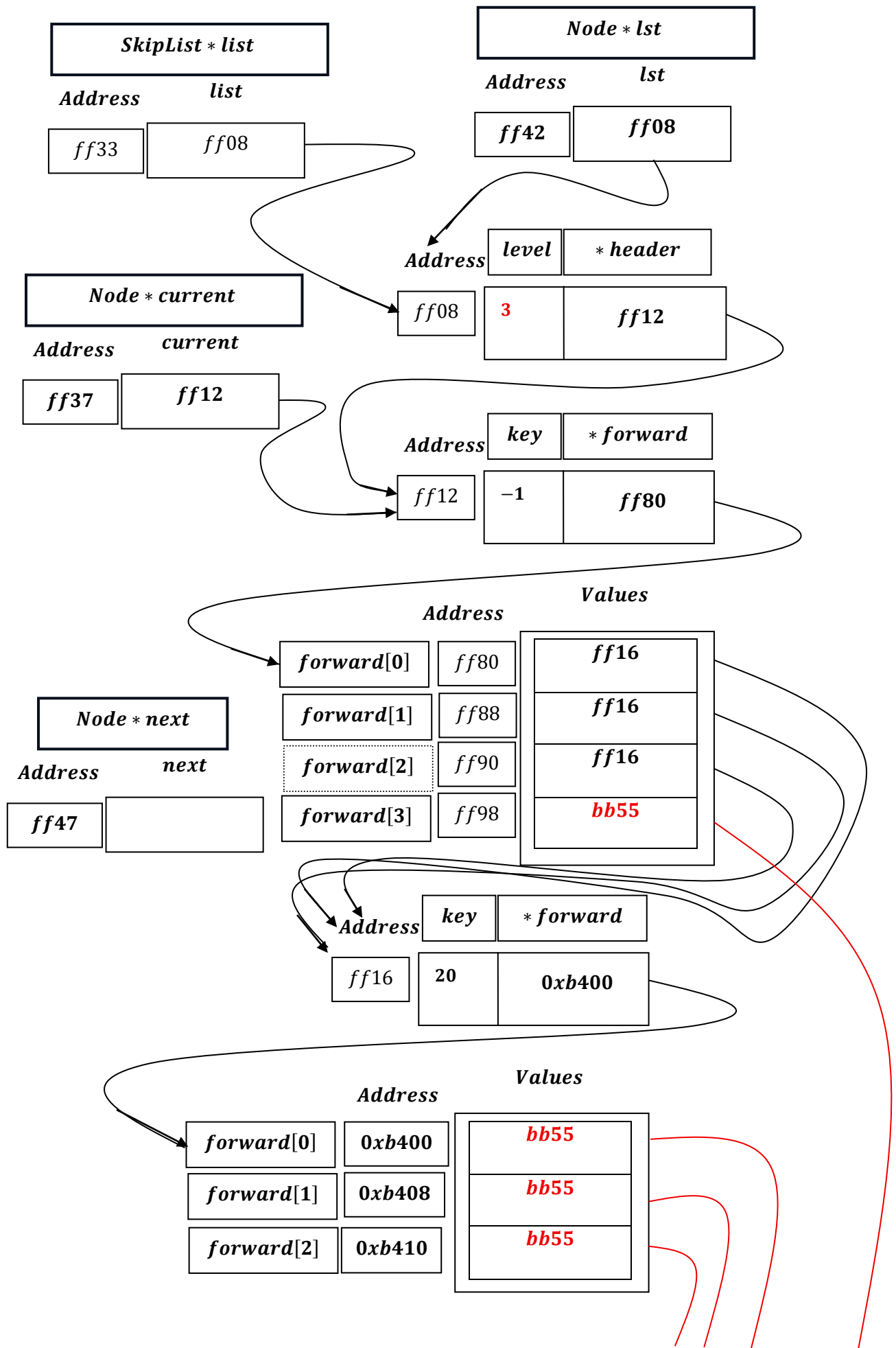
```
void destroyList(struct SkipList *lst)
```

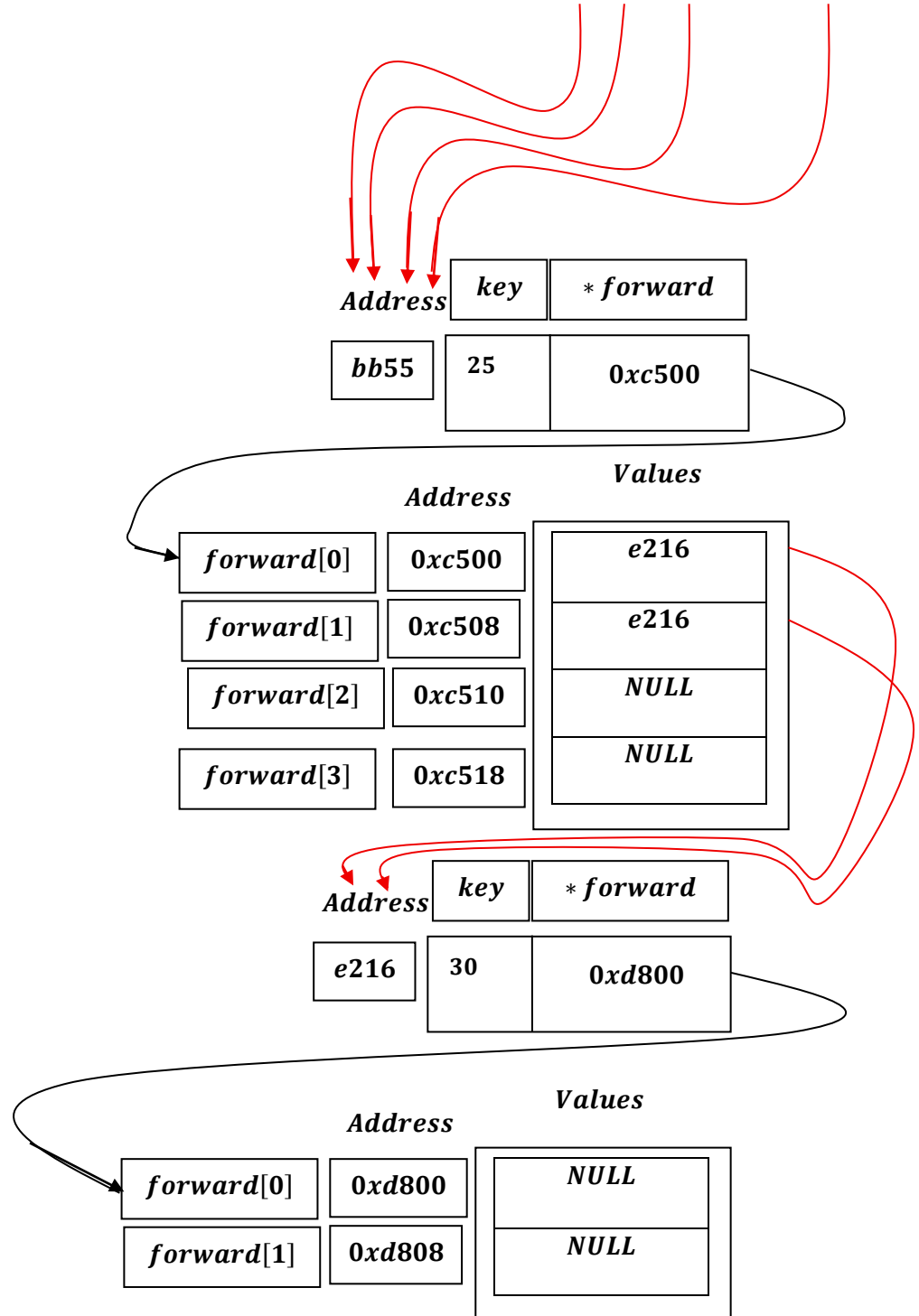
destroyList(list := lst: ff08) → Assigned

```
struct Node *current = lst->header;
```

current = lst → header = ff12.

```
struct Node *next;
```





```
while (current != NULL) {
    // 1. Save the pointer to the next node
    next = current->forward[0];

    // 2. Free the dynamically allocated array of pointers
    free(current->forward);
    // 3. Free the node itself
    free(current);

    // 4. Move to the next node
    current = next;
}
```

While(current: ff12 $\neq$ NULL)[True] :

next = current: ff12 $\rightarrow$ forward[0]: ff16

free(current: ff12 $\rightarrow$ forward)

*The pointer **current** $\rightarrow$ **forward** holds the starting address of this chunk, which is **ff80**. This is where **forward[0]** lives.*

***The Offset:** The address **ff88** is simply the location of the second element in that array (**forward[1]**). It is exactly 8 bytes after the starting address, sitting inside the same block of memory.*

*When the program executes **free(current $\rightarrow$ forward)**, it tells the system to deallocate the entire chunk of memory that begins at **ff80**. Because **ff88** is just a part of that continuous chunk, it gets completely freed at the exact same time.*

It will not add any offsets and delete each block of memory but all at once.

```
free(current);
```

free(current: ff12)

```
current = next;
```

current = next = ff16 .

*SkipList * list*

Address list

ff33 ff08

*Node * lst*

Address lst

ff42 ff08

*Address level * header*

ff08 3 ff12

Address Values

ff12

*Node * next*

Address next

ff47 ff16

*Node * current*

Address current

ff37 ff16

*Address key * forward*

ff16 20 0xb400

Values

Address

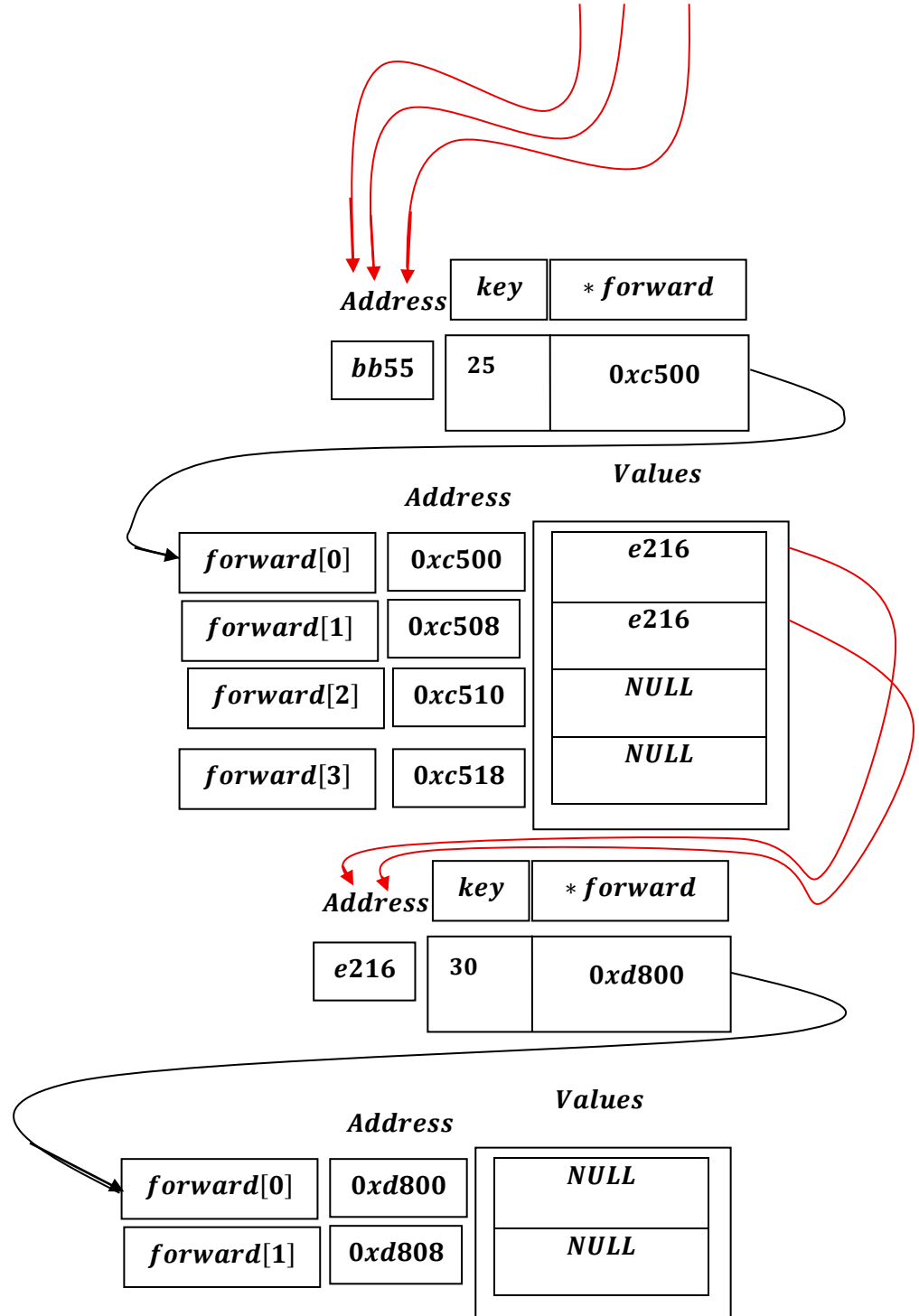
ff80
ff88
ff90
ff98

Values

Address

forward[0] 0xb400
forward[1] 0xb408
forward[2] 0xb410

bb55
bb55
bb55



While(current: ff16 $\neq$ NULL)[True] :

next = current: ff16 $\rightarrow$ forward[0]: bb55

free(current: ff16 $\rightarrow$ forward)

The pointer **current** $\rightarrow$ **forward** holds the starting address of this chunk, which is **0xb400**. This is where **forward[0]** lives.

The Offset: The address **0xb408** is simply the location of the second element

in that array (*forward[1]*). It is exactly 8 bytes after the starting address, sitting inside the same block of memory.

When the program executes *free(current → forward)*, it tells the system to deallocate the entire chunk of memory that begins at *0xb400*. Because *0xb408* is just a part of that continuous chunk, it gets completely freed at the exact same time.

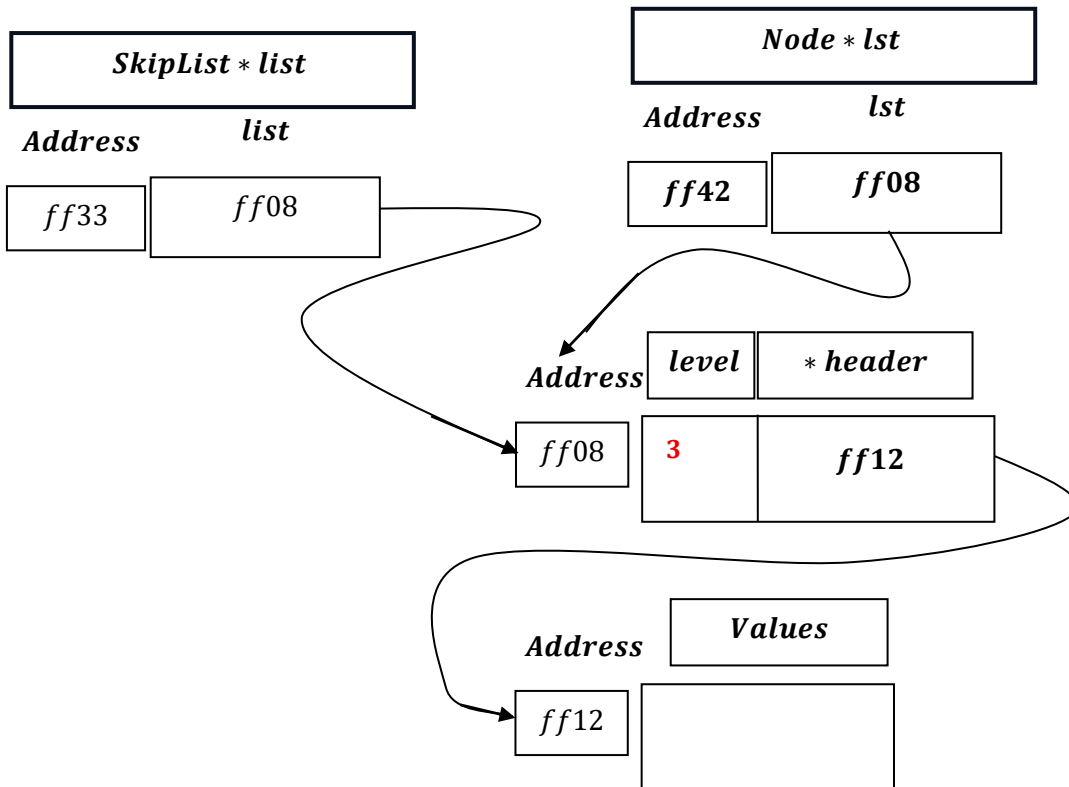
It will not add any offsets and delete each block of memory but all at once.

```
free(current);
```

free(current: ff16)

```
current = next;
```

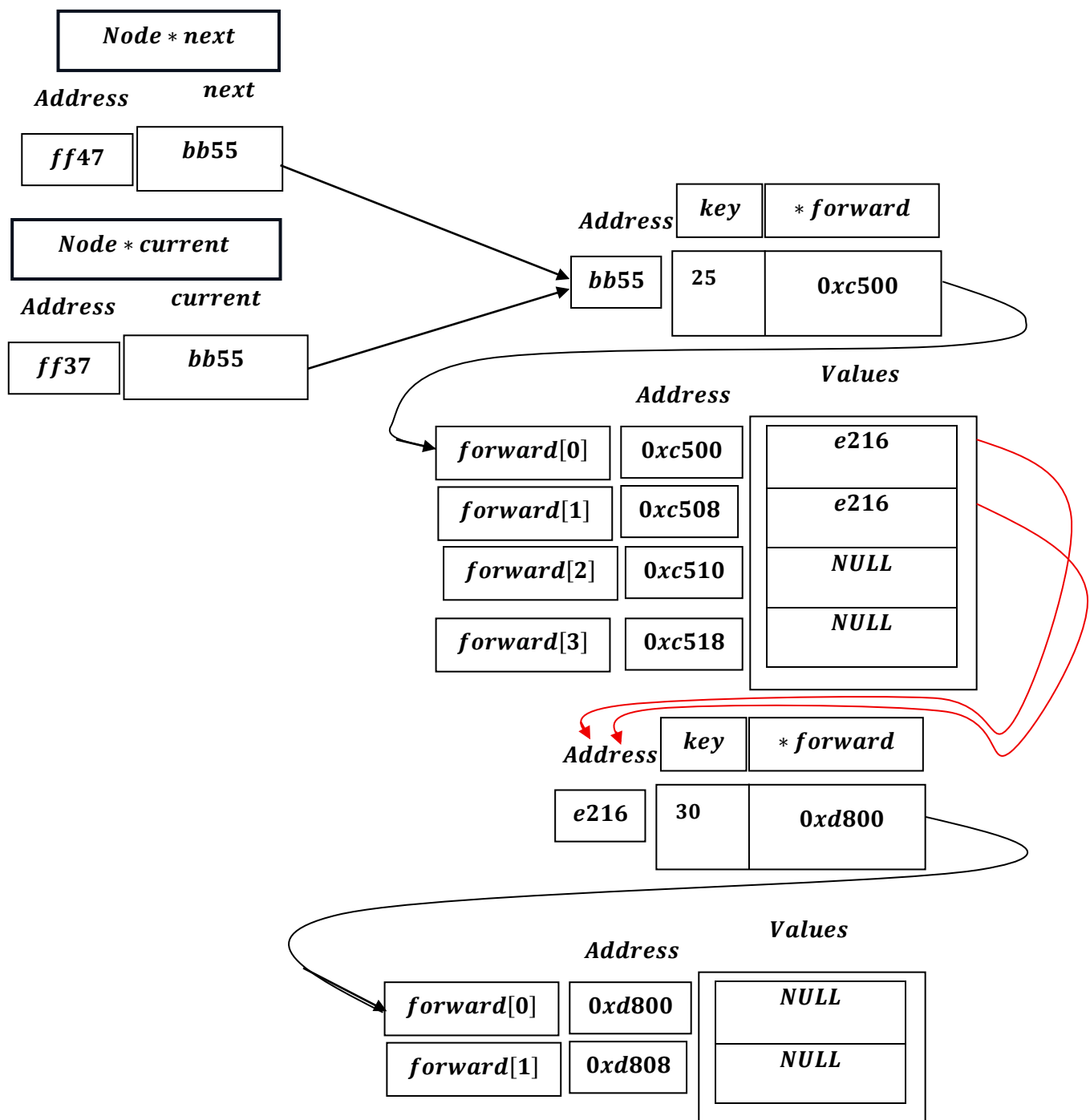
current = next = bb55 .



Address	Values
ff80	
ff88	
ff90	
ff98	

Address	Values
ff16	

Address	Values
0xb400	
0xb408	
0xb410	



While(current: bb55 $\neq$ NULL)[True] :

next = current: bb55 $\rightarrow$ forward[0]: e216

free(current: bb55 $\rightarrow$ forward)

The pointer **current $\rightarrow$ forward** holds the starting address of this chunk, which is **0xc500**. This is where **forward[0]** lives.

***The Offset:** The address **0xc508** is simply the location of the second element in that array (**forward[1]**). It is exactly 8 bytes after the starting address, sitting inside the same block of memory.*

*When the program executes **free(current → forward)**, it tells the system to deallocate the entire chunk of memory that begins at **0xc500**. Because **0xc508** is just a part of that continuous chunk, it gets completely freed at the exact same time.*

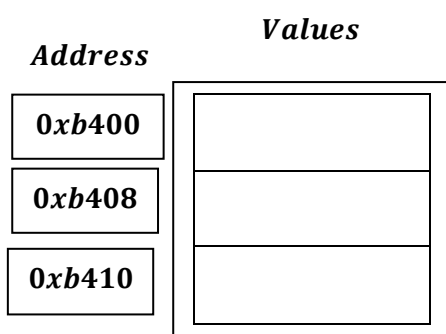
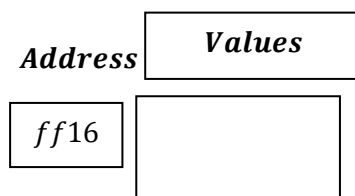
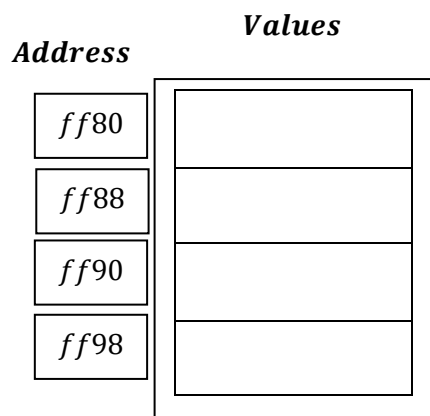
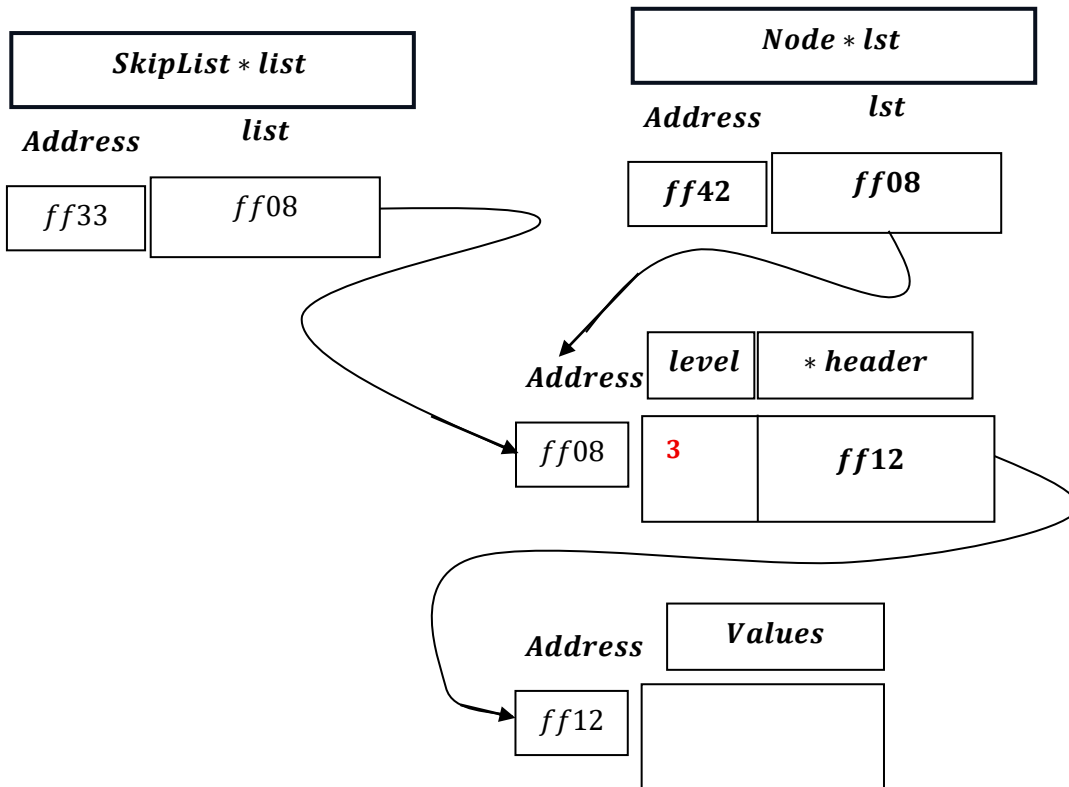
It will not add any offsets and delete each block of memory but all at once.

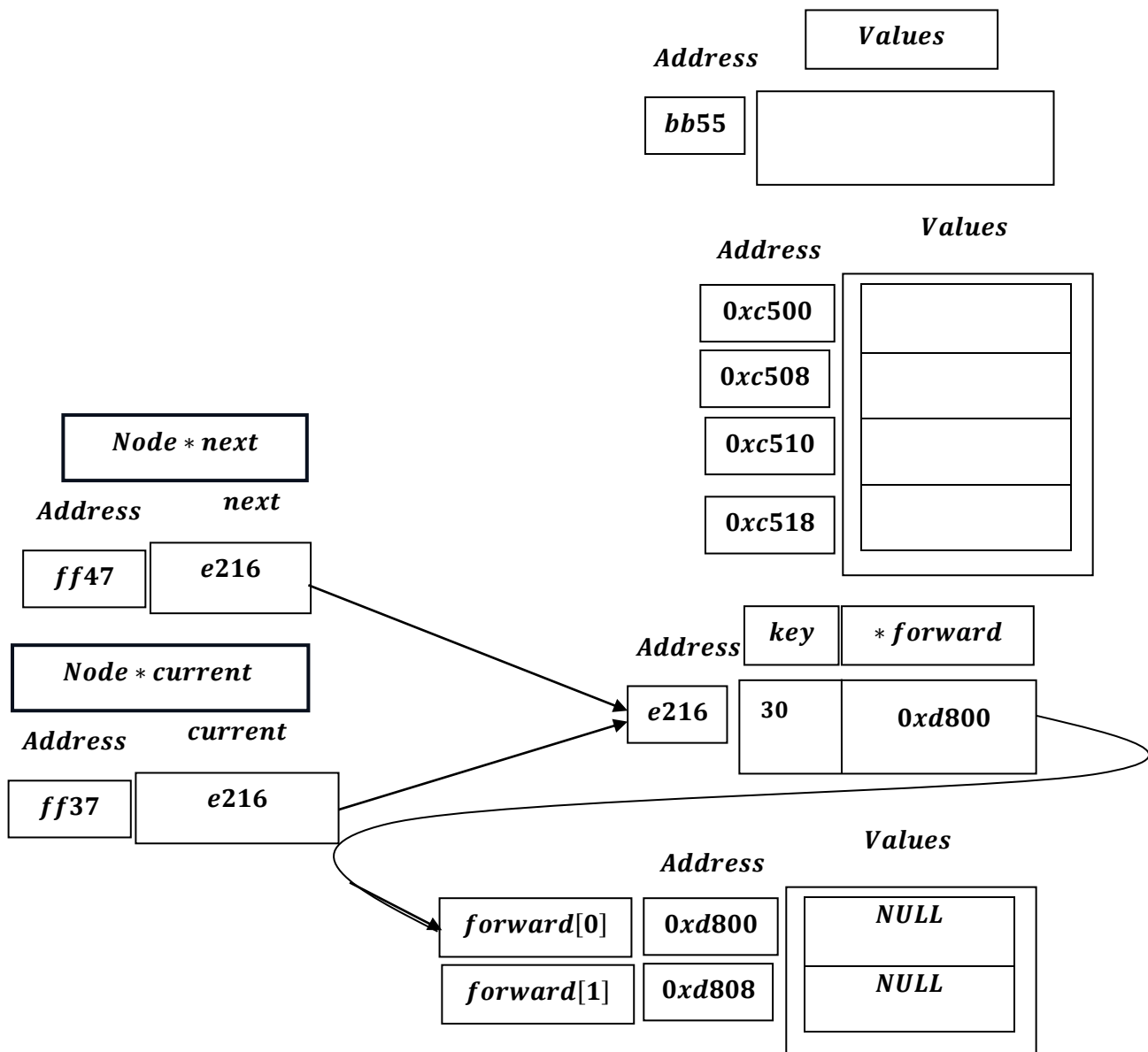
```
free(current);
```

free(current: bb55)

```
current = next;
```

current = next = e216 .





While(*current*: `e216` $\neq$ `NULL`)[*True*] :

next = *current*: `e216` $\rightarrow$ `forward[0]`: `NULL`

free(*current*: `e216` $\rightarrow$ `forward`)

The pointer *current* $\rightarrow$ *forward* holds the starting address of this chunk, which is `0xd800`. This is where `forward[0]` lives.

***The Offset:** The address **0xd808** is simply the location of the second element in that array (**forward[1]**). It is exactly 8 bytes after the starting address, sitting inside the same block of memory.*

*When the program executes **free(current → forward)**, it tells the system to deallocate the entire chunk of memory that begins at **0xd800**. Because **0xd808** is just a part of that continuous chunk, it gets completely freed at the exact same time.*

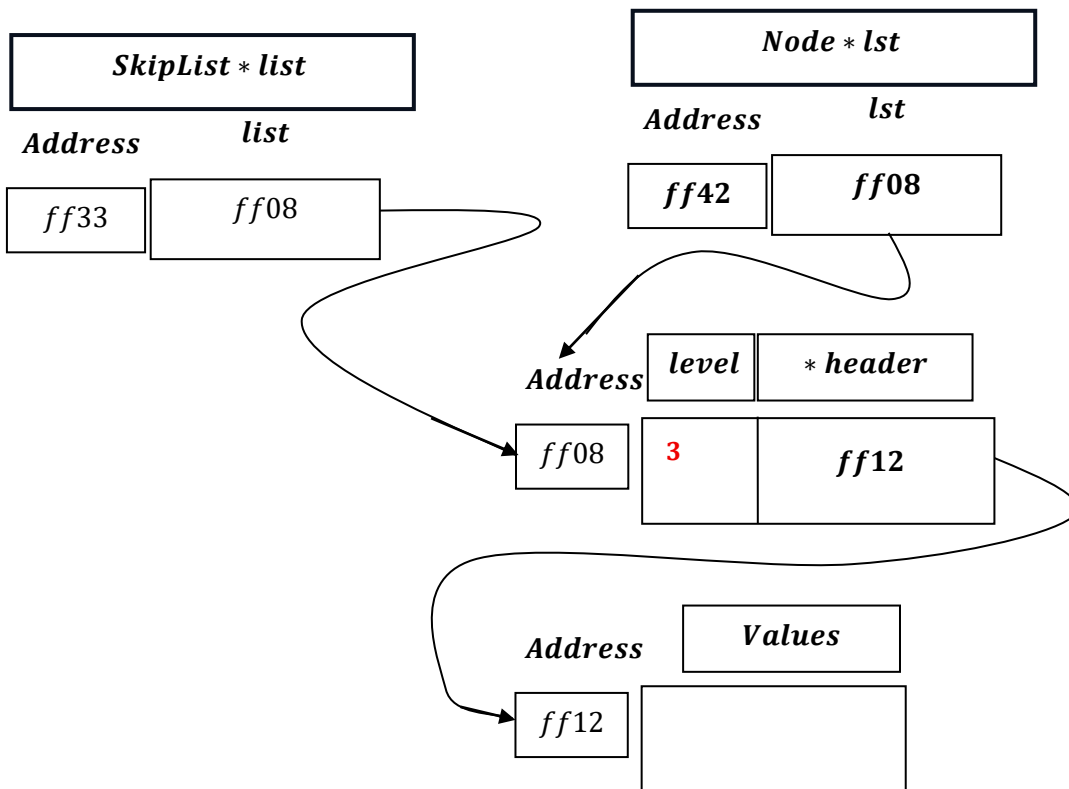
It will not add any offsets and delete each block of memory but all at once.

```
free(current);
```

free(current: e216)

```
current = next;
```

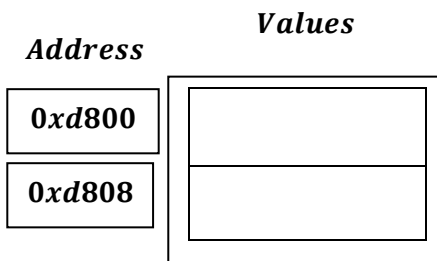
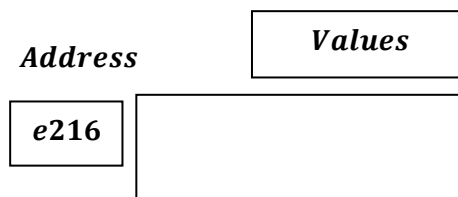
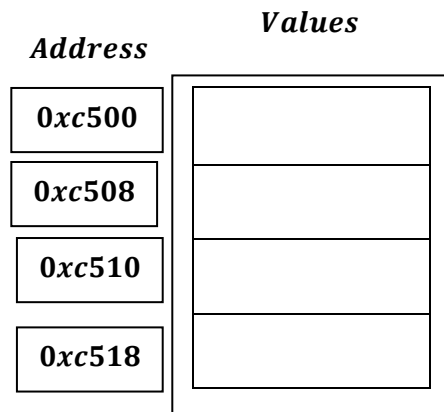
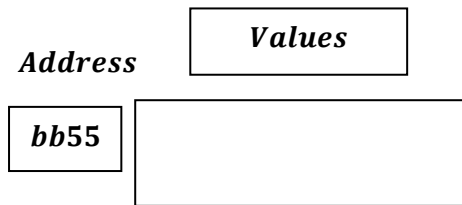
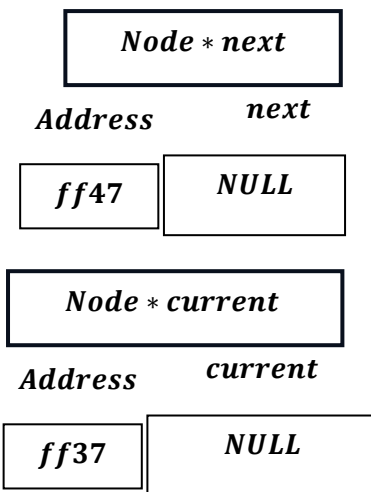
current = next = NULL .



Address	Values
ff80	
ff88	
ff90	
ff98	

Address	Values
ff16	

Address	Values
0xb400	
0xb408	
0xb410	

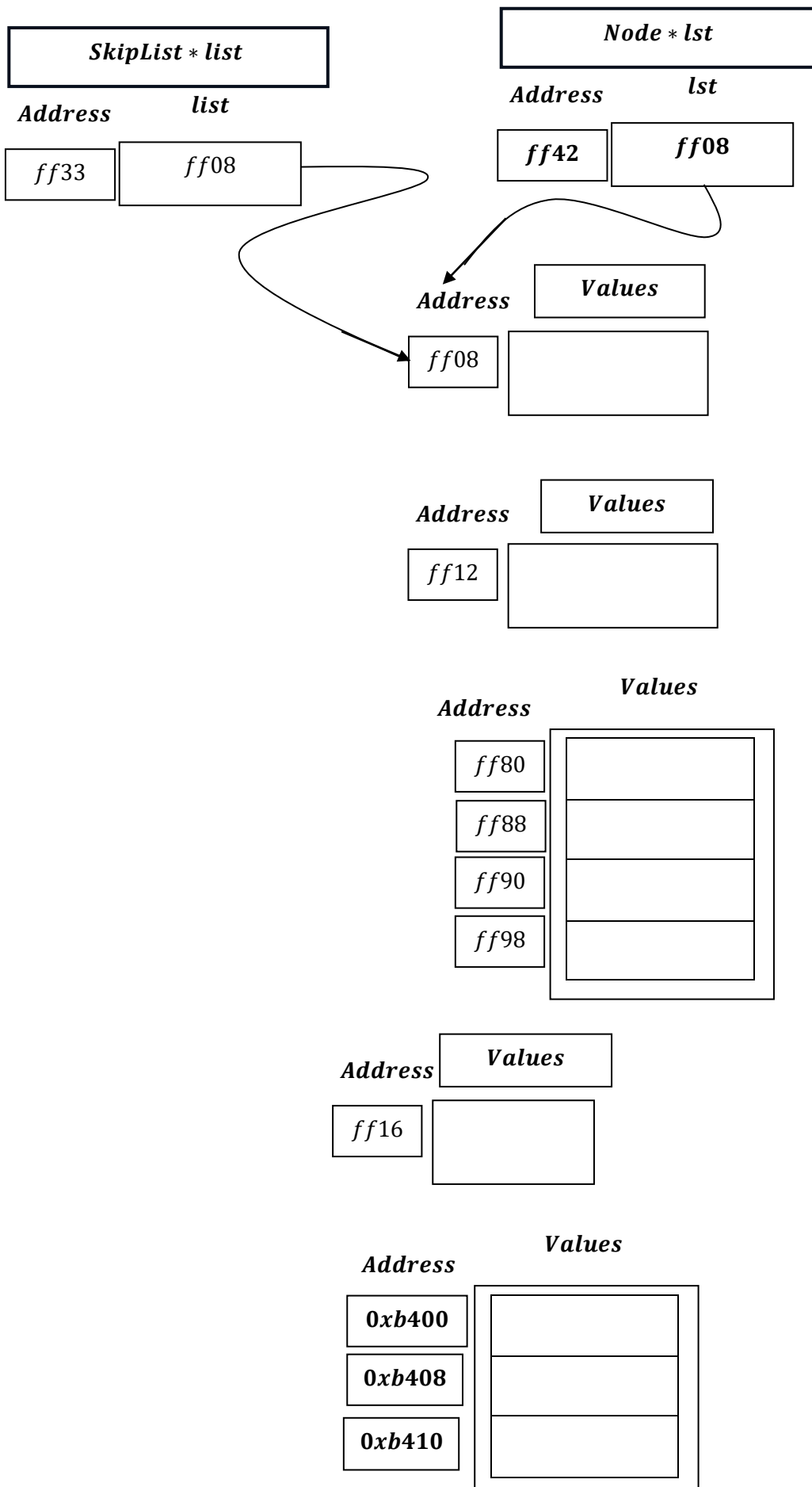


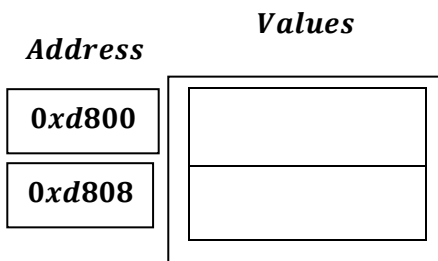
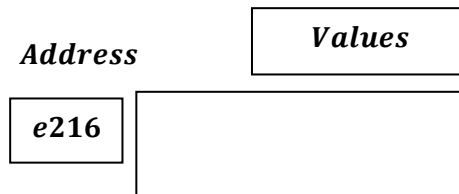
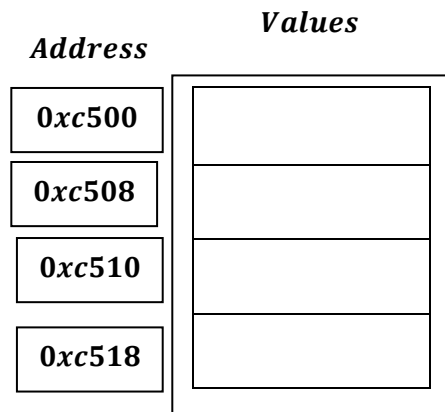
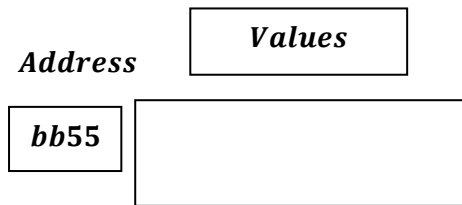
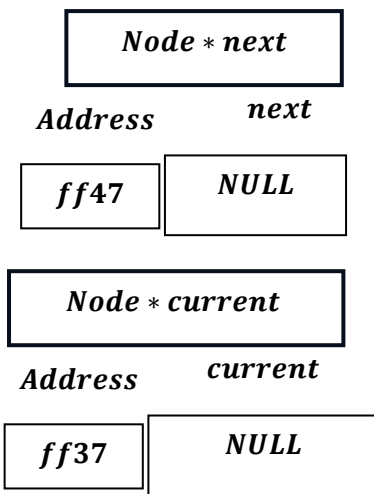
While(current: NULL ≠ NULL) [False, Loop Exits]

Now we execute the final freeing:

```
free(lst);
```

free(lst: ff08)

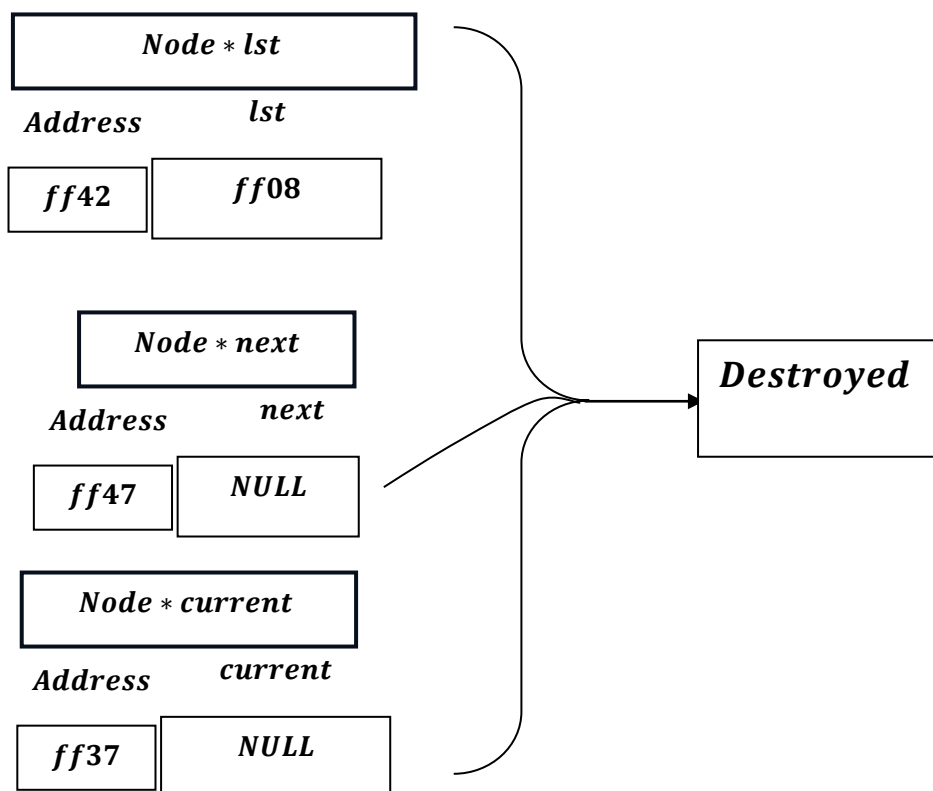




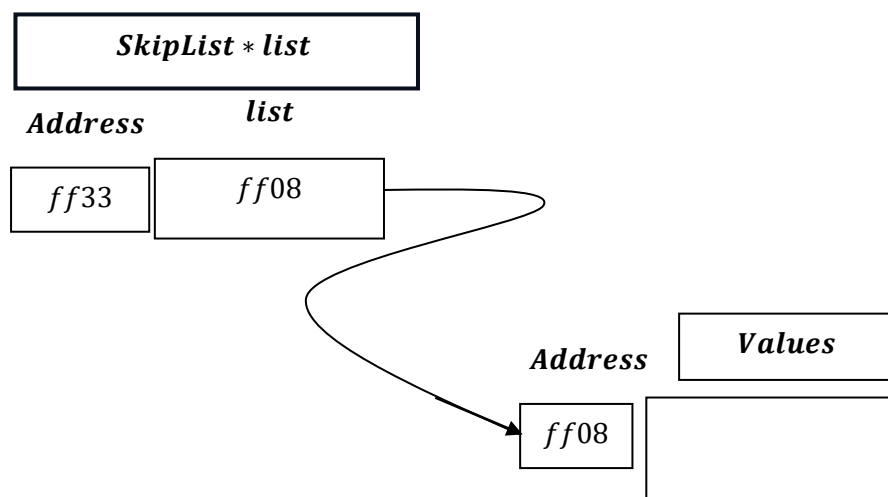
And, then it prints:

```
cout << "Memory successfully freed. Skip List destroyed." << endl;
```

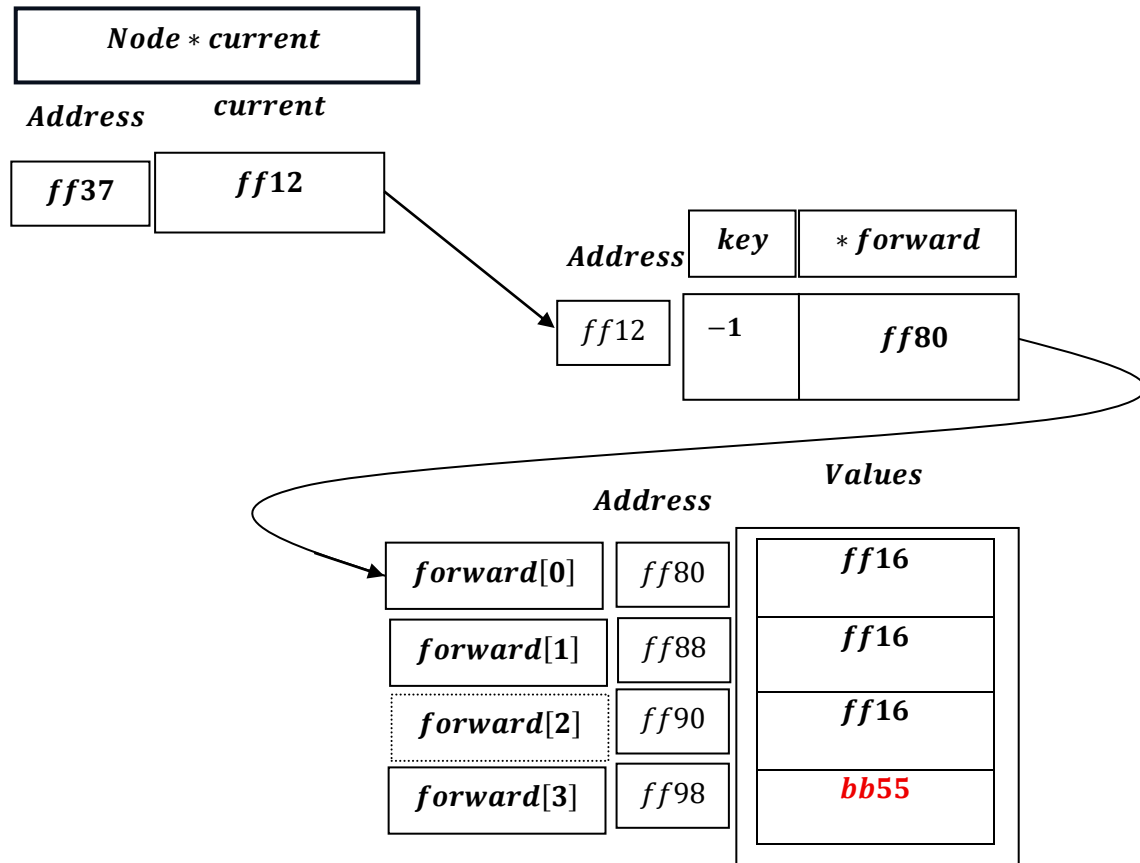
And , as function ends.



And we are left with:



Note how we are freeing the elements:

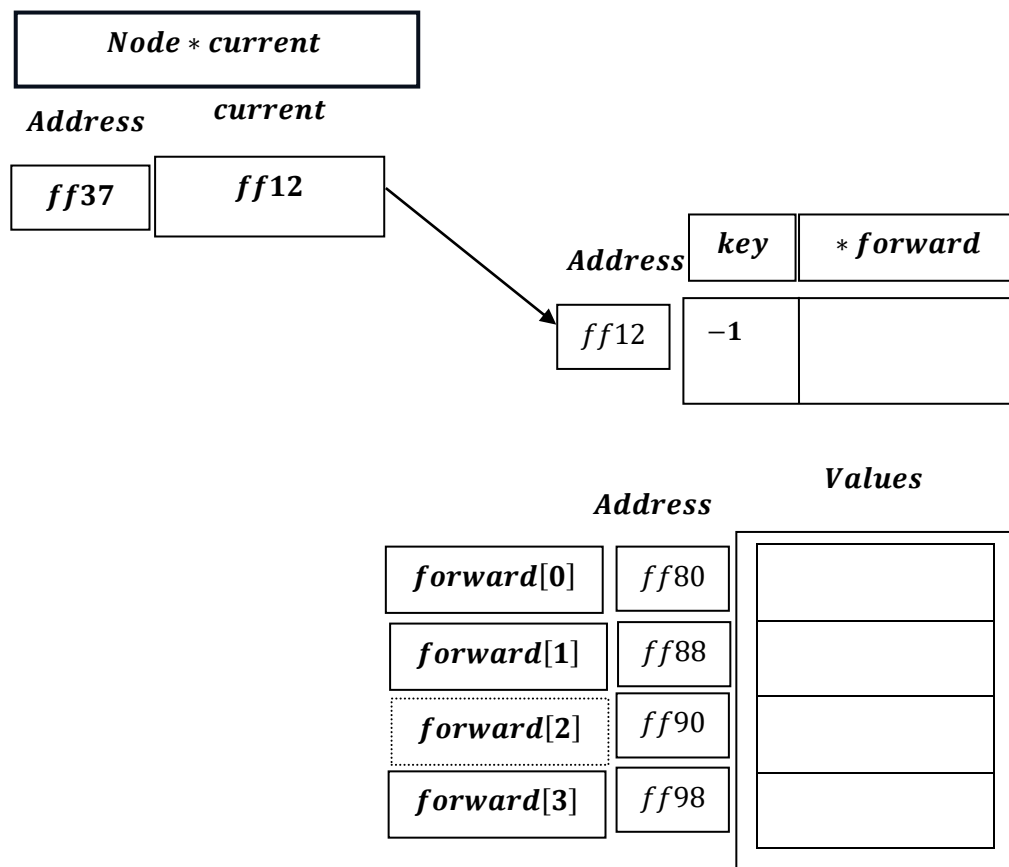


First current → forward is freed

```
free(current->forward);
```

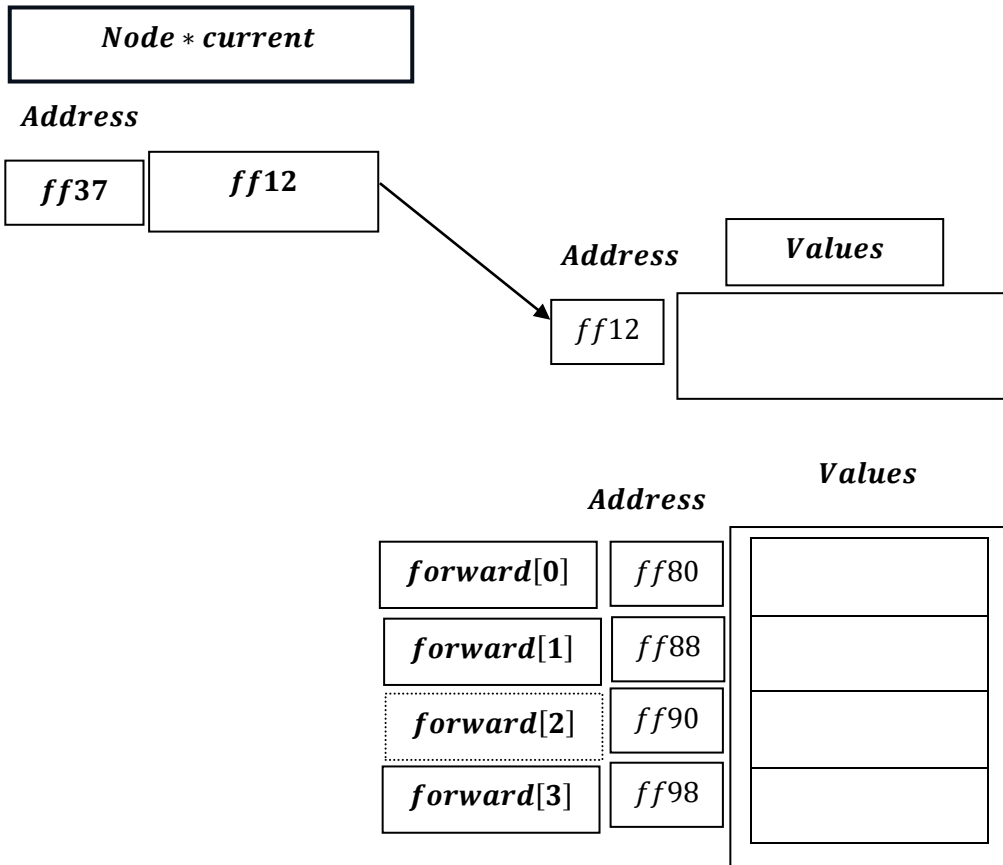
The pointer **current** → **forward** holds the starting address of this chunk, which is **ff80**. And next addresses [**ff88**, **ff90**, **ff98**] automatically gets freed all at once.

forward is dynamically allocated as a single contiguous array. The elements are contiguous, with an **8 – byte** stride between the starting addresses of consecutive elements because each element stores an **8 – byte pointer**. Therefore, **free(current → forward)** deallocates the entire array allocation in one operation.



Next , **current** is freed , this wipes out value of **Key**.

```
free(current);
```



So, there are two way freeing of memory.

Skip List
–Destroy List
–Time Complexity Analysis

Program:

```
void destroyList(struct SkipList *lst) {
    // Start at the header
    struct Node *current = lst->header;
    struct Node *next;

    // We only need to traverse level 0 (the "local train" level)
    // because it is the only level guaranteed to contain EVERY node.
    while (current != NULL) {
        // 1. Save the pointer to the next node
        next = current->forward[0];

        // 2. Free the dynamically allocated array of pointers
        free(current->forward);

        // 3. Free the node itself
        free(current);

        // 4. Move to the next node
        current = next;
    }

    // Finally, free the main Skip List structure
    free(lst);
    cout << "Memory successfully freed. Skip List destroyed." << endl;
}

...
case 5:
    cout << "Destroying the current list..." << endl;
    destroyList(list);
    // CRITICAL: We must give the program a fresh, empty list
    // so it doesn't crash if the user tries to insert again.
    break;
```

Time Complexity Analysis

Unlike the **Insertion**, **Search**, and **Deletion** modules, the **destroyList** function

does not rely on probability, coin flips, or navigating express lanes.

Because its only goal is to completely wipe the data structure from memory, it uses a straightforward, brute – force sweep.

Because of this, the Time Complexity for the Destroy module does not fluctuate. **The Best, Worst, and Average** cases are all exactly **$O(n)$** .

The Traversal Strategy: Level 0 Only

To destroy the list, the algorithm must ensure it does not miss a single node.

- **The Code:** It initializes a pointer at the header and uses a **while (current != NULL)** loop to walk forward. To move to the next node, it strictly uses **next = current → forward[0]**.
- **The Logic:** Level 0 (the ground floor) acts as the "local train". It is the only level mathematically guaranteed to contain every single node in the Skip List. By ignoring the upper – level express lanes entirely and just walking across Level 0, the algorithm guarantees it will visit exactly n nodes (plus the dummy header) one by one.
- **The Complexity:** Because it visits every node exactly once, the traversal takes **$O(n)$** time.

The Deallocation Strategy: The $O(1)$ Two – Step Wipe

At every single node, the program performs a two – step freeing process:

- 1. **$free(current \rightarrow forward)$** : The **$forward$** array holding all the pointers for that node was dynamically allocated as a single, contiguous array in memory. The program does not need a loop to delete **$forward[0]$** , then **$forward[1]$** , etc. It hands the starting address of the chunk to the operating system, which deallocates the entire array in one single **$O(1)$** operation.*
- 2. **$free(current)$** : Once the pointer array is gone, this command wipes out the node structure itself (which holds the **key**), also in **$O(1)$** time.*

Final Calculation

*The algorithm visits n nodes. At each node, it performs the two – step **$O(1)$** deallocation. Finally, it drops out of the loop and executes **$free(lst)$** to destroy the main Skip List container.*

$$n \text{ nodes} \times O(1) \text{ deletion operations} = O(n)$$

*Therefore, no matter how perfectly balanced or catastrophically unbalanced the Skip List is, the time complexity to destroy it is universally and strictly **$O(n)$** .*

Skip List
–Destroy List
–Space Complexity Analysis

Program:

```
void destroyList(struct SkipList *lst) {
    // Start at the header
    struct Node *current = lst->header;
    struct Node *next;

    // We only need to traverse level 0 (the "local train" level)
    // because it is the only level guaranteed to contain EVERY node.
    while (current != NULL) {
        // 1. Save the pointer to the next node
        next = current->forward[0];

        // 2. Free the dynamically allocated array of pointers
        free(current->forward);

        // 3. Free the node itself
        free(current);

        // 4. Move to the next node
        current = next;
    }

    // Finally, free the main Skip List structure
    free(lst);
    cout << "Memory successfully freed. Skip List destroyed." << endl;
}

...
case 5:
    cout << "Destroying the current list..." << endl;
    destroyList(list);
    // CRITICAL: We must give the program a fresh, empty list
    // so it doesn't crash if the user tries to insert again.
    break;
```

Space Complexity Analysis

1. Input Space Complexity

Just like the other modules, the input space evaluates the memory of the parameters passed into the function and the overarching data structure it acts upon.

A. Parametric Input Space: $O(1)$

- **The Parameter:** The function signature is `void destroyList(struct SkipList * lst)`. It takes exactly one argument: a memory pointer addressing the main Skip List container.
- **The Math:** This single pointer requires a fixed, microscopic amount of memory (typically 8 bytes) to hold the address.
- **The Conclusion:** Because the size of this parameter never scales up – whether you are destroying a list of 5 nodes or 5 million nodes – the parametric input space is strictly $O(1)$.

B. Full Data Holistic Input Space: $O(n)$

- **The Structure:** The function operates on the fully built Skip List currently sitting in the computer's memory.
- **The Math:** As proven in the general Space Complexity analysis, a Skip List holds n elements using an expected average of $2n$ pointers.
- **The Conclusion:** Because the physical memory footprint of the data structure being destroyed scales directly with the number of elements inside it, the full data holistic input space is $O(n)$.

2. Auxiliary Space Complexity: $O(1)$

Auxiliary space strictly measures the extra, temporary memory the algorithm asks the system for in order to execute its internal logic.

- **The Variables:** *Inside the `destroyList` function, the algorithm declares exactly two local pointer variables:*
`struct Node * current` and `struct Node * next`.
- **No Arrays or Recursion:** *Unlike the Insert and Delete modules, the Destroy module does not need to remember a complex vertical path, so it does not allocate an `update[ ]array`. It simply uses the `next` pointer to remember the address of the upcoming node on **Level 0** before it wipes the `current` node out of existence.*
- **The Conclusion:** *Because the function allocates exactly zero arrays, makes zero recursive calls, and only asks the system for enough memory to hold two temporary tracking pointers, the auxiliary space complexity is a strict, absolute **$O(1)$** .*

Final Total Space Complexity

To find the total space complexity of the module, we add the dominant input space to the auxiliary space:

- **Total Space Complexity** =

Either:

$$\text{Full Data Holistic Space } (O(n)) + \text{Auxiliary Space } (O(1)) = O(n)$$

Or:

$$\begin{aligned} &\text{Input Space}(\text{Full Data Holistic Space } (O(n)) \\ &\quad + \text{Parametric Input Space } (O(1))) + \text{Auxiliary Space } (O(1)) \\ &= O(n) \end{aligned}$$

Algorithm: SkipListDestroy(LST)

1. [Initialize Pointers]

SET CURRENT := HEADER[LST].

Declare a node pointer NEXT.

2. [Traverse Level 0 to Free Nodes]

Repeat while CURRENT != NULL:

// Save the pointer to the next node

SET NEXT := FORWARD[0][CURRENT].

// Free the dynamically allocated array of pointers

CALL free(FORWARD[CURRENT]).

// Free the node itself

CALL free(CURRENT).

// Move to the next node

SET CURRENT := NEXT.

[End of while Loop]

3. [Free the Main Structure]

CALL free(LST).

4. [Success Message]

Write: "Memory successfully freed. Skip List destroyed."

5. [Exit]

END OF ALGORITHM.

Worst Case Scenario – The All Tails Scenario (The Flat List)

AND

The All Heads Scenario (The Traffic Jam)

1. The "All Tails" Scenario (The Flat List)

In this scenario, every single time a node was inserted, the `randomLevel()` coin flip immediately resulted in "Tails" (1). No node was ever promoted.

*The Header node is built to reach **Level 3**, but because no actual data nodes grew taller than **Level 0**, all the upper – level express lanes just point to empty space (NULL).*

```
[ SKIP LIST : ALL TAILS (MAX_LEVEL = 3) ]
```

```
Level 3: [Header] -----> NULL
Level 2: [Header] -----> NULL
Level 1: [Header] -----> NULL
Level 0: [Header] ---> [ 20 ] ---> [ 25 ] ---> [ 30 ] ---> NULL
```

Why it causes $O(n)$ time:

If we search for 30, the pointer drops instantly from Level 3 down to Level 0. Once on the ground floor, it has no choice but to physically step through 20, then 25, then 30. It operates exactly like a standard, singly linked list.

*SkipList * list*

Address *list*

ff33 *ff08*

Address *level* ** header*

ff08 **3** *ff12*

Address *key* ** forward*

ff12 -1 *ff80*

Address

Values

<i>forward[0]</i>	<i>ff80</i>	<i>ff16</i>
<i>forward[1]</i>	<i>ff88</i>	<i>NULL</i>
<i>forward[2]</i>	<i>ff90</i>	<i>NULL</i>
<i>forward[3]</i>	<i>ff98</i>	<i>NULL</i>

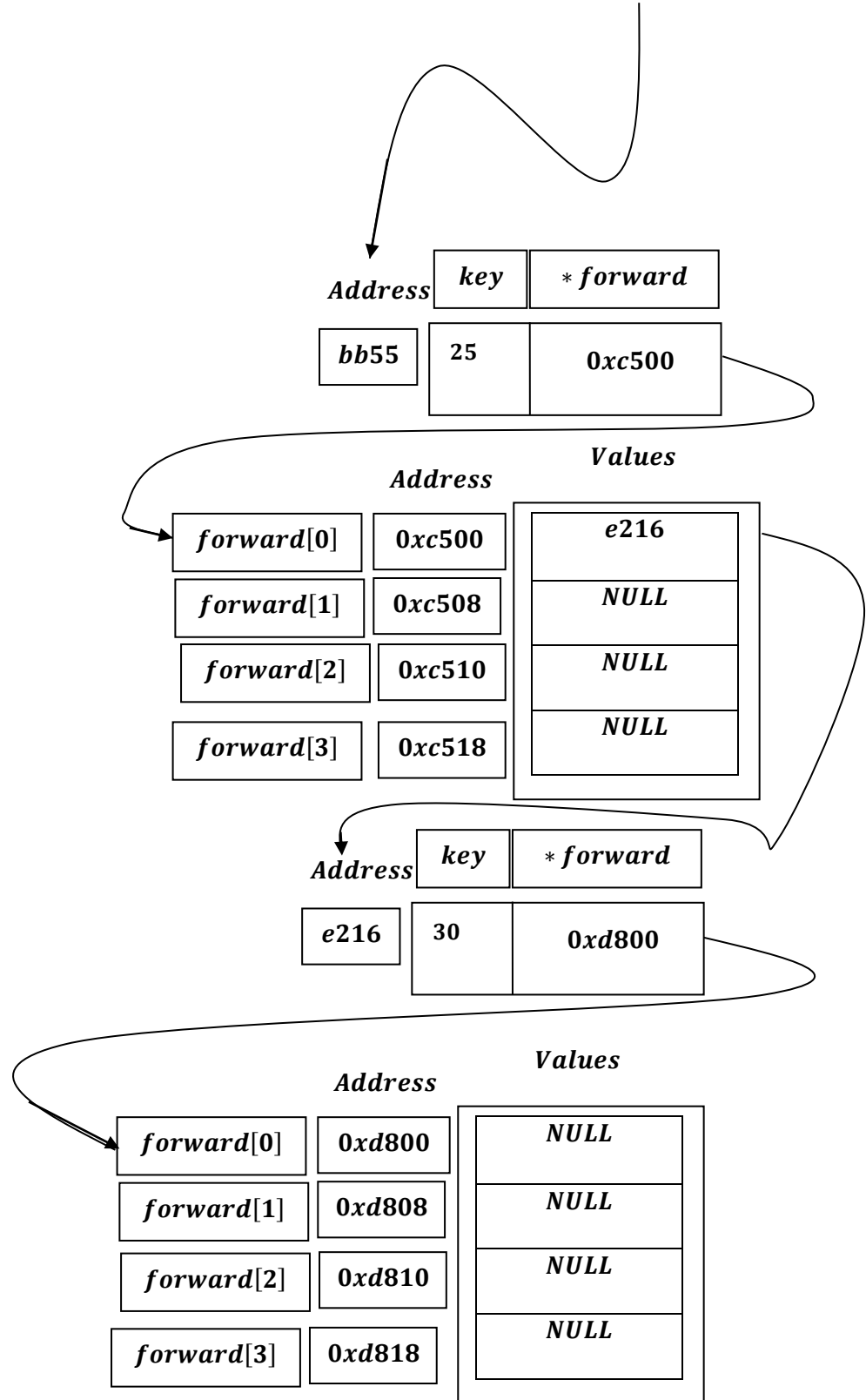
Address *key* ** forward*

ff16 20 *0xb400*

Address

Values

<i>forward[0]</i>	<i>0xb400</i>	<i>bb55</i>
<i>forward[1]</i>	<i>0xb408</i>	<i>NULL</i>
<i>forward[2]</i>	<i>0xb410</i>	<i>NULL</i>
<i>forward[3]</i>	<i>0xb418</i>	<i>NULL</i>



The structure of a singly linked list.

2. The "All Heads" Scenario (The Traffic Jam)

In this scenario, the random number generator suffered a massive anomaly.

Every single time a node was inserted, it flipped Heads(0) repeatedly, growing to the absolute MAX_LEVEL cap.

Every single node exists on every single floor.

```
[ SKIP LIST : ALL HEADS (MAX_LEVEL = 3) ]
```

```
Level 3: [Header] ---> [ 20 ] ---> [ 25 ] ---> [ 30 ] ---> NULL
Level 2: [Header] ---> [ 20 ] ---> [ 25 ] ---> [ 30 ] ---> NULL
Level 1: [Header] ---> [ 20 ] ---> [ 25 ] ---> [ 30 ] ---> NULL
Level 0: [Header] ---> [ 20 ] ---> [ 25 ] ---> [ 30 ] ---> NULL
```

- **The Height of the Nodes:** The "All Heads" scenario as a situation where the random number generator consistently flips heads, "promoting every single node to the absolute maximum level (e.g., MAX_LEVEL = 3)".

And say, randomLevel or rLevel is also 3, in All Heads Scenario.

- **The Clogged Express Lane:** Because of this promotion, "every single node in the list has a pointer at level 3".
- **The Horizontal Traversal:** Because every node has a Level 3 pointer, the header's **forward[3]** pointer just points to Node 1. Node 1's **forward[3]** pointer points to Node 2. Node 2's **forward[3]** points to Node 3.

Similarly,

- **For Level 2 (*forward[2]*):** *Because every node has a Level 2 pointer, the header's *forward[2]* pointer just points to Node 1. Node 1's *forward[2]* pointer points to Node 2. Node 2's *forward[2]* points to Node 3.*
- **For Level 1 (*forward[1]*):** *Because every node has a Level 1 pointer, the header's *forward[1]* pointer just points to Node 1. Node 1's *forward[1]* pointer points to Node 2. Node 2's *forward[1]* points to Node 3.*
- **For Level 0 (*forward[0]*):** *Because every node has a Level 0 pointer, the header's *forward[0]* pointer just points to Node 1. Node 1's *forward[0]* pointer points to Node 2. Node 2's *forward[0]* points to Node 3.*

This is the ultimate proof of why the "All Heads" scenario is a structural disaster.

*SkipList * list*

Address *list*

ff33 *ff08*

Address *level* ** header*

ff08 3 *ff12*

Address *key* ** forward*

ff12 -1 *ff80*

Address *Values*

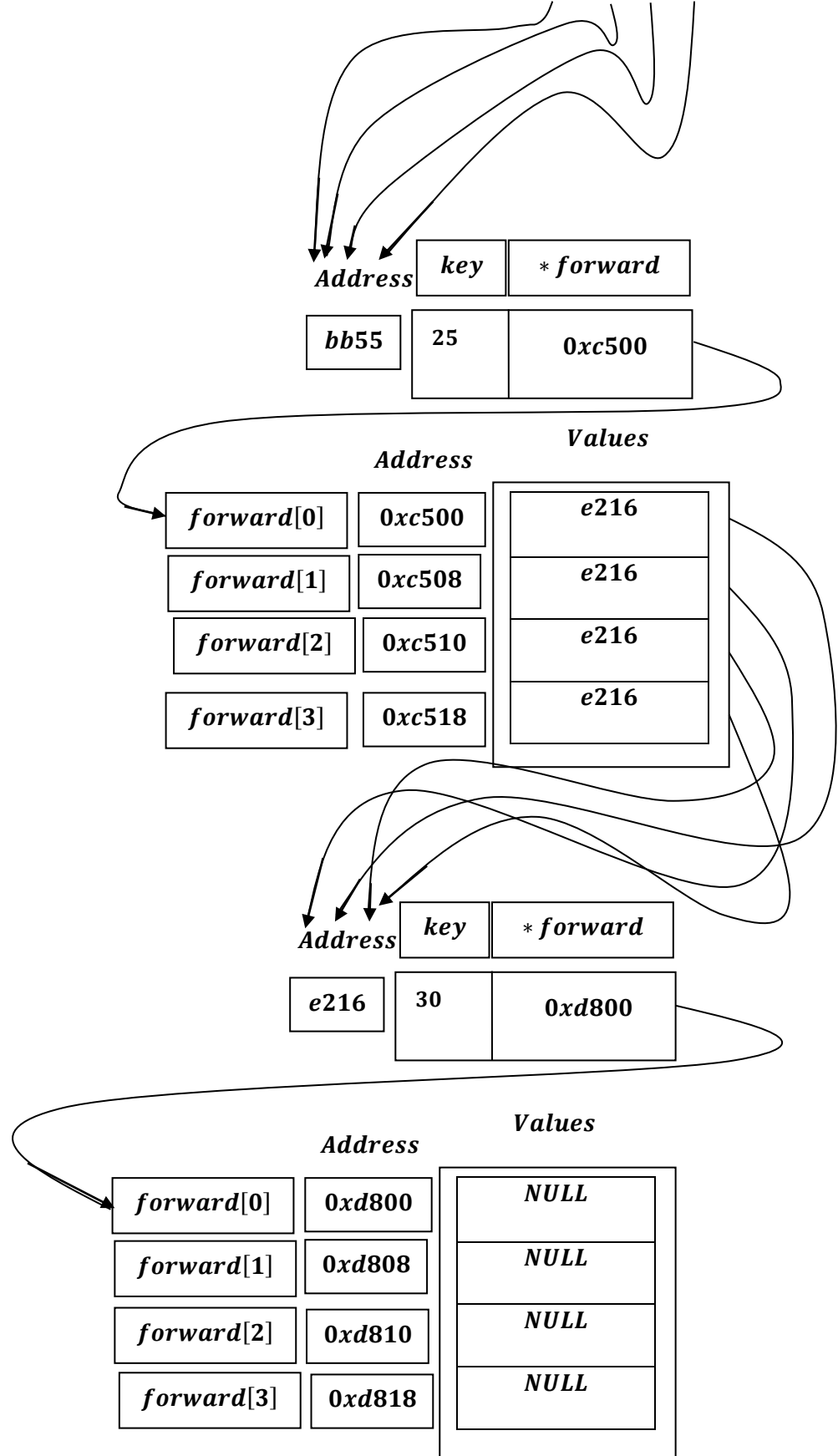
<i>forward</i>	<i>Address</i>	<i>Values</i>
<i>forward</i> [0]	<i>ff80</i>	<i>ff16</i>
<i>forward</i> [1]	<i>ff88</i>	<i>ff16</i>
<i>forward</i> [2]	<i>ff90</i>	<i>ff16</i>
<i>forward</i> [3]	<i>ff98</i>	<i>ff16</i>

Address *key* ** forward*

ff16 20 *0xb400*

Address *Values*

<i>forward</i>	<i>Address</i>	<i>Values</i>
<i>forward</i> [0]	<i>0xb400</i>	<i>bb55</i>
<i>forward</i> [1]	<i>0xb408</i>	<i>bb55</i>
<i>forward</i> [2]	<i>0xb410</i>	<i>bb55</i>
<i>forward</i> [3]	<i>0xb410</i>	<i>bb55</i>



Why it causes $O(n)$ time:

Level 3 (The Traffic Jam): The algorithm starts at the top and is forced to evaluate 20, then 25, then 30 horizontally. It does a full $O(n)$ amount of work just crossing the top floor.

Dropping Down (Levels 2, 1, and 0): Once the pointer finally reaches Node 30 at the end of the list on Level 3, the for loop drops down to Level 2. Because the pointer is already sitting exactly where it needs to be, the lower – level horizontal loops terminate instantly with zero steps taken. This repeats for Level 1 and Level 0.

Level 2, Level 1, and Level 0 are completely skipped horizontally because all the heavy lifting was already accidentally done on the top floor.

- **Top Floor (Level 3):** The pointer starts at the Header, moves right to **Node 1 (20)**, moves right to **Node 2 (25)**, and finally moves right to **Node 3 (30)**. It has now reached the end of the list on **Level 3**. The **while** loop terminates.
- **Dropping to Level 2:** The outer **for** loop executes **$i--$** , dropping the floor down to **Level 2**.
 - **The Catch:** The **current** pointer does not reset! It is already physically sitting on Node 30.
 - When the inner while loop asks, Should I move right?, it checks Node 30's **forward[2]** pointer. Because **Node 30** is the last node, that pointer is **NULL**.
 - The condition fails instantly, and zero horizontal steps are taken.

- **Dropping to Level 1 and Level 0:** The exact same thing happens.

The floor drops, but because the pointer is already parked

at the very end of the list(NULL), the while loop condition instantly fails on every lower floor.

How?

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

*Here is exactly how that code executes as the **for** loop drops down:*

1. **Level 2 (i = 2):** The program checks the condition **current → forward[2] ≠ NULL**. Because Node 30 is the last node, **current → forward[2] is NULL**. The condition fails instantly, the **while** loop terminates with zero steps taken, and the program simply records **update[2] = current** (the pointer address of Level3 where Node 30 is sitting, here **e216**).
2. **Level 1 (i = 1):** The for loop drops down, but current is still Node 30. It checks **current → forward[1] ≠ NULL**. But **current → forward[1] is NULL**. The condition fails instantly, the while loop is skipped, and it records **update[1] = current** (the pointer address of Level3 where Node 30 is sitting, here **e216**).
3. **Level 0 (i = 0):** The exact same thing happens. It checks **current → forward[0] ≠ NULL**. It is NULL. The condition fails instantly, the while loop is skipped, and it records **update[0] = current** (Node 30).

Insert Element – When All Heads Scenario

Now Come to Insert Element:

```
current = current->forward[0];
```

current = current → forward[0] = NULL.

```
int rlevel = randomLevel();
```

rlevel = randomLevel() = 3[MAX_LEVEL]

*Runs Theoretically $O(\log n)$ times
But in practical scenario, it is
fixed constant = $O(1)$ as it cannot
go above MAX_LEVEL which is a fixed
constant.*

```
if (rlevel > lst->level) {  
    for (int i = lst->level + 1; i <= rlevel; i++) {  
        update[i] = lst->header;  
    }  
    lst->level = rlevel;  
}
```

Here rlevel = 3 > lst → level = 3, completely skipped.

```
struct Node *n = createNode(key, rlevel);  
for (int i = 0; i <= rlevel; i++) {  
    n->forward[i] = update[i]->forward[i];  
    update[i]->forward[i] = n;  
}  
cout << "Successfully inserted key " << key << endl;  
}
```

→ Now, it will create a new Node.

for $i = 0; i \leq rlevel() = 3$:

$n \rightarrow forward[0] = update[0]$ i.e. e216 → forward[0] = NULL

update[0] i.e. e216 → forward[0] = n i.e. New Node's Address

$i = 0 + 1 = 1$

for i = 1; i ≤ rlevel() = 3:

n → forward[1] = update[1] i. e. e216 → forward[1] = NULL

update[1] i. e. e216 → forward[1] = n i. e. New Node's Address

i = 1 + 1 = 2

for i = 2; i ≤ rlevel() = 3:

n → forward[2] = update[2] i. e. e216 → forward[2] = NULL

update[2] i. e. e216 → forward[2] = n i. e. New Node's Address

i = 2 + 1 = 3

for i = 3; i ≤ rlevel() = 3:

n → forward[3] = update[3] i. e. e216 → forward[3] = NULL

update[3] i. e. e216 → forward[3] = n i. e. New Node's Address

i = 3 + 1 = 4

for i = 4; i ≤ rlevel() = 4 [Loop Exits]

Hence , new Node is adjusted at very end of the list.

In practical Scenario: MAX_LEVEL = RLEVEL = O(1)

as MAX_LEVEL is constant.

Therefore , for loop ran from 0 to some constant `c` resulting O(1).

In Theoretical Scenario: MAX_LEVEL = RLEVEL = O(log n)

Therefore , for loop ran from 0 to log n resulting O(log n).

At both the scenarios: $T_{Search} + T_{coinflip} + T_{insert\&Rewire} =$

= Search: O(n) + CoinFlip: O(log n) or O(1) + Insert: O(log n) or O(1) = O(n)

Delete Element – When All Heads Scenario

```
current = current->forward[0];
```

current = current → forward[0] = NULL.

```
if (current != NULL && current->key == key) {
    // Rewire the pointers to bypass the deleted node
    for (int i = 0; i <= lst->level; i++) {
        if (update[i]->forward[i] != current) {
            break; // If a level doesn't point to current, higher levels
won't
            // either
        }
        update[i]->forward[i] = current->forward[i];
    }

    // Free memory using free()
    free(current->forward);
    free(current);

    // Reduce the maximum level of the list if layers become empty
    while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
        lst->level--;
    }
    cout << "Successfully deleted key " << key << endl;
}
```

As Current is NULL ,

- *whole rewiring is skipped.*
- *The freeing will get skipped.*
- *while loop i.e.Reducing Maximum Level also gets skipped*

```
else {
    cout << "Key " << key << " not found. Cannot delete." << endl;
}
}
```

And we run: ``Key is not found``, hence:

Hence : Search: $O(n) + O(1) = O(n)$

Same thing for All Tails Scenario:

Search:

As, at first current will be pointing to Header Node : ff12 and

as , ff12 $\rightarrow$ forward[3] = NULL , ff12 $\rightarrow$ forward[2] = NULL ,

ff12 $\rightarrow$ forward[1] = NULL.

Therefore:

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

update[1] = ff12

update[2] = ff12

update[3] = ff12

When at Level = 0 :

current $\rightarrow$ e216

update[0] = e216

After Search , Time Complexity = $O(n)$

Insert Element – When All Tails Scenario

```
current = current->forward[0];
```

Current = Current → forward[0] = NULL

```
if (current == NULL || current->key != key) {
    int rlevel = randomLevel();

    // If the random level is greater than the list's current max
    level,
    // initialize the update array for the new levels to point to
    the header.
    if (rlevel > lst->level) {
        for (int i = lst->level + 1; i <= rlevel; i++) {
            update[i] = lst->header;
        }
        lst->level = rlevel;
    }
}
```

As Current = NULL(True):

rlevel = randomLevel(), Tails therefore return 0. $\left[\begin{array}{l} \text{At first} \\ \text{coin flip,} \\ \text{hence strict} \\ 0(1). \end{array} \right]$

if(rlevel = 0 > lst → level = 3)[True]:

for $\left(i = \text{lst} \rightarrow \text{level} + 1[3 + 1 = 4]; i \leq \text{rlevel} = 0 \left[\begin{array}{l} \text{Loop} \\ \text{Condition} \\ \text{Fails} \\ \text{Hence Exit} \end{array} \right] \right)$

lst → level = rlevel = 0.

```

struct Node *n = createNode(key, rlevel);
for (int i = 0; i <= rlevel; i++) {
    n->forward[i] = update[i]->forward[i];
    update[i]->forward[i] = n;
}
cout << "Successfully inserted key " << key << endl;
}

```

→ Now , it will create a new Node.

for $i = 0; i \leq rlevel() = 0$:

$n \rightarrow forward[0] = update[0]$ i. e. $e216 \rightarrow forward[0] = NULL$

$update[0]$ i. e. $e216 \rightarrow forward[0] = n$ i. e. New Node's Address

$i = 0 + 1 = 1$

for $i = 0; i \leq rlevel() = 0$ [Loop Exits]

Hence , new Node is adjusted at very end of the list.

*As , loop strictly runs only Once , hence Time Complexity of Insertion and Rewiring is strictly constant: **$O(1)$** .*

$$\begin{aligned}
 &\therefore T_{Search} + T_{coinflip} + T_{insert\&Rewire} = \\
 &= \text{Search: } O(n) + \text{CoinFlip: } O(1) + \text{Insert: } O(1) = O(n)
 \end{aligned}$$

Delete Element – When All Tails Scenario

```
current = current->forward[0];
```

current = current → forward[0] = NULL.

```
if (current != NULL && current->key == key) {
    // Rewire the pointers to bypass the deleted node
    for (int i = 0; i <= lst->level; i++) {
        if (update[i]->forward[i] != current) {
            break; // If a level doesn't point to current, higher levels
won't
                // either
        }
        update[i]->forward[i] = current->forward[i];
    }

    // Free memory using free()
    free(current->forward);
    free(current);

    // Reduce the maximum level of the list if layers become empty
    while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
        lst->level--;
    }
    cout << "Successfully deleted key " << key << endl;
}
```

As Current is NULL ,

- *whole rewiring is skipped.*
- *The freeing will get skipped.*
- *while loop i.e.Reducing Maximum Level also gets skipped*

```
else {
    cout << "Key " << key << " not found. Cannot delete." << endl;
}
}
```

And we run: ``Key is not found``, hence:

Hence : Search: $O(n) + O(1) = O(n)$

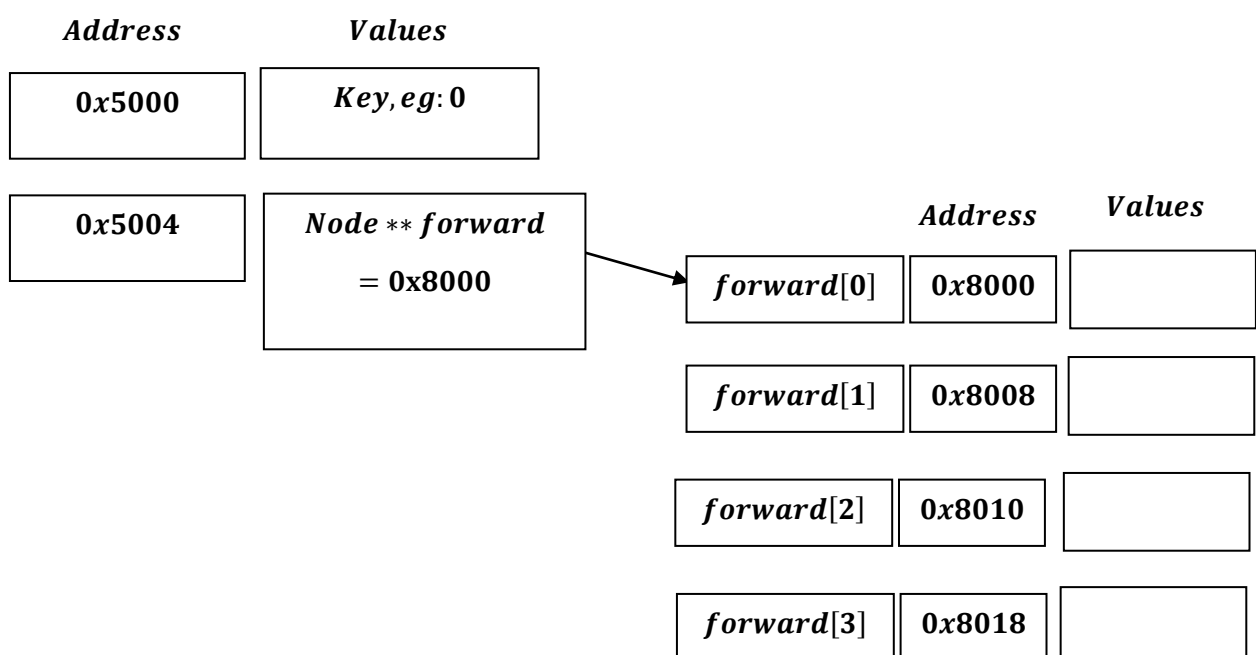
Skip List – Memory Referencing Overview

```
struct Node {  
    int key;  
    // Array of pointers to next nodes, one for each level  
    struct Node **forward;  
};  
struct Node *n = createNode(key, rlevel);
```

Formation of Struct Node – An Memory Layout Example

```
n->key = key;  
  
    // Allocate memory for the forward pointer array (level + 1  
elements)  
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level  
+ 1));
```

Struct * Node



→ $baseaddress + offset = 0x5000 + 0 [bytes] = \text{Where key address is stored.}$

→ $baseaddress + offset = 0x5000 + 4 [bytes] = \text{** forward, where}$
 $forward[0]'s \text{ address is stored.}$

The Compiler's "Secret" Calculation

When the compiler sees `node->key = 0;`, it thinks:

- "Get the base memory address stored in the pointer node." (e.g., `0x5000`)
- "Look up the definition of the Node struct. The key member is at the beginning, so its offset is 0."
- "Calculate the final address: $base\ address + offset \Rightarrow 0x5000 + 0 \Rightarrow 0x5000$."
- "Write the value `0` to the memory location starting at `0x5000`."

When the compiler sees an assignment to the pointer itself,

such as `node->forward = 0x8000;` (where `0x8000` is the memory address of the newly allocated dynamic array), it uses that exact same logical sequence!

- Get the base memory address stored in the pointer `node`." (e.g., `0x5000`)
- Look up the definition of the Node struct. The forward pointer comes directly after the integer key variable (which is 4 bytes long). So, the offset for forward is 4."
- "Calculate the final address:

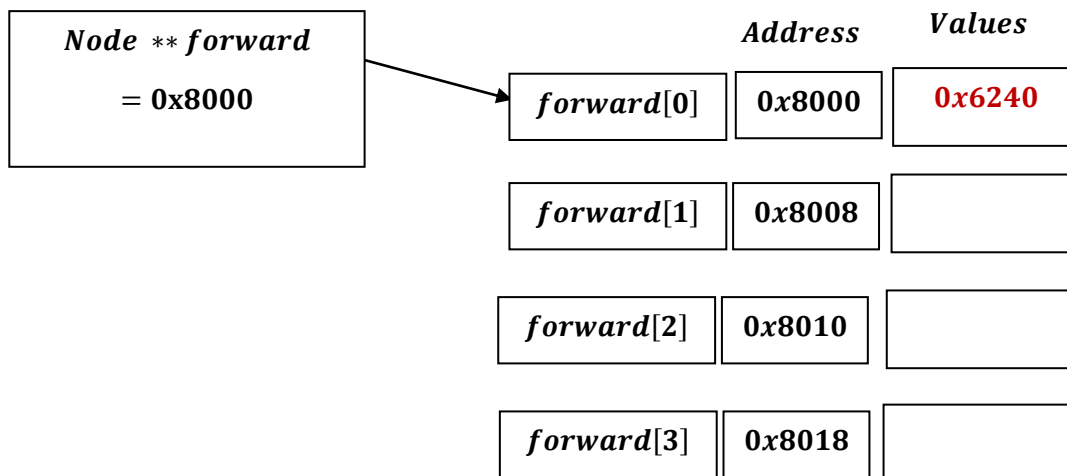
$base\ address + offset \Rightarrow 0x5000 + 4 \Rightarrow 0x5004$."

"Write the value `0x8000` (the 8-byte pointer address) to the memory location starting at `0x5004`."

The "Double Jump" (When we add the brackets [])

It is important to note that writing to `node->forward` is just setting up the map. If the compiler sees `node->forward[0] = 0x6240;`, it has to do a two-step operation:

1. Step 1 (The Struct Offset): It does the exact calculation above ($0x5000 + 4 = 0x5004$) to read the value stored there, which tells it the array lives at `0x8000`.
2. Step 2 (The Array Offset): It takes that new base address (`0x8000`), looks at the index `[0]`, calculates the array offset ($0 * 8 \text{ bytes} = 0$), and finally writes the target address (`0x6240`) into `0x8000`.



Now, as :

```
struct Node **forward;
```

And ,when we write:

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level + 1));
```

This means:

if $level = 3$, then $level + 1 = 4$ and Node pointer variable of 8 bytes in 64 bit architecture.

Then it translates to:

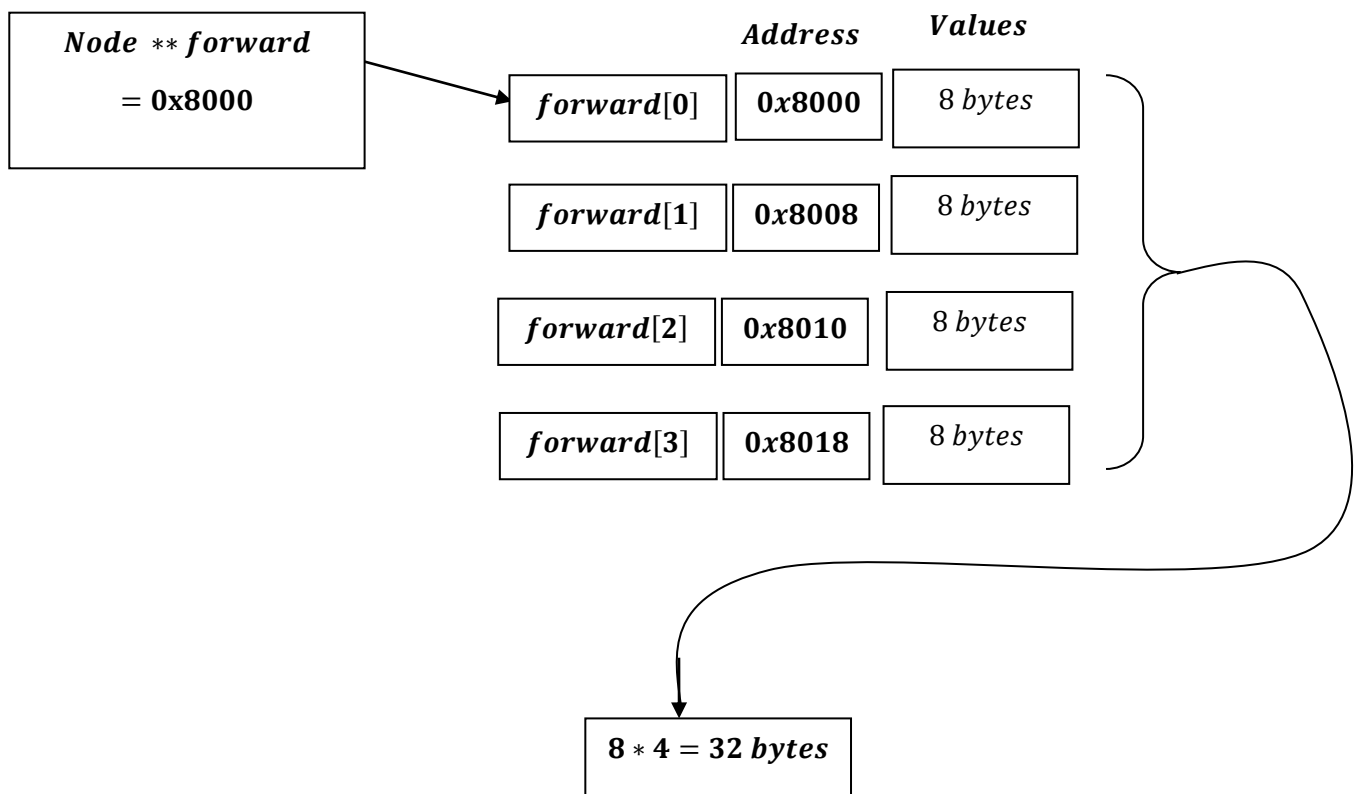
*$n \rightarrow forward = (struct Node **)malloc(size\ of(8\ bytes) * (4));$*

We are telling the compiler: "Take these raw 32 bytes (8×4) that malloc just created, and label them as a block that will hold pointers."

*As we are assigning, $n \rightarrow forward = (struct Node **)malloc(size\ of(8\ bytes) * (4));$*

hence, $n \rightarrow forward$ will hold the first address of the memory block i.e.

$forward[0]$.



Now when we, write: $n \rightarrow forward[0]$, compiler translates it into:

$(n \rightarrow forward + 0)$.*

Similarly:

$n \rightarrow \text{forward}[1]$, compiler translates it into : $*(n \rightarrow \text{forward} + 1)$

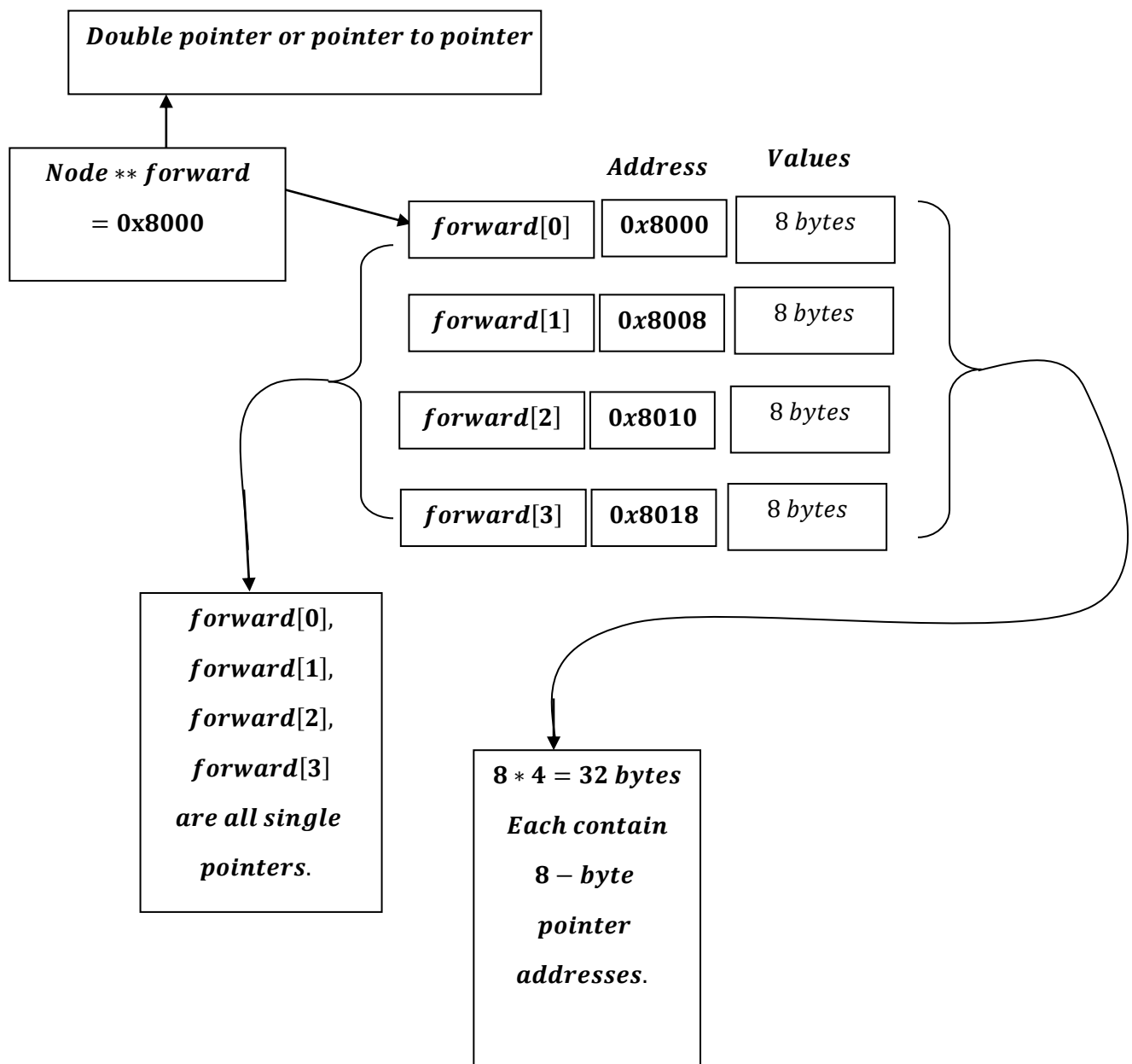
$n \rightarrow \text{forward}[2]$, compiler translates it into : $*(n \rightarrow \text{forward} + 2)$

$n \rightarrow \text{forward}[3]$, compiler translates it into : $*(n \rightarrow \text{forward} + 3)$

When the compiler evaluates the $+ 0$ inside $(n \rightarrow \text{forward} + 0)$:

→ It asks: "**What type of data does this base address point to?**"

- Because forward is a $\text{Node} **$ (pointer to a pointer), the compiler knows it points to an 8 – byte pointer.



$$\text{Formula} = \text{Base Address} + (\text{Index} \times \text{Size in Bytes})$$

Note:

$n \rightarrow \text{forward}: 0x8000$

$(n \rightarrow \text{forward} + 0)$ translates as: $0x8000 + 0 \times 8 \text{ bytes} := 0x8000$, where value(8 bytes address) at $n \rightarrow \text{forward}[0]$ is stored. $[0_{10}(\text{Decimal}) = 0_{16}(\text{Hex})]$

$(n \rightarrow \text{forward} + 1)$ translates as: $0x8000 + 1 \times 8 \text{ bytes} := 0x8008$, where value(8 bytes address) at $n \rightarrow \text{forward}[1]$ is stored. $[8_{10}(\text{Decimal}) = 8_{16}(\text{Hex})]$

$(n \rightarrow \text{forward} + 2)$ translates as: $0x8000 + 2 \times 8 \text{ bytes} := 0x8010$, where value(8 bytes address) at $n \rightarrow \text{forward}[2]$ is stored. $[16_{10}(\text{Decimal}) = 10_{16}(\text{Hex})]$

$(n \rightarrow \text{forward} + 3)$ translates as: $0x8000 + 3 \times 8 \text{ bytes} := 0x8018$, where value(8 bytes address) at $n \rightarrow \text{forward}[3]$ is stored. $[24_{10}(\text{Decimal}) = 18_{16}(\text{Hex})]$

```
struct SkipList {  
    int level;           // Current maximum level in the list  
    struct Node *header; // Pointer to the header node  
};
```

And, we have :

```
lst->header = createNode(-1, MAX_LEVEL);
```

And After call, we have: –

```
struct Node *createNode(int key, int level) {  
    // Allocate memory for the node  
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));  
    n->key = key;
```

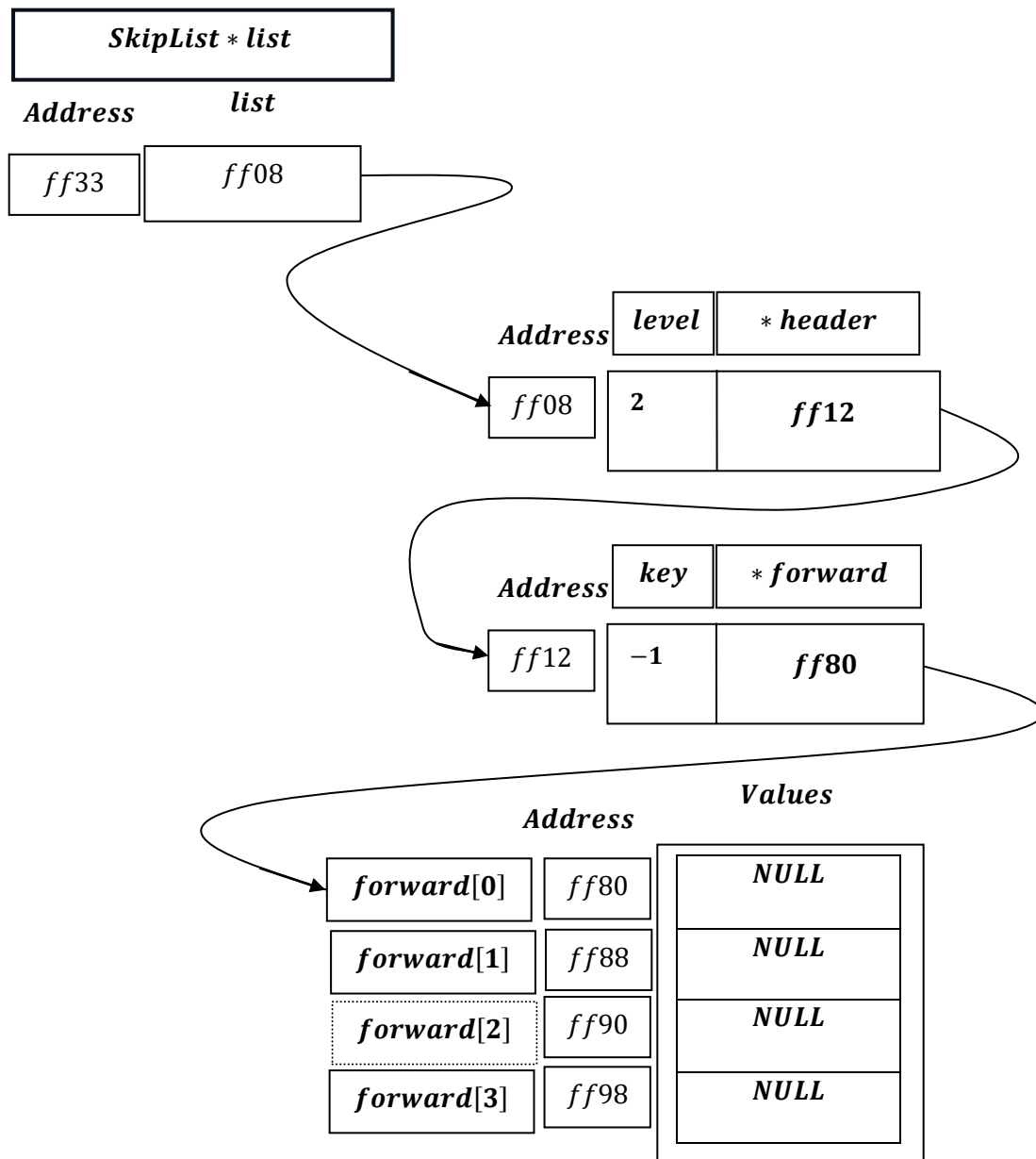
```

    // Allocate memory for the forward pointer array (level + 1
elements)
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}

```

Therefore, we have seen diagrammatically:



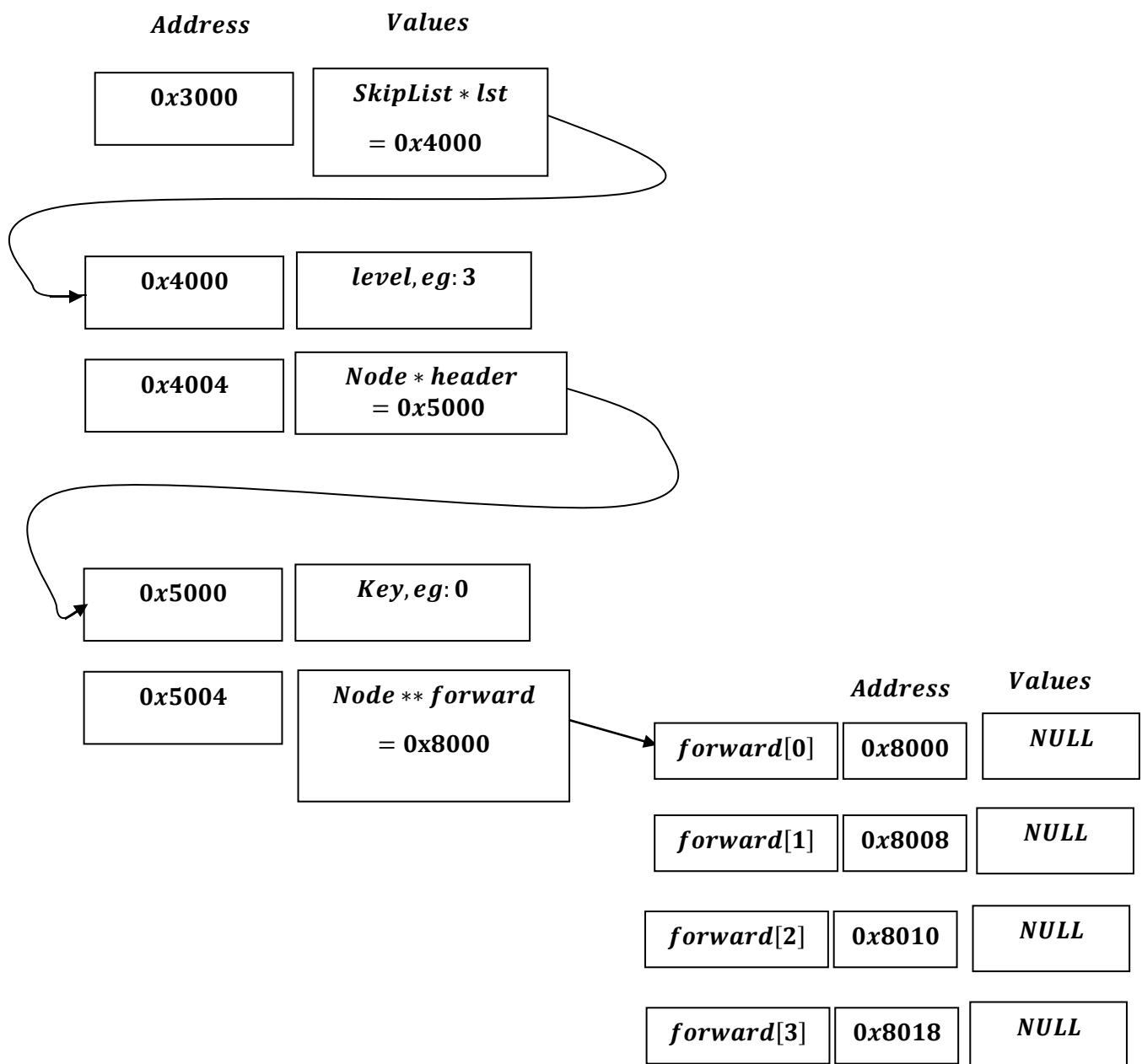
But its actually looks like *[Lets change the addresses]*:

```
struct SkipList {  
    int level;           // Current maximum level in the list  
    struct Node *header; // Pointer to the header node  
};
```

*So, compiler will first allocate : $Base(0x4000) + offset(0) = 0x4000$
for member integer variable **Level** and as it is integer ,
it will allocate 4 bytes for it.*

*And then it will allocate for $Node * header$ at address : $Base(0x4000) + offset(4)$
 $= 0x4004$ for member pointer variable : header.*

Therefore ,it will look like :



***SkipList's* pointer variable ** lst* will keep on pointing at 0x4000** ***Fixed
At
Level***

even during next **** header*** address is calculated i.e. *Base Address + Offset* [0x4000 + 4 = 0x4004].

*i. e. to move next address just + 4 offset is address but **SkipList * lst** will keep on pointing fixed at **LEVEL's address[0x4000]** and doesn't point to **Node * header**.*

Similarly,

***Node's** pointer variable * **header** will keep on pointing at **0x5000** $\left[\begin{array}{c} \text{Fixed} \\ \text{At} \\ \text{Key} \end{array} \right]$*

*even during next ** **forward** address is calculated i. e. Base Address + Offset [0x5000 + 4 = 0x5004].*

*i. e. to move next address just + 4 offset is address but **Node * header** will keep on pointing fixed at **Key's Address[0x5000]** and doesn't point to **Node * header**.*

*Same for node's double pointer variable ** **Forward** , which will keep pointing at node $\rightarrow$ **forward[0]**'s address fixed : 0x8000 and approaching next addresses of **node $\rightarrow$ forward[1]** , **node $\rightarrow$ forward[2]** , **node $\rightarrow$ forward[3]** even , **node $\rightarrow$ forward[0]** , only math is used :*

** (**n $\rightarrow$ forward + 0**) translates as: $0x8000 + 0 \times 8 \text{ bytes} := 0x8000$, where value(8 bytes address) at **n $\rightarrow$ forward[0]** is stored. $[0_{10}(\text{Decimal}) = 0_{16}(\text{Hex})]$*

** (**n $\rightarrow$ forward + 1**) translates as: $0x8000 + 1 \times 8 \text{ bytes} := 0x8008$, where value(8 bytes address) at **n $\rightarrow$ forward[1]** is stored. $[8_{10}(\text{Decimal}) = 8_{16}(\text{Hex})]$*

** (**n $\rightarrow$ forward + 2**) translates as: $0x8000 + 2 \times 8 \text{ bytes} := 0x8010$, where value(8 bytes address) at **n $\rightarrow$ forward[2]** is stored. $[16_{10}(\text{Decimal}) = 10_{16}(\text{Hex})]$*

** (**n $\rightarrow$ forward + 3**) translates as: $0x8000 + 3 \times 8 \text{ bytes} := 0x8018$, where value(8 bytes address) at **n $\rightarrow$ forward[3]** is stored. $[24_{10}(\text{Decimal}) = 18_{16}(\text{Hex})]$*

Similarly for **update**:

```
struct Node *update[MAX_LEVEL + 1];
```

update[3 + 1 = 4] ,

MAX_LEVEL evaluates to 3. Consequently, the array size calculation is update[3 + 1 = 4].

This means the array is allocated to hold exactly 4 elements.

Before memset call, the update array may look like :

Array Element	Hypothetical Stack Address	Value (Garbage Data)	Pointer Status
update[0]	0x7fff0000 to 0x7fff0007	0x4a3f8921bce45099	Wild Pointer (Points to a random, invalid address)
update[1]	0x7fff0008 to 0x7fff000f	0x0000000000000002	Wild Pointer
update[2]	0x7fff0010 to 0x7fff0017	0x7fffee1245a00881c	Wild Pointer
update[3]	0x7fff0018 to 0x7fff001f	0x0000000000000000	Accidentally NULL (Just by chance)

More simpler way:

	Address	Values	
update[0]	0x7fff0000	0x4a3f8921bce45099	[8 bytes]
update[1]	0x7fff0008	0x0000000000000002	[8 bytes]
update[2]	0x7fff0010	0x7fffee1245a00881c	[8 bytes]
update[3]	0x7fff0018	0x0000000000000000	[8 bytes]

```
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

“memset” stands for Memory Set. It takes a block of memory and rapidly fills it with a specific value. Here ‘0’ or NULL.

	Address	Values
update[0]	0x7fff0000	NULL
update[1]	0x7fff0008	NULL
update[2]	0x7fff0010	NULL
update[3]	0x7fff0018	NULL

And how compiler will translate :

→ $update[0] = * (update + 0)$

→ $update[1] = * (update + 1)$

→ $update[2] = * (update + 2)$

→ $update[3] = * (update + 3)$

And how data will get stored:

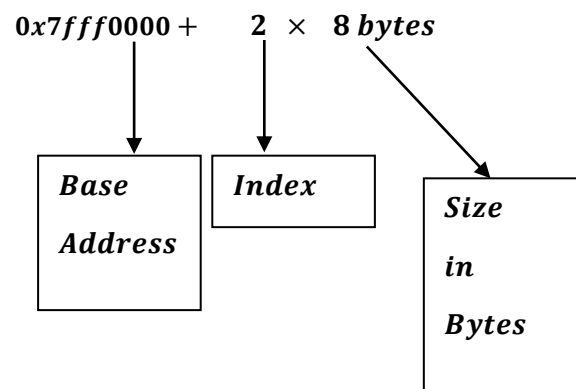
** (update + 0) translates as: $0x7fff0000 + 0 \times 8 \text{ bytes} := 0x7fff0000$, where value(8 bytes address) at **update[0]** is stored. [$0_{10}(\text{Decimal}) = 0_{16}(\text{Hex})$]*

** (update + 1) translates as: $0x7fff0000 + 1 \times 8 \text{ bytes} := 0x7fff0008$, where value(8 bytes address) at **update[1]** is stored. [$8_{10}(\text{Decimal}) = 8_{16}(\text{Hex})$]*

** (**update** + 2) translates as: $0x7fff0000 + 2 \times 8 \text{ bytes} := 0x7fff0010$, where value(8 bytes address) at **update[2]** is stored. [$16_{10}(\text{Decimal}) = 10_{16}(\text{Hex})$]*

** (**n** → **forward** + 3) translates as: $0x7fff0000 + 3 \times 8 \text{ bytes} := 0x7fff0018$, where value(8 bytes address) at **update[3]** is stored. [$24_{10}(\text{Decimal}) = 18_{16}(\text{Hex})$]*

And



```
struct Node *update[MAX_LEVEL + 1];
```

*Node * update is single pointer variable and it will allocate 8 bytes $\times$ (level + 1), if level = 3, then: $8 \times 4 = 32$ bytes of memory and why 8 bytes as * update is a local temporary variable created for rewiring needed for operation like: **insertion** and **deletion** which will hold 8 byte addresses.*

Now, we have got an overview of memory referencing but Why 8 byte addresses? For pointer variables and why 4 bytes addresses for integer variable? The true memory allocation will be discussed in next document.

Skip List

–Memory Allocation

According to 64 – bit Architecture

Basically, we are dealing with integer variable and pointer variable

In 64 – bit architecture :

- *A integer variable is of 4 bytes.*
- *A pointer variable is of 8 – bytes.*

```
struct Node {  
    int key;  
    // Array of pointers to next nodes, one for each level  
    struct Node **forward;  
};  
struct Node *n = createNode(key, rlevel);
```

Formation of Struct Node – An Memory Layout Example

```
n->key = key;  
  
    // Allocate memory for the forward pointer array (level + 1  
elements)  
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level  
+ 1));
```

Say,

int key starts with address : $(0x5000)_{16}$

$$\therefore 5 \times 16^3 + 0 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = (20480)_{10} \Rightarrow \frac{(20480)_{10}}{4 \text{ bytes}} \text{ i.e.}$$

$$\frac{(20480)_{10}}{(4)_{10}} = 5120_{10} \text{ and remainder : } 0, \text{ hence divisible by 4 bytes,}$$

$\therefore$ Data value will get store in $(0x5000)_{16}$.

But as ,int key is of 4 bytes it will hold (0 – 3)offset.

$$(0x5000)_{16} - \text{'0' offset i.e. } (0x5000 + 0) = (0x5000)_{16} \\ = \text{first byte of the integer(key)}$$

$$(0x5001)_{16} - \text{'1' offset i.e. } (0x5000 + 1) = (0x5001)_{16} \\ = 2^{\text{nd}} \text{ byte of the integer(key)}$$

$$(0x5002)_{16} - \text{'2' offset i.e. } (0x5000 + 2) = (0x5002)_{16} \\ = 3^{\text{rd}} \text{ byte of the integer(key)}$$

$$(0x5003)_{16} - \text{'3' offset i.e. } (0x5000 + 3) = (0x5003)_{16} \\ = 4^{\text{th}} \text{ byte of the integer(key)}$$

Say, *key* is = 20

Decimal = $(20)_{10}$

Hexadecimal = $(20)_{10} = (0x14)_{16}$

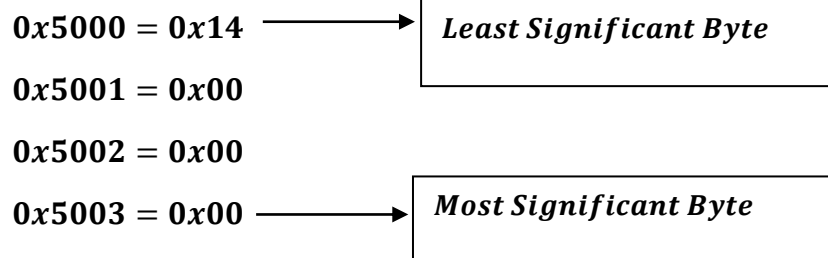
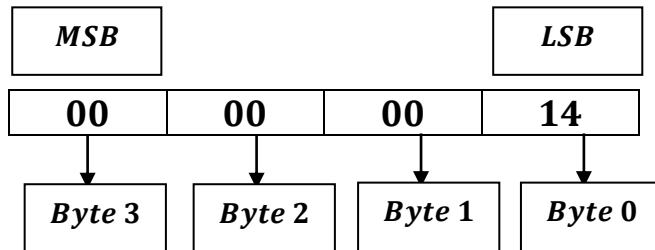
Hex: 0x00000014

Note : 0x00000014 has 8 hexadecimal digits.

Since 1 hexadecimal digit = 4 bits.

$$\Rightarrow 8 \times 4 \text{ bits} = 32 \text{ bits.}$$

$\Rightarrow 32 \text{ bits} \div 8 = 4 \text{ bytes} .$



As we know everything is stored in Memory address at binary format.

$0x14 = 00010100_2$ [8 bits]

$[00000000 \ 00000000 \ 00000000 \ 00010100]_2 = 20_{10}$ [8 × 4 = 32 bits]

$0x5000 = 00010100$ [8 bits]

$0x5001 = 00000000$ [8 bits]

$0x5002 = 00000000$ [8 bits]

$0x5003 = 00000000$ [8 bits]

Next, we have, to start with Address : 0x5004 and we have pointer variable Node ** forward , which will store 8 byte value.

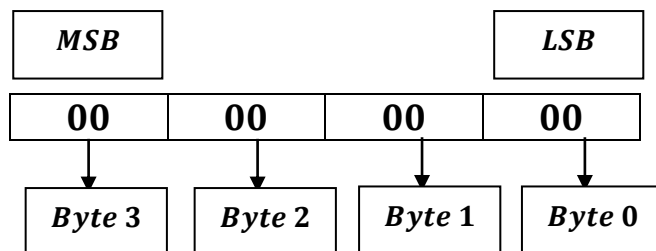
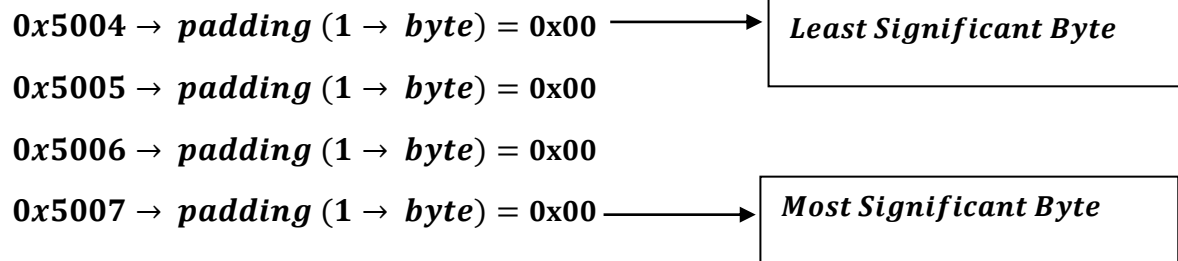
$$\therefore 5 \times 16^3 + 0 \times 16^2 + 0 \times 16^1 + 4 \times 16^0 = (20484)_{10}$$

$$\Rightarrow \frac{(20484)_{10}}{8 \text{ bytes}} \text{ i.e. } \frac{(20484)_{10}}{(8)_{10}} = 2560_{10} \text{ and}$$

remainder : 4 , hence not divisible by 8 bytes .

This will lead to extra 4 bytes padding.

*[Earlier we have started with address divisible by 4 bytes ,
hence next address divisible by 8 bytes after $(8 - 4 = 4)$ i.e. 4 bytes.]*



Represented in 8 bits as:

$$0x5004 \rightarrow \text{padding } (1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$$

$$0x5005 \rightarrow \text{padding } (1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$$

$$0x5006 \rightarrow \text{padding } (1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$$

$$0x5007 \rightarrow \text{padding } (1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$$

*Next, we have, to start with Address : 0x5008 and we have pointer variable Node ** forward , which will store 8 byte value.*

$$\therefore 5 \times 16^3 + 0 \times 16^2 + 0 \times 16^1 + 8 \times 16^0 = (20488)_{10}$$

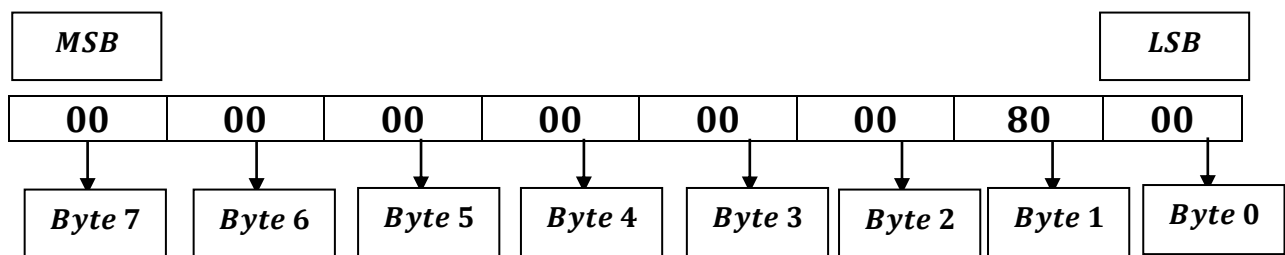
$$\Rightarrow \frac{(20488)_{10}}{8 \text{ bytes}} \text{ i.e. } \frac{(20488)_{10}}{(8)_{10}} = 2561_2 \text{ and}$$

remainder : 0 , hence divisible by 8 bytes .

And it will store : 0x8000 [8 byte address]

$$\text{Note: } (0x8000)_{16} = (1000 \ 0000 \ 0000 \ 0000)_2$$

$$0x8000 = 0x00 \ 00 \ 00 \ 00 \ 00 \ 00 \ 80 \ 00 \text{ [8 hexadecimal digits]}$$



0x5008 → forward double pointer (1 → byte) = 0x00

0x5009 → forward double pointer (1 → byte) = 0x80

0x500A → forward double pointer (1 → byte) = 0x00

0x500B → forward double pointer (1 → byte) = 0x00

0x500C → forward double pointer (1 → byte) = 0x00

0x500D → forward double pointer (1 → byte) = 0x00

0x500E → forward double pointer (1 → byte) = 0x00

0x500F → forward double pointer (1 → byte) = 0x00

*Least
Significant
Byte*

*Most
Significant
Byte*

$$\text{Note: } (0x80)_{16} = 1000 \ 0000_2$$

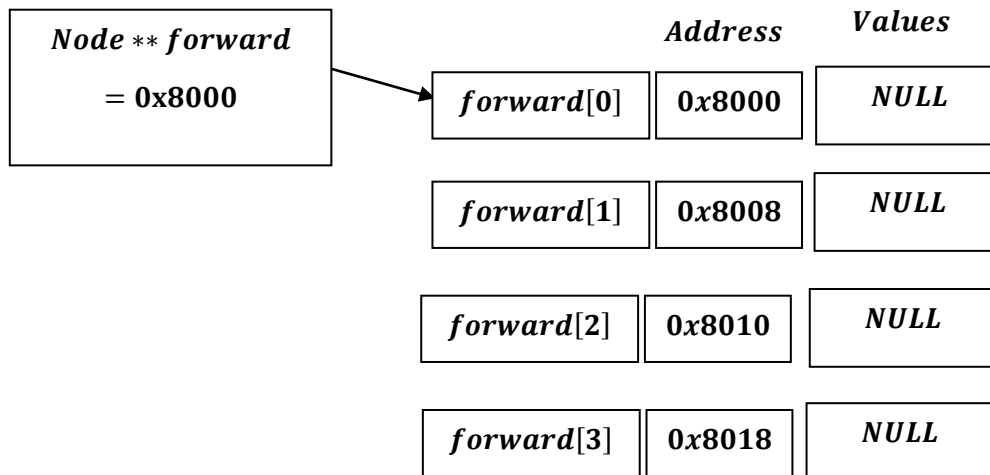
Represented in 8 bits as:

0x5008 → *forward double pointer* (1 → *byte*) = 00000000[8 bits]
0x5009 → *forward double pointer* (1 → *byte*) = 1000 0000 [8 bits]
0x500A → *forward double pointer* (1 → *byte*) = 00000000[8 bits]
0x500B → *forward double pointer* (1 → *byte*) = 00000000[8 bits]
0x500C → *forward double pointer* (1 → *byte*) = 00000000[8 bits]
0x500D → *forward double pointer* (1 → *byte*) = 00000000[8 bits]
0x500E → *forward double pointer* (1 → *byte*) = 00000000[8 bits]
0x500F → *forward double pointer* (1 → *byte*) = 00000000[8 bits]

Now , we can say that:

<i>Addresses</i>	<i>Offset</i>	<i>Size</i>	<i>Content</i>	<i>Description</i>
0x5000 – 0x5003	0 – 3	4 bytes	<i>Integer : key</i>	<i>Integer value(Key)</i>
0x5004 – 0x5007	4 – 7	4 bytes	<i>padding</i>	<i>Empty space (alignment)</i>
0x5008 – 0x500F	8 – 15	8 bytes	<i>double pointer (pointer to Pointer variable named Forward</i>	<i>Address of forward[0] which is a single pointer variable.</i>

Now ,when we come to:



0x8000 is divisible by 8 (byte)hence:

0x8000 → forward single pointer[forward[0]] (1 → byte) = 0x00

0x8001 → forward single pointer[forward[0]] (1 → byte) = 0x00

0x8002 → forward single pointer[forward[0]] (1 → byte) = 0x00

0x8003 → forward single pointer[forward[0]] (1 → byte) = 0x00

0x8004 → forward single pointer[forward[0]] (1 → byte) = 0x00

0x8005 → forward single pointer[forward[0]](1 → byte) = 0x00

0x8006 → forward single pointer[forward[0]](1 → byte) = 0x00

0x8007 → forward single pointer[forward[0]] (1 → byte) = 0x00

Again, 0x8008 is divisible by 8 (bytes)hence:

$0x8008 \rightarrow \text{forward single pointer}[\text{forward}[1]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8009 \rightarrow \text{forward single pointer}[\text{forward}[1]] (1 \rightarrow \text{byte}) = 0x00$
 $0x800A \rightarrow \text{forward single pointer}[\text{forward}[1]] (1 \rightarrow \text{byte}) = 0x00$
 $0x800B \rightarrow \text{forward single pointer}[\text{forward}[1]] (1 \rightarrow \text{byte}) = 0x00$
 $0x800C \rightarrow \text{forward single pointer}[\text{forward}[1]] (1 \rightarrow \text{byte}) = 0x00$
 $0x800D \rightarrow \text{forward single pointer}[\text{forward}[1]] (1 \rightarrow \text{byte}) = 0x00$
 $0x800E \rightarrow \text{forward single pointer}[\text{forward}[1]] (1 \rightarrow \text{byte}) = 0x00$
 $0x800F \rightarrow \text{forward single pointer}[\text{forward}[1]] (1 \rightarrow \text{byte}) = 0x00$

Again, 0x8010 is divisible by 8 (bytes) hence:

$0x8010 \rightarrow \text{forward single pointer}[\text{forward}[2]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8011 \rightarrow \text{forward single pointer}[\text{forward}[2]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8012 \rightarrow \text{forward single pointer}[\text{forward}[2]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8013 \rightarrow \text{forward single pointer}[\text{forward}[2]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8014 \rightarrow \text{forward single pointer}[\text{forward}[2]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8015 \rightarrow \text{forward single pointer}[\text{forward}[2]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8016 \rightarrow \text{forward single pointer}[\text{forward}[2]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8017 \rightarrow \text{forward single pointer}[\text{forward}[2]] (1 \rightarrow \text{byte}) = 0x00$

Again, 0x8018 is divisible by 8 (bytes) hence:

$0x8018 \rightarrow \text{forward single pointer}[\text{forward}[3]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8019 \rightarrow \text{forward single pointer}[\text{forward}[3]] (1 \rightarrow \text{byte}) = 0x00$
 $0x801A \rightarrow \text{forward single pointer}[\text{forward}[3]] (1 \rightarrow \text{byte}) = 0x00$
 $0x801B \rightarrow \text{forward single pointer}[\text{forward}[3]] (1 \rightarrow \text{byte}) = 0x00$
 $0x801C \rightarrow \text{forward single pointer}[\text{forward}[3]] (1 \rightarrow \text{byte}) = 0x00$
 $0x801D \rightarrow \text{forward single pointer}[\text{forward}[3]] (1 \rightarrow \text{byte}) = 0x00$

$$0x801E \rightarrow \textit{forward single pointer}[\textit{forward}[3]](1 \rightarrow \textit{byte}) = 0x00$$

0x801F $\rightarrow$ *forward single pointer*[*forward*[3]] (1 $\rightarrow$ *byte*) = **0x00**

$0x8000 \rightarrow \text{forward single pointer}[\text{forward}[0]] (1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

0x8001 $\rightarrow$ *forward single pointer*[*forward*[0]] (1 $\rightarrow$ *byte*) = 00000000[8 bits]

$0x8002 \rightarrow \text{forward single pointer}[\text{forward}[0]] \ (1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x8003 \rightarrow \textit{forward single pointer}[\textit{forward}[0]] \ (1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$

0x8004 $\rightarrow$ *forward single pointer*[*forward*[0]] (1 $\rightarrow$ *byte*) = 00000000[8 bits]

$$0x8005 \rightarrow \textit{forward single pointer}[\textit{forward}[0]](1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$$
$$0x8006 \rightarrow \textit{forward single pointer}[\textit{forward}[0]](1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$$

$0x8007 \rightarrow \text{forward single pointer}[\text{forward}[0]] \ (1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$$0x8008 \rightarrow \textit{forward single pointer}[\textit{forward}[1]] \ (1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$$
$$0x8009 \rightarrow \text{forward single pointer}[\text{forward}[1]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$$
$$0x800A \rightarrow \textit{forward single pointer}[\textit{forward}[1]] (1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$$
$$0x800B \rightarrow \text{forward single pointer}[\text{forward}[1]] (1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$$
$$0x800C \rightarrow \textit{forward single pointer}[\textit{forward}[1]](1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$$
$$0x800D \rightarrow \textit{forward single pointer}[\textit{forward}[1]](1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$$
$$0x800E \rightarrow \textit{forward single pointer}[\textit{forward}[1]](1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$$

0x800F $\rightarrow$ *forward single pointer*[*forward*[1]] (1 $\rightarrow$ *byte*) = 00000000[8 bits]

$0x8010 \rightarrow \text{forward single pointer}[\text{forward}[2]] \ (1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$$0x8011 \rightarrow \textit{forward single pointer}[\textit{forward}[2]](1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$$

$0x8012 \rightarrow \textit{forward single pointer}[\textit{forward}[2]] \ (1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$

$0x8013 \rightarrow \textit{forward single pointer}[\textit{forward}[2]] \ (1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$

$0x8015 \rightarrow \text{forward single pointer}[\text{forward}[2]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x8016 \rightarrow \text{forward single pointer}[\text{forward}[2]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x8017 \rightarrow \text{forward single pointer}[\text{forward}[2]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x8018 \rightarrow \text{forward single pointer}[\text{forward}[3]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x8019 \rightarrow \text{forward single pointer}[\text{forward}[3]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x801A \rightarrow \text{forward single pointer}[\text{forward}[3]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x801B \rightarrow \text{forward single pointer}[\text{forward}[3]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x801C \rightarrow \text{forward single pointer}[\text{forward}[3]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x801D \rightarrow \text{forward single pointer}[\text{forward}[3]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x801E \rightarrow \text{forward single pointer}[\text{forward}[3]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x801F \rightarrow \text{forward single pointer}[\text{forward}[3]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

Now , we can say that:

<i>Addresses</i>	<i>Offset</i>	<i>Size</i>	<i>Content</i>	<i>Description</i>
$0x8000 - 0x8007$	$0 - 7$	8 bytes	$\text{forward}[0]$	<i>Pointer variable store 8 byte address value.</i>
$0x8008 - 0x800F$	$8 - 15$	8 bytes	$\text{forward}[1]$	<i>Pointer variable store 8 byte address value.</i>
$0x8010 - 0x8017$	$16 - 23$	8 bytes	$\text{forward}[2]$	<i>Pointer variable store 8 byte address value.</i>
$0x8018 - 0x801F$	$24 - 31$	8 bytes	$\text{forward}[3]$	<i>Pointer variable store 8 byte address value.</i>

Now , why so?

*As it is **64 – bit architecture**. 8 bytes = (8 bytes $\times$ 8 bits) = 64 bit . $\left[\begin{array}{l} \text{As, 1 byte} \\ = 8 \text{ bits} \end{array} \right]$*

When 4 byte for integer is added which is (4 byte $\times$ 8 bits) = 32 bits .

*It will add another 8 byte padding – to make it (8 bytes $\times$ 8 bits) = 64 bit ,
if and only if the 4 byte integer is followed by 8 byte pointer to make
8 byte alignment.*

More accurately:

On a typical 64 – bit architecture, an 8 – byte pointer occupies 64 bits.

*A 4 – byte integer occupies 32 bits. If the integer is followed by an 8 – byte
–aligned pointer, the compiler may insert 4 bytes of padding so that
the pointer begins at an 8 – byte – aligned offset.*

For 32 – bit architecture ,

Pointer Variable → 4 bytes [32 bit]

Integer Variable → 4 bytes [32 bit]

Say, **key** is = 20

Decimal = (20)₁₀

Hexadecimal = (20)₁₀ = (0x14)₁₆

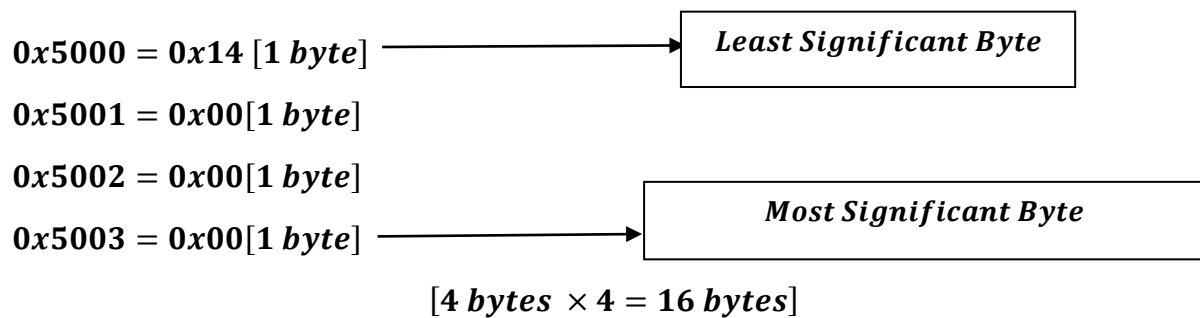
Say int **key** starts with address : (0x5000)₁₆

$$\therefore 5 \times 16^3 + 0 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = (20480)_{10} \Rightarrow \frac{(20480)_{10}}{4 \text{ bytes}} \text{ i. e.}$$

$$\frac{(20480)_{10}}{(4)_{10}} = 5120_{10} \text{ and remainder : 0 , hence divisible by 4 bytes ,}$$

∴ Data value will get store in (0x5000)₁₆.

In 4 byte(32 bit) hexadecimal := 0x00 00 00 14



Represented in 8 bits as:

0x5000 = 00010100 [8 bits]

0x5001 = 00000000 [8 bits]

0x5002 = 00000000 [8 bits]

0x5003 = 00000000 [8 bits]

$$[8 \text{ bits} \times 4 = 32 \text{ bits}]$$

Next we have 4 – byte pointer variable , hence :

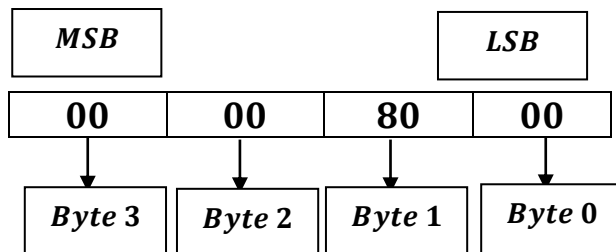
$$\therefore 5 \times 16^3 + 0 \times 16^2 + 0 \times 16^1 + 4 \times 16^0 = (20484)_{10}$$

$$\Rightarrow \frac{(20484)_{10}}{4 \text{ bytes}} \text{ i.e. } \frac{(20484)_{10}}{(4)_{10}} = 5121_{10} \text{ and}$$

remainder : 0 , hence divisible by 4 bytes .

And it will store : 0x8000 [4 byte address]

0x8000 = 0x00 00 80 00 [4 hexadecimal digits]



0x5004 → forward double pointer (1 → byte) = 0x00

0x5005 → forward double pointer (1 → byte) = 0x80

0x5006 → forward double pointer (1 → byte) = 0x00

0x5007 → forward double pointer (1 → byte) = 0x00

Least
Significant
Byte

Most
Significant
Byte

Represented in 8 bit format as :

0x5004 → forward double pointer (1 → byte) = 00000000[8 bits]

0x5005 → forward double pointer (1 → byte) = 1000 0000 [8 bits]

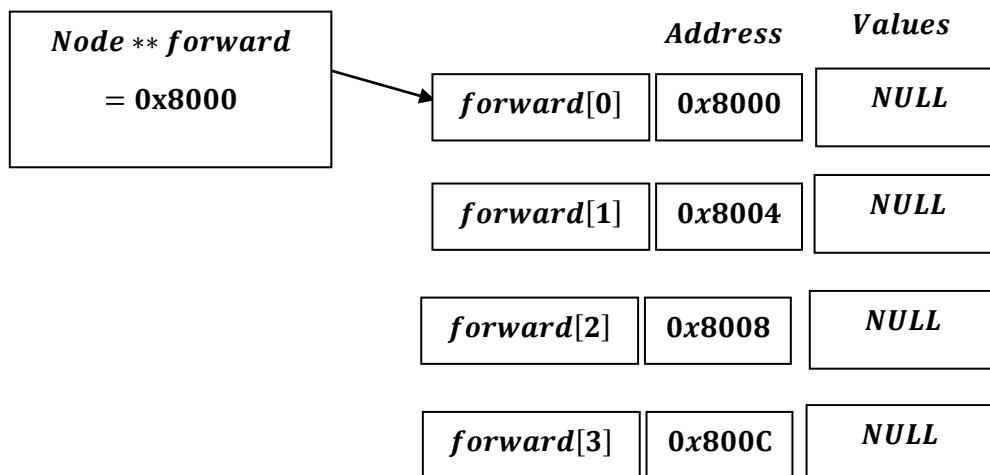
0x5006 → forward double pointer (1 → byte) = 00000000[8 bits]

0x5007 → forward double pointer (1 → byte) = 00000000[8 bits]

Now , we can say that:

<i>Addresses</i>	<i>Offset</i>	<i>Size</i>	<i>Content</i>	<i>Description</i>
0x5000 – 0x5003	0 – 3	4 bytes	<i>Integer : key</i>	<i>Integer value(Key)</i>
0x5004 – 0x5007	4 – 7	4 bytes	<i>double pointer (pointer to Pointer) variable named Forward</i>	<i>Address of forward[0] which is a single pointer variable.</i>

Now , when we come to:



0x8000 is divisible by 4 (byte) hence:

0x8000 → forward single pointer[forward[0]] (1 → byte) = 0x00

0x8001 → forward single pointer[forward[0]] (1 → byte) = 0x00

0x8002 → forward single pointer[forward[0]] (1 → byte) = 0x00

0x8003 → forward single pointer[forward[0]] (1 → byte) = 0x00

Represented in 8 bit format as :

0x8000 → forward single pointer[forward[0]] (1 → byte) = 00000000[8 bits]

0x8001 → forward single pointer[forward[0]] (1 → byte) = 00000000[8 bits]

0x8002 → forward single pointer[forward[0]] (1 → byte) = 00000000[8 bits]

0x8003 → forward single pointer[forward[0]] (1 → byte) = 00000000[8 bits]

0x8004 is divisible by 4 (byte) hence:

0x8004 → forward single pointer[forward[1]] (1 → byte) = 0x00

0x8005 → forward single pointer[forward[1]] (1 → byte) = 0x00

0x8006 → forward single pointer[forward[1]] (1 → byte) = 0x00

0x8007 → forward single pointer[forward[1]] (1 → byte) = 0x00

Represented in 8 bit format as :

0x8004 → forward single pointer[forward[1]] (1 → byte) = 00000000[8 bits]

0x8005 → forward single pointer[forward[1]] (1 → byte) = 00000000[8 bits]

0x8006 → forward single pointer[forward[1]] (1 → byte) = 00000000[8 bits]

0x8007 → forward single pointer[forward[1]] (1 → byte) = 00000000[8 bits]

Again, 0x8008 is divisible by 4 (bytes) hence:

0x8008 → forward single pointer[forward[2]] (1 → byte) = 0x00

0x8009 → forward single pointer[forward[2]] (1 → byte) = 0x00

0x800A → forward single pointer[forward[2]] (1 → byte) = 0x00

0x800B → forward single pointer[forward[2]] (1 → byte) = 0x00

Represented in 8 bit format as :

0x8008 → forward single pointer[forward[2]] (1 → byte) = 00000000[8 bits]

0x8009 → forward single pointer[forward[2]] (1 → byte) = 00000000[8 bits]

0x800A → forward single pointer[forward[2]] (1 → byte) = 00000000[8 bits]

0x800B → forward single pointer[forward[2]] (1 → byte) = 00000000[8 bits]

Again, 0x801C is divisible by 4 (bytes) hence:

0x800C → forward single pointer[forward[3]] (1 → byte) = 0x00

0x800D → forward single pointer[forward[3]] (1 → byte) = 0x00

0x800E → forward single pointer[forward[3]] (1 → byte) = 0x00

0x800F → forward single pointer[forward[3]] (1 → byte) = 0x00

Represented in 8 bit format as :

0x800C → forward single pointer[forward[3]] (1 → byte) = 00000000[8 bits]

0x800D → forward single pointer[forward[3]] (1 → byte) = 00000000[8 bits]

0x800E → forward single pointer[forward[3]] (1 → byte) = 00000000[8 bits]

0x800F → forward single pointer[forward[3]] (1 → byte) = 00000000[8 bits]

<i>Addresses</i>	<i>Offset</i>	<i>Size</i>	<i>Content</i>	<i>Description</i>
0x8000 – 0x8003	0 – 3	4 bytes	<i>forward[0]</i>	<i>Pointer variable store 8 byte address value.</i>
0x8004 – 0x8007	4 – 7	4 bytes	<i>forward[1]</i>	<i>Pointer variable store 4 byte address value.</i>
0x8008 – 0x800B	8 – 11	4 bytes	<i>forward[2]</i>	<i>Pointer variable store 4 byte address value.</i>
0x800C – 0x800F	12 – 15	4 bytes	<i>forward[3]</i>	<i>Pointer variable store 4 byte address value.</i>

On a typical 32 – bit architecture, a 4 – byte pointer occupies 32 bits.

A 4 – byte integer also occupies 32 bits. If the integer is followed by a 4 – byte – aligned pointer, no padding is normally required because the 4 – byte integer already ends at an offset that is aligned to 4 bytes. Thus, the pointer can immediately follow the integer while maintaining 4 – byte alignment.

***Adaptation of derivation of William Pugh's Skip List
Algorithm [1 – based indexing] to our program
[0 – based indexing]***

0 – based indexing:

- ***The Ground Floor : Level 0. Every single node has at least 1 pointer.
(0 coin flips needed, Probability = $p^0 = 1$.)***
- ***The Next Floor : Level 1. To reach Level 1, a node has to win exactly 1
coin flip. (1 coin flips needed, Probability = p^1 .)***
- ***The i^{th} Floor : Level i . To reach Level i , a node has to win exactly i
coin flip. (i coin flips needed, Probability = p^i .)***

Or, we may write :

- ***Level L : Required L consecutive coin flips .Probability is p^L .***

For a list of n elements, the expected number of nodes at level L is given by:

$$***$E[\text{nodes at level } L] = n \cdot p^L$***$$

Analysis suggests that ideally we should start a search at the level L where we

expect $\frac{1}{p}$ nodes i.e. if $p = \frac{1}{2}$ means it has 2 nodes on left or right (equally likely events).

$$***$n \cdot p^L = \frac{1}{p}$***$$

$$***$\Rightarrow p^L \times p = \frac{1}{n}$***$$

$$\Rightarrow p^{L+1} = \frac{1}{n}$$

$$\Rightarrow n \times p^{L+1} = 1$$

$$\text{or, roughly } n \times p^L = 1$$

$$\text{or, } p^L = \frac{1}{n}$$

Solving for L:

$$n = \frac{1}{p^L}$$

$$\Rightarrow n = \frac{1}{p^L}$$

$$\Rightarrow n = \left(\frac{1}{p}\right)^L$$

By definition of logarithm [$b^y = x$, then $\log_b(x) = y$]:

$$\Rightarrow L = \log_{\frac{1}{p}}(n)$$

formally William Pugh write it as:

$$\Rightarrow L(n) = \log_{\frac{1}{p}}(n)$$

Because we are using 0 – based indexing, the ideal expected maximum index (the highest floor we expect to exist) is actually one less than the total number of levels, meaning the expected maximum level index is:

$$L(n) - 1.$$

2. Deriving the expected Maximum Level in a List of `n` Elements and Expected Search Cost through probabilistic analysis:

A **random variable** has a fixed but unpredictable value and a predictable probability distribution and average. If X is a random variable, $\text{Prob}\{X = t\}$ denotes the probability that X equals t and $\text{Prob}\{X > t\}$ denotes the probability that X is greater than t .

Example 1:

Translation: "What is the probability that my die roll (X) exactly equals my target number (t)?"

$\text{Prob}\{X = 6\}$ is exactly $\frac{1}{6}$.

Example 2:

A die has 6 possible outcomes: 1, 2, 3, 4, 5, 6. The question asks for the probability of rolling greater than 3. The winning numbers are 4, 5, and 6. That gives you 3 winning outcomes out of 6 total possibilities.

$$\text{Prob}\{X > 3\} = \frac{3}{6} = \frac{1}{2}.$$

Definition:

Pugh sets up three variables:

- X : A random variable representing the time it takes to run Algorithm A_X .
- Y : A random variable representing the time it takes to run Algorithm A_Y .
- t : A specific time limit (for example, 5 milliseconds)

Definition 1: Probabilistic Equality ($=_{\text{prob}}$)

The formal definition is: $X =_{\text{prob}} Y$ if for all t , $\text{Prob}\{X > t\} = \text{Prob}\{Y > t\}$.

- *The Translation: Algorithm X and Algorithm Y are probabilistically identical.*
- *What it means: No matter what time limit (t) we pick, both algorithms have the exact same chance of taking longer than that limit.*
- *Example: If there is a 10% chance that Algorithm X takes longer than 5 milliseconds, there is also exactly a 10% chance that Algorithm Y takes longer than 5 milliseconds. If they match for every possible time limit, they are $=_{\text{prob}}$.*

Definition 2: Probabilistic Inequality ($\leq_{\text{prob}}$)

The formal definition is: $X \leq_{\text{prob}} Y$ if for all t , $\text{Prob}\{X > t\} \leq \text{Prob}\{Y > t\}$.

We define $X \leq_{\text{prob}} Y$ to be true if and only if for any value t , the probability that X exceeds t is less than the probability that Y exceeds t .

- *The Translation: "Algorithm X is probabilistically faster than (or equal to) Algorithm Y."*
- *What it means: Pick any time limit (t). The chance that Algorithm X misses that deadline is always smaller than (or equal to) the chance that Algorithm Y misses that deadline.*
- *The Pizza Analogy: Imagine X and Y are pizza delivery places, and t is 30 minutes. If Place X has a 5% chance of taking longer than 30 minutes, and Place Y has a 20% chance of taking longer than 30 minutes, then Place X is probabilistically faster. If this remains true whether t is 10 minutes, 30 minutes, or 2 hours, then $X \leq_{\text{prob}} Y$.*

Applying the Bounds to the 0 – Based Skip List

Let M be a random variable representing the actual maximum level present in a list of n elements.

Probability that a single node exceeds level k : Because we are using 0 – based indexing, reaching level k requires k coin flips.

To exceed level k (i.e., reach at least level $k + 1$), it requires $k + 1$ coin flips. Therefore, the probability is p^{k+1} .

Probability that no node in a list of n nodes exceeds level k : $(1 - p^{k+1})^n$.

Bernoulli's Inequality : $(1 - x)^n \geq 1 - nx$

Imagine rolling a 100 sided die. The chance of rolling a `1` is 1%

which means $x = 1\% = \frac{1}{100} = 0.01$.

if we roll it 3 times ($n = 3$), what is the chance we never roll a 1?

- **Exact Math**: $(1 - 0.01)^3 = 0.99^3 \approx 0.970299$
- **Shortcut Math**: $1 - (3 \times 0.01) = 1 - 0.03 = 0.97$

Its visible that 0.970299 is strictly greater than 0.97.

Probability that the maximum level M is greater than k :

$$\text{Prob}(M > k) = 1 - (1 - p^{k+1})^n$$

Here, $(1 - p^{k+1})^n$ is the exact chance that the list's height is k or shorter (failure rate), so subtracting it from 1 (100%) gives the probability that $M > k$.

Using Bernoulli's Inequality, where $(1 - x)^n \geq 1 - nx$ for $x \in [0, 1]$, we get $1 - (1 - x)^n \leq nx$. Plugging this in, we get:

$$\text{Prob}(M > k) = 1 - (1 - p^{k+1})^n < n \times p^{k+1}$$

$$\left[\begin{array}{l} \text{As, } (1 - p^{k+1})^n \text{ is strictly greater than} \\ 1 - np^{k+1} \text{ i. e., } (1 - p^{k+1})^n > 1 - np^{k+1} \\ \text{, the inverted formula} \\ \text{becomes strictly less than:} \\ 1 - (1 - p^{k+1})^n < n \times p^{k+1} \end{array} \right]$$

Negative Binomial Distribution and the Overhang:

A binomial distribution is a statistical probability model that counts the number of successes in a fixed number of independent yes or no trials. NB stands for Negative Binomial distribution. Pugh defines $NB(s, p)$ as the number of failures $(1 - p)$ we see while getting our s^{th} success in a sequence of independent Bernoulli trials with success probability p .

- **Average:** $\mathbb{E}[NB(s, p)] = \frac{s(1-p)}{p}$.
- **Variance:** $\text{Var}(NB(s, p)) = \frac{s(1-p)}{p^2}$.

In the formula $NB(1, 1 - p)$:

- **The 1 (The Target):** This is the s in the formula, meaning we are waiting for exactly 1 success.
- **$1 - p$:** This is the probability of success. Therefore:

$$\mathbb{E}[NB(1, 1 - p)] = \frac{1 \times (1 - (1 - p))}{1 - p} = \frac{p}{1 - p}$$

- **$1 - (1 - p)$ is the probability of failure.**

The Test Subject k (0 – Based Adjustment)

k is an arbitrary variable used to test boundaries. We want to test a level k that sits exactly i floors above our expected target.

For 0 – based indexing, the ideal target maximum index is $L(n) - 1$. Therefore, we set our test level k to:

$$k = (L(n) - 1) + i$$

Substituting this into our probabilistic upper bound:

$$\begin{aligned} \text{Prob}(M > k) &< n \times p^{k+1} \\ \Rightarrow \text{Prob}(M > (L(n) - 1) + i) &< n \times p^{(L(n)-1+i)+1} \\ \Rightarrow \text{Prob}(M > (L(n) - 1) + i) &< n \times p^{L(n)-1+1+i} \\ \Rightarrow \text{Prob}(M > L(n) - 1 + i) &< n \times p^{L(n)+i} \end{aligned}$$

Now, expand the exponents:

$$n \times p^{L(n)+i} = n \times p^{L(n)} \times p^i$$

We know $L(n) = \log_{\frac{1}{p}}(n)$. By taking the reciprocal, we know that

$p^{L(n)} = \frac{1}{n}$. Plugging this in:

$$n \times p^{L(n)} \times p^i = n \times \frac{1}{n} \times p^i = p^i$$

So, we have proven:

$$\text{Prob}(M > L(n) - 1 + i) < p^i$$

Pugh also observes that $\text{Prob}(NB(1, 1 - p) + 1 > i) = p^i$.

$NB(1, 1 - p) + 1$ represents the random extra number of levels by which the maximum node level can extend beyond the target, corresponding to an unusually long sequence of successful promotions.

Because both probabilities equal p^i , we can establish a formal probabilistic upper bound for M :

$$M \leq_{\text{prob}} (L(n) - 1) + NB(1, 1 - p) + 1$$

$$\therefore L(n) - 1 + NB(1, 1 - p) + 1$$

$$\Rightarrow L(n) + NB(1, 1 - p) - 1 + 1$$

$$\Rightarrow L(n) + NB(1, 1 - p)$$

$$\therefore M \leq_{\text{prob}} L(n) + NB(1, 1 - p)$$

Translated into plain English: The maximum height of the entire skip list (M) will safely be less than or equal to our ideal target height, plus a tiny, rapidly shrinking statistical streak of extra coin flips.

Taking the expected value of M (the average of M):

We know,

$$\mathbb{E}[NB(1, 1 - p)] = \frac{p}{1 - p}$$

$$\therefore \mathbb{E}[L(n) + NB(1, 1 - p)] = L(n) + \frac{p}{1 - p}$$

$$\text{Hence, } E[M] \leq L(n) + \frac{p}{1 - p}$$

4. Expected Search Cost

To find the expected search cost, we trace the search path backward (up and to the left) and sum the expected movements.

- *Upward movements from Level 0 to $L(n) - 1$:*

This requires exactly $L(n) - 1$ upward movements.

- *Leftward movements below $L(n) - 1$:*

For every upward move, the expected number of leftward moves is governed by $NB(L(n) - 1, p)$.

$$\mathbb{E}[NB(L(n) - 1, p)] = \frac{(L(n) - 1)(1 - p)}{p}$$

Summing the cost to climb to the target level:

$$\begin{aligned}\mathbb{E}[(L(n) - 1) + NB(L(n) - 1, p)] &= (L(n) - 1) + \frac{(L(n) - 1)(1 - p)}{p} \\ &= (L(n) - 1) \left(1 + \frac{1 - p}{p}\right) \\ &= (L(n) - 1) \left(\frac{p + 1 - p}{p}\right) \\ &= \frac{L(n) - 1}{p}\end{aligned}$$

Leftward movements remaining once at $L(n) - 1$:

Bounded by the number of nodes with a level $\geq L(n) - 1$.

This follows a Binomial distribution $B\left(n, \frac{1}{np}\right)$.

$$\mathbb{E}\left[B\left(n, \frac{1}{np}\right)\right] = n \times \frac{1}{np} = \frac{1}{p}$$

Extra upward movements from $L(n) - 1$ to the maximum level:

As derived earlier, this distance is $NB(1, 1 - p)$ for 0 - based indexing.

$$\mathbb{E}[NB(1, 1 - p)] = \frac{p}{1 - p}$$

Total Expected Search Cost:

Taking expectations gives:

$$\mathbb{E}[\text{search cost}] = \frac{L(n) - 1}{p} + \frac{1}{p} + \frac{p}{1 - p}$$

Simplify the equation:

$$\Rightarrow \mathbb{E}[\text{search cost}] = \frac{L(n) - 1 + 1}{p} + \frac{p}{1 - p}$$

$$\Rightarrow \mathbb{E}[\text{search cost}] = \frac{L(n)}{p} + \frac{p}{1 - p}$$

Plugging in $p = 0.5$ (or $\frac{1}{2}$), we get:

$$\begin{aligned} &= \frac{\log_2(n)}{0.5} + \frac{0.5}{1 - 0.5} \\ &= 2 \times \log_2(n) + \frac{0.5}{0.5} \\ &= 2 \times \log_2(n) + 1 \end{aligned}$$

Because $0 < p < 1$ and the fractional terms resolve to constants, we can conclude that the expected search cost drops the constants and becomes strictly:

$$= O(\log_2 n)$$

Note:

- $\frac{1}{p}$ is a constant. If $p = 0.5$, this is forever exactly 2.
- $\frac{p}{1-p}$ is a constant. If $p = 0.5$, this becomes $\frac{0.5}{0.5}$, which is forever exactly 1.

$$E[\text{search cost}] = \frac{1}{p} \times L(n) + \frac{p}{1-p}$$

$$E[\text{search cost}] = \frac{1}{p} \times \log_{\frac{1}{p}}(n) + \frac{p}{1-p}$$

*Hence, final search cost := **$O(\log n)$***

This derivation according to William Pugh – from his skip list Algorithm.

In a skip list, a node is assigned level i or higher with probability p^{i-1} (or p^i depending on 0 – based vs. 1 – based indexing).

0 – based vs. 1 – based indexing → What does it mean ?

1 – based indexing:

- *When William Pugh wrote the theoretical math for the Skip List he started counting the levels at 1.*
- *The Ground Floor : Level 1. Every single node has at least 1 pointer. (0 coin flips needed, Probability = $p^0 = 1$.)*
- *The Next Floor : Level 2. To reach Level 2, a node has to win exactly 1 coin flip. (1 coin flips needed, Probability = p^1 .)*
- *The i^{th} Floor : Level i . To reach Level i , a node has to win exactly $i - 1$ coin flip. ($i - 1$ coin flips needed, Probability = p^{i-1} .)*

Or, we may write :

- *Level L : Required $L - 1$ consecutive coin flips . Probability is p^{L-1} .*

For a list of n elements, the expected number of nodes at level L is given by:

$$***$E[\text{nodes at level } L] = n \cdot p^L$***$$

Analysis suggests that ideally we should start a search at the level `L` where we expect $\frac{1}{p}$ nodes i. e. if $p = \frac{1}{2}$ means it has 2 nodes on left or right (equally likely events).

$$n \cdot p^L = \frac{1}{p}$$

$$\Rightarrow p^L \times p = \frac{1}{n}$$

$$\Rightarrow p^{L+1} = \frac{1}{n}$$

$$\Rightarrow n \times p^{L+1} = 1$$

$$\text{or, roughly } n \times p^L = 1$$

$$\text{or, } p^L = \frac{1}{n}$$

Solving for L:

$$n = \frac{1}{p^L}$$

$$\Rightarrow n = \frac{1}{p^L}$$

$$\Rightarrow n = \left(\frac{1}{p}\right)^L$$

By definition of logarithm [$b^y = x$, then $\log_b(x) = y$]:

$$\Rightarrow L = \log_{\frac{1}{p}}(n)$$

formally William Pugh write it as:

$$\Rightarrow L(n) = \log_{\frac{1}{p}}(n)$$

2. Deriving the expected Maximum Level in a List of `n` Elements and Expected Search Cost through probabilistic analysis:

A **random variable** has a fixed but unpredictable value and a predictable probability distribution and average. If X is a random variable, $\text{Prob}\{X = t\}$ denotes the probability that X equals t and $\text{Prob}\{X > t\}$ denotes the probability that X is greater than t .

Example 1:

Translation: "What is the probability that my die roll (X) exactly equals my target number (t)?"

$\text{Prob}\{X = 6\}$ is exactly $\frac{1}{6}$.

Example 2:

A die has 6 possible outcomes: 1, 2, 3, 4, 5, 6. The question asks for the probability of rolling greater than 3. The winning numbers are 4, 5, and 6. That gives you 3 winning outcomes out of 6 total possibilities.

$$\text{Prob}\{X > 3\} = \frac{3}{6} = \frac{1}{2}.$$

Definition:

Pugh sets up three variables:

- *X : A random variable representing the time it takes to run Algorithm A_X .*
- *Y : A random variable representing the time it takes to run Algorithm A_Y .*
- *t : A specific time limit (for example, 5 milliseconds)*

Definition 1: Probabilistic Equality ($=_{prob}$)

The formal definition is: $X =_{prob} Y$ if for all t , $Prob\{X > t\} = Prob\{Y > t\}$.

- *The Translation: "Algorithm X and Algorithm Y are probabilistically identical."*
- *What it means: No matter what time limit (t) we pick, both algorithms have the exact same chance of taking longer than that limit.*
- *Example: If there is a 10% chance that Algorithm X takes longer than 5 milliseconds, there is also exactly a 10% chance that Algorithm Y takes longer than 5 milliseconds. If they match for every possible time limit, they are $=_{prob}$.*

Definition 2: Probabilistic Inequality ($\leq_{prob}$)

The formal definition is: $X \leq_{prob} Y$ if for all t , $Prob\{X > t\} \leq Prob\{Y > t\}$.

We define $X \leq_{prob} Y$ to be true if and only if for any value t , the probability that X exceeds t is less than the probability that Y exceeds t .

- *The Translation: "Algorithm X is probabilistically faster than (or equal to) Algorithm Y."*
- *What it means: Pick any time limit (t). The chance that Algorithm X misses that deadline is always smaller than (or equal to) the chance that Algorithm Y misses that deadline.*
- *The Pizza Analogy: Imagine X and Y are pizza delivery places, and t is 30 minutes. If Place X has a 5% chance of taking longer than 30 minutes, and Place Y has a 20% chance of taking longer than 30 minutes, then Place X is probabilistically faster. If this remains true whether t is 10 minutes, 30 minutes, or 2 hours, then $X \leq_{prob} Y$.*

Let M be a random variable representing the actual maximum level present in a list of n elements.

Probability that a single node exceeds level k : $(p^k)^{1 \text{ node}} = p^k$ `k` due to 1 -
based
indexing

Probability that no node in a list of n nodes exceeds level k :

- Probability that no node in a list exceeds level $k \Rightarrow 1 - p^k$
- And total number of nodes is n , hence: $(1 - p^k)^n$

Bernoulli's Inequality : $(1 - x)^n \geq 1 - nx$

Imagine rolling a 100 sided die. The chance of rolling a 1 is 1%

which means $x = 1\% = \frac{1}{100} = 0.01$.

if we roll it 3 times ($n = 3$), what is the chance we never roll a 1?

- **Exact Math**: $(1 - 0.01)^3 = 0.99^3 \approx 0.970299$
- **Shortcut Math**: $1 - (3 \times 0.01) = 1 - 0.03 = 0.97$

Its visible that 0.970299 is strictly greater than 0.97.

“Pugh uses this rule to simplify his probability equation”

Probability that the maximum level M is greater than k :

$$\text{Prob}(M > k) = 1 - (1 - p^k)^n$$

$1 - (1 - p^k)^n$ denotes ``At least one succeeds`` i. e.

Maximum Level $M > \text{Level } k$ and $(1 - p^k)^n$ is the exact chance that the list's height is `k` which is shorter than Maximum height (M), then that the maximum height (M) is greater than k is just 100% minus that failure rate $(1 - p^k)^n$. Here 1 is 100% minus failure rate $(1 - p^k)^n$.

Now, we know ,Bernoulli's Inequality : $(1 - x)^n \geq 1 - nx$ and we have $1 - (1 - p^k)^n$.

Therefore , : $(1 - x)^n \geq 1 - nx$ for $x \in [0, 1]$ (i. e. $0 \leq x \leq 1$)
 $\Rightarrow 1 - (1 - x)^n \leq nx$,

Now plugging this in $\text{Prob}(M > k) = 1 - (1 - p^k)^n$, we get:

$$\text{Prob}(M > k) = 1 - (1 - p^k)^n < n \times p^k \left[\begin{array}{l} \text{As, } (1 - p^k)^n \text{ is strictly greater than} \\ 1 - np^k \text{ i. e., } (1 - p^k)^n > 1 - np^k \\ \text{, the inverted formula} \\ \text{becomes strictly less than:} \\ 1 - (1 - p^k)^n < n \times p^k \end{array} \right]$$

Rewriting $n \times p^k$ using $n = \left(\frac{1}{p}\right)^{L(n)} = p^{-L(n)}$:

$$n \times p^k = \left(\frac{1}{p^k}\right)^{L(n)} = p^{k-L(n)}$$

A binomial distribution is a statistical probability model that counts the number of successes in a fixed number of independent yes – or – no trials.

NB stands for Negative Binomial distribution. Pugh defines $NB(s, p)$ as the number of failures $(1 - p)$ we see while getting our s^{th} success in a sequence of independent Bernoulli trials with success probability p .

Where average:

$$\mathbb{E}[NB(s, p)] = \frac{s(1 - p)}{p}$$

Variance:

$$\text{Var}(NB(s, p)) = \frac{s(1 - p)}{p^2}$$

If ,

The 1 (The Target): This is the s in the formula. It means we are waiting for exactly 1 success. As soon as we get 1 success, the game is over, and we stop counting.

And , $1 - p$ is the probability of success .

Then, it gives rise to $NB(1, 1 - p)$.

where , $s = 1$ and $p = 1 - p$,

$$\text{As, } \mathbb{E}[NB(s, p)] = \frac{s(1 - p)}{p}$$

$$\therefore \mathbb{E}[NB(1, 1 - p)] = \frac{1 \times (1 - (1 - p))}{1 - p} = \frac{p}{1 - p}$$

Where, $1 - (1 - p)$ is the probability of failure .

We know,

- *The i^{th} Floor : Level i . To reach Level i , a node has to win exactly $i - 1$ coin flip. ($i - 1$ coin flips needed, Probability = p^{i-1} .)*

Pugh needs a statistical function that perfectly equals p^i when we ask it for the probability of being greater than i .

To be taller than i , node must reach Level $i + 1$. Level $i + 1$ requires exactly $(i + 1) - 1 = i$ winning coin flips.

Therefore, the probability that a single node is taller than Level i is exactly p^i .

We know,

$$\text{Prob}(M > k) < n \times p^k$$

Note:

- *M (The Reality): This is the actual, physical maximum height that our skip list ended up growing to.*
- *$L(n)$ (The Target): This is the expected, ideal baseline search level.*
Expected nodes at level $L(n)$ or higher = $\frac{1}{p}$.

If there are n elements and each node has probability p of rising from level i to level $i + 1$, then the expected number of nodes with level at least L is

$$np^{L-1} [\mathbb{E}[N_{\geq L}] = np^{L-1}]. \text{ Pugh defines } L(n) = \log_{\frac{1}{p}} n$$

as the level where the expected number of nodes at level $L(n)$ or higher is $\frac{1}{p}$. It is the ideal starting level for a search.

- ***k (The Test Subject): This is an arbitrary variable used to test boundaries.***

$$i = k - L(n).$$

By ,Rearranging this ,we get : $k = L(n) + i$.

Now ,we want to test a level `k` that sits exactly `i` floors above our expected target $L(n)$.

So,we set our test level to $k = L(n) + i$.

$$\begin{aligned} \text{Prob}(M > k) &< n \times p^k \\ \Rightarrow \text{Prob}(M > L(n) + i) &< n \times p^{L(n)+i} \end{aligned}$$

$$\text{Now, } n \times p^{L(n)+i} = n \times p^{L(n)} \times p^i$$

$$\text{But , } L(n) = \log_{\frac{1}{p}}(n)$$

$$\text{So, } p^{L(n)} = p^{\log_{\frac{1}{p}} n}$$

We know, $\left(\frac{1}{p}\right)^L = n$ and we know, $L = \log_{\frac{1}{p}}(n)$, Therefore: $\left(\frac{1}{p}\right)^{\log_{\frac{1}{p}}(n)} = n$,

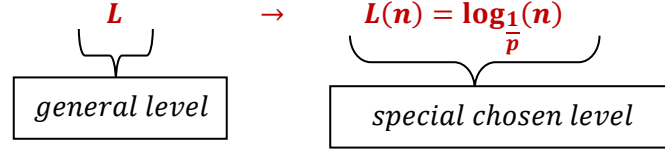
since $\frac{1}{p} = p^{-1}$, we have $p^{-\log_{\frac{1}{p}}(n)} = n$, Taking reciprocal , we get : $p^{\log_{\frac{1}{p}}(n)} = n^{-1} = \frac{1}{n}$.

$$\therefore p^{L(n)} = p^{\log_{\frac{1}{p}} n} = \frac{1}{n}$$

$$\text{Now,plugging it in: } n \times p^{L(n)} \times p^i = n \times \frac{1}{n} \times p^i = p^i.$$

Also, from here we can also prove that : $\mathbb{E}[N_{\geq L}] = np^{L-1} = n \times p^L \times p^{-1}$
 $= n \times p^L \times p^{-1} = n \times p^L \times \frac{1}{p}$, and we know $L(\text{general level}) = \log_{\frac{1}{p}}(n)$,

then Pugh names that particular value as $L(n)$. We can think of it as :



i. e., When $L = L(n)$, then: $n \times p^{L(n)} \times \frac{1}{p} = n \times p^{\log_{\frac{1}{p}}(n)} \times \frac{1}{p} = n \times \frac{1}{n} \times \frac{1}{p} = \frac{1}{p}$.

$$\therefore \mathbb{E}[N_{\geq L}] = \frac{1}{p},$$

Now , coming back:

$$\begin{aligned} \text{Prob}(M > k) &< n \times p^k \\ \Rightarrow \text{Prob}(M > L(n) + i) &< n \times p^{L(n)+i} \\ \Rightarrow \text{Prob}(M > L(n) + i) &< n \times p^{L(n)} \times p^i \\ \Rightarrow \text{Prob}(M > L(n) + i) &< p^i \end{aligned}$$

Pugh also observes :

$$\text{Prob}(NB(1, 1 - p) + 1 > i) = p^i$$

Where, $NB(1, 1 - p) + 1$ represents the few incredibly lucky nodes that randomly flipped ``Heads`` over and over again, growing past the $L(n)$ target.

More precisely, $NB(1, 1 - p) + 1$ represents the random extra number of levels by which the maximum node level can extend beyond $L(n)$, corresponding to an unusually long sequence of successful promotions (the node keeps getting promoted with probability p).

We know,

$$\begin{aligned}\mathbb{E}[NB(1, 1 - p)] &= \frac{p}{1 - p} \\ \therefore \mathbb{E}[L(n) + NB(1, 1 - p)] &= L(n) + \frac{p}{1 - p} \\ \therefore \mathbb{E}[L(n) + NB(1, 1 - p) + 1] &= L(n) + \frac{p}{1 - p} + 1 \\ &= L(n) + \frac{p + 1 - p}{1 - p} \\ &= L(n) + \frac{1}{1 - p}\end{aligned}$$

$$\text{Hence , } E[M] \leq L(n) + \frac{1}{1 - p}$$

Lets rearrange:

$$\begin{aligned}Prob(M > k) &< n \times p^k \\ \Rightarrow Prob(M > L(n) + i) &< n \times p^{L(n)+i} \\ \Rightarrow Prob(M > L(n) + i) &< n \times p^{L(n)} \times p^i \\ \Rightarrow Prob(M > L(n) + i) &< p^i\end{aligned}$$

And ,

$$Prob(NB(1, 1 - p) + 1 > i) = p^i$$

Hence,

$$Prob(M > L(n) + i) < Prob(NB(1, 1 - p) + 1 > i)$$

This allows Pugh to establish a formal probabistic upper bound for M:

$$\text{Therefore , } M \leq_{prob} L(n) + NB(1, 1 - p) + 1$$

The symbol $\leq_{\text{prob}}$ means is probabilistically less than or equal to. Pugh is establishing a rock – solid upper limit on how tall the skip list (M) can possibly get.

Where,

- M : The actual, physical maximum level of the tallest node in our skip list of n elements.
- $\leq_{\text{prob}}$: "Probabilistically less than or equal to." It means, "I guarantee the height will almost never exceed the right side of this equation".
- $L(n)$: The Expected Target. This is the base height we expect a list of n elements to reach.
- Where, $NB(1, 1 - p) + 1$ represents the few incredibly lucky nodes that randomly flipped "Heads" over and over again, growing past the $L(n)$ target.

Translated into plain English, this formula means:

"The maximum height of the entire skip list (M) will safely be less than or equal to our ideal target height ($L(n)$), plus a tiny, rapidly shrinking statistical streak of extra coin flips ($NB(1, 1 - p) + 1$)."

Now,

$$\text{Expected value of } M \text{ or Average of } M \text{ i.e. } E[M] \leq L(n) + \frac{1}{1 - p}$$

Definition (binomial distributions – $B(t, p)$). Let t be a non – negative integer and p be a probability. The term $B(t, p)$ denotes a random variable equal to the number of successes seen in a series of t independent random trials where the probability of a success in a trial is p .

Where average:

$$\mathbb{E}[B(t, p)] = t \times p$$

Variance:

$$\text{Var}(B(t, p)) = t \times p(1 - p)$$

Where,

- $t \rightarrow$ The trials or the total number of attempts we get to flip coins.
- $p \rightarrow$ The probability of success on any single attempt.
- $1 - p \rightarrow$ The probability of failure.

$$\text{We are at: } M \leq_{\text{prob}} L(n) + NB(1, 1 - p) + 1$$

So the maximum level might extend to:

$$L(n) + NB(1, 1 - p) + 1$$

$$\text{Expected value of } M \text{ or Average of } M \text{ i.e. } E[M] \leq L(n) + \frac{1}{1 - p}$$

- $L(n) - 1 \rightarrow$ Upward movements from Level 1 to $L(n)$.
- $NB(L(n) - 1, p) \rightarrow$ Leftward movements needed while finding those $L(n) - 1$ upward moves.

$$\therefore \mathbb{E}[(L(n) - 1 + NB(L(n) - 1, p))], \text{ we know } \mathbb{E}[NB(s, p)] = \frac{s(1 - p)}{p}$$

$$\therefore NB((L(n) - 1, p)) = \frac{(L(n) - 1)(1 - p)}{p}$$

$$\Rightarrow \mathbb{E}[(L(n) - 1 + NB(L(n) - 1, p))] = L(n) - 1 + \frac{(L(n) - 1)(1 - p)}{p}$$

$$\begin{aligned}
&= L(n) - 1 \left(1 + \frac{(1-p)}{p} \right) \\
&= L(n) - 1 \left(\frac{p + 1 - p}{p} \right) \\
&= L(n) - 1 \left(\frac{1}{p} \right) \\
&= \frac{L(n) - 1}{p}
\end{aligned}$$

- $B\left(n, \frac{1}{np}\right) \rightarrow$ *Leftward movements remaining once at $L(n)$, bounded by the number of nodes with level $\geq L(n)$.*

$$p^{L(n)-1} = p^{L(n)} \times p^{-1} = p^{\frac{\log_1(n)}{p}} \times \frac{1}{p} = \frac{1}{n} \times \frac{1}{p} = \frac{1}{np}, \text{ Therefore , } B\left(n, \frac{1}{np}\right).$$

$$\therefore \mathbb{E} \left[B\left(n, \frac{1}{np}\right) \right] = n \times \frac{1}{np} = \frac{1}{p} [As, \mathbb{E}[B(t, p)] = t \times p]$$

- $NB(1, 1-p) + 1 \rightarrow$ *Extra upward movements from $L(n)$ to the unusually high maximum level.*

Where pugh defines M as the maximum level and establishes:

$$M \leq_{prob} L(n) + NB(1, 1-p) + 1$$

Therefore ,once we have already reached $L(n)$, the additional upward distance is:

$$[L(n) + NB(1, 1-p) + 1] - L(n) = NB(1, 1-p) + 1.$$

$$As, \mathbb{E}[NB(s, p)] = \frac{s(1-p)}{p}$$

$$\therefore \mathbb{E}[NB(1, 1-p)] = \frac{1 \times (1 - (1-p))}{1-p} = \frac{p}{1-p}$$

$$\therefore \mathbb{E}[NB(1, 1-p) + 1] = \frac{p}{1-p} + 1 = \frac{p + 1 - p}{1-p} = \frac{1}{1-p}$$

$\therefore$ Cost of the entire search is $:=$

Total cost to climb out of a list of `n` elements

$$\leq \text{prob}(L(n) - 1 + NB(L(n) - 1, p))$$

reach $L(n)$

$$+ B\left(n, \frac{1}{np}\right)$$

left movements at or above $L(n)$

$$+ NB(1, 1-p) + 1$$

climb above $L(n)$

This is what Pugh means by a probabilistic upper bound on the total length of the search path.

Then taking expectations gives :

$\mathbb{E}[\text{search cost}]$

$$= \mathbb{E}[(L(n) - 1 + NB(L(n) - 1, p))] + \mathbb{E}\left[B\left(n, \frac{1}{np}\right)\right] + \mathbb{E}[NB(1, 1-p) + 1]$$

$$= \frac{L(n) - 1}{p} + \frac{1}{p} + \frac{1}{1-p}$$

$$\begin{aligned}
&= \frac{L(n)}{p} + \frac{1}{1-p} \\
&= \frac{L(n)}{p} + \frac{1}{1-p} \\
&= \frac{\log_{\frac{1}{p}}(n)}{p} + \frac{1}{1-p}
\end{aligned}$$

Plugging $p = 0.5$, we get:

$$\begin{aligned}
&= \frac{\log_{\frac{1}{0.5}}(n)}{0.5} + \frac{1}{1-0.5} \\
&= \frac{\log_{\frac{1}{2}}(n)}{\frac{1}{2}} + \frac{1}{1-\frac{1}{2}} \\
&= 2 \times \log_2 n + 2 \\
&= O(2 \times \log_2 n + 2) \\
&= O(\log_2 n)
\end{aligned}$$

Or we can say as $0 < p < 1$ since p is a probability, hence $\frac{1}{1-p}$ and $\frac{1}{p}$ are all constants, hence, $\mathbb{E}[\text{search cost}] = O(\log n)$.

13. *Skip List*

–*Class Tweaks*

```
class Node {
    public: // Must explicitly declare public so SkipList can access
data
    int key;
    Node **forward;

    // Constructor handles node memory allocation via 'new'
    Node(int k, int level) {
        key = k;
        forward = new Node*[level + 1];
        for (int i = 0; i <= level; i++) {
            forward[i] = nullptr;
        }
    }
}
```

*forward = new Node * [level + 1];*

is same as:

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level +
1));
```

*When the compiler reads **new Node * [level + 1];**, it reads it from left to right like a set of instructions:*

1. **new (The Action):** *The compiler tells the OS, "Go to the Heap and carve out a contiguous block of memory."*

2. **Node * (The Data Type):** This answers the question: What is going to be stored in this memory block? It is storing single pointers (**Node ***). Since every single pointer is 8 bytes.
3. **[level + 1] (The Size):** This is how many single pointers we need. If the node's random level is 3, it calculates $3 + 1 = 4$ slots.
4. **The Math:** The compiler automatically calculates **4 slots $\times$ 8 bytes = 32 bytes**.
5. **forward = (The Assignment):** Finally, **new** returns the base memory address of that 32 – byte block (**e.g., 0x8000**) and stores it inside our double pointer variable, **forward**.

```
void clearList() {
    Node *current = header->forward[0];

    // Traverse level 0 and delete every node
    while (current != nullptr) {
        Node *next = current->forward[0];
        delete current; // Calls ~Node() automatically
        current = next;
    }

    // Reset the header to empty
    for (int i = 0; i <= MAX_LEVEL; i++) {
        header->forward[i] = nullptr;
    }
    level = 0;

    cout << "Memory successfully freed. Skip List cleared." <<
endl;
}
```

```
~Node() {
    delete[] forward;
}
```

*The Two – Step Mechanism of **delete***

*When the CPU executes **delete current**;, the C++ runtime performs two distinct actions behind the scenes:*

```
delete current;  
├── 1. Call Destructor (~Node()) → Cleans up internal resources (e.g., delete[] forward)  
└── 2. Free Memory (operator delete) → Releases the Node's own memory back to the Heap
```

Now,

```
~Node() {  
    delete[] forward;  
}
```

is same as:

```
free(current->forward);
```

Now freeing memory by delete keyword:

is same as:

```
free(current);
```
